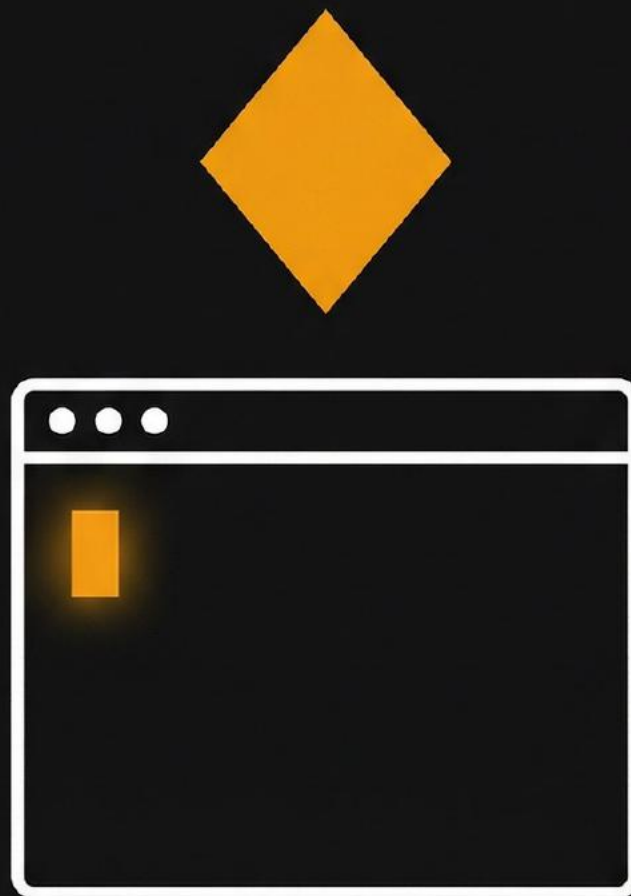


CLAUDE CODE FOR PRODUCT MANAGERS

Harnessing Agentic AI for Strategic Outcomes



Robin Jones

Claude Code for Product Managers

Harnessing Agentic AI for Strategic Outcomes

Robin Jones

Self-Published

© 2026 Robin Jones. All rights reserved.

Claude Code for Product Managers

[Copyright](#)

[Preface](#)

[Who This Book Is For](#)

[What You'll Learn](#)

[What This Book Is Not](#)

[How to Use This Book](#)

[A Note on Currency](#)

[1 Why Claude Code for PMs—Not Claude.ai](#)

[1.1 Choosing the Right Tool Saves Hours Every Week](#)

[1.2 How Claude Code Works: Autonomy Over Conversation](#)

[1.3 Deciding When to Use Which Tool](#)

[1.4 The Three PM Use Cases That Demand Claude Code](#)

[1.5 Know the Trade-offs Before You Start](#)

[1.6 The Two-Tool Mental Model](#)

[2 Setup, Safety Modes, and Cost Management](#)

[2.1 Getting Claude Code Running in 15 Minutes](#)

[2.2 Running Your First Investigation](#)

[2.3 Controlling What Claude Code Can Do](#)

[2.4 Understanding What You're Paying For](#)

[2.5 Tracking and Controlling Spend](#)

[3 Understanding Your Product's Implementation](#)

[3.1 The PM's Codebase Problem](#)

[3.2 Building Your Mental Map of the Codebase](#)

[3.3 Four Questions That Answer Almost Anything](#)

[3.4 Reading Code Without Reading Code](#)

[3.5 Making Claude Code Remember Your Context](#)

[3.6 Knowing When to Escalate to Engineering](#)

[4 Investigating Bugs and User Issues](#)

[4.1 What You Can Triage Without Engineering Help](#)

[4.2 Translating User Complaints into Testable Queries](#)

[4.3 From User Report to Engineering Handoff in 30 Minutes](#)

[4.4 Recognizing Bug Types Before Engineering Does](#)

[4.5 Bug Reports Engineering Actually Wants to Read](#)

- [4.6 Knowing When to Stop](#)
- [5 Market and Competitive Analysis](#)
 - [5.1 Why Claude Code Beats Claude.ai for Research](#)
 - [5.2 Building Competitive Intelligence You Can Update](#)
 - [5.3 Tracking Competitor Pricing Without Spreadsheets](#)
 - [5.4 Creating Defensible Market Estimates](#)
 - [5.5 Separating Signal from Noise in Industry Trends](#)
- [6 Customer Feedback and UX Research Synthesis](#)
 - [6.1 The Feedback Synthesis Challenge](#)
 - [6.2 Getting Your Data Ready for Analysis](#)
 - [6.3 Finding Patterns Across Hundreds of Responses](#)
 - [6.4 Extracting Insights from Interview Transcripts](#)
 - [6.5 From Insight to Action](#)
- [7 Requirements, Personas, and Documentation](#)
 - [7.1 Creating Personas from Scattered Research](#)
 - [7.2 Generating Comprehensive Stories from Feature Concepts](#)
 - [7.3 Writing PRDs That Stay Current](#)
 - [7.4 Keeping Product Docs in Sync with Implementation](#)
- [8 Anatomy of a Skill—Structure and Design Patterns](#)
 - [8.1 From Ad-Hoc Prompts to Repeatable Workflows](#)
 - [8.2 What Is a Skill?](#)
 - [8.3 Skill Components](#)
 - [8.4 The SKILL.md File in Detail](#)
 - [8.5 Design Patterns for PM Skills](#)
 - [8.6 Determinism and Repeatability](#)
 - [8.7 Skill Composition](#)
- [9 Building Your PM Skill Library](#)
 - [9.1 The Starter Kit: Five Essential PM Skills](#)
 - [9.2 Creating Your First Skill Step by Step](#)
 - [9.3 Keeping Your Skills Working Over Time](#)
 - [9.4 Making Skills Adapt to Different Requests](#)
 - [9.5 Tracking Whether Skills Save You Time](#)
- [10 MCP: Connecting to Your PM Stack](#)
 - [10.1 How MCP Eliminates Export-Import Cycles](#)
 - [10.2 Configuring Your First Integration](#)
 - [10.3 Querying and Updating Jira Without Leaving Claude Code](#)
 - [10.4 Pulling Context and Posting Updates Automatically](#)
 - [10.5 Extracting Design Specs Programmatically](#)
 - [10.6 Pulling Metrics Without Writing SQL](#)

- [11 Subagents for Specialized Tasks](#)
 - [11.1 How Subagents Enable Parallel Work](#)
 - [11.2 Four Patterns for Common PM Workflows](#)
 - [11.3 Coordinating Multiple Agents Effectively](#)
 - [11.4 Limitations and When to Avoid Subagents](#)
- [12 Collaborating with Engineering](#)
 - [12.1 Where PM and Engineering Usage Overlaps](#)
 - [12.2 Transitioning Work Without Losing Context](#)
 - [12.3 Maintaining Context Both Roles Can Use](#)
 - [12.4 Building Skills That Serve the Whole Team](#)
- [13 Failure Modes and Boundaries](#)
 - [13.1 Common PM Failure Modes](#)
 - [13.2 Knowing When You've Extracted the Available Value](#)
 - [13.3 Tasks That Belong to Engineering](#)
 - [13.4 Habits That Make Claude Code Work Long-Term](#)
- [Appendix A: Command Reference](#)
 - [Commands You'll Use Every Session](#)
 - [Launching Claude Code with Options](#)
 - [Controlling Claude Code's Autonomy](#)
 - [Faster Navigation and Control](#)
 - [Where Settings Live](#)
 - [Custom Slash Commands](#)
 - [Quick Reference Card](#)
- [Appendix B: SKILL.md Template](#)
 - [Minimal Template](#)
 - [Comprehensive Template](#)
 - [Section One](#)
 - [Section Two](#)
 - [Field Reference](#)
 - [Section Guidance](#)
 - [File Structure](#)
 - [Common Mistakes](#)
- [Appendix C: Sample CLAUDE.md for PM Workflows](#)
 - [File Locations](#)
 - [Sample CLAUDE.md](#)
 - [Section-by-Section Guidance](#)
 - [Size Guidelines](#)
 - [Quick Start](#)
- [Appendix D: Prompt Templates](#)

[Investigation Prompts](#)
[Generation Prompts](#)
[Analysis Prompts](#)
[Quick Customization Guide](#)

Copyright

© 2026 Robin Jones. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Published by: Self-Published

First Edition: 2026

Disclaimer: The information in this book is provided “as is” without warranty of any kind. The author and publisher assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

Claude Code is a product of Anthropic PBC. This book is not affiliated with, endorsed by, or sponsored by Anthropic.

Preface

Who This Book Is For

This book is for product managers who own products with codebases they don't read.

If you've ever waited for engineering to answer "how does this feature work?" or spent hours manually synthesizing customer feedback that could be structured programmatically, this book will show you a better way.

You don't need to write code professionally. You don't need deep CLI experience. You need to be comfortable with the idea that some PM tasks are better done by an agent with file system access than by a conversational AI.

What You'll Learn

By the end of this book, you'll be able to:

- Choose the right AI tool (Claude.ai vs. Claude Code) for each PM task
- Set up and configure Claude Code safely, with cost controls
- Investigate your codebase to answer questions independently
- Synthesize research and feedback into structured artifacts
- Build reusable workflows (skills) for recurring PM tasks
- Connect to your PM stack (Jira, Slack, Figma) via MCP
- Collaborate effectively with engineering while using AI tools

What This Book Is Not

This is not a comprehensive Claude Code manual. It's not a technical reference. It's not for engineers.

This book focuses ruthlessly on PM use cases. If it doesn't solve a problem you actually have, it's not here.

How to Use This Book

If you're new to Claude Code: Read Part I (Foundations) to understand tool selection and setup, then jump to whichever part addresses your most pressing need.

If you're already using Claude Code: Skip to Part II (Codebase Intelligence) or Part IV (Skills) depending on whether you're investigating or automating.

If you're skeptical: Read Chapter 1. If the decision framework doesn't resonate, this tool might not be for you yet.

Each chapter includes exact prompts you can copy, cost estimates, and failure modes. Write in the margins. Skip sections that don't apply. Come back when they do.

A Note on Currency

AI tools evolve quickly. This book focuses on durable concepts—the decision framework for tool selection, the structure of repeatable workflows, the collaboration patterns with engineering—while providing specific examples that may need adaptation as Claude Code changes.

When features change, the principles remain: use the right tool for the task, control your costs, fail gracefully, respect engineering expertise.

Let's begin.

1 Why Claude Code for PMs— Not Claude.ai

1.1 Choosing the Right Tool Saves Hours Every Week

A customer reports a critical bug in your checkout flow. Engineering is heads-down on the quarterly roadmap. You need to triage: real bug or user error? What's the scope? Which team owns it? You wait for engineering to context-switch, investigate, and report back. Two days pass. The bug sits in triage.

Or: You open Claude Code in your repository, ask “How does checkout validation work? Show me where a null payment method could slip through,” and get file references with plain English explanations in 10 minutes. You write up the investigation with specific code locations. Engineering confirms and fixes without the discovery phase. Total elapsed time: 15 minutes.

The difference between these scenarios is tool selection. Most PMs default to Claude.ai for everything because it's familiar, then wonder why AI hasn't changed their workflow. The web interface handles conversational tasks well. It fails at the work that matters most to PMs: investigating codebases, generating artifacts that live in repositories, and building repeatable processes.

You have three ways to access Claude: the web interface at Claude.ai, the API, and Claude Code. The API is for engineers building integrations. Claude.ai and Claude Code are both for PMs, but they solve different problems. Choosing wrong is expensive.

Claude Code is an agent with file system access, shell commands, and the ability to persist context across sessions. These capabilities matter when your work involves files, repositories, or repeatable processes (regardless of whether you write code professionally).

Here's the hidden cost of the wrong choice. You spend 30 minutes on Claude.ai explaining your codebase context, asking it to analyze customer feedback stored in a CSV, and requesting a synthesis document. It gives you a thoughtful response. You copy-paste into a Google Doc. Two weeks later, you need to run the same analysis on new feedback. You start from scratch: same 30 minutes of context, same manual export, same copy-paste. You've now spent an hour on a task that Claude Code handles in five minutes with a reusable skill.

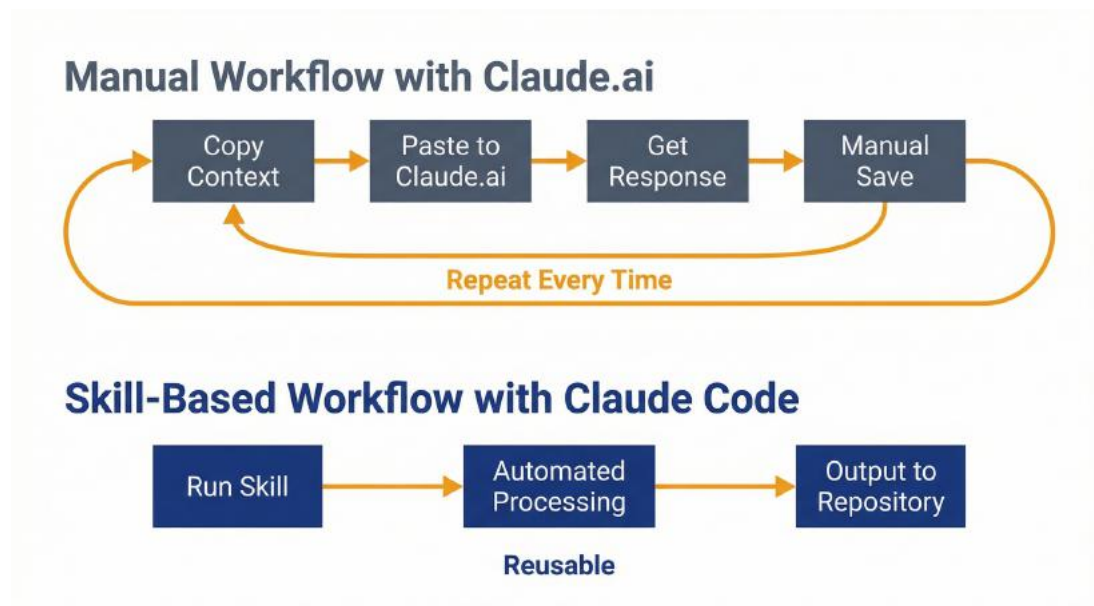


Figure 1.1: Workflow Comparison – Use this to understand the hidden cost of wrong tool selection. Left: Manual copy-paste workflow repeating context, manual export, and reformatting each time. Right: Skill-based automation where setup happens once and subsequent runs take 5 minutes versus 30 minutes.

Or you need to understand how your checkout flow works because a critical bug report came in. You ask engineering, but they're in sprint planning. You try Claude.ai, but you can't upload the entire codebase, and without context, its answers are generic. You wait. Engineering gets back to you tomorrow. The bug sits in triage another day. If you'd used Claude Code, you would have had a root cause hypothesis in 10 minutes, complete with references to the specific files and functions involved.

The tool selection problem centers on task persistence and data access. When work lives in files, involves iteration, or happens more than once,

the web interface forces manual overhead that compounds with each repetition. Claude Code eliminates that overhead by operating directly in your workspace.

Most PMs discover this through waste. They spend weeks using Claude.ai before realizing they've been copying the same context into every conversation, manually reformatting outputs, and redoing work that should be automatic. The switching cost feels high: installation, learning commands, understanding permission modes. But the waste cost is higher, and it accrues silently.

The decision isn't about technical ability. It's about whether your work involves files you'll touch repeatedly, or whether this is a one-time question. Answer that honestly, and the tool picks itself.

1.2 How Claude Code Works: Autonomy Over Conversation

Claude Code is an agentic runtime. That's the technical term. For product managers, it means Claude gets to act instead of just respond. When you ask Claude.ai a question, it thinks and answers. When you ask Claude Code a question, it thinks, reads files, runs commands, generates artifacts, checks its work, and then answers. The difference is autonomy.

This matters because PM work is largely about information synthesis across scattered sources. Requirements live in Jira. Customer feedback lives in Zendesk exports. Implementation details live in the codebase. Market research lives in bookmarked URLs and messy notes. Getting a coherent answer requires pulling from all of these, and doing it manually burns hours.

Claude Code operates in your project directory with five capabilities that Claude.ai fundamentally lacks. Understanding these explains when you need the agent, not just the conversation.

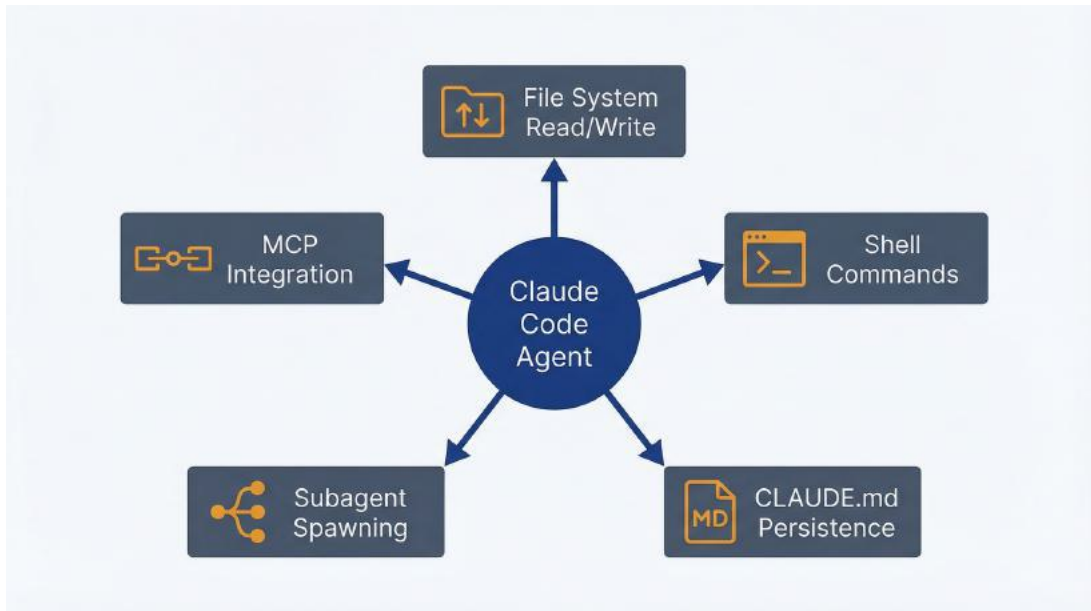


Figure 1.2: Five Core Capabilities – Use this to understand what Claude Code can do that Claude.ai cannot. File system read/write access, shell command execution, persistent project context via CLAUDE.md, subagent spawning for parallel work, and external system integration via MCP. These capabilities enable codebase investigation, artifact generation, and repeatable workflows.

File system read/write access. Claude Code can open any file in your project, read it, understand it, and write new files or modify existing ones. This sounds basic until you realize Claude.ai requires you to copy-paste everything. Need to reference a PRD while analyzing feedback? With Claude.ai, you paste the PRD, paste the feedback, ask for synthesis, then manually save the output. With Claude Code, you point it at `docs/prd.md` and `data/feedback.csv`, and it generates `analysis/synthesis.md` directly in your repo. No clipboard, no manual formatting, no risk of paste errors.

For PMs, this means investigation outputs live where they belong: in version control, alongside the code they reference, accessible to the team. Your codebase analysis becomes a markdown file that engineering can read, comment on, and update.

Shell command execution. Claude Code can run terminal commands. For PMs, this primarily means git operations (checking history, comparing branches, seeing who changed what) and running analysis scripts you wouldn't write yourself but can explain the requirements for.

Concrete example: You need to understand when a particular feature was introduced and who worked on it. You could ask engineering to run `git log` and interpret the results, or you could ask Claude Code: “When was the checkout flow’s two-step verification added, and what was the reasoning?” It runs the git commands, reads the relevant commits and file changes, and tells you the story. No engineering interruption required.

This makes terminal capabilities available through natural language, no expertise required.

Persistent project context via CLAUDE.md. Every time you open Claude.ai, you start fresh. Every time you start Claude Code in a project directory, it reads `CLAUDE.md` if one exists: a file containing project-specific context that persists across all sessions.

For a PM, this file might contain your product domain glossary, key user journeys mapped to code areas, team conventions, links to external documentation, and answers to frequently-investigated questions. You build this context once, and every subsequent session starts informed. No more “here’s what our product does” preamble every conversation.

This is the difference between a smart assistant and a team member. Team members have context. Claude Code with a well-maintained `CLAUDE.md` has context.

Subagent spawning. Claude Code can launch additional Claude instances to work on subtasks in parallel or with specialized focus. As a PM, you rarely invoke this directly, but it underpins complex workflows.

Example: You ask Claude Code to prepare a competitive analysis comparing your product to three competitors across features, pricing, and positioning. Instead of doing this sequentially, it can spawn three subagents (one per competitor) to research in parallel, then aggregate the results into a comparison matrix. What would take you three hours of tab-switching takes 10 minutes.

The cost here is real (you’re running multiple Claude instances simultaneously), but the time savings often justify it for high-value

research tasks.

External system integration via MCP. Model Context Protocol lets Claude Code connect to external tools: Jira, Slack, Figma, databases, analytics platforms. This is the newest capability and the least essential for getting started, but it eliminates the export-import cycle that kills momentum.

Instead of exporting Jira tickets to CSV, uploading to Claude.ai, and copy-pasting results back, Claude Code queries Jira directly, analyzes in place, and updates tickets with findings. The workflow stays in one environment. Chapter 10 covers this in detail, but for now, know that it's possible and powerful once you've mastered the basics.

These five capabilities share a pattern: they eliminate manual handoffs between thinking and doing. Claude.ai thinks; you do the work. Claude Code thinks and does, then shows you the results for approval. One-off questions work fine either way. Repeatable PM workflows benefit from the autonomy.

The agent loop is how Claude Code actually works under the hood. You give it a goal. It plans an approach. It executes a step (reading a file, running a command, generating an artifact). It observes the result. It decides whether to iterate or stop. Then it reports back.

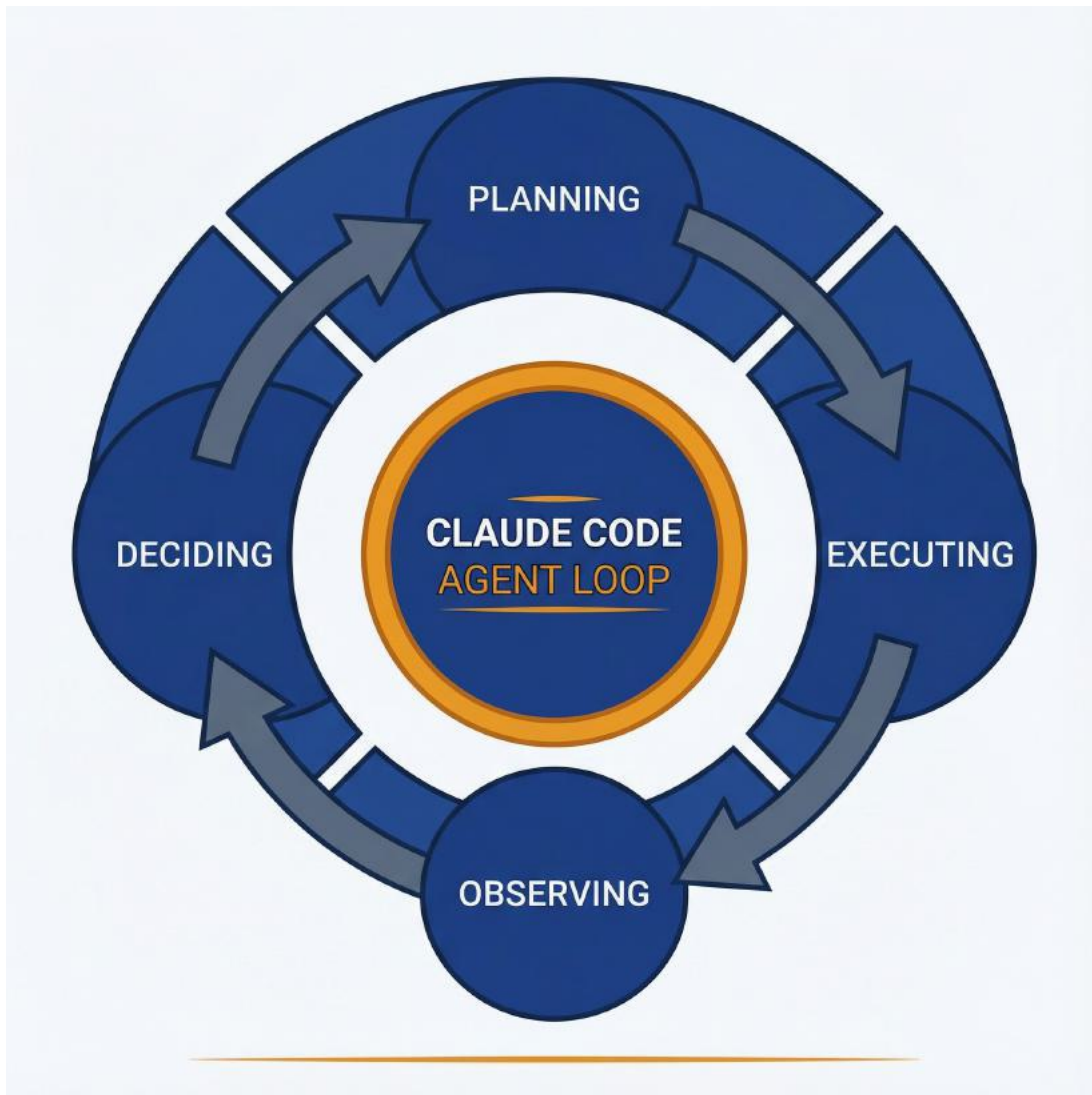


Figure 1.3: The Agent Loop – Use this to understand how Claude Code operates. Continuous cycle: receive goal → plan approach → execute step (read file, run command, generate artifact) → observe result → decide to iterate or stop → report back. This loop is why Claude Code sessions feel different from conversations—you watch an agent work through problems.

This loop is why Claude Code sessions feel different from Claude.ai conversations. You watch an agent work through a problem, seeing each step, and maintaining veto power over actions that matter. The verbosity can feel excessive at first (“I’m going to read this file now”), but it builds trust. You see what it’s doing before it commits changes.

For PMs used to tools that either require manual steps for everything or

operate as black boxes, this middle ground takes adjustment. Claude Code shows its work while doing the work itself. It's a collaborative model where you maintain oversight.

Claude Code is an agent with workspace access and persistence. It's the same model as Claude.ai with different capabilities: file operations, shell commands, and context retention. Those capabilities solve specific PM problems, which is why the next section provides a decision framework for choosing the right tool.

1.3 Deciding When to Use Which Tool

Choosing between Claude.ai and Claude Code comes down to three questions: Does the task require reading or writing files? Will you do this task again? Does the output need to live in version control? If any answer is yes, start with Claude Code. If all three are no, use Claude.ai.

Most PMs get this backwards. They default to Claude.ai because it's familiar, then discover friction when they need to reference files or repeat a workflow. Starting with the right tool saves the migration cost later.

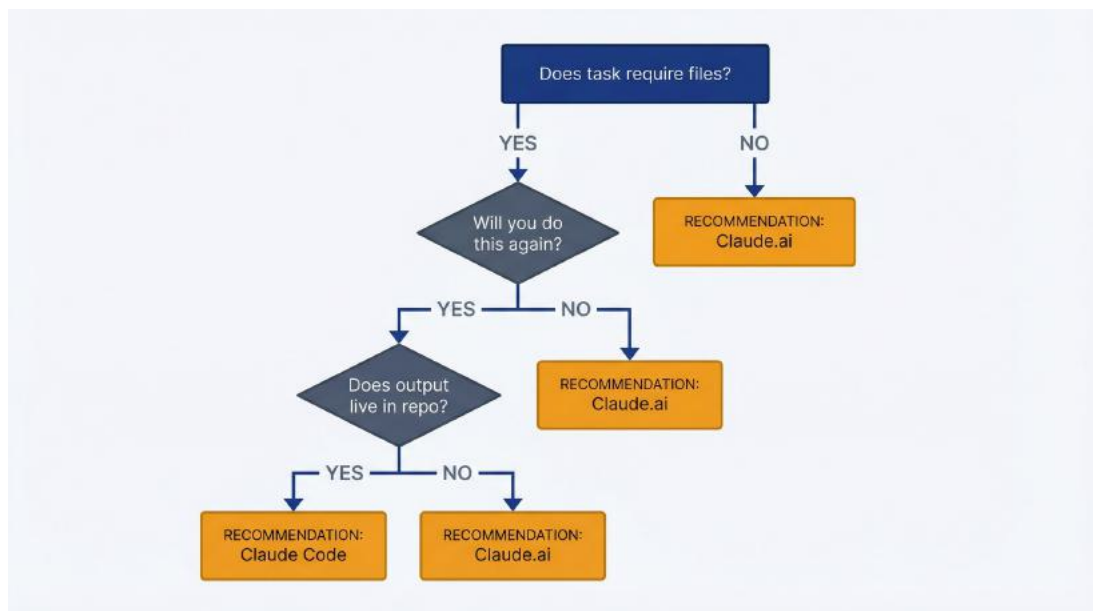


Figure 1.4: Tool Selection Decision Tree – Use this to choose the right tool for each task. Three questions: Does it require files? Will you do it again? Does output need version control? Any “yes” → Claude Code. All

“no” → Claude.ai. Default to Claude Code in read-only mode when uncertain.

Here’s a reference table for twelve common PM scenarios. Bookmark this page.

Task	Tool	Reasoning
Draft stakeholder email	Claude.ai	One-time conversational task, no files needed
Investigate bug report with codebase access	Claude Code	Requires reading code, git history, producing artifact
Brainstorm feature ideas	Claude.ai	Exploratory conversation, doesn’t need persistence
Analyze customer feedback CSV	Claude Code	File input, structured output, likely repeatable
Explain technical concept from docs	Claude.ai	Quick answer, no file operations
Generate release notes from git history	Claude Code	Needs git commands, file output, repeatable monthly
Write PRD section based on research notes	Claude.ai	If notes are in your head; Claude Code if in files
Understand how feature X is implemented	Claude Code	Requires codebase navigation and context persistence
Competitive feature comparison	Claude Code	If repeatable; Claude.ai for one-time

Synthesize user interview transcripts	Claude Code	Multiple file input, structured output artifact
Review and suggest improvements to API documentation	Claude Code	Needs to read docs in repo, suggest file edits
Create user personas from scratch	Claude.ai	Initial draft is conversational; use Code if data-driven

The pattern: Claude.ai for thinking, Claude Code for doing. Tasks that require copying output into another tool signal a better fit for Claude Code.

The “Will I do this again?” heuristic is the fastest decision rule. If you’ll run this analysis weekly, create this report monthly, or investigate this type of issue repeatedly, the setup cost of Claude Code pays back immediately. Even if the first run takes longer (learning the commands, structuring the prompt, saving the skill), the second run is 10× faster.

PMs consistently underestimate repetition. You think you’re doing one competitive analysis, but you’re actually establishing a quarterly review process. You think you’re writing one user story, but you’re actually defining a template you’ll use for the next 50. If there’s any chance you’ll repeat the task, bias toward Claude Code and build the reusable workflow now.

Conversely, if this is genuinely one-time (drafting a specific message, exploring a vague concept, getting a quick answer), Claude.ai is faster. No setup, no learning curve, just ask and move on. The overhead of launching Claude Code in a directory, granting permissions, and structuring file outputs only makes sense when you’ll amortize that cost.

When you’re genuinely uncertain, default to Claude Code in read-only mode. Launch it with the `--permission-mode plan` flag, which prevents any modifications. Ask your question. If Claude Code’s answer involves “I

would read these files and generate this artifact,” you’re in the right tool. If the answer is purely conversational with no file references, you could have used Claude.ai. Over time, this calibration becomes instinctive.

One edge case: collaborative work. If the output needs to be reviewed by engineering, live in the repository, or feed into CI/CD processes, Claude Code is required regardless of repetition. An investigation summary that lives in `docs/` alongside the code it references is more valuable than the same summary in a Slack thread. File-based artifacts persist, link to specific commits, and integrate with team workflows. Conversational outputs disappear.

The framework helps you match tool capabilities to task requirements. Both tools use the same model. The difference lies in file access, command execution, and persistence. Most PM work involves files, iteration, and collaboration. Claude.ai handles 30% of PM use cases elegantly, and Claude Code handles the other 70% that change how you work.

1.4 The Three PM Use Cases That Demand Claude Code

Three categories of PM work require Claude Code’s capabilities. If your task falls into any of these, the decision is made.

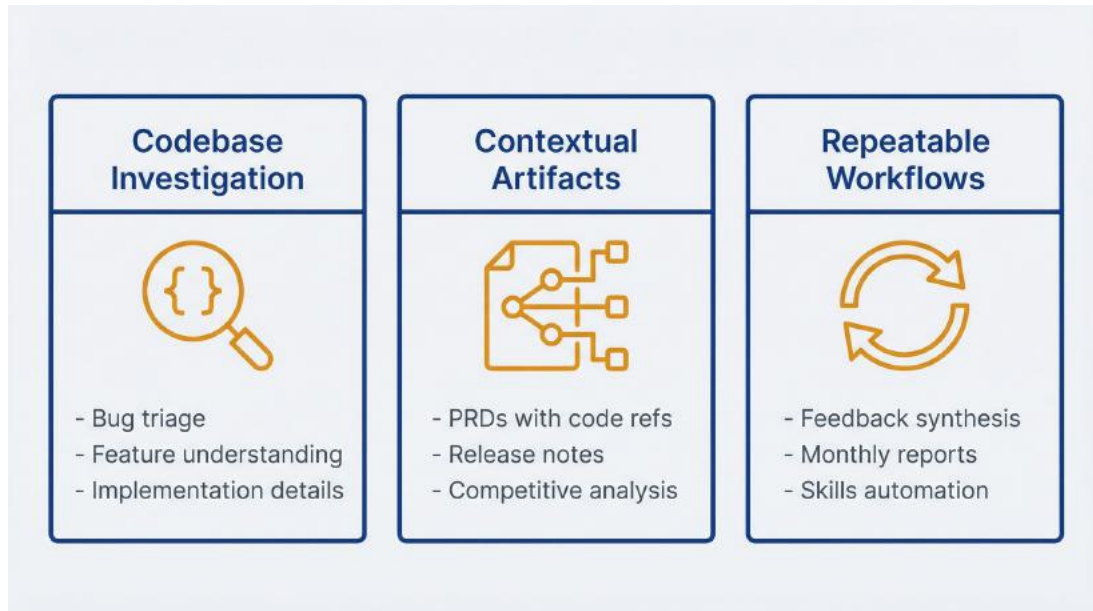


Figure 1.5: Three PM Use Cases – Use this to identify when Claude Code is essential. Codebase investigation (understand implementation without reading code), contextual artifact generation (documents that reference and sync with code), and repeatable workflows (skills that encode recurring PM processes). If your work involves any of these, Claude Code becomes essential infrastructure.

Codebase investigation. You need to understand how something works, but you can't read code fluently and engineering has other priorities. This is the canonical Claude Code use case for PMs.

A customer reports that discount codes stack when they shouldn't. Engineering is heads-down on the quarterly goal. You need to triage: Is this a real bug or user error? If it's real, how critical? What's the likely scope?

Open Claude Code in the repository. Ask: "How does discount code validation work? Show me where multiple discounts could be applied to a single order." Within minutes, you have file references, function explanations in plain English, and a hypothesis about the bug. You write up the investigation with specific code locations. Engineering sees the report, confirms the issue, and fixes it without the usual discovery phase. Total PM time: 15 minutes. Engineering time saved: an hour of investigation plus multiple back-and-forth clarifications.

This pattern repeats constantly. “How does authentication work?” “What happens when a payment fails?” “Where does the app check subscription status?” These questions require reading code, tracing data flow, and understanding implementation details. Claude Code provides these capabilities through natural language queries.

The output lives in `docs/investigations/` or alongside the relevant ticket. Future you benefits when a similar question arises. Future teammates benefit from the documented institutional knowledge. The investigation becomes an artifact, not a lost conversation.

Contextual artifact generation. You need to create a document that references your codebase, references external research, or needs to stay synchronized with code changes. The document’s value comes from its integration with the repository.

Examples: A PRD that cites current API capabilities and links to the relevant files. Release notes generated from git history and Jira tickets, written in your product voice. Competitive analysis that updates quarterly with the same structure and lives in `research/competitors/`. Documentation that stays current as implementation evolves.

Claude.ai can write these documents, but you provide the context manually, save the output manually, and update it manually when things change. Claude Code reads from your repository, pulls from external sources via MCP, generates the artifact in place, and can re-run the workflow when you need the next update.

The first quarterly competitive analysis takes 30 minutes to set up as a skill. The second one takes 5 minutes to run. The third, fourth, and fifth take 5 minutes each. You’ve built a repeatable process that produces consistent output, and the artifact lives in version control where the team can see its evolution.

Repeatable workflows. You do the same type of task regularly: weekly feedback synthesis, monthly release planning, quarterly roadmap updates, per-feature user story generation. The specific inputs change, but the process is stable.

This is where skills matter. A skill is a documented workflow stored in `.claude/skills/` that Claude Code executes on demand. You define the process once: inputs required, steps to execute, output format, quality criteria. Then you invoke it: “Run feedback-synthesis on data/customer-feedback-nov-2024.csv” and Claude Code executes the workflow.

Without skills, you repeat the same instructions every time (explaining the categorization scheme, the output format, the synthesis depth). With skills, the workflow is encoded. You focus on inputs and review outputs. The cognitive overhead drops from “remember exactly how I did this last month” to “run the thing.”

Chapters 8 and 9 cover skills in depth. For now, know that if you do something more than twice, you’re a candidate for building a skill. The effort investment pays back on the third run.

These three use cases (codebase investigation, contextual artifact generation, and repeatable workflows) account for most high-value PM applications of Claude Code. If your work involves none of these, Claude.ai likely suffices. If your work involves all three, Claude Code becomes essential infrastructure.

1.5 Know the Trade-offs Before You Start

Claude Code solves real problems, but it imposes costs and constraints that Claude.ai avoids.

Learning curve. You need to install it, understand permission modes, learn basic commands, and develop intuition about when to use which capabilities. This takes hours of hands-on practice. The web interface requires no installation and works immediately. For PMs who occasionally need AI assistance but rarely work with repositories, the switching cost may exceed the benefit.

Expect the first week to feel inefficient. You’ll fumble with commands, forget to set read-only mode when you meant to, and wonder if this is worth it. By week two, if you’re using it for the right tasks, the efficiency gains become apparent. By week four, you’ll resent having to use

Claude.ai because it feels limited by comparison. But those first few sessions are legitimately awkward.

Cost visibility and budgeting. Claude.ai has subscription pricing: flat monthly fee, unlimited usage within rate limits. Claude Code uses API pricing (pay per token consumed). For exploration-heavy PM work, token consumption can be unpredictable. Reading a large codebase, spawning multiple subagents, or running complex synthesis tasks can burn through your monthly budget in ways that subscription pricing masks.

Chapter 2 covers cost management in detail. The short version: budget \$50-150/month for typical PM usage, monitor with `/cost` after each session, and use `/compact` to reduce context when costs climb. If you're uncomfortable with variable costs or lack budget authority, Claude.ai's fixed pricing may be preferable despite its limitations.

Enterprise deployment. Many organizations restrict API access, require security reviews for tools with file system access, or prohibit connecting to external services. Claude Code requires an Anthropic API key and runs with local file permissions. If your organization's security policies forbid this, you're stuck with whatever official tools they approve (likely the web interface only).

This isn't a limitation of Claude Code; it's a reality of enterprise IT governance. Advocating for approved usage can work, especially if you demonstrate value on a personal project first, but expect a multi-week approval process.

When Claude.ai is genuinely better. Quick questions. Brainstorming sessions. Draft review where you paste in content and want conversational feedback. Anything exploratory where you haven't yet defined the output or don't need file operations. The web interface is faster to open, requires no mental overhead about permissions, and works from any device without installation.

If you find yourself using Claude Code for tasks that fit the web interface better, you're optimizing the wrong thing. Both tools exist because they serve different needs. Use the right one for each task instead of forcing everything through your preferred interface.

1.6 The Two-Tool Mental Model

Two tools, same intelligence, different capabilities. Choose based on task requirements, not on which interface feels more familiar.

Claude.ai: Conversational assistant. You talk, it responds. Every conversation starts fresh. Outputs stay in the conversation history unless you manually copy them elsewhere. Ideal for thinking through problems, getting quick answers, drafting content that doesn't need to integrate with files or repositories. Zero setup cost, zero learning curve, flat subscription pricing.

Claude Code: Autonomous agent with workspace access. You set a goal; it reads files, runs commands, generates artifacts, and reports results. Conversations build on persistent project context from `CLAUDE.md`. Outputs land in your file system, version controlled and accessible to your team. Ideal for codebase investigation, artifact generation, and repeatable workflows. Requires installation and learning, variable API costs, higher cognitive overhead.

The decision framework:

- Does this task involve files? → Claude Code
- Will I do this task again? → Claude Code
- Does the output need to live in the repository? → Claude Code
- Is this a one-time conversational task? → Claude.ai
- Am I brainstorming without defined outputs? → Claude.ai

Most PMs will use both tools regularly. Claude.ai for 30% of tasks, Claude Code for 70%. The mistake is using Claude.ai for everything because it feels easier, then wondering why AI hasn't actually changed your workflow. The workflows that change are the ones where artifacts matter, repetition matters, and integration with your repository matters.

Claude Code handles those. That's why this book exists: to show you how to identify those workflows, build them correctly, and compound the value over time. Chapter 2 covers setup and safety. Let's get you running.

2 Setup, Safety Modes, and Cost Management

2.1 Getting Claude Code Running in 15 Minutes

You've decided Claude Code solves problems you have. Now you need it running without breaking anything. This section gets you from zero to working session in 15 minutes, assuming you have a terminal and can follow commands.

Installation uses platform-specific commands. Choose the one that matches your operating system:

macOS, Linux, or WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows (PowerShell):

```
irm https://claude.ai/install.ps1 | iex
```

macOS with Homebrew:

```
brew install --cask claude-code
```

Any platform with Node.js 18+:

```
npm install -g @anthropic-ai/claude-code
```

The curl/irm method is recommended. It downloads a native binary that works immediately. The npm option requires Node.js 18 or higher and takes longer but works across all platforms. If your organization restricts software installation, you'll need IT approval before proceeding.

Installation takes about two minutes. If you see permission errors on macOS or Linux with the curl method, the script will prompt for your

password. Windows PowerShell may require running as Administrator.

Authentication is handled within Claude Code after installation. You have two options:

Option 1: Claude subscription (Pro, Max, or Team plan). After launching Claude Code, you'll be prompted to authenticate, or you can use:

```
/login
```

This slash command opens your browser to authenticate with your Claude account credentials. Usage is included in your subscription with no per-token billing and no surprise charges. This is the simpler option for most PMs.

Option 2: Anthropic API with usage-based billing. If you don't have a Claude subscription, get an API key from console.anthropic.com/settings/keys. Create a key, copy it immediately (you won't see it again), then set it as an environment variable. Consult your operating system documentation for setting environment variables permanently.

Critical warning: If you have an `ANTHROPIC_API_KEY` environment variable set, Claude Code will use it automatically for API-based billing (pay-as-you-go) and ignore your subscription, even if you have a Pro or Max plan. You'll be billed at API rates instead of using your subscription. If you're using a subscription, make sure this environment variable is not set. Use `/status` to verify which authentication method is active.

Authentication is stored locally in your home directory configuration. Never commit API keys to version control or share them with teammates. Each person needs their own authentication for usage tracking and billing clarity.

Verifying installation: Navigate to any directory and run:

```
claude doctor
```

This checks your installation, shows your version, and confirms

everything is configured correctly. If you see “command not found,” the installation failed. Try reinstalling or check that the installation directory is in your PATH.

Common installation failures:

Network restrictions. Corporate firewalls sometimes block downloads from claude.ai or npm registries. If installation hangs or fails with network errors, you may need to configure your organization’s proxy or request an exception. IT can usually resolve this within a day if you provide the installation URL.

Permission denied. On macOS and Linux, the curl installation script may need elevated permissions to install the binary in a system directory. The script will prompt for your password. If using npm, you may need `sudo npm install -g @anthropic-ai/claude-code` for global installation.

Windows execution policy. PowerShell may block the installation script due to execution policy restrictions. If you see this error, run `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser` before the installation command, or ask IT to adjust the policy.

Node.js version mismatch. If using the npm installation method, Claude Code requires Node.js 18 or higher. Check with `node --version`. If you have an older version, update Node.js first using a version manager like `nvm` (macOS/Linux) or `nvm-windows` (Windows).

Authentication failures. If `/login` opens your browser but authentication fails, check that you have an active Claude Pro, Max, or Team subscription. If using an API key, verify you copied the entire key without extra spaces or line breaks. Regenerate if necessary, since old keys don’t expire automatically.

Configuration uses a hierarchical settings system with three levels:

- **Global:** `~/.claude/settings.json` (applies to all projects)
- **Project:** `.claude/settings.json` (team-shared, committed to git)
- **Local:** `.claude/settings.local.json` (personal overrides, gitignored)

You'll rarely edit these files directly when starting out, but knowing the structure helps later. The files store authentication, default permission mode, and other preferences.

To set a default permission mode (recommended for PMs), create `~/.claude/settings.json` with:

```
{  
  "defaultMode": "plan"  
}
```

This makes every session start in read-only mode until you override it, which is a safe default while learning. Section 2.3 explains why this matters.

Installation is done. You have Claude Code installed, authenticated, and configured. If you hit issues beyond these common failures, check code.claude.com/docs for troubleshooting guides. Most PMs get through installation without problems. The time sink is typically waiting for IT approval, not technical difficulties.

2.2 Running Your First Investigation

Open your terminal and navigate to any git repository you work with. This could be your product's main repo, a documentation repo, or even a personal project. Claude Code works in any directory with files, but it's most useful in repositories you need to understand.

```
cd /path/to/your/repository  
claude
```

The session launches. You see a welcome message, information about what Claude Code can access (the current directory and its subdirectories), and a prompt waiting for input.

The session interface is minimal by design. You have three elements:

1. The prompt: A `>` symbol where you type requests in natural language. No special syntax required. Write as if you're messaging a colleague who

can read files and run commands.

2. Output and status indicators: Claude Code shows you what it's doing. "Reading file X." "Running command Y." "Generating artifact Z." This verbosity feels excessive at first (you know what you asked for), but it builds trust. You see each action before it happens, which matters when operations modify files.

3. Token usage indicators: After each response, you see tokens consumed and approximate cost. This is your budget awareness feedback loop. Ignore it during your first session; start paying attention by session three when you understand what drives costs.

Understanding what Claude Code sees when it starts: Every file and subdirectory in your current location. Git history if you're in a repository. Any `CLAUDE.md` file for persistent context. Environment variables it's allowed to access. It does not automatically read every file (that would consume enormous tokens), but it can read any file you reference or that it determines is relevant to your request.

Try this first request:

What is this repository? Give me a 3-sentence summary of what it does.

Claude Code will read key files like `README.md`, `package.json`, or similar metadata, then provide a summary. The response shows which files it consulted. This teaches you the pattern: you ask questions, it decides what to read, it shows you what it read, then it answers.

Basic commands you need immediately:

`/help`: Shows available commands. You won't memorize these, but you'll reference this list regularly in your first week.

`/status`: Shows current session state, including permission mode, files modified, tokens consumed, and estimated cost.

`/exit`: Ends the session. Your conversation history is saved locally for reference, but context doesn't persist into the next session unless you've

created a `CLAUDE.md` file.

Try a second request:

Show me the five most recently modified files in this repository.

Claude Code runs a git or file system command, displays the results, and explains what it found. You're seeing shell command access in action: capabilities you have through natural language without knowing the exact command syntax.

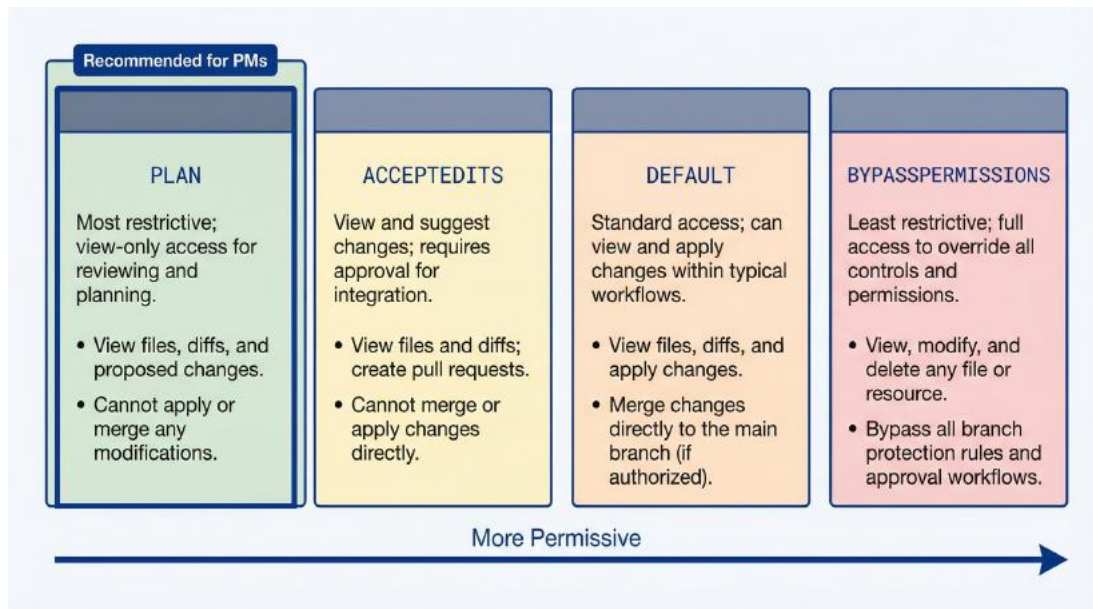
Your first session should last about five minutes. Ask two or three exploratory questions about the repository you're in. Check `/status` to see token consumption. Exit with `/exit`. That's it. You've confirmed the installation works and experienced the core interaction pattern.

The next session will feel less foreign. By the third session, you'll stop thinking about the interface and focus on the work. The learning curve for basic usage is measured in hours, not days, assuming you're working with repositories you already understand at a product level.

2.3 Controlling What Claude Code Can Do

Claude Code can modify files, run arbitrary shell commands, and delete things if you ask it to. This power requires guardrails until you understand what you're doing.

Four permission modes control what Claude Code can do autonomously. As a PM, you'll primarily use three of them.



Permission modes spectrum showing four levels from most restrictive (plan) to least restrictive (bypassPermissions). Plan mode: read files, run read-only commands. AcceptEdits mode: adds file modification. Default mode: adds shell commands with prompts. BypassPermissions mode: full autonomy, no prompts.

The diagram above shows the permission hierarchy. Each mode includes all capabilities of the modes above it, plus additional powers:

plan mode (read-only analysis): Claude Code can read files, run read-only commands like `git log` or `ls`, and answer questions. It cannot modify files, install dependencies, run tests, or execute commands that change state. This is your recommended default mode as a PM until you have a specific need for modifications.

Use this when: Investigating codebases, analyzing data, understanding implementation, triaging bugs. Ninety percent of PM work falls into this category.

acceptEdits mode (auto-approve file modifications): Claude Code can read files and modify them without prompting for each edit. It will still prompt before running shell commands. This mode is efficient when you trust Claude Code to make file changes but want oversight on command execution.

Use this when: Generating release notes, creating PRD documents, updating CLAUDE.md, building skills. Tasks where you need file outputs but aren't running builds or tests.

default mode (prompts for all operations): Claude Code prompts for permission before modifying files or running commands. This provides maximum control (you approve every action) at the cost of more interruptions.

Use this when: Learning Claude Code and want to see every action before it happens, or working on sensitive changes where you want explicit approval.

bypassPermissions mode (complete autonomy): Claude Code can do anything you can do in the terminal without prompting. Run tests, install packages, start servers, make git commits, push to remote repositories. This mode is powerful and risky.

Use this when: Rarely, as a PM. The main use case is working with an engineer to test something or debug an issue where you need full system access and trust Claude Code completely. Most PM workflows never need this mode. This mode requires the `--dangerously-skip-permissions` flag when launching. It's intentionally difficult to enable.

Setting permission mode happens in several ways:

1. **Default in settings.json** (section 2.1) applies to all sessions
2. **Shift+Tab during a session** cycles through the three common modes: default → acceptEdits → plan
3. **Command-line flag when launching:** `claude --permission-mode plan`

The Shift+Tab cycling is fastest once you're in a session. If you start in default mode and realize you just want to investigate without risk of modifications, press Shift+Tab twice to cycle into plan mode. Note that `bypassPermissions` mode cannot be enabled via Shift+Tab. You must use the `--dangerously-skip-permissions` flag when launching, which serves as a safety mechanism.

Plan mode for safe exploration is your training wheels. If you configure it as your default (section 2.1), every session starts safely. You can always press Shift+Tab to cycle to a more permissive mode if you discover you need file modification capabilities. You cannot undo file modifications or executed commands after they complete.

Concrete scenario: You’re investigating a bug and ask Claude Code to show you how authentication works. In `plan` mode, it reads the relevant files and explains. Perfect—you got what you needed. In `bypassPermissions` mode, if you’d phrased your request ambiguously, Claude Code might have tried to run the authentication flow, potentially hitting external services or modifying data. This is unlikely with a clear request, but `plan` mode removes the possibility entirely.

When to use each mode as a PM:

Your Task	Mode	Why
Understand how feature X works	<code>plan</code>	Read-only investigation
Triage a bug report	<code>plan</code>	Read code and git history only
Generate release notes document	<code>acceptEdits</code>	Need to write file output
Create a new skill in <code>.claude/skills/</code>	<code>acceptEdits</code>	Writing skill configuration files
Analyze customer feedback CSV	<code>plan</code>	Read data, answer questions, no file output needed
Update CLAUDE.md with new context	<code>acceptEdits</code>	Modifying repository file Executing test

Run tests to reproduce a bug	default or acceptEdits	commands requires approval
Investigate database schema	plan	Read-only unless you need to run migrations

Setting default modes in configuration was covered in 2.1, but worth repeating: adding `"defaultMode": "plan"` to `~/.claude/settings.json` makes every session start safely. You can always press Shift+Tab to cycle to a more permissive mode when you need file modification or command execution. Defaulting to the most restrictive mode prevents accidents.

What “accidents” look like: You ask Claude Code to “fix the typo in the README” while in `bypassPermissions` mode. It reads `README.md`, identifies the typo, corrects it, and (because you didn’t specify otherwise) makes a git commit with an auto-generated message and pushes to your current branch. This might be helpful, or it might violate your team’s commit conventions and require a force-push to undo. In `acceptEdits` mode, it would have fixed the typo in the file and stopped, letting you review before committing.

These modes exist because Claude Code is powerful enough to cause problems if used carelessly. As a PM, you’re rarely operating close to the edge of what it can do, so restrictive defaults keep you safe while learning.

One clarification: “safe” means “won’t modify things unexpectedly.” Claude Code in any mode will answer questions accurately based on what it reads, but it cannot guarantee its understanding is correct. If you ask “How does authentication work?” and it misinterprets the code flow, giving you wrong information, that’s not a safety issue—that’s an accuracy issue. Permission modes prevent unintended actions, not wrong answers. Section 3.6 addresses limitations of codebase investigation.

2.4 Understanding What You’re Paying For

Claude Code costs money every time you use it, but how you pay depends on your authentication method. If you're using a Claude subscription (Pro, Max, or Team), usage is included and you can ignore most of this section. If you're using the API with usage-based billing, read carefully.

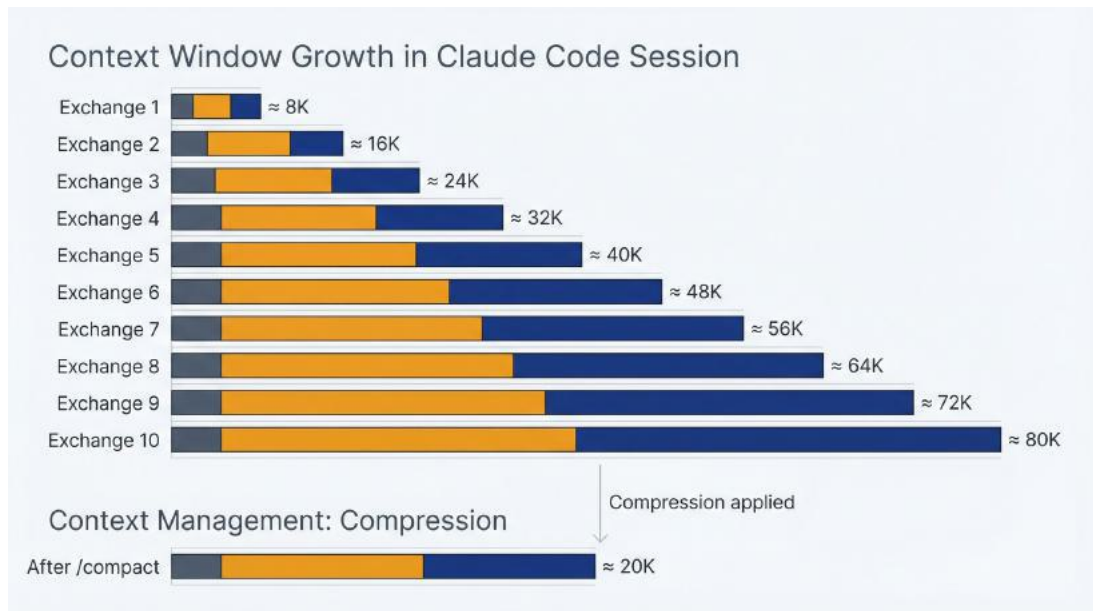
For Claude subscription users: Your usage is bundled into your monthly subscription fee. There are no per-token charges, no surprise bills, and no need to track costs obsessively. The `/cost` command (section 2.5) still shows token consumption for awareness, but it doesn't translate to additional charges. Skip to section 2.5.

For API users: Everything Claude reads or writes counts against your token budget. Input tokens are text you send (your prompts and any files Claude reads to answer). Output tokens are text Claude generates (responses, file contents it writes, explanations). Anthropic's API pricing is roughly \$3 per million input tokens and \$15 per million output tokens for Claude Sonnet, the standard model. Opus costs more, Haiku costs less, but Sonnet is the default for Claude Code and handles PM tasks well.

Translating tokens to practical usage: A token is roughly three-quarters of a word. A 500-word document is about 650 tokens. A typical codebase file might be 200-2,000 tokens depending on length. Your prompt asking Claude Code to investigate something is 50-200 tokens. The response is 200-1,000 tokens.

Do the math: Asking Claude Code to read five files (5,000 tokens input) and provide a summary (500 tokens output) costs approximately \$0.02. Asking it to read fifty files and generate a comprehensive report might cost \$0.50. A deep investigation session where you iterate multiple times across dozens of files might consume 200,000 tokens total, costing \$1-2.

The context window: what it is, why it matters. Every active Claude Code session maintains a context window: the running history of your conversation, all files read, and all outputs generated. This context is how Claude Code "remembers" what you discussed earlier in the session. The context window has a size limit (200,000 tokens for Claude Sonnet at publication time, though this increases regularly).



Context window growth during a session. Shows a bar chart where each exchange adds to the context: initial prompt (small), file reads (medium), responses (medium), follow-up questions (cumulative). By exchange 10, the context has grown significantly. A second bar shows the effect of `/compact`, reducing the accumulated context to a compressed summary.

The diagram illustrates why long sessions get expensive. Each exchange adds to the context, and every new message must process the entire accumulated history. After ten exchanges, you're paying to re-read everything from the beginning with each new question.

For PMs, this manifests as two practical constraints:

1. Long sessions get expensive. Every message you send and every response Claude generates stays in context. After twenty exchanges, you might have 50,000 tokens of context. Each new prompt reads that entire context before responding, so you're paying to re-process previous conversation history with each request. This compounds quietly.

2. Sessions can exceed the context window. If you investigate a large codebase, ask Claude Code to read many files, and iterate extensively, you'll hit the context limit. Claude Code will warn you and can use `/compact` to summarize and reduce context, but be aware that extremely long sessions require management.

Approximate costs per task type:

Task	Tokens	Cost	Duration
Quick question about a file	2,000 - 5,000	\$0.01 - \$0.02	1 minute
Feature investigation (how does X work?)	20,000 - 50,000	\$0.10 - \$0.25	5-10 minutes
Bug triage with codebase analysis	30,000 - 100,000	\$0.15 - \$0.50	10-20 minutes
Generate release notes from git history	40,000 - 80,000	\$0.20 - \$0.40	5-10 minutes
Synthesize customer feedback CSV	50,000 - 150,000	\$0.25 - \$0.75	10-15 minutes
Build a new skill with iteration	60,000 - 120,000	\$0.30 - \$0.60	15-30 minutes
Deep codebase exploration session	100,000 - 300,000	\$0.50 - \$1.50	30-60 minutes

These are estimates. Your actual consumption depends on codebase size, question complexity, and iteration depth. Use /cost (section 2.5) after your first few sessions to calibrate your intuition.

Why PMs burn tokens faster than engineers. Engineers know

what they're looking for. They ask pointed questions: "Show me the implementation of `user.authenticate()`." They get an answer, act on it, and exit. Total tokens: 10,000. Cost: \$0.05.

PMs are exploring. You don't know which file contains the relevant logic. You ask broad questions: "How does authentication work in this app?" Claude Code reads multiple files to build a complete picture. You ask follow-up questions to clarify edge cases. You iterate on the explanation until you understand. Total tokens: 80,000. Cost: \$0.40.

This isn't inefficiency. It's the nature of PM investigation. You're building understanding from zero, while engineers are augmenting existing knowledge. Budget for exploration costs. The alternative is interrupting engineers repeatedly, which has its own cost in team productivity.

Monthly budgeting for typical PM usage patterns (API users only):

Real-world data from Claude Code users shows typical spending: -
Average usage: ~\$100-200/month for daily active users - **Light usage** (5-10 sessions/month, mostly quick questions): \$10-30 - **Moderate usage** (15-25 sessions/month, mix of investigation and generation): \$50-100 - **Heavy usage** (daily sessions, deep investigations, skill building): \$100-200 - **Power usage** (multiple daily sessions, extensive codebase work): \$200-400

If you're spending more than \$400/month as a PM, you're either working on an enormous codebase, running extremely long sessions unnecessarily, or using Claude Code for tasks better suited to other tools. Section 2.5 covers cost management tactics.

The cost feels abstract until you see it on your Anthropic bill. Start by budgeting \$100/month and monitoring actual usage through `/cost`. Adjust up or down based on value delivered. If Claude Code saves you five hours of work in a month, \$100 is a rounding error compared to your fully-loaded cost per hour. If you're spending \$100 and getting minimal value, you're using it wrong, likely for tasks that Claude.ai handles better.

For subscription users: Your costs are fixed at your subscription rate (\$20/month for Pro, \$40/month for Max as of publication). Token

consumption is interesting for understanding session efficiency but doesn't affect your bill.

Token awareness matters whether you're paying per token or using a subscription. Subscription users still benefit from efficient sessions: shorter conversations mean faster answers and less context to manage. API users have the added incentive of direct cost control. Either way, you learn to ask better questions and exit sessions when you have what you need, rather than meandering through conversations that provide diminishing returns.

2.5 Tracking and Controlling Spend

Claude Code gives you three commands to monitor and control spending within sessions. Use them habitually until cost awareness becomes automatic.

/cost: Checking current session spend. Run this command at any point to see tokens consumed (input and output separately), current session cost estimate, and context window usage. This is your budget dashboard.

For subscription users, the dollar amounts shown are estimates of what the session would cost at API rates. This is useful for understanding session efficiency even though you're not paying per token. For API users, these are actual charges.

Use `/cost` after your first few questions in each session to establish a baseline. Use it again before asking something you know will be expensive, like reading fifty files for a comprehensive analysis. Use it at the end of sessions to build intuition about what different types of work cost.

The output shows token counts and estimated costs:

```
Session costs:
Input tokens: 45,230 ($0.14)
Output tokens: 12,100 ($0.18)
Total: $0.32
```

For API users: This tells you exactly what you've spent. If you're ten minutes into a session and already at \$2, something is wrong. Either you're investigating a massive codebase, iterating excessively, or asking questions that require reading far more context than necessary. Adjust your approach or accept the cost if the value justifies it.

For subscription users: This tells you session efficiency. High token counts mean longer sessions and more context to manage, which affects responsiveness even if it doesn't affect your bill.

/compact: Summarizing context to reduce tokens. When session costs climb (or context gets unwieldy) and you need to continue working, `/compact` triggers Claude Code to summarize the conversation history and file contents read so far, replacing the full context with a compressed version. This reduces the context window size and lowers the token cost of subsequent messages.

You can optionally specify what to preserve: `/compact Focus on preserving authentication implementation details` tells Claude Code what matters most in the summary.

Concretely: You've spent 30 minutes investigating a complex feature and accumulated 80,000 tokens of context. Each new question now processes that entire context. For API users, this might cost \$0.25 even for simple requests. For subscription users, it means slower responses. Run `/compact`, and Claude Code condenses the context to 20,000 tokens, keeping the key information and discarding redundancy. Subsequent questions are faster and cheaper.

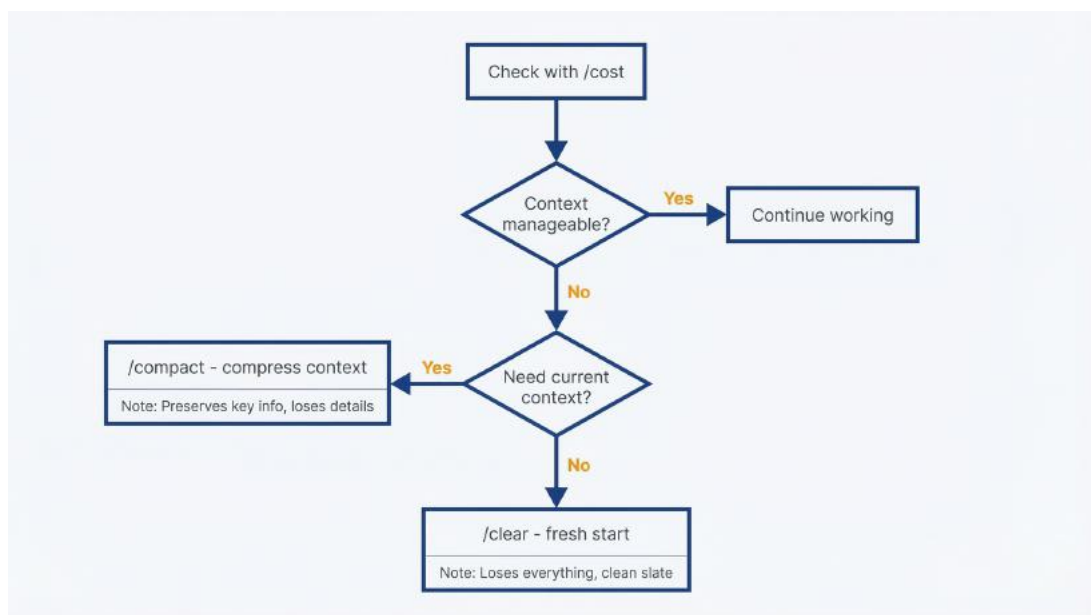
When to use `/compact`:

- Session costs exceed your budget for the task
- You're switching topics within a session and don't need prior context
- You want to continue working but reduce per-message cost
- `/cost` shows context size approaching window limits

Trade-off: Compaction loses details. If you summarize extensive investigation notes, then later ask a question that requires those details, Claude Code won't have them. You might need to re-read files, which

costs tokens again. Use `/compact` when you've finished one phase of work and are moving to another, not in the middle of iterative investigation.

`/clear`: Starting fresh without losing learnings. This command ends the current session and starts a new one in the same directory, with zero context from the previous conversation. Unlike `/exit` followed by `relaunch`, `/clear` is faster and keeps you in the same working state.



Decision flowchart for cost commands. Start: Check session state with `/cost`. If context is manageable and costs acceptable, continue working. If costs high but need to continue on same topic, use `/compact` to compress. If switching to unrelated task or context too bloated, use `/clear` to start fresh. Shows the trade-offs: `/compact` preserves key context but loses details; `/clear` loses everything but gives clean slate.

Use `/clear` when:

- You've finished one task and want to start another unrelated task
- Session costs are high and `/compact` isn't sufficient
- You've gone down an unproductive path and want a clean slate
- Context from previous discussion is confusing current responses

Trade-off: You lose all conversation history. If you haven't saved outputs to files, those outputs are gone. Before running `/clear`, make sure any valuable information from the session is captured in files or notes.

Setting spend alerts and limits (API users): Claude Code doesn't have built-in spend alerts, but you can monitor usage through your Anthropic account dashboard at console.anthropic.com. Set up billing alerts there to notify you when monthly spending exceeds thresholds you define: \$100, \$200, \$300, whatever fits your budget.

Check your dashboard weekly for the first month to calibrate expectations. Most PMs are surprised at how quickly usage adds up when investigating large codebases or running frequent sessions. This is expected. The goal is intentional spending on high-value work, not minimizing costs at the expense of productivity.

Monthly budgeting workflow (API users):

1. Decide your monthly budget based on expected usage (section 2.4 guidelines)
2. Set a billing alert at 80% of that budget in your Anthropic dashboard
3. Use `/cost` after each session to track daily consumption
4. Review actual vs. budget weekly for the first month
5. Adjust budget or usage patterns based on value delivered

If you consistently hit your budget limit mid-month, either increase the budget (if the value justifies it) or reduce session frequency and scope. If you're spending 30% of your budget, you're either being too conservative or have overestimated your needs. Consider using Claude Code more proactively.

Subscription users: Your costs are fixed. Use `/cost` to understand session efficiency, but don't worry about the dollar amounts shown. They're estimates, not charges.

Cost management is about intentionality, not minimization. For API users, spending \$200/month on a tool that saves you 10 hours of work is phenomenal ROI. For subscription users, your \$20-40/month is already spent, so use Claude Code proactively to get value from it. Use these commands to maintain awareness, make informed decisions about session scope, and (for API users) avoid bill shock. After your first month, cost management becomes automatic. You'll know intuitively what tasks consume tokens and whether the efficiency justifies the approach.

You have Claude Code installed, you understand how to run safe sessions, and you know how to monitor costs. Chapter 3 shows you how to use these capabilities for codebase investigation: the highest-value PM use case and the skill that changes how you work with engineering.

3 Understanding Your Product's Implementation

3.1 The PM's Codebase Problem

You own the product but can't read the code. This creates a dependency loop that slows everything down. A customer reports confusing behavior in your pricing calculator. Engineering is midway through sprint. You need answers. Is this a bug or working as designed? If it's a bug, how severe? What's the likely fix complexity? You can't answer these questions yourself, so you interrupt engineering. They context-switch, investigate, report back. Two hours for them, two days for you waiting. The bug sits in triage.

Engineering shouldn't be your only interface to the codebase you're responsible for. You don't need to read code fluently. You need to ask questions and get accurate answers without derailing someone's sprint. That's what Claude Code provides.

The latency cost of context-switching engineers is expensive in ways that don't show up on sprint boards. Every "quick question" is a 15-minute interruption once you account for context switching. Five questions a day is an hour of engineering time plus your own waiting time. Multiply across a quarter and you've burned weeks of productivity on questions you could have answered yourself.

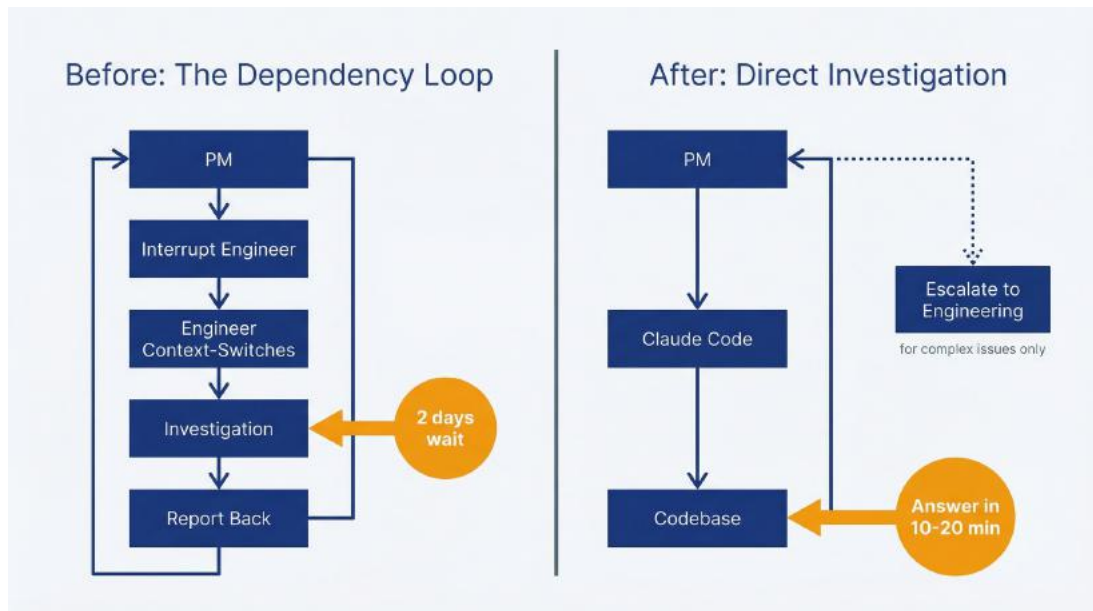


Figure 3.1: Investigation Workflow Comparison – Use this to understand the time savings from direct codebase investigation versus the traditional engineering dependency loop. Left side shows old workflow taking two days with multiple context switches. Right side shows direct PM investigation via Claude Code in 10-20 minutes.

The alternative isn't learning to code. The alternative is asking the codebase directly through an agent that translates implementation into product language. You still escalate complex issues to engineering, but you handle first-pass investigation yourself. This chapter shows you how.

3.2 Building Your Mental Map of the Codebase

Before investigating specific features, spend 20 minutes understanding how the codebase is organized. This is your map. Without it, every investigation starts from zero. With it, you know where to look and what questions to ask.

Run this session once when you first start using Claude Code in a new repository. Update it quarterly or when major architectural changes land. The output becomes a reference document that saves time on every subsequent investigation.

Launch Claude Code in plan mode at the repository root:


```
cd /path/to/your/repository  
claude --permission-mode plan
```

Plan mode is critical here (you're learning, not modifying). Use this prompt:

Prompt: Architecture Overview

Explain this codebase's architecture to a product manager. Cover:

- What the main components are and what each owns
- How data flows through the system
- Where user-facing features typically live
- How frontend and backend connect (if applicable)
- Key abstractions or patterns I should understand

Keep explanations non-technical. Focus on mental models that help me navigate the codebase when investigating features.

Claude Code will read key files (usually README.md, package configuration, directory structure, and a sampling of core modules) and provide a summary. The response takes 5-10 minutes and costs \$0.10-0.25 for typical repositories.

What to expect in the output:

Component breakdown. “The frontend is a React application in `src/client/` that handles all UI. The backend is an Express API in `src/server/` that manages business logic and database access. The `src/shared/` directory contains code used by both.”

Data flow explanation. “When a user submits a form, the React component sends a request to `/api/endpoint`, which routes to a controller in `src/server/controllers/`, calls a service in `src/server/services/` for business logic, and queries the database through models in `src/server/models/`.”

Feature location patterns. “User-facing features are typically implemented as React components in `src/client/components/` with corresponding API endpoints in `src/server/routes/` and business logic in

`src/server/services/.`”

Key abstractions. “The codebase uses a repository pattern: database access goes through repository classes in `src/server/repositories/` rather than direct queries. Authentication uses JWT tokens validated by middleware in `src/server/middleware/auth.js`.”

This isn’t comprehensive documentation. It’s a starting orientation. You’re building intuition about where things live and how they connect. When you later investigate “how does checkout work,” you’ll know to look at client components, server routes, and payment service integration.

Save the output for future reference. Copy the response into a file like `docs/architecture-overview.md` or add it to your `CLAUDE.md` (section 3.5). This becomes your quick-reference guide. When you need to refresh your memory in three months, you read the doc instead of running the session again.

Follow-up questions that add value:

After the initial overview, ask clarifying questions about areas relevant to your product domain:

Where does user authentication happen? How do we determine what a logged-in user can access?

How do we handle payments? What third-party services are we integrated with?

Where is pricing logic implemented? Is it frontend, backend, or both?

These questions cost another 5-10 minutes and \$0.10-0.20, but they fill in details specific to your product. Generic architecture matters less than knowing where critical business logic lives.

When this session isn’t sufficient: Small repositories (under 50 files) with clear structure don’t need this. You can learn the layout through investigation as you go. Massive monoliths (thousands of files) require multiple focused sessions rather than one overview. You’ll need to understand specific subsystems separately.

This alignment session is your foundation. It pays back immediately the first time you investigate a feature and already know which directories to focus on. Engineers take this knowledge for granted because they work in the codebase daily. You're building the same mental map, just through conversation instead of code reading.

3.3 Four Questions That Answer Almost Anything

Most PM questions about the codebase fall into four patterns. Learn these patterns and you can investigate almost anything without engineering help.

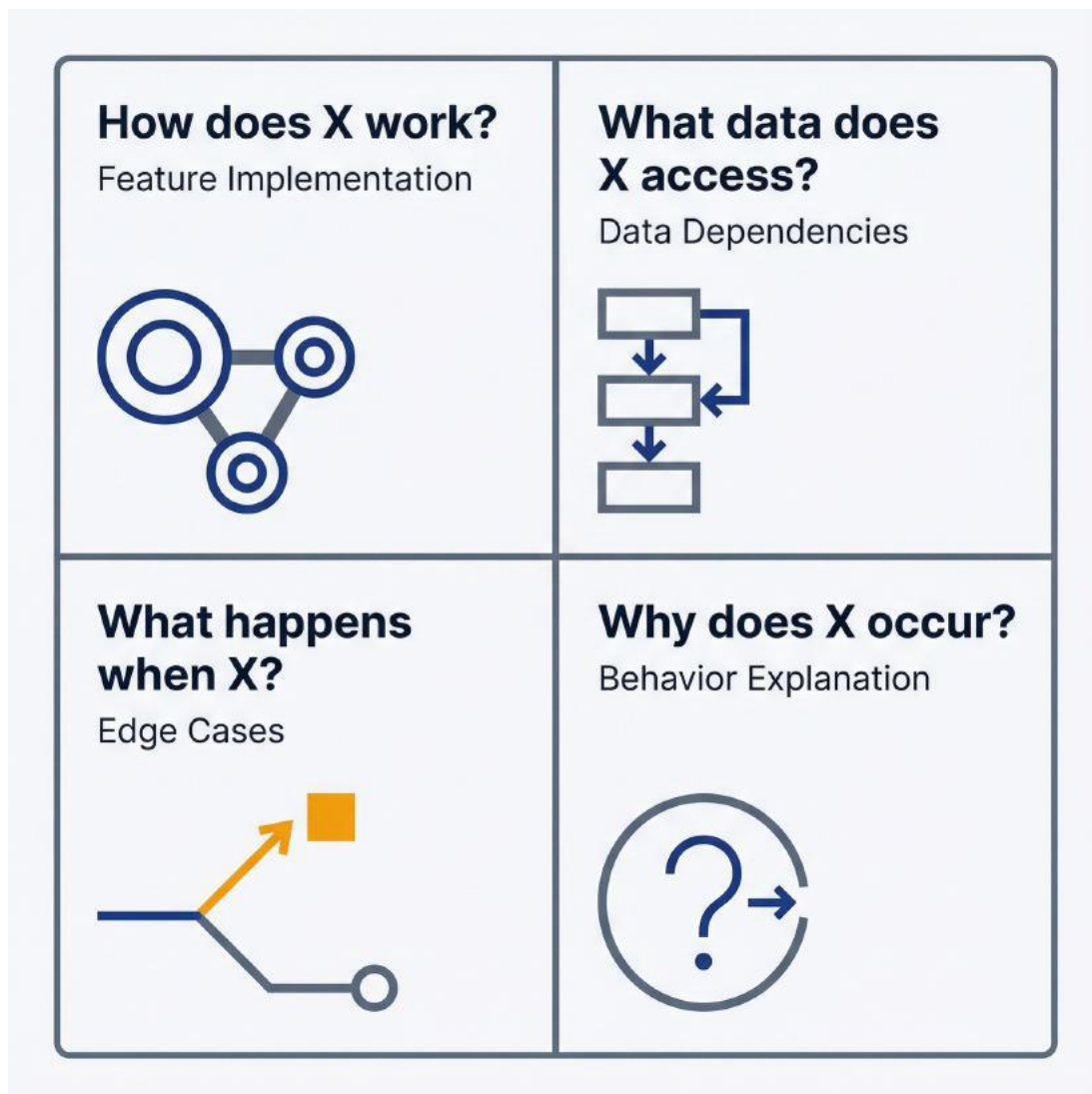


Figure 3.2: Four Investigation Patterns – Use this to structure your codebase queries. The 2x2 grid covers: “How does X work?” (feature implementation), “What data does X access?” (data dependencies), “What happens when X?” (edge cases), and “Why does X occur?” (behavior explanation). These four patterns handle 90% of PM investigations.

Pattern 1: “How does [feature] work?”

You need to understand implementation to answer product questions. A customer asks why discount codes can’t be combined with promotions. Engineering built this, but you don’t know the reasoning. Instead of asking engineering to explain, ask the codebase.

Launch Claude Code in plan mode and use this prompt:

Prompt: Feature Implementation

How does discount code validation work in this application?
Specifically:

- Where is discount code logic implemented?
- What rules determine if a discount code can be applied?
- Can discount codes be combined with other promotions? Why or why not?
- What happens when a user enters an invalid code?

Explain in product terms, not code details.

Claude Code traces the feature from user action (entering a code) through validation logic to outcome (discount applied or error shown). You get file references with plain English explanations. “Discount validation happens in `src/services/discount-validator.js:45–78`. The system checks if the code is active, not expired, and meets minimum cart value requirements. Combination with promotions is prevented by a business rule in line 92 that rejects codes when `cart.hasActivePromotion` is true.”

This gives you the answer (can’t combine because of explicit business logic) and the context to explain to customers or stakeholders. It also shows you where to look if requirements change. You now know which

file owns discount logic.

Pattern 2: “What data does [feature] access?”

Understanding data dependencies helps you assess impact of changes and identify privacy or performance concerns. You’re planning a feature that needs user purchase history. What data is available? Where does it come from? What are the privacy implications?

Prompt: Data Access Investigation

What user data does the purchase history feature access? Show me:

- What specific data fields are read (order details, payment info, shipping addresses, etc.)
- Where this data is stored (database tables, external APIs, local storage)
- Who has access to this data in the code (which services or components)
- Any privacy or security considerations I should know about

You learn that purchase history pulls from the `orders` table (order ID, date, items, total), joins with the `products` table for current product names, and excludes payment details for security. The data is accessed only by authenticated users viewing their own history, enforced by middleware in `src/middleware/auth.js:34`. This tells you what’s possible for your new feature and what constraints apply.

Pattern 3: “What happens when [condition]?”

Edge cases and error scenarios matter for product quality but are hard to discover without seeing implementation. What happens when a payment fails midway through checkout? When a user’s session expires during form submission? When inventory runs out between adding to cart and completing purchase?

Prompt: Edge Case Investigation

What happens when a payment fails during checkout? Walk me through:

- What the user sees (error messages, page state)
- What happens to their cart and order
- Whether we retry, or they have to start over
- How we handle partial completions (if order is created but payment fails)

Claude Code reads error handling code, payment integration logic, and state management to explain the behavior. You discover that payment failures show a generic error, preserve the cart, and allow retry without losing progress. Or you discover gaps where edge cases aren't handled well, which become backlog items.

Pattern 4: “Why does [behavior] occur?”

Users report something confusing or counterintuitive. You need to know if it's intentional or a bug before triaging. Why does the search filter reset when clicking “Show More”? Why do timestamps show in UTC instead of user's timezone? Why does the form accept invalid phone numbers?

Prompt: Behavior Explanation

Users report that the search filter resets when they click “Show More” to load additional results. Why does this happen? Is it intentional or a bug?

Explain what in the code causes this behavior and whether there's a product reason for it.

You learn that “Show More” triggers a full page refresh (legacy implementation) which resets all component state including filters. It's not intentional. It's a technical limitation from old code. Now you can triage this as a bug with context for engineering about root cause.

Making these patterns work:

Be specific in prompts. “How does authentication work?” is too broad. Claude Code might read 20 files and give you a system overview when you needed to know one detail. “How does the app determine if a user is logged in when they visit a protected page?” is focused and gets a direct answer.

Request product language. Always include “explain in product terms” or “focus on user-facing behavior” to prevent code-heavy explanations you can’t parse. Claude Code defaults to technical detail unless you specify otherwise.

Ask follow-up questions. The first answer gives you direction. Follow-ups dig deeper: “You mentioned a business rule in line 92. What’s the reasoning for that rule?” or “You said payment failures preserve the cart. Where is that logic implemented?”

Reference file locations. Claude Code provides file paths and line numbers. Use these when you need engineering help. “I investigated the discount code issue and found the logic in `discount-validator.js:92`. Can you explain the reasoning for preventing promotion combinations?” This is more effective than “Hey, how do discount codes work?”

These four patterns handle 90% of feature investigations. You’re not reading code. You’re asking informed questions and getting implementation details translated into product context. The investigation might take 10-20 minutes and cost \$0.15-0.50, but it answers questions that would otherwise require interrupting engineering or remaining ignorant of implementation reality.

3.4 Reading Code Without Reading Code

You can understand implementation without parsing syntax. Claude Code translates code into concepts, diagrams, and plain English. This section covers techniques for building mental models from codebases you can’t read directly.

Asking for plain English summaries. Code is precise but opaque. Natural language is interpretable but potentially ambiguous. Ask Claude Code to bridge the gap:

Prompt: Plain English Summary

Summarize what the `processCheckout` function does in plain English. Focus on:

- The steps it performs in order
- What inputs it requires
- What it returns or produces
- What could go wrong and how it handles errors

Output: “The function takes a cart object and payment information. First, it validates the cart has items and totals are correct. Second, it processes payment through the Stripe integration. If payment succeeds, it creates an order record in the database, sends a confirmation email, and clears the cart. If payment fails, it logs the error and returns failure without creating an order. If database writes fail after successful payment, it queues the order for manual reconciliation.”

You now understand checkout flow without reading a line of code. You can explain it to stakeholders, identify improvement opportunities, and ask meaningful questions about edge cases.

Requesting flowcharts and sequence diagrams. Visual representations often clarify complex logic better than text. Claude Code can’t generate actual images, but it can produce text-based diagrams that show flow:

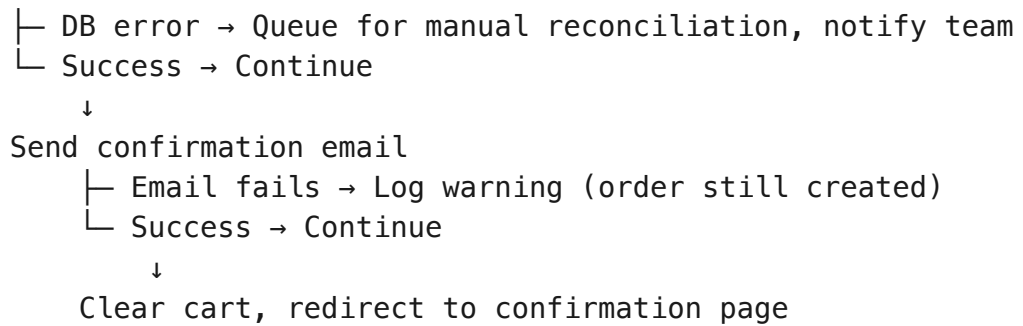
Prompt: Flow Diagram Request

Create a text-based flowchart showing what happens when a user submits a checkout form. Show decision points, success/failure paths, and external service calls.

Output might be:

```

User submits checkout form
↓
Validate cart (items exist, prices current)
├─ Invalid → Return error to user
└─ Valid → Continue
    ↓
    Process payment via Stripe API
    ├─ Payment fails → Log error, show user message, preserve cart
    └─ Payment succeeds → Continue
        ↓
        Create order in database
  
```

This shows you every decision point, failure mode, and dependency. You can walk through customer scenarios. “What if their payment succeeds but email sending fails?” The diagram shows the order is created and the email failure is logged but doesn’t block completion. This is important product knowledge.

Identifying the “important” files for a feature. Large codebases have hundreds or thousands of files. Most are irrelevant to any given feature. Ask Claude Code to filter:

Prompt: Critical Files Identification

I need to understand the checkout feature. What are the five most important files I should know about? For each file, explain what it owns and why it matters to checkout.

Claude Code analyzes the codebase and prioritizes: “1. `CheckoutForm.tsx` - The React component users interact with, handles form submission and displays errors. 2. `checkout.service.js` - Contains business logic for processing checkout, validates cart and orchestrates payment. 3. `stripe-integration.js` - Handles all Stripe API calls for payment processing. 4. `order.model.js` - Database model for orders, defines what data we store. 5. `email-notifications.js` - Sends confirmation emails and handles template rendering.”

Now when you investigate checkout issues or plan improvements, you know where to look. Instead of navigating blindly, you start with context.

Building a mental model without syntax knowledge. Your goal isn’t to read code. It’s to understand how features behave, how data flows, and where decisions are made. Focus your questions on concepts:

- “What are the main steps in the user registration process?”
- “How does the app decide which pricing tier to show a user?”
- “What external services does the search feature depend on?”
- “Where do we store user preferences, and how are they loaded?”

These questions reveal system behavior and architecture without requiring you to parse function syntax, understand loops, or trace variable assignments. Claude Code does the parsing and gives you the conceptual model.

Limitations of AI-generated explanations. Claude Code interprets code based on what it reads, but it can’t run the code or guarantee its interpretation is correct. If logic is complex, has subtle bugs, or depends on runtime conditions Claude Code can’t see, the explanation might be incomplete or wrong.

Treat explanations as high-confidence hypotheses, not absolute truth. When precision matters (debugging a critical issue, assessing security implications, planning major changes), validate with engineering. When you need general understanding (answering stakeholder questions, triaging bugs, assessing feasibility), Claude Code’s explanations are reliable enough to act on.

You’re reading code without reading code by asking the right questions and accepting summaries instead of syntax. This is sufficient for 80% of PM needs. The remaining 20% (deep debugging, security audits, performance optimization) still requires engineering expertise. Know the boundary and you won’t overstep.

3.5 Making Claude Code Remember Your Context

Every Claude Code session starts by reading `CLAUDE.md` if one exists in your project. This file contains context that persists across all sessions: your project’s institutional knowledge that doesn’t belong in code comments or external docs.

For PMs, a well-maintained `CLAUDE.md` transforms Claude Code from a

smart assistant into a team member who knows your product. The first session without `CLAUDE.md` requires explaining your domain. Every session with `CLAUDE.md` starts informed. The setup time pays back immediately.

Purpose: Persistent context that survives sessions. Claude.ai forgets everything when you close the window. Claude Code forgets conversation history when you exit a session. But `CLAUDE.md` persists. Anything you want Claude Code to know in every session goes here.

This isn't a dumping ground for all documentation. It's focused context that improves investigation quality and reduces repetitive explanation. Think of it as onboarding documentation for an AI teammate.

What to include for PM workflows:

Product domain glossary. Your product has terminology that's ambiguous or specific to your domain. Define it once:

Product Terminology

- **Credit**: Internal currency users earn through referrals. Not related to payment credits.
- **Workspace**: A team's shared environment. Users can belong to multiple workspaces.
- **Pipeline**: Sales pipeline, not data pipeline. Refers to deal stages in CRM.
- **Activation**: When a user completes profile setup and creates their first project. Different from account activation (email verification).

This prevents Claude Code from misinterpreting terms that have multiple meanings. When you ask about “activation rates,” it knows you mean profile completion, not email verification.

Key user journeys mapped to code areas. Connect product flows to implementation:

User Journeys

New User Onboarding

1. User signs up → ``src/auth/signup.js``
2. Email verification → ``src/services/email-verification.js``

3. Profile setup → ``src/components/ProfileSetup.tsx``
4. First project creation → ``src/services/project-creator.js``

Checkout Flow

- Entry point: ``src/components/Checkout/CheckoutForm.tsx``
- Payment processing: ``src/services/payment-processor.js``
- Order creation: ``src/models/order.js``
- Confirmation: ``src/services/order-notifications.js``

When you ask Claude Code to investigate part of onboarding, it knows where to look without exploration. This saves tokens and time.

Team conventions and communication norms. How does your team work? What should Claude Code know about your practices?

Team Conventions

- All customer-facing strings use the i18n system in ``src/locales/``. Don't suggest hardcoded strings.
- Pricing logic lives exclusively in ``src/services/pricing/``. Frontend never calculates prices.
- We use Jira ticket IDs in commit messages: "Fix checkout bug (PM-1234)"
- Database migrations require approval from data team before running
- Feature flags are managed in ``src/config/features.js``

This prevents Claude Code from suggesting changes that violate team practices. If you ask for help generating release notes, it knows to look for Jira IDs in commits.

Links to external documentation. Point to resources Claude Code can't access but you reference frequently:

External Resources

- API documentation: <https://docs.internal.com/api>
- Design system: <https://www.figma.com/design-system>
- Product strategy doc: [Google Docs link]
- Analytics dashboard: <https://analytics.internal.com>

When discussing features, consider consistency with the design system and alignment with product strategy.

These links don't help Claude Code directly (it can't follow them), but they remind you to check these resources when making decisions.

Placement: Where to put CLAUDE.md. The primary and recommended location is your project root: `/path/to/your/repo/CLAUDE.md`. This makes the file visible to anyone browsing the repository and establishes it as team documentation.

Claude Code reads CLAUDE.md files hierarchically. First from your home directory (`~/claude/CLAUDE.md` for personal preferences that apply across all projects), then from your project root. You can also place CLAUDE.md files in subdirectories for context specific to that part of the codebase, though this is uncommon.

Most PM use cases favor a single CLAUDE.md in the project root. The content you're adding (domain knowledge, user journey mappings, conventions) benefits engineers and designers too, not just Claude Code. Make it visible and treat it as living documentation the team maintains together.

Updating CLAUDE.md as your understanding grows. Treat this file as living documentation. When you investigate a feature and learn something valuable, add it. When team conventions change, update it. When you notice Claude Code making the same mistake repeatedly (misinterpreting a term, missing a pattern), add clarification.

The file pays for itself when it prevents one repeated explanation or helps you onboard a new PM who can read your CLAUDE.md to understand product context. Version control tracks changes, so you see how understanding evolves over time.

Example structure for a PM-focused CLAUDE.md:

```
# Product Context for Claude Code
```

```
## About This Product
```

```
[2-3 sentences: What does this product do? Who uses it?]
```

```
## Product Terminology
```

[Key terms and their specific meanings in your domain]

User Journeys

[Critical user flows mapped to code locations]

Business Rules

[Important product logic that isn't obvious from code:

- Why do we limit X to Y?
- What's the reasoning for Z behavior?
- What are the constraints on feature A?]

Team Conventions

[How we work, what to avoid suggesting, what practices to follow]

External Resources

[Links to docs, designs, strategy, analytics]

Common PM Questions

[Frequently investigated topics with quick answers or pointers:

- "How does pricing work?" → See src/services/pricing/
- "Where is email content managed?" → src/templates/email/]

Start minimal. Add a glossary and one or two key user journeys. Expand over time as you discover what context improves investigation quality. A 50-line CLAUDE.md with focused, high-value content beats a 500-line dump of everything you know.

3.6 Knowing When to Escalate to Engineering

Claude Code reads code, interprets it, and explains it in plain English. This works remarkably well for most PM needs, but it has boundaries you need to recognize. Overconfidence in AI-generated explanations creates problems when you act on incomplete or incorrect information.

Runtime behavior vs. static analysis. Claude Code reads source code: the static text of your application. It can't run the code, observe actual behavior, or see runtime state. This matters when behavior depends on configuration, environment variables, database state, or external service responses.

Concrete example: You ask how the app determines which features a user can access. Claude Code reads the permission checking code and explains the logic: "The app checks if `user.role` is 'admin' and grants access to admin features if true." But it can't tell you what roles are actually assigned in production, whether there are data quality issues causing incorrect role values, or if an environment variable overrides this logic in certain deployments.

For understanding how something is supposed to work, Claude Code is reliable. For understanding how it actually works in production right now, you need runtime data: logs, database queries, monitoring dashboards, or engineering help.

Configuration and environment dependencies. Applications behave differently based on configuration files, environment variables, and deployment contexts. Development, staging, and production might run the same code but behave differently due to configuration.

Claude Code can read configuration files and identify dependencies, but it can't tell you which configuration is active in which environment without being explicitly told. If you ask "How do we handle failed payments?" and the answer depends on whether Stripe is in test mode or production mode, Claude Code explains both code paths but might not clarify which applies where.

Make configuration explicit when investigating environment-sensitive features. "How do we handle failed payments in production?" prompts Claude Code to look for production configuration.

When to escalate to engineering. You can investigate independently, but some situations require engineering expertise:

Security-sensitive questions. "Is this authentication implementation secure?" or "Could a user access another user's data?" require security

expertise, not just code reading. Claude Code can identify obvious issues (passwords stored in plain text, SQL injection vulnerabilities if they're blatant), but subtle security problems require human review.

Performance issues. “Why is this feature slow?” involves profiling, database query analysis, and runtime behavior observation. Claude Code can identify potentially expensive operations by reading code (nested loops, N+1 queries), but actual performance depends on data volume, database indexes, and infrastructure that it can't see.

Complex debugging. If a bug is intermittent, data-dependent, or involves race conditions, Claude Code's static analysis won't find root cause. Use it for initial investigation (narrowing down relevant code areas), then hand off to engineering with context.

Architectural decisions. “Should we refactor this to use microservices?” or “Is this the right database for our scale?” require engineering judgment about trade-offs, team capabilities, and long-term maintenance. Claude Code can explain current architecture and identify pain points, but it can't make strategic technical decisions.

Production system access. Anything requiring database queries, log analysis, or monitoring dashboard review is outside Claude Code's scope. It works with code and files, not live systems.

Avoiding overconfidence in explanations. Treat Claude Code's explanations as high-confidence interpretations, not absolute truth. When accuracy matters (customer-facing communications, roadmap decisions, security assessments), validate findings with engineering before acting.

Use this mental model: Claude Code is a junior engineer who can read code and explain what they see but has no production experience and hasn't talked to the original authors. You trust their reading but verify anything critical.

Practical boundaries for PM usage:

Handle with Claude Code	Escalate to Engineering
– How does feature X work? – What files implement Y? – What data does Z access? – What happens when user does B? – What are edge cases for E?	– Is feature X secure? – Why is Y slow in production? – Should we refactor Z? – Why is B failing for some users? – How do we test E in staging?
Pattern: Structure questions → Claude Code. Quality/Performance/Production → Engineering	

Figure 3.3: Escalation Boundaries – Use this to know when to investigate independently versus when to involve engineering. Left column shows Claude Code territory (structure and behavior questions): How does X work, What files implement Y, What data does Z access. Right column shows Engineering territory (quality, performance, production questions): Is X secure, Why is Y slow, Should we refactor Z.

You can handle with Claude Code	Escalate to engineering
How does feature X work?	Is feature X secure?
What files implement Y?	Why is Y slow in production?
What data does Z access?	Should we refactor Z?
Where is configuration A defined?	What's configuration A in prod?
What happens when user does B?	Why is B failing for some users?
Can I combine feature C with D?	How should we build C+D integration?
What are edge cases for E?	How do we test E in staging?

The pattern: Questions about what exists and how it's structured are Claude Code territory. Questions about quality, performance, correctness,

and production behavior require engineering.

This doesn't diminish Claude Code's value. It focuses your usage on high-return investigations while maintaining appropriate humility about AI limitations. You'll still save hours of engineering time by handling first-pass investigation yourself. You'll just avoid the trap of treating AI explanations as oracle truth.

You can now investigate your codebase without depending on engineering for every question. You understand how to orient yourself with architecture overviews, apply feature investigation patterns, build mental models without reading code, create persistent context in `CLAUDE.md`, and recognize when to escalate. Chapter 4 shows you how to apply these capabilities specifically to bug investigation: the PM workflow where codebase access delivers immediate, measurable value.

4 Investigating Bugs and User Issues

4.1 What You Can Triage Without Engineering Help

A support ticket arrives: “My payment didn’t go through but I got charged.” Engineering is shipping a release. The customer is escalating to your VP. You need to triage: real bug, user error, or edge case? You need scope: one customer or systemic? You need context: what should engineering know before they investigate?

Traditional PM response: Forward the ticket to engineering with “Can you take a look?” and wait. The engineer context-switches, investigates, and reports back. If it’s user error, you’ve interrupted a sprint for nothing. If it’s a real bug, you’ve provided no context beyond the raw customer complaint. Either way, the feedback loop takes hours or days.

Alternative: Open Claude Code, investigate the payment flow yourself, identify the code path that could cause this behavior, and write up findings before assigning to engineering. Total time: 20 minutes. Engineering gets a focused investigation with file references and hypothesis. They confirm and fix without the discovery phase.

Bug investigation is where Claude Code delivers immediate, measurable PM value. You’re not fixing bugs. That’s engineering’s job. You’re gathering context, forming hypotheses, and reducing back-and-forth. Good triage saves engineering hours per week. Bad triage wastes everyone’s time with incomplete handoffs that require follow-up questions.

Your role in bug investigation:

Is this a real bug or user error? Many “bugs” are users not understanding intended behavior, incorrect configurations, or expired

sessions. Investigation often reveals the system worked correctly and the user misunderstood. Catching this saves engineering a wasted cycle.

What's the scope and severity? One customer affected or many? Data loss involved or cosmetic issue? Intermittent or reproducible? This context determines priority. Claude Code helps you understand what conditions trigger the issue and how widespread those conditions might be.

What context does engineering need? File locations, relevant code paths, similar past issues, reproduction steps you've identified. The more context you provide, the faster engineering resolves the issue. Raw customer complaints with no investigation create more work, not less.

You're not expected to fix bugs. You're expected to triage intelligently, provide useful context, and avoid wasting engineering time on issues you could have resolved or clarified yourself. Claude Code makes this possible without reading code.

4.2 Translating User Complaints into Testable Queries

User language doesn't match code language. Customers describe symptoms: "The page is broken." "My discount didn't apply." "I can't log in." Engineers need specifics: error messages, reproduction steps, environment details, data states. Your job is translation.

Translating user language to technical queries. Start by extracting the concrete from the vague. A report says "The checkout page is broken." What specifically is broken? Did they see an error message? What were they trying to do? What happened instead of what they expected?

If you have access to the original support ticket, extract these details. If not, make reasonable assumptions and note them in your investigation. Then ask Claude Code to help identify what could cause the described symptom.

Launch Claude Code in plan mode:

claude --permission-mode plan

Prompt: Symptom Translation

A user reports: “[paste the exact complaint]”

What code paths could cause this behavior? Identify:

- Which files handle this feature
- What could go wrong at each step
- What error states the user might encounter
- What data or conditions might trigger this issue

For example, with “My discount code didn’t apply even though it’s valid”:

A user reports: “My discount code didn’t apply even though it’s valid. I entered it correctly and it just said ‘invalid code’.”

What code paths could cause this behavior in the discount or checkout flow? Identify potential failure points: code validation logic, expiration checks, usage limits, cart requirements, case sensitivity.

Claude Code reads the relevant files and returns something like:
“Discount validation happens in `src/services/discount-validator.js`. The code checks: (1) code exists in database, (2) not expired based on `valid_until` date, (3) usage count below `max_uses`, (4) cart meets `minimum_cart_value`. Any false check returns ‘invalid code’ with no specific message. Case sensitivity is enforced, so ‘SUMMER20’ and ‘summer20’ are different codes.”

You now have a testable hypothesis. The generic “invalid code” message masks the actual failure reason. The customer’s code might be expired, over usage limit, or entered in wrong case. All of these would appear valid to the user but be rejected by the system.

Identifying relevant log patterns and error messages. If your application logs errors, ask Claude Code what logging exists:

What logging or error tracking exists for discount code validation?
Where would I find logs for failed discount applications?

Claude Code identifies logging statements, error handlers, and where error details are captured. You learn that failed validations log to `logs/discount-errors.log` with reason codes, or that failures aren't logged at all. The absence of logging is itself useful information for improving debugging.

Distinguishing frontend vs. backend vs. data issues. Not all bugs are code bugs. Some are configuration issues, data quality problems, or environment mismatches.

Prompt: Issue Layer Identification

A user experiences [symptom]. Help me determine if this is likely:

- Frontend issue (display, client-side logic, browser compatibility)
- Backend issue (server logic, API responses, business rules)
- Data issue (database state, user record problems, missing data)
- Configuration issue (environment settings, feature flags, third-party services)

What evidence would distinguish these?

For the discount code example, Claude Code might explain: “Frontend issue if the code isn't being sent correctly (check network requests). Backend issue if validation logic has bugs (the code we reviewed). Data issue if the discount code record in the database is malformed or missing. Configuration issue if discount service is disabled in this environment or Stripe isn't processing properly.”

This narrows your investigation. If the customer says “I entered the code and got an error immediately,” that's likely frontend or backend validation. If they say “It seemed to work but the discount didn't show on my receipt,” that's likely a later processing step, maybe backend calculation or data persistence.

Cost and time for reproduction investigation. A typical reproduction investigation session (translating user complaint, identifying relevant code, narrowing to likely causes) takes 10-20 minutes and consumes 15,000-40,000 tokens. Cost for API users: \$0.08-0.20. Cost for subscription users: included. Compare to the alternative: a vague

ticket that engineering bounces back for more information, adding a day to resolution time.

4.3 From User Report to Engineering Handoff in 30 Minutes

Structured investigation beats ad-hoc exploration. Follow this workflow to move from user report to actionable engineering handoff.

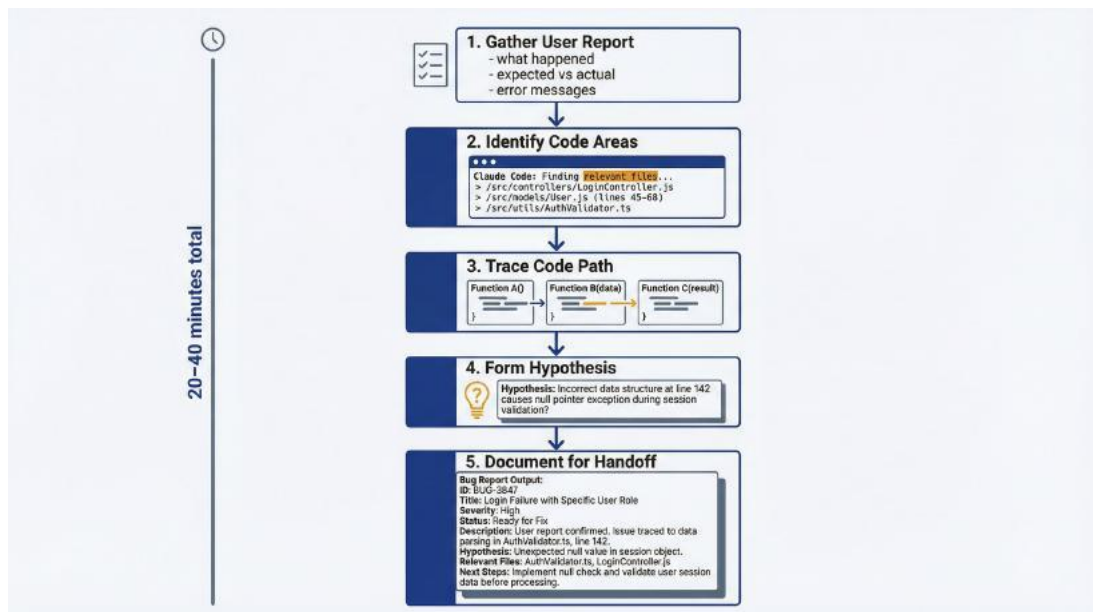


Figure 4.1: Bug Investigation Workflow – Use this to structure your bug triage process. Five steps: 1) Gather user report details (5 min), 2) Identify relevant code areas with Claude Code, 3) Trace the code path, 4) Form a testable hypothesis, 5) Document findings for engineering handoff. Total time: 20-40 minutes.

Step 1: Gather user report details. Before touching Claude Code, extract everything you can from the report:

- What the user was trying to do
- What they expected to happen
- What actually happened
- Any error messages they saw (exact text if possible)
- When it happened
- What browser/device/environment they were using

- Whether they can reproduce it or it was one-time

This takes five minutes and prevents wasted investigation cycles. If you investigate “login is broken” without knowing the user was on an outdated browser, you’ll trace code that works correctly.

Step 2: Ask Claude Code to identify relevant code areas. With context gathered, start your investigation session:

```
claude --permission-mode plan
```

Prompt: Code Area Identification

I’m investigating a user-reported issue: [one-sentence summary]

Context:

- User action: [what they were doing]
- Expected: [what should have happened]
- Actual: [what happened instead]
- Error message (if any): [exact text]

Identify the code areas likely involved. Show me:

- Entry point (where this user action starts in the code)
- Key files in the execution path
- Where errors are caught and handled
- Where this could fail

Claude Code maps the relevant code territory. For a payment failure investigation, you might get: “User clicks pay → CheckoutButton.tsx dispatches action → checkout.service.js validates cart → payment-processor.js calls Stripe API → response handled in payment-response-handler.js → order created in order.service.js. Error handling exists at each step; the Stripe integration has specific error mapping in stripe-errors.js.”

Step 3: Request explanation of the suspected code path. Once you know where to look, ask for details:

Explain what `payment-processor.js` does step by step. Focus on:

- What could cause it to fail
- What happens when Stripe returns an error
- Whether the user sees specific error messages or generic ones
- What gets logged when payment fails

Claude Code reads the file and translates: “The processor validates the payment method exists, checks the cart total matches the amount to charge, calls Stripe’s charge API, and handles the response. Stripe errors are caught and mapped. ‘card_declined’ becomes ‘Your card was declined’, ‘insufficient_funds’ becomes the same message for privacy. All failures log the Stripe error code to `payments.log` but show the generic message to users. If Stripe returns success but the order creation fails afterward, there’s a reconciliation queue that catches orphaned payments.”

Now you’re getting somewhere. The user said they were charged but the payment “didn’t go through.” This could mean: Stripe charged successfully, but order creation failed, triggering the reconciliation queue. The customer was charged but saw an error because the failure happened after payment.

Step 4: Formulate hypothesis. Based on your investigation, form a testable hypothesis:

“The user was likely charged successfully by Stripe, but the subsequent order creation failed. This could be a database error, timeout, or validation issue. The system showed a generic error message despite successful payment. The reconciliation queue should have this case. If so, the payment exists in Stripe and the user needs a manual order creation or refund.”

This is actionable. Engineering can check the reconciliation queue, verify in Stripe, and resolve the specific case while also investigating why order creation failed.

Step 5: Document findings for engineering. Write up your investigation:

Prompt: Investigation Summary

Summarize my investigation into this bug for engineering handoff. Include:

- What the user reported
- What I investigated
- Files and code areas I reviewed
- My hypothesis for root cause
- Specific questions for engineering
- Suggested next steps

Claude Code generates a summary you can paste into Jira, Slack, or email. Add the file references it found, your hypothesis, and what engineering should verify. This handoff respects engineering time. They're not starting from zero.

Complete workflow duration: 20-40 minutes for a moderately complex bug. Token consumption: 30,000-80,000 tokens. For API users, this costs \$0.15-0.40. A thorough investigation that saves engineering an hour of discovery is worth far more than that.

4.4 Recognizing Bug Types Before Engineering Does

Certain bug categories recur across products. Recognizing these patterns helps you investigate faster and communicate more precisely with engineering. You're not debugging code. You're pattern matching symptoms to known issue types.

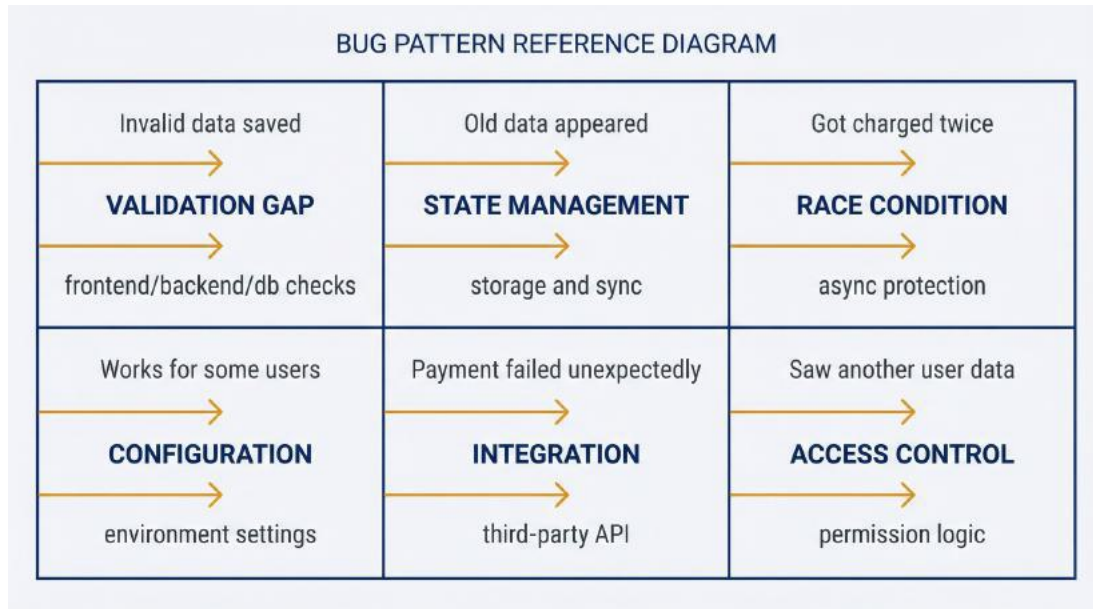


Figure 4.2: Bug Pattern Reference – Use this to quickly categorize user-reported issues. Six common patterns: Validation Gap (invalid data saved), State Management (old data appeared), Race Condition (double-charged), Configuration Mismatch (works for some users), Integration Failure (payment failed unexpectedly), Access Control (saw another user’s data). Match symptoms to patterns for faster investigation.

Data validation gaps. User input gets through that shouldn’t. Form accepts invalid email addresses. Price field allows negative numbers. Required fields can be bypassed. These bugs stem from validation logic that’s incomplete or inconsistent between frontend and backend.

Prompt: Validation Investigation

How is [field/input] validated in this feature? Check:

- Frontend validation (what the form checks before submission)
- Backend validation (what the server checks when receiving data)
- Database constraints (what the schema enforces)

Are there gaps where invalid data could get through?

Claude Code compares validation at each layer. You might find the frontend checks email format but the backend doesn’t, allowing API calls to bypass frontend validation. This is your hypothesis: the symptom (invalid data in database) matches the pattern (validation gap).

State management issues. UI shows stale data. Changes don't persist after navigation. User sees someone else's data. These bugs involve how the application tracks and synchronizes state.

Prompt: State Investigation

How is [feature] state managed? Trace:

- Where state is stored (local, session, server)
- How state is updated after user actions
- When state is refreshed or synchronized
- What could cause state to be stale or incorrect

State bugs are subtle because the code “works” but just doesn't coordinate correctly. Claude Code can identify that the user profile is cached locally but updated server-side, and the cache isn't invalidated on update. Symptom matches pattern.

Race conditions (conceptual identification). Two operations that shouldn't happen simultaneously collide. User clicks button twice and gets double-charged. Concurrent edits overwrite each other. These are timing bugs that require specific conditions to trigger.

You can't debug race conditions with static analysis, but you can identify susceptibility:

Could [feature] have race conditions? Look for:

- User actions that trigger async operations
- Buttons or forms without click protection
- Concurrent access to shared resources
- Operations that should be atomic but aren't

Claude Code might find: “The checkout button triggers async payment processing but doesn't disable on click. Multiple clicks could dispatch multiple payment calls. There's no idempotency key in the Stripe call to prevent duplicate charges.”

You can't confirm this is the root cause without runtime testing, but you've identified the pattern. Your bug report says: “Suspected race

condition. Checkout button lacks click protection and payment call may not be idempotent. User may have double-clicked.”

Configuration mismatches. Feature works in some environments but not others. Staging behaves differently from production. Certain users experience issues others don’t. These bugs stem from environment-specific settings.

Prompt: Configuration Investigation

What configuration controls [feature]? Show me:

- Environment variables that affect behavior
- Feature flags or toggles
- Settings that differ between environments
- User-specific or account-specific settings

You learn that the feature uses a third-party API that has different credentials per environment, and the production credential expired. Or a feature flag enables the feature for beta users only, and the affected user isn’t in the beta group.

Third-party integration failures. External services return errors, timeout, or behave unexpectedly. Payment processor declines valid cards. Email service fails silently. Analytics events don’t appear.

Prompt: Integration Investigation

How does this feature integrate with [third-party service]? Show me:

- Where API calls are made
- How errors from the service are handled
- Whether failures are logged and how
- What the user sees when the integration fails

Claude Code traces the integration. You find that Stripe errors are logged but the user sees a generic “something went wrong” message. Or that email sending is fire-and-forget with no error handling, so failures silently disappear.

Permission and access control bugs. User can access something they shouldn't. User can't access something they should. Admin-only features appear for regular users.

Prompt: Access Control Investigation

How are permissions checked for [feature]? Show me:

- Where access control logic lives
- What conditions grant access
- How the system determines user role/permissions
- What happens when access is denied

You discover that permission checking happens in the frontend only, so users can bypass it with direct API calls. Or that the permission check has an OR that should be an AND. Symptom matches pattern: access control logic is flawed.

Pattern reference table:

User reports...	Likely pattern	Investigation focus
“Invalid data got saved”	Validation gap	Compare frontend/backend/database validation
“Old data appeared” / “Changes didn’t save”	State management	Trace state storage, updates, synchronization
“Got charged twice”	Race condition	Look for async operations without protection
“Works for some users but not me”	Configuration mismatch	Check user-specific or environment settings
“Payment failed but should have worked”	Third-party integration	Trace external API calls and error handling
“I could see another	Access control	Review permission logic and enforcement

user's data”

Recognizing patterns speeds investigation from “what could be wrong” to “is it this specific thing?” You’re not debugging. You’re categorizing symptoms to communicate precisely with engineering.

4.5 Bug Reports Engineering Actually Wants to Read

Your investigation is only valuable if you communicate it effectively. Engineering needs specific information to act quickly. A well-structured bug report with investigation context reduces resolution time from days to hours.

What engineering needs:

Reproduction steps. How can they trigger the issue? Numbered steps from starting state to error. “1. Create new user account. 2. Add item to cart. 3. Apply discount code ‘SUMMER20’. 4. Click checkout. 5. Observe error message.” If you couldn’t reproduce, say so and describe what the user reported.

Environment. Where did this happen?

Production/staging/development. Browser and version. Device type. User account type (new vs. existing, free vs. paid). Anything that might affect behavior.

Expected vs. actual. What should happen? What happened instead? Be specific. “Expected: Discount applies and cart total updates. Actual: Error message ‘Invalid code’ appears despite code being valid in admin panel.”

Error messages. Exact text, not paraphrased. Error codes if visible. Screenshots if the user provided them. Log entries if you have access.

Using Claude Code output to enrich reports. Your investigation produced file references, code explanations, and hypotheses. Include them:

Prompt: Bug Report Generation

Based on my investigation, generate a bug report including:

- Summary of the issue
- Reproduction steps (based on my understanding)
- Environment details
- Technical context (files involved, code paths)
- My hypothesis for root cause
- Suggested investigation points for engineering
- Open questions I couldn't answer

Claude Code synthesizes your session into structured output. You'll still need to review and edit (add user-reported details it didn't see, remove speculation you're not confident in), but it creates a solid starting draft.

Including relevant code snippets without pretending expertise.

You found that the discount validation in `discount-validator.js:45-78` could cause this issue. Include the reference without pretending you understand every line:

"I investigated the discount validation flow and found the logic in `discount-validator.js:45-78`. Based on my reading, the code checks [list conditions]. The user's scenario might fail at [specific condition] because [hypothesis]. I'm not certain this is the root cause, but it's where I'd look first."

This provides useful context without overstepping. You're saying "I looked here and this seems relevant" not "I know exactly what's wrong." Engineers appreciate the context and won't resent appropriate humility.

Template: The PM-investigated bug report:

Summary

[One sentence describing the issue]

User Report

- ****User action****: [What they were trying to do]
- ****Expected****: [What should have happened]
- ****Actual****: [What happened instead]

- **Error message**: [Exact text if available]

Environment

- **Where**: Production / Staging / etc.
- **User type**: [Account characteristics]
- **Browser/device**: [If known]
- **When**: [Date/time if relevant]

Reproduction Steps

1. [Step 1]
2. [Step 2]
3. [Observe: specific behavior]

(Note if reproduction was confirmed by PM or inferred from user report)

PM Investigation

Files Reviewed

- `path/to/file.js` - [What this file handles]
- `path/to/other.js` - [What this file handles]

Findings

[What you learned from reading the code. What the flow looks like.
Where things could fail.]

Hypothesis

[Your best guess at root cause, stated as hypothesis not conclusion]

Open Questions

- [Thing you couldn't determine from code]
- [Thing that needs runtime verification]

Suggested Next Steps

- [] Verify [specific thing] in logs
- [] Check [specific state or data]
- [] Test [specific scenario]

Priority Assessment

[Your recommendation: P1/P2/P3 and why. Scope assessment—one user or many?]

What not to include. Don't include lengthy code dumps you don't

understand. Don't state hypotheses as conclusions. Don't assign blame to specific commits or engineers. Don't editorialize about how "obvious" the fix is. Stick to observations, hypotheses, and questions.

A bug report like this turns a vague "something's broken" into actionable engineering work. You've done the first 30% of investigation. Engineering does the remaining 70%, but that remaining work is focused and efficient instead of starting from scratch.

4.6 Knowing When to Stop

Investigation has diminishing returns. At some point, further PM investigation wastes your time and delays engineering involvement. Recognize the boundaries.

Signs you've gathered enough context:

You can explain the flow. You understand what the feature does, how data moves through it, and where in the code it happens. Even if you can't read the code fluently, you can describe the path from user action to outcome.

You have a testable hypothesis. You've formed a reasonable guess about what's wrong. "The discount validation fails because the code is case-sensitive and the user entered lowercase." "The payment succeeded but order creation failed due to a database timeout." Hypotheses don't have to be right—they give engineering a starting point.

You've identified relevant files. You can point to specific file paths where the issue likely lives. This saves engineering the exploration phase.

You know what you don't know. Your investigation hit limits: runtime behavior you can't see, data states you can't verify, configuration you can't access. Documenting these gaps helps engineering focus their investigation.

If you've hit these four criteria, you have enough. Write the bug report and hand off.

Signs you're in over your head:

The code is too complex to summarize. If Claude Code's explanations don't make sense even in plain English, the logic may be beyond what investigation can clarify. Hand off to engineering.

You're going in circles. You've asked variations of the same question three times and aren't getting new information. The issue requires runtime debugging, not code reading.

It's security-sensitive. Authentication, authorization, data access, encryption: anything where getting it wrong has serious consequences. Your hypothesis could be dangerously wrong. Engineering security review is mandatory.

It requires production access. Checking database state, reading production logs, testing with real user data. Claude Code reads code, not live systems. You've hit the boundary.

You've spent more than an hour. For most bugs, if an hour of investigation hasn't yielded a clear hypothesis, the issue is complex enough that engineering should take over. Your time has diminishing returns past this point.

Handoff protocols that respect engineering time. When you hand off, communicate clearly:

"I spent 30 minutes investigating. Here's what I found: [summary]. My hypothesis is [hypothesis]. I stopped because [reason, such as needing production data or code being too complex]. Can you take it from here?"

This tells engineering what you've done, what you think, and where to start. They're not duplicating your work or wondering what you already tried.

The diminishing returns curve. The first 20 minutes of investigation yield 80% of the context value. You learn the relevant files, form a hypothesis, and identify gaps. The next 40 minutes yield perhaps 15% more value: confirming details, ruling out alternatives, documenting edge cases. Beyond an hour, you're likely iterating without progress.

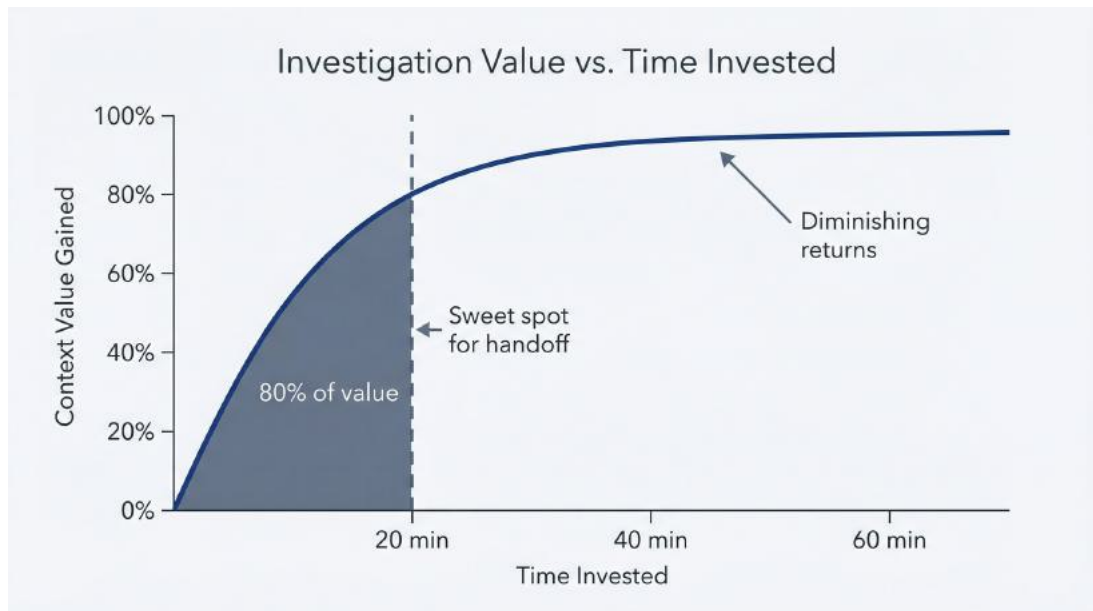


Figure 4.5: Investigation Value Curve – Use this to know when to stop investigating and hand off to engineering (the 20-minute rule). The curve rises steeply in the first 20 minutes to capture 80% of context value, then flattens significantly. A bold vertical line at the 20-minute mark labeled “THE HANDOFF POINT” with callout: “Stop here—the engineer takes it from here.” Green zone (0-20 min) shows high PM value; yellow zone (20-40 min) shows moderate gains; gray zone beyond 40 minutes indicates diminishing returns.

For straightforward bugs: 15-20 minutes of investigation, then hand off.

For complex bugs: 30-45 minutes maximum, then hand off with clear documentation of what you tried.

For critical/urgent bugs: 10 minutes to gather initial context, then parallel work with engineering rather than sequential handoff.

When to skip investigation entirely. Some issues don’t warrant PM investigation:

- Security vulnerabilities reported by users: escalate to security team immediately
- Production outages: engineering handles incident response, not PMs
- Issues requiring immediate hotfix: investigation delays action
- Bugs with clear reproduction and obvious impact: just file the ticket

Investigation adds value when context-gathering saves more engineering time than it costs PM time. For obvious or urgent issues, skip to handoff.

You can now investigate bugs without depending on engineering for initial triage. You understand how to translate user complaints into technical queries, follow a structured investigation workflow, recognize common bug patterns, write useful bug reports, and know when to stop. This capability changes your relationship with engineering. You're a partner in investigation, not just a ticket router.

Chapter 5 shifts focus from codebase work to research: using Claude Code for market and competitive analysis with persistent, versionable outputs.

5 Market and Competitive Analysis

5.1 Why Claude Code Beats Claude.ai for Research

Your stakeholders want a competitive analysis by Friday. You have three options. First: spend two days manually compiling information into a Google Doc that becomes outdated next quarter. Second: use Claude.ai for a quick conversation that produces decent output but disappears when you close the tab. Third: use Claude Code to generate structured, versionable artifacts that live in your repository and can be updated systematically.

For one-off questions (“What’s Competitor X’s pricing model?”), Claude.ai is sufficient. For research that needs to persist, be referenced in PRDs, updated quarterly, and shared across teams, Claude Code is the correct tool. The difference isn’t capability. It’s persistence and repeatability.

Why use Claude Code instead of Claude.ai for research?

Persistence. Research outputs become files in your repository. They survive session closures, can be referenced by file path in documentation, and integrate with your team’s existing workflow. The competitive matrix you generate today exists as `research/competitive-analysis-2026-01.md` next month when your VP asks to see it again.

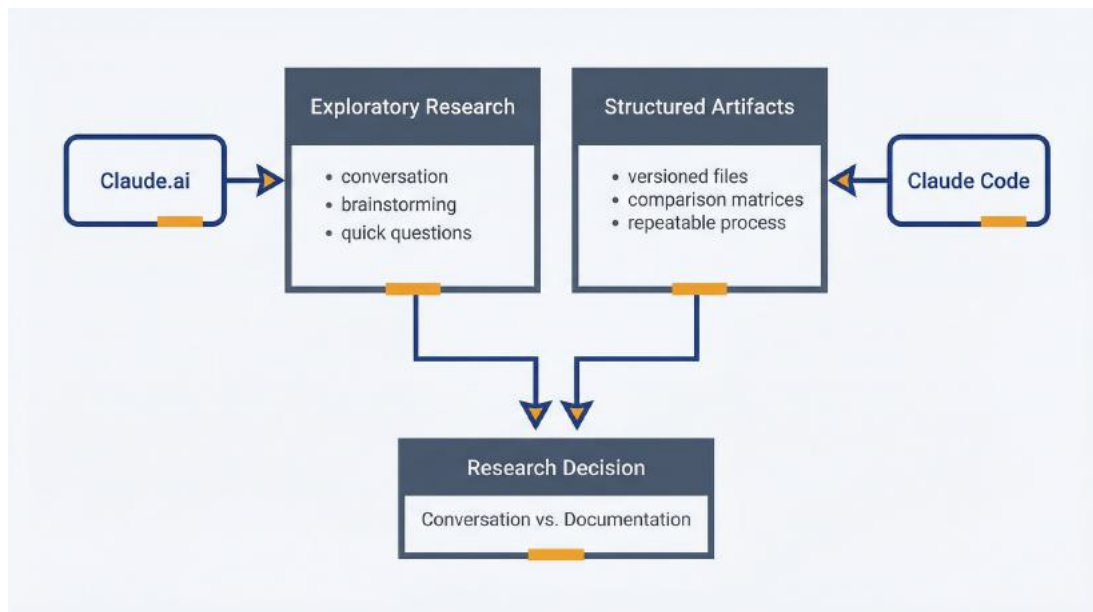
File output. Claude Code writes structured documents: markdown tables, comparison matrices, bullet lists formatted consistently. Claude.ai gives you text you need to copy-paste and reformat. Claude Code produces artifacts ready to commit.

Repeatability. You build the research methodology once (prompts, output formats, analysis frameworks), then execute it consistently.

Quarterly competitive reviews use the same process, making results comparable over time. You're not reinventing research structure every cycle.

Versionability. Research artifacts in git track changes. You see what changed between Q1 and Q2 competitive analysis. You can diff pricing comparisons to spot trends. This matters when presenting to stakeholders who ask “How has this landscape shifted?”

When web-based Claude is sufficient: Exploratory research where you're still figuring out what questions to ask. Quick fact-checking. Brainstorming research directions. Anything where conversation is more valuable than artifact generation. Claude.ai excels at iterative dialogue. Use it for thinking, Claude Code for documenting.



Research tool selection: Claude.ai for exploratory conversation vs. Claude Code for structured, versioned artifacts

The session to generate a competitive analysis costs \$0.30-0.75 and takes 15-25 minutes. Compare to two days of manual work or the cost of outdated research informing bad product decisions. The ROI is immediate if you're doing market research more than once a quarter.

5.2 Building Competitive Intelligence You Can

Update

You need to know who you're competing with, what they offer, and how you're differentiated. Manual research involves browsing competitor websites, taking notes, trying to keep everything in a spreadsheet that's inconsistent and incomplete. Claude Code systematizes this.

Setting up a competitive analysis workspace. Create a directory structure for research:

```
mkdir -p research/competitors
cd research/competitors
```

This keeps research separate from code, makes it easy to find, and allows for organization by date or topic. You might structure it:

```
research/
├── competitors/
│   ├── competitive-matrix-2026-01.md
│   ├── competitor-a-profile.md
│   ├── competitor-b-profile.md
│   └── competitor-c-profile.md
├── pricing/
└── market-sizing/
```

Launch Claude Code in your repository root with file write access:

```
claude
```

Prompt template: Structured competitor profiling. The key to useful competitive analysis is consistency. Every competitor profile should answer the same questions in the same format. Define your structure, then execute it repeatedly.

Prompt: Competitor Profile

Research [Competitor Name] using web search and create a structured profile in `research/competitors/[competitor-name]-profile.md`.

Include the following sections:

- **Overview:** One paragraph describing what they do, who they serve, company stage
- **Product offerings:** Core products and key features (bullet list)
- **Pricing model:** How they charge, what tiers exist, approximate price points
- **Target market:** Who they sell to (company size, industry, user roles)
- **Key differentiators:** What they emphasize in positioning (3-5 bullets)
- **Strengths:** Where they're strong relative to us
- **Weaknesses:** Where they're weak or have gaps (3-5 bullets)
- **Recent developments:** Product launches, funding, partnerships (last 6 months)
- **Last updated:** [Today's date]

Use web search to gather current information. Keep each section concise (this is a reference doc, not a deep dive).

Claude Code searches the web for information, synthesizes findings, and writes a structured markdown file. The output takes 5-10 minutes to generate and costs \$0.15-0.35 depending on web search volume.

What you get:

A consistent, structured profile you can reference. Each competitor has the same sections, making comparison straightforward. When your CEO asks "What does Competitor X do differently?" you open their profile and read the differentiators section.

Output format: Comparison matrices and feature tables.

Individual profiles are useful, but stakeholders want comparison. Once you have 3-5 competitor profiles, generate a matrix:

Prompt: Competitive Matrix

Based on the competitor profiles in `research/competitors/`, create a comparison matrix in `research/competitors/competitive-matrix-2026-01.md`.

Create a markdown table comparing:

- Core features (rows: authentication, permissions, integrations, mobile support, API access, reporting, etc.)
- Competitors (columns: Us, Competitor A, Competitor B, Competitor C, Competitor D)
- Values: ✓ (full support), ~ (partial/limited), x (not available), ? (unknown)

Include a pricing comparison table:

- Rows: Free tier, Starter, Professional, Enterprise
- Columns: Same competitors
- Values: Price/month or “Contact sales” or “N/A”

Add brief analysis after each table (2-3 sentences) highlighting key patterns.

Claude Code reads your existing competitor profiles, extracts relevant information, and formats it into scannable tables. The matrix shows gaps: features competitors have that you don’t, pricing tiers where you’re not competitive, strengths you should emphasize.

Example output:

Feature Comparison

Feature	Us	Competitor A	Competitor B	Competitor C
SSO/SAML	✓	✓	~	x
Role-based access	✓	✓	✓	~
API access	✓	✓ (limited)	✓	x
Mobile apps	~	✓	✓	x
Custom integrations	✓	~	x	x
Advanced reporting	✓	✓	~	~

****Analysis**:** We're competitive on enterprise features (SSO, RBAC, API) but lag on mobile. Competitors A and B have stronger mobile offerings. Competitor C is weak across the board, likely targeting SMB with simpler needs.

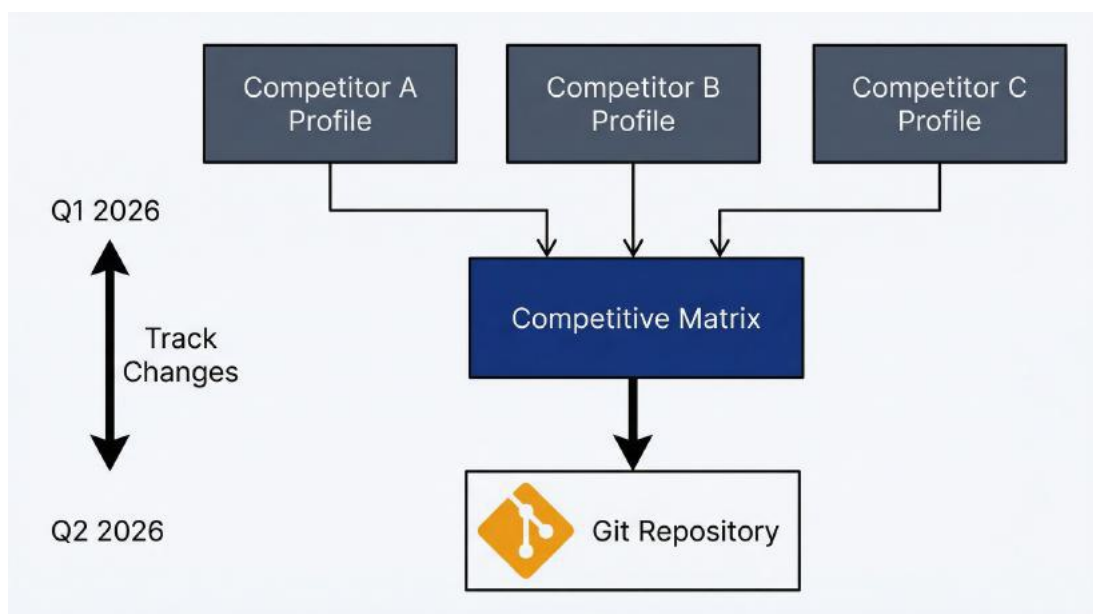
Storing results as versionable artifacts. Commit these files to git:

```
git add research/competitors/  
git commit -m "Add Q1 2026 competitive analysis"
```

Now your research is versioned. Three months later, update the profiles and generate a new matrix dated Q2. Diff the files to see what changed:

```
git diff competitive-matrix-2026-01.md competitive-matrix-2026-04.md
```

You see that Competitor A added mobile features, Competitor B raised prices, and a new competitor entered the market. This historical view shows landscape evolution, not just current state.



Competitive analysis workflow: individual profiles feed into comparison matrices, tracked over time with version control

Cost and time for competitive landscape mapping. Initial setup (5 competitor profiles + matrix): 45-60 minutes, 80,000-150,000 tokens, \$0.40-0.75. Quarterly updates (refresh profiles + regenerate matrix): 20-30 minutes, 40,000-70,000 tokens, \$0.20-0.35. Compare to 8-12 hours of manual research per cycle.

When this approach fails: Niche industries where public information is scarce. Competitors with opaque pricing (everything is “contact sales”). Fast-moving markets where information becomes stale weekly. In these cases, Claude Code still structures what you know, but you’ll need to supplement with sales calls, analyst reports, or customer interviews.

5.3 Tracking Competitor Pricing Without Spreadsheets

Pricing intelligence requires systematic collection and structured comparison. Your competitor charges \$49/month, but for what features? What limits? How does that compare to your \$59/month tier? Manual research captures pricing but misses context. Claude Code captures both.

Gathering pricing intelligence systematically. Pricing pages change. Promotions launch. Discounts appear. You need a snapshot methodology that you can repeat to track changes over time.

Prompt: Pricing Deep Dive

Research [Competitor Name]’s current pricing model using web search and create a detailed breakdown in `research/pricing/[competitor-name]-pricing-2026-01.md`.

Include:

- **Pricing tiers:** Name, monthly price, annual price (if different)
- **Feature limits per tier:** Users, storage, API calls, projects, etc.
- **Key feature gates:** What features are locked to higher tiers?
- **Add-ons:** Optional extras and their pricing
- **Discounts:** Annual discount percentage, volume discounts, nonprofit/edu pricing
- **Free tier:** What’s included, what limits apply
- **Enterprise tier:** Is pricing public or “contact sales”? What’s included?
- **Billing model:** Per user, per feature, usage-based, flat rate?

Use web search to gather current information. If something isn’t publicly documented, note it as “Not disclosed.”

Add a “Last updated” date at the top.

Claude Code searches pricing pages, extracts details, and structures them consistently. You get a pricing breakdown that’s far more detailed than “they charge \$49/month.” You know what that \$49 includes, what it

excludes, and how it compares to their other tiers.

Prompt template: Pricing model comparison. Once you have individual competitor pricing docs, compare them:

Prompt: Pricing Comparison Table

Based on pricing files in `research/pricing/`, create a comparison table in `research/pricing/pricing-comparison-2026-01.md`.

Create a table with:

- Rows: Pricing tiers (Free, Starter/Basic, Professional/Standard, Enterprise)
- Columns: Us, Competitor A, Competitor B, Competitor C
- Values: Price/user/month (or total monthly price if not per-user)

Below the pricing table, create a feature limits comparison:

- Rows: Key limiting factors (users, storage, API calls, projects)
- Same columns
- Values: Actual limits (e.g., “10 users”, “100GB”, “1000/day”)

Add analysis:

- Which competitor is cheapest/most expensive at each tier?
- Where are we priced competitively vs. premium vs. discount?
- What’s the value proposition at each price point?

This surfaces pricing strategy differences. Maybe Competitor A is cheaper on sticker price but more expensive per user. Maybe Competitor B offers more generous limits at the same price, indicating you should reconsider your caps. Maybe you’re the only one with a true free tier, which is a competitive advantage you’re not emphasizing enough.

Analyzing positioning through messaging audit. Pricing isn’t just numbers. It’s how competitors position value. What do they emphasize? What problems do they claim to solve? What customer types do they reference?

Prompt: Positioning Analysis

Research [Competitor Name]'s website using web search and analyze their positioning in `research/competitors/[competitor-name]-positioning.md`.

Extract and analyze from their homepage, product pages, and marketing content:

- **Primary headline:** What's their main value proposition?
- **Key messaging themes:** What concepts appear repeatedly? (e.g., "fast", "secure", "easy", "enterprise-grade")
- **Customer types referenced:** Who do they say they serve?
- **Problem statements:** What customer pain points do they address?
- **Differentiation claims:** What makes them "different" or "better"?
- **Social proof:** What case studies, logos, testimonials do they feature?
- **Call to action:** What do they want visitors to do? (Trial, demo, contact sales)

Provide a 2-3 sentence summary of their positioning strategy.

You learn that Competitor A positions as "enterprise-grade security for regulated industries" while Competitor B positions as "the fastest way to get started (no setup required)." These are different value propositions attracting different customers. Your positioning needs to either differentiate from both or target an underserved segment.

Tracking changes over time with dated artifacts. Positioning shifts. Competitors reposition when they enter new markets, after funding rounds, or in response to competition. Tracking this reveals strategy.

Save positioning analyses with dates: `competitor-a-positioning-2026-01.md`, `competitor-a-positioning-2026-04.md`. When you compare versions, you see messaging evolution. Maybe Competitor A stopped mentioning "small teams" and started emphasizing "enterprise scale." They're moving upmarket. This informs your own positioning decisions.

Time and cost for pricing analysis: Deep dive on one competitor's

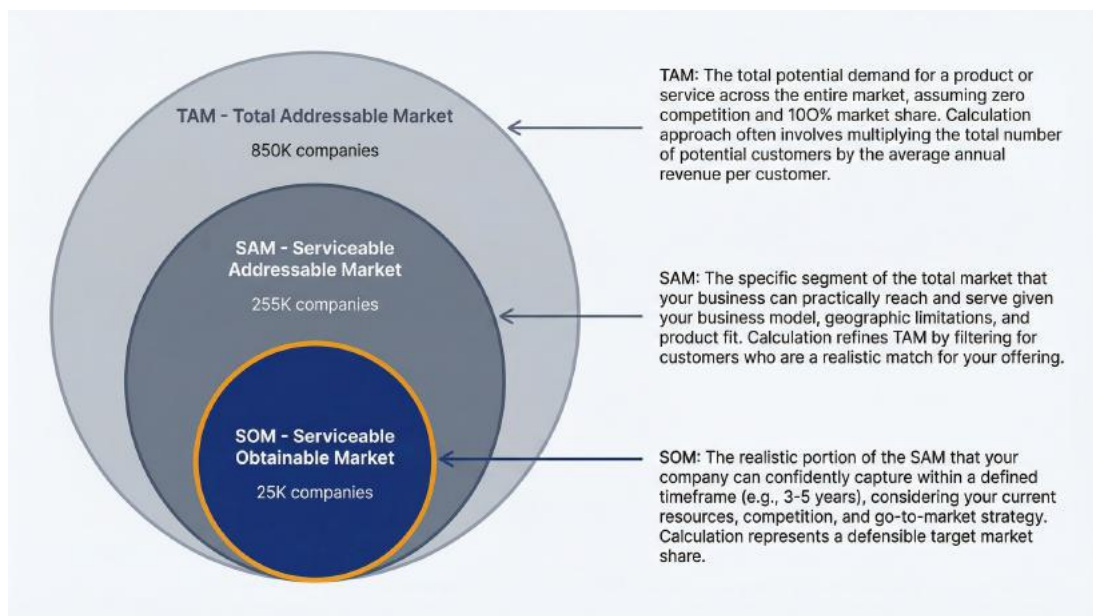
pricing: 10-15 minutes, 20,000-35,000 tokens, \$0.10-0.18. Pricing comparison table for 5 competitors: 10 minutes, 15,000-25,000 tokens, \$0.08-0.13. Positioning analysis per competitor: 15-20 minutes, 25,000-40,000 tokens, \$0.13-0.20.

When this fails: Competitors with complex, customized pricing where public information doesn't reflect actual deals. Opaque enterprise pricing with no listed numbers. In these cases, supplement Claude Code research with sales intelligence: what prospects tell you about competitor quotes, win/loss analysis, conversations with former competitor customers.

5.4 Creating Defensible Market Estimates

Stakeholders want numbers. How big is this market? How many potential customers? What's our addressable opportunity? Market sizing requires assumptions, data sources, and structured reasoning. Claude Code doesn't magically produce accurate numbers. It structures your methodology so your assumptions are visible and defensible.

Using Claude Code to structure TAM/SAM/SOM analysis. Total Addressable Market, Serviceable Addressable Market, Serviceable Obtainable Market: the classic VC framework. You need to estimate these, document your reasoning, and make it reviewable.



TAM/SAM/SOM nested market sizing: from total addressable market down to obtainable opportunity

Prompt: Market Sizing Framework

Help me size the market for [product/category] using a TAM/SAM/SOM framework. Create research/market-sizing/market-size-2026.md.

For each metric, provide:

- **Definition:** What this metric means for our product
- **Calculation approach:** How we'll estimate it
- **Data sources:** Where numbers come from (industry reports, census data, proxies)
- **Assumptions:** What we're assuming to be true
- **Estimated range:** Low/medium/high estimates
- **Reasoning:** Why this calculation is reasonable

Structure:

1. **TAM** - Total Addressable Market (if everyone who could use this did)
2. **SAM** - Serviceable Addressable Market (realistic subset we can serve)
3. **SOM** - Serviceable Obtainable Market (what we could capture in 1-3 years)

Show your work. Make assumptions explicit and conservative.

Claude Code helps you think through the logic:

Total Addressable Market (TAM)

****Definition**:** All companies in North America with 50-5000 employees that need [product category].

****Calculation approach**:**

- Total companies with 50-5000 employees in North America: ~850,000 (US Census Bureau, 2024)
- Percentage needing [product category]: ~30% (industry reports suggest

penetration in similar tools)

– TAM in companies: $850,000 \times 30\% = 255,000$ companies

****Revenue TAM**:**

– Assuming average deal size of \$5,000/year

– $255,000$ companies $\times$ \$5,000 = \$1.275 billion

****Assumptions**:**

– Company size range captures target market (eliminates too-small and enterprise)

– 30% need category based on analogous product adoption rates

– \$5,000/year is realistic average (mix of SMB and mid-market)

****Estimated range**:**

– Low: \$850M (20% need, \$4K average)

– Medium: \$1.3B (30% need, \$5K average)

– High: \$2.1B (40% need, \$6K average)

You're not hiding assumptions in a spreadsheet. They're documented and reviewable. When your CFO challenges the 30% penetration rate, you can discuss it explicitly and update the model.

Prompt template: Market sizing with assumptions. The key is documenting methodology, not producing a single “magic number.”

Prompt: Market Sizing Methodology Review

Review my market sizing assumptions in research/market-sizing/market-size-2026.md. For each assumption, evaluate:

- Is the data source credible?
- Is the logic sound?
- Are there obvious gaps or questionable leaps?
- What sensitivity analysis would help? (What if this assumption is off by 50%?)

Provide constructive feedback to strengthen the analysis.

Claude Code challenges your reasoning. It might point out: “The 30% penetration assumption isn’t sourced. It’s described as ‘industry reports suggest.’ Can you cite a specific report or provide a proxy calculation?”

Or: “The calculation assumes all target companies spend the same amount, but SMB vs. mid-market pricing likely differs significantly. Consider segmenting the calculation.”

This feedback helps you produce defensible market sizing, not just numbers that sound good.

Segmentation frameworks and their application. Markets aren’t monolithic. You have segments by company size, industry, geography, or use case. Segmentation shows where to focus.

Prompt: Market Segmentation

Segment the [product category] market by [company size / industry / geography / use case]. Create a segmentation analysis in `research/market-sizing/segmentation-2026.md`.

For each segment, estimate:

- **Size:** How many potential customers?
- **Characteristics:** What defines this segment?
- **Needs:** What do they care about most?
- **Willingness to pay:** Price sensitivity (high/medium/low)
- **Competition:** Who else targets this segment?
- **Our fit:** How well do we serve this segment today?
(Strong/Medium/Weak/None)

Rank segments by attractiveness: $\text{Size} \times \text{Willingness to pay} \times \text{Our fit}$.

You get a prioritized list of segments with clear reasoning. Maybe “Mid-market healthcare” scores high because it’s large, willing to pay premium for compliance features you have, and underserved by competitors focused on SMB or enterprise. This informs go-to-market strategy.

Documenting methodology for stakeholder review. Market sizing documents become references. Your board deck references them. Your sales team uses them to understand target market. Documenting methodology makes the numbers trustworthy.

Include a “How to update this analysis” section in your market sizing doc:

How to Update This Analysis

To refresh these estimates:

1. Check US Census Bureau data for updated company counts
2. Review [specific industry reports] for updated penetration rates
3. Validate average deal size against actual sales data
4. Re-run calculations with updated inputs
5. Update "Last reviewed" date

****Review cadence**:** Annually, or when entering new markets.

This lets someone else (or future you) update the analysis without reinventing methodology.

Time and cost: Initial market sizing analysis: 30-45 minutes, 50,000-80,000 tokens, \$0.25-0.40. Segmentation analysis: 20-30 minutes, 30,000-50,000 tokens, \$0.15-0.25. Updates with new data: 15-20 minutes, 20,000-35,000 tokens, \$0.10-0.18.

Limitations: Claude Code doesn't have proprietary market data. It uses publicly available information, which means estimates may be rough. For high-stakes decisions like fundraising or M&A, supplement with paid analyst reports, surveys, or consultants. For product strategy and internal planning, Claude Code's structured approach is sufficient.

5.5 Separating Signal from Noise in Industry Trends

Industry trends shape product strategy. AI capabilities evolving. Privacy regulations tightening. Customer expectations shifting. You need to track trends, separate hype from signal, and synthesize implications for your roadmap. This is research work that benefits from Claude Code's synthesis and documentation capabilities.

Aggregating trend data from multiple sources. Trends aren't single articles. They're patterns across sources. You read analyst reports, tech blogs, customer conversations, conference talks. Synthesizing this into coherent trend briefs is time-consuming. Claude Code systematizes it.

Create a trends workspace:

```
mkdir -p research/trends
```

Collect sources (paste article text, copy report sections, summarize conversations) into individual files:

```
research/trends/  
├── ai-tooling-gartner-2026.txt  
├── privacy-regulations-summary.txt  
├── customer-feedback-themes.txt  
└── saas-trends-2026.txt
```

Then ask Claude Code to synthesize:

Prompt: Trend Synthesis

Review the source files in `research/trends/` and synthesize key themes into `research/trends/trend-brief-2026-01.md`.

For each major trend identified:

- **Trend name:** Concise label
- **Description:** What's changing (2-3 sentences)
- **Evidence:** What sources support this? (cite specific files)
- **Timing:** Is this happening now, soon, or later? (1 year / 2-3 years / 3+ years)
- **Relevance to us:** How does this affect our product? (High / Medium / Low)
- **Implications:** What should we consider doing? (2-3 bullets)

Prioritize by relevance. Limit to top 5-7 trends to avoid overwhelming the brief.

Claude Code reads your source files, identifies recurring themes, and produces a structured trend brief. You get synthesis, not just aggregation.

Prompt template: Industry trend synthesis. The key is connecting trends to action.

Example output:

Trend: AI-Powered Productivity Features Becoming Table Stakes

****Description**:** Customers increasingly expect AI assistance built into products: autocomplete, suggestions, smart defaults. What was a differentiator 18 months ago is becoming an expected baseline feature across SaaS tools.

****Evidence**:**

- Gartner report (ai-tooling-gartner-2026.txt) notes 67% of B2B buyers expect AI features
- Customer feedback (customer-feedback-themes.txt) includes 12 mentions of "why doesn't this suggest..." or "competitors have AI for this"
- SaaS trends report (saas-trends-2026.txt) lists AI features as top 3 investment area

****Timing**:** Happening now. Customers are comparing products based on AI capabilities today.

****Relevance to us**:** High. We have minimal AI features and competitors are shipping them.

****Implications**:**

- Evaluate which workflows would benefit most from AI assistance
- Prioritize AI features that improve core workflows vs. novelty
- Consider partnership vs. build-in-house for AI capabilities
- Monitor customer churn related to "missing features" that might be AI-related

This connects dots across sources and makes trends actionable. You're not just reporting "AI is hot." You're showing evidence, timing, and specific implications for your product.

Separating signal from noise in trend reports. Not every trend matters. Some are hype. Some are real but irrelevant to your market. Some are important but far future. Claude Code helps you filter.

Prompt: Trend Validation

Review the trends in research/trends/trend-brief-2026-01.md. For each trend, assess:

- **Signal vs. noise:** Is this backed by multiple credible sources, or is it

one article's speculation?

- **Relevance filter:** Does this affect our target customer segment, or a different market?
- **Urgency:** Do we need to act this quarter, this year, or just monitor?

Flag any trends that appear to be hype without substance. Suggest which trends deserve immediate roadmap consideration vs. long-term monitoring.

This prevents chasing every shiny trend. You get a reality check. “The ‘blockchain for everything’ trend appears in one source with no customer validation. Recommend deprioritizing. The ‘privacy-first architecture’ trend appears in regulatory analysis, customer feedback, and competitive positioning. This is signal. Recommend roadmap review.”

Creating actionable trend briefs. The output should drive decisions, not sit in a folder. Structure trend briefs for stakeholder consumption:

Prompt: Executive Trend Summary

Based on `research/trends/trend-brief-2026-01.md`, create a one-page executive summary in `research/trends/trend-summary-exec-2026-01.md`.

Format:

- **Top 3 trends we must respond to:** Brief description + specific action recommended
- **Trends to monitor:** 2-3 emerging trends not yet urgent
- **Trends we're ignoring and why:** 1-2 trends that don't apply to us

Keep it scannable. Executives should read this in 3 minutes and understand implications.

This becomes a slide in your strategy review, a reference in roadmap planning, or a brief for your exec team. It's action-oriented, not just informational.

Time and cost: Synthesizing 5-8 source documents into a trend brief: 20-30 minutes, 40,000-70,000 tokens, \$0.20-0.35. Creating executive summary from trend brief: 10 minutes, 15,000-25,000 tokens, \$0.08-

0.13. Total for trend analysis cycle: 30-40 minutes, \$0.30-0.50.

When to refresh: Quarterly for fast-moving industries, annually for slower markets. Ad-hoc when major industry events occur (new regulations, major competitor moves, technology breakthroughs).

Limitations: Claude Code synthesizes what you give it. If your sources are low-quality or biased toward one perspective, the synthesis will be too. Garbage in, garbage out. Combine Claude Code's synthesis with diverse, credible sources: analyst reports, customer research, industry experts, competitive intelligence.

You can now use Claude Code for systematic market research that produces versionable, updateable artifacts instead of one-off documents that disappear. Chapter 6 applies similar synthesis techniques to customer feedback and UX research: turning raw feedback data into actionable product insights.

6 Customer Feedback and UX Research Synthesis

6.1 The Feedback Synthesis Challenge

You have 847 support tickets from last quarter, 200+ NPS responses, 15 customer interview transcripts, and a Slack channel where users post feature requests daily. Your VP wants insights. What themes are emerging? What should we prioritize? What are customers actually telling us?

Traditional approach: Read through everything, highlight things that seem important, make a spreadsheet with categories, try to remember what you read three hours ago, produce a summary that's 60% what you remember clearly and 40% what stuck in your mind because it was recent or dramatic. This takes days and misses patterns that appear across scattered sources.

You need systematic synthesis: consistent categorization, frequency analysis, connections between themes, supporting evidence for each insight. Manual synthesis at scale is unreliable. Your memory filters for recency and salience, not actual frequency or importance. You categorize inconsistently as you develop better understanding midway through. You miss weak signals buried in volume.

Claude Code systematizes feedback synthesis. It reads all your data, applies consistent categorization, tracks frequency, identifies co-occurring themes, and produces structured reports with supporting quotes. The same analysis methodology executes repeatedly. Quarterly NPS analysis uses the same process every quarter, making results comparable over time.

Volume problem: Too much feedback, not enough patterns. As your product grows, feedback volume overwhelms manual analysis. You need to process hundreds or thousands of data points to find signal.

Claude Code processes data exhaustively, not selectively. It reads all 847 tickets, not a sample. It categorizes all 200 NPS responses consistently.

Format problem: Data scattered across tools and formats.

Feedback lives in Zendesk, Intercom, Google Forms, email, Slack, sales call notes, Twitter mentions. Different formats, different structures. Synthesizing requires getting everything into analyzable form first: export, convert, consolidate. Claude Code handles multiple file formats natively: CSV from Zendesk, JSON from Intercom, plain text from interview transcripts, markdown from meeting notes.

Prioritization problem: What matters most? Raw feedback doesn't come with priority scores. You need to identify what appears frequently, what causes actual pain versus minor annoyance, what blocks workflows versus creates inconvenience. Synthesis reveals patterns. The 15% of feedback that mentions slow export might be the critical path issue your customers politely mention once rather than the loud feature request five users repeatedly demand.

Claude Code solves these problems for \$0.40-0.90 per synthesis session and 30-60 minutes of your time. Compare to two days of manual work that produces inconsistent results you can't easily repeat or update.

When this is overkill: If you have 20 pieces of feedback, read them yourself. If the feedback is highly structured already (numeric survey responses, yes/no questions), use a spreadsheet. Claude Code adds value when you have unstructured qualitative data at volume (text feedback, interview transcripts, open-ended responses) where pattern identification requires actually reading everything.

6.2 Getting Your Data Ready for Analysis

Claude Code reads files, not databases or SaaS tools directly. You need to export your feedback data into formats Claude Code can process. This takes 10-20 minutes of setup work, then becomes repeatable for future synthesis cycles.

Exporting from common sources: Most feedback tools offer CSV or

JSON export. The mechanics vary by tool, but the goal is the same: get your data into a file.

Zendesk: Navigate to Reporting → Explore → Create report → Select tickets → Filter by date range → Export as CSV. The export includes ticket subject, description, comments, status, tags, custom fields. You get one row per ticket with all interaction history concatenated. File size varies: 1,000 tickets typically produces 5-15MB depending on comment volume.

Intercom: Go to Reporting → Conversations → Export conversations → Select date range and filters → Download CSV or JSON. CSV format is simpler for analysis; JSON includes richer metadata if you need it. Typical export: 500 conversations = 3-8MB.

Survey tools (Typeform, Google Forms, SurveyMonkey): All offer CSV export. Responses export as one row per respondent with each question as a column. Open-ended text responses are what you're after: the columns with paragraph-length answers, not multiple choice.

NPS tools: Export the numerical scores and the "Why did you give this score?" text responses. The text is more valuable than the number for synthesis. You want to understand what's driving the score.

Slack: For feature requests or feedback in channels, copy relevant threads into a text file or use Slack's export feature. Format doesn't matter much. Claude Code can process channel exports (JSON) or plain text files where you've pasted conversations. For small volumes, manual copy-paste into a text file is faster than figuring out Slack's export API.

Interview transcripts: If you recorded and transcribed user interviews, you likely have text files or Word docs. Convert Word docs to plain text (.txt) or markdown (.md). Claude Code handles both. If you use transcription services like Otter.ai or Rev, export as plain text.

Email feedback: Forward relevant customer emails to a folder, then export that folder. Or copy-paste emails into a text file with clear separators between messages. Format: one file per email thread or one consolidated file with dividers like --- between messages.

File formats Claude Code handles well:

CSV is ideal for structured feedback with consistent fields. Each row is one feedback item (ticket, survey response, NPS comment). Columns are attributes (date, customer name, feedback text, category, rating). Claude Code reads CSV natively and can analyze rows systematically.

JSON works for more complex exports with nested data. Intercom exports include conversation threads, user metadata, tag hierarchies. Claude Code parses JSON structure and extracts relevant fields.

Plain text (.txt) is the universal format. Interview transcripts, email threads, Slack conversations: anything can become plain text. Structure it minimally by separating feedback items with blank lines or dividers. Add basic metadata inline if useful: “User: enterprise_customer_47 / Date: 2026-01-15 / Feedback: [text]”

Markdown (.md) works like plain text but with light formatting. Use headings to separate feedback sources or interviews. Useful for interview transcripts where you want to preserve speaker labels and question structure.

What Claude Code doesn’t handle: Binary formats without conversion. Word documents (.docx), PDFs with scanned text, images of handwritten notes, and audio files need conversion to text first. For PDFs, extract text using PDF export tools. For audio, transcribe using Otter.ai, Rev, or similar services.

Cleaning data before analysis: You don’t need perfect data, but basic cleaning improves results.

Remove PII (personally identifiable information). Customer names, email addresses, phone numbers, account IDs: anything that identifies individuals. Replace with generic labels: “Customer_1”, “Enterprise_user”, “Trial_account”. You’re analyzing patterns, not tracking individuals. PII removal prevents accidental exposure if you share analysis outputs and respects privacy.

If your export has a “Name” column, replace values with generic IDs. If feedback text includes “Hi Sarah” or mentions emails, find-and-replace

or redact. Claude Code doesn't automatically scrub PII. That's your responsibility before analysis.

Standardize formats within files. If your CSV has inconsistent date formats (some "1/15/2026", some "January 15, 2026"), pick one format. If some feedback uses smart quotes and some uses straight quotes, this doesn't matter. If responses mix languages, note this. Claude Code handles multilingual text but may categorize differently across languages.

Remove empty rows and non-feedback data. CSV exports often include header rows, footer rows with export metadata, and empty rows between sections. Delete these. Each row should be one feedback item. Text files should contain only feedback, not export timestamps or system messages.

Privacy and PII considerations: Feedback data is sensitive. Customers shared opinions trusting you'll use them to improve the product, not expose their identities.

Default to anonymization. Unless you have specific need to track feedback by individual (account risk analysis, high-touch enterprise customers), remove identifying information. This makes sharing synthesis outputs safer. You can post theme reports in Slack or attach to PRDs without privacy concerns.

Be explicit about what you're analyzing. If you're using Claude Code via API, data is processed by Anthropic's systems and not used for training. If you're concerned about sensitive feedback (medical, financial, legal domains), review Anthropic's data handling policies or consider on-premise alternatives.

Don't analyze feedback that customers provided under confidentiality. Enterprise customer interviews often include NDAs or confidentiality agreements. If you promised not to share specific feedback externally, don't put it into systems where you're uncertain about data handling. When in doubt, exclude that data or consult your legal team.

Typical preparation workflow:

1. Export data from source tools (10-15 minutes for multiple sources)

2. Remove PII using find-and-replace in spreadsheet or text editor (5-10 minutes)
3. Clean obvious data issues—empty rows, format inconsistencies (5 minutes)
4. Save files in a project directory: `feedback/2026-q1/` with descriptive names like `zendesk-tickets.csv`, `nps-responses.csv`, `interview-transcripts.txt`

Total preparation time: 20-40 minutes for a typical quarterly synthesis. This is one-time work per synthesis cycle. Once data is prepared, analysis runs quickly.

How much data is too much? Claude Code can process large files, but token consumption scales with file size. A single file with 1,000 feedback items (roughly 200,000-500,000 words total) costs \$1.00-2.50 to process completely. Splitting large datasets into smaller files can help manage costs. Analyze each file separately, then aggregate themes.

If you have more than 2,000 feedback items, consider either: - Sampling (analyze a random subset, still more systematic than manual review) - Batch processing (split by time period or source, analyze separately, then compare) - Stratified synthesis (analyze segments separately: enterprise vs. SMB, new users vs. existing, etc.)

For typical PM use cases (quarterly synthesis, post-release feedback review, thematic analysis for roadmap planning), you're usually working with 100-1,000 feedback items. This is well within Claude Code's efficient range.

6.3 Finding Patterns Across Hundreds of Responses

You have feedback prepared. Now you need themes: the recurring topics, problems, requests, and sentiments that appear across your data. Thematic analysis identifies patterns and quantifies how often each appears.

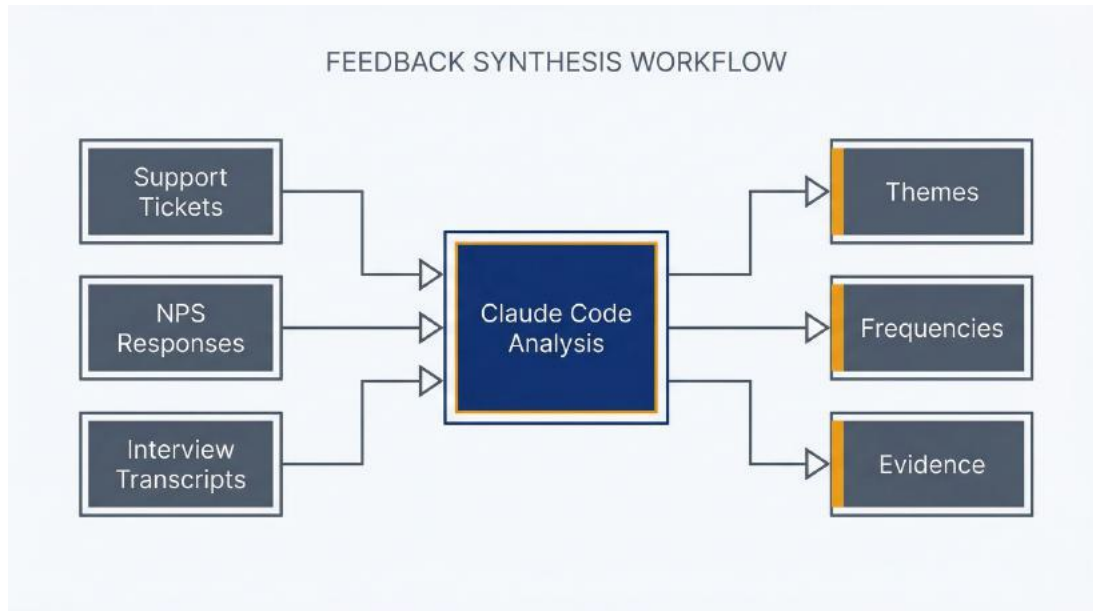


Figure 6.1: Feedback Synthesis Workflow – Use this to transform scattered customer data into actionable insights. Raw data sources (support tickets, NPS, interviews) flow through Claude Code analysis to produce structured themes with frequencies, sentiment, and supporting evidence.

Prompt template: Feedback categorization. Start by asking Claude Code to identify themes. You can either define categories in advance if you know what you're looking for, or let themes emerge from the data.

Launch Claude Code in plan mode—you're reading and analyzing, not writing files yet:

```
claude --permission-mode plan
```

Prompt: Emergent Theme Identification

Read all the feedback in `feedback/2026-q1/` and identify the top 10-15 themes that appear repeatedly.

For each theme:

- **Theme name:** Concise label (3-5 words)
- **Description:** What this theme represents (1 sentence)
- **Frequency:** How many feedback items mention this theme
- **Representative quotes:** 2-3 example quotes that illustrate this

theme

Prioritize by frequency. Include only themes that appear at least 5 times unless they represent critical issues.

Claude Code reads your feedback files, identifies recurring topics, counts occurrences, and provides structured output. You get something like:

`## Theme: Slow Export Performance`

`**Description**`: Users report that exporting large datasets takes too long or times out.

`**Frequency**`: 87 mentions across tickets and NPS responses

`**Representative quotes**`:

- `- "Tried to export 50k records and it timed out after 10 minutes. This is unusable for our reporting needs."`
- `- "Export feature is frustratingly slow. Takes 5+ minutes for medium-sized exports."`
- `- "Please fix export speed—this is blocking our monthly reports."`

`## Theme: Missing Bulk Edit Capabilities`

`**Description**`: Users want to edit multiple items simultaneously rather than one at a time.

`**Frequency**`: 64 mentions

`**Representative quotes**`:

- `- "I need to update 200 records with the same tag. Having to do this one-by-one is painful."`
- `- "Why can't I select multiple items and apply changes? This is standard in every tool we use."`
- `- "Bulk editing would save me hours per week."`

Defining categories: User-defined vs. emergent. You have two approaches to categorization.

Emergent themes: Let Claude Code identify what appears in the data without preconceived categories. This is exploratory; you discover what customers are actually talking about. Use this when you don't have strong

hypotheses about feedback themes or when you want to validate whether your assumed categories match reality.

Predefined categories: Give Claude Code specific categories to use. This works when you have established taxonomy (feature requests, bugs, usability issues, performance complaints, documentation gaps) and want to classify feedback consistently over time.

Prompt: Predefined Category Classification

Read all feedback in `feedback/2026-q1/` and classify each item into these categories:

- Feature requests
- Bug reports
- Performance issues
- Usability/UX problems
- Documentation gaps
- Integration requests
- Pricing/billing questions
- Other

For each category:

- Count how many feedback items fall into it
- List the top 5 most frequently mentioned specific issues within that category
- Provide 2-3 representative quotes

If feedback mentions multiple categories, count it in each applicable category.

This produces a structured breakdown showing that you received 127 feature requests (with top requests being X, Y, Z), 89 bug reports (concentrated in areas A, B, C), 64 performance issues, etc.

Which approach to use: Emergent for discovery and validation. Predefined for consistency and comparison across time periods. You might use emergent analysis once to establish categories, then use those categories as predefined taxonomy in future quarters to track how theme

distribution changes.

Frequency analysis and prioritization. Raw frequency matters but isn't everything. Ten complaints about a critical workflow blocker may matter more than 50 complaints about a cosmetic annoyance. Claude Code helps you layer context onto frequency.

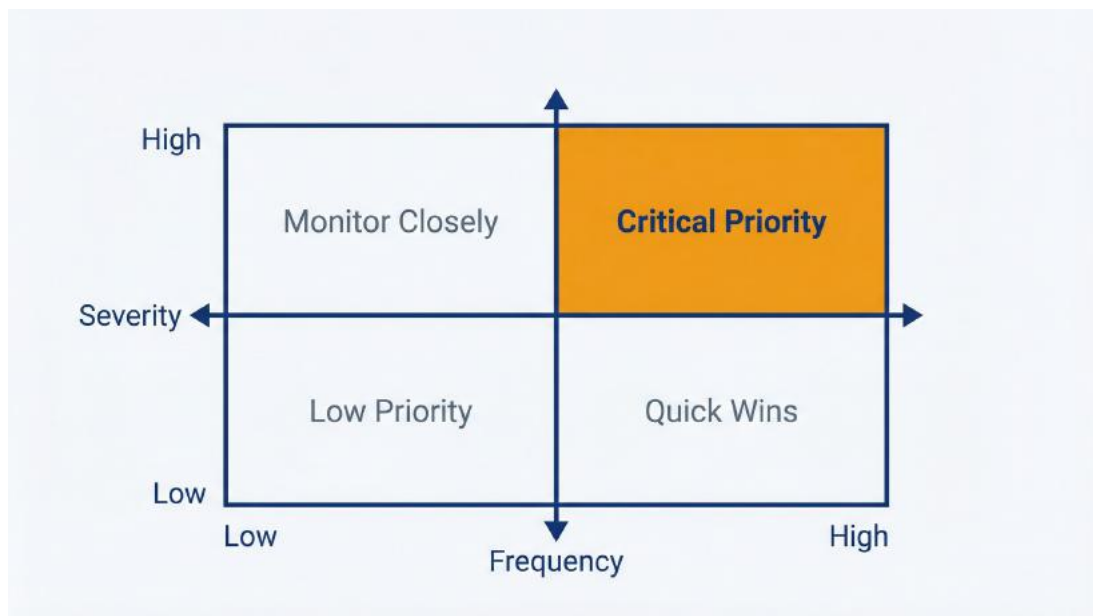


Figure 6.2: Prioritization Matrix – Use this to identify which feedback themes demand immediate attention. Frequency on x-axis, severity on y-axis. High-frequency + high-severity quadrant (top right) indicates critical priority items that should drive roadmap decisions.

Prompt: Frequency and Impact Analysis

Review the themes identified in the previous analysis. For each theme, assess:

Frequency tier:

- High: Mentioned by >10% of feedback items
- Medium: Mentioned by 5-10% of feedback items
- Low: Mentioned by <5% but still notable

Severity signals (based on language used):

- Critical: Language like “blocking”, “can’t do my job”, “showstopper”,

- “considering alternatives”
- High: Language like “frustrating”, “time-consuming”, “painful”, “needs fixing”
 - Medium: Language like “would be nice”, “improvement”, “suggestion”
 - Low: Language like “minor issue”, “not urgent”, “nice to have”

Create a prioritization matrix: Frequency × Severity. Highlight themes that are both frequent and severe.

This surfaces the “quiet killers”: issues that many customers mention with high-severity language but that you might overlook if you only chase the loudest feedback. It also deprioritizes noise, things many people mention but with low-impact language suggesting it’s not actually blocking them.

Sentiment overlay on themes. Beyond frequency and severity, you want to know sentiment. Are customers angry about this or mildly annoyed? Grateful when it works or neutral?

Prompt: Sentiment Analysis by Theme

For each major theme identified, analyze the sentiment of feedback mentioning that theme:

- **Positive:** Customers satisfied, praising, thanking
- **Neutral:** Factual reports, suggestions without emotion
- **Negative:** Frustration, disappointment, anger
- **Mixed:** Both positive and negative (e.g., “Love the feature but performance is bad”)

Provide sentiment distribution (e.g., “Slow Export Performance: 5% positive, 15% neutral, 75% negative, 5% mixed”) and note any patterns in sentiment by customer segment if visible in the data.

Sentiment reveals urgency. A theme with 90% negative sentiment demands attention regardless of frequency. A theme with mixed sentiment might indicate the feature works well for some use cases but fails for others—suggesting targeted improvements rather than wholesale changes.

Output format: Theme report with supporting quotes. The synthesis output should be scannable and actionable. Not a 50-page document, but a structured report you can reference in roadmap planning.

Prompt: Generate Theme Report

Based on the thematic analysis, create a feedback synthesis report in `feedback/2026-q1/theme-report.md`.

Structure:

1. **Executive Summary** (1 paragraph): Top 3 themes by frequency and top 3 by severity
2. **Detailed Themes** (one section per theme):
 - Theme name and description
 - Frequency and percentage of total feedback
 - Severity assessment
 - Sentiment distribution
 - Representative quotes (3-5 quotes)
 - Customer segments affected (if identifiable)
 - Suggested action or area for investigation
3. **Cross-Cutting Patterns** (1 paragraph): Themes that frequently co-occur
4. **Weak Signals** (bullet list): Low-frequency themes worth monitoring

Keep total report under 6 pages. Prioritize clarity over completeness.

Claude Code generates a structured report file. Switch to normal mode to allow file writing:

```
/exit  
claude
```

Run the prompt again. Claude Code writes the theme report to your repository. Commit it:

```
git add feedback/2026-q1/theme-report.md  
git commit -m "Add Q1 2026 feedback synthesis report"
```

Time and cost for thematic analysis: Reading and analyzing 500 feedback items across multiple files: 20-30 minutes, 60,000-120,000 tokens, \$0.30-0.60. Generating structured theme report: 10-15 minutes, 25,000-45,000 tokens, \$0.13-0.23. Total for complete thematic synthesis: 30-45 minutes, \$0.45-0.85.

When thematic analysis fails: If feedback is too heterogeneous and every customer wants something completely different, themes don't emerge. This suggests your product serves too many use cases or your customer base is too diverse to synthesize meaningfully. Segment first (by industry, company size, user role), then analyze segments separately.

If feedback is extremely sparse (50 items covering 30 different topics), statistical patterns don't exist. Just read the feedback manually. Thematic analysis adds value at scale, not for small datasets.

If feedback quality is low (one-word responses, "good/bad" without explanation), there's nothing to synthesize. This is a data collection problem, not an analysis problem. Improve how you gather feedback with better survey questions, interview protocols, and support ticket templates.

6.4 Extracting Insights from Interview Transcripts

Feedback tickets tell you what broke. UX research tells you why users struggle and what they're trying to accomplish. Synthesizing research requires different techniques than feedback. You're analyzing narratives, not categorizing complaints.

Analyzing interview transcripts. User interviews produce rich qualitative data: stories, workflows, pain points, mental models. Synthesis extracts insights from these narratives.

Prepare transcripts: one file per interview or one consolidated file with clear separators. Label speakers consistently (Interviewer: / Participant: or Q: / A:). Remove identifying information.

Typical structure:

Interview with Enterprise_Customer_12

Date: 2026-01-15

Role: Operations Manager

Company size: 500 employees

Q: Walk me through how you currently handle [workflow].

A: We start by exporting data from our CRM, which is already frustrating because...

[rest of transcript]

Launch Claude Code:

```
claude --permission-mode plan
```

Prompt: Interview Synthesis

Read the interview transcript in `research/interviews/interview-[ID].txt` and synthesize key insights.

Extract:

- **Pain points:** What frustrates this user? What's broken or difficult? (bullet list with quotes)
- **Workflows:** What is the user trying to accomplish? What's their process? (narrative description)
- **Workarounds:** What hacks or alternative tools do they use to compensate? (bullet list)
- **Mental models:** How does the user think about the problem space? What terminology do they use? (paragraph)
- **Unmet needs:** What would make this user's work easier? What gaps exist? (bullet list)
- **Success criteria:** How does this user define success for this workflow? (bullet list)

Focus on direct insights from what the user said, not interpretation. Use quotes to support each point.

Claude Code reads the transcript and produces structured output. You get pain points with supporting quotes, workflows described in the user's

own words, and needs clearly articulated.

Prompt template: Interview synthesis. For individual interviews, the above works well. For multiple interviews (10-20 transcripts from a research study), you need aggregation:

Prompt: Multi-Interview Synthesis

Read all interview transcripts in research/interviews/ and synthesize patterns across interviews.

For each category below, identify themes that appear across multiple interviews (not individual one-off mentions):

Pain Points (recurring problems):

- Theme name
- How many interviews mentioned this (e.g., “8 of 12 interviews”)
- Representative quotes from 2-3 different interviews
- Severity assessment based on language used

Workflow Patterns (how users accomplish tasks):

- Common workflow described
- How many users follow this pattern
- Variations or alternative approaches mentioned

Unmet Needs (what users wish existed):

- Need description
- How many users expressed this
- Context or use cases where this need arises

Segment Differences (if applicable):

- Note any patterns that differ by user role, company size, industry, or other visible segments

Prioritize by frequency—focus on themes appearing in at least 3 interviews unless a single mention represents a critical insight.

This produces cross-interview patterns. You discover that 9 of 12 interviews mentioned slow data export as a pain point, 7 mentioned manual data entry as a workaround for lack of integrations, 5 expressed need for real-time collaboration features.

Identifying pain points and unmet needs. The distinction matters:

Pain points are problems with current state: things that are broken, frustrating, slow, confusing. These map to bugs, performance improvements, usability fixes. Example: “Export takes too long” is a pain point with your product.

Unmet needs are gaps: things users want but don’t have. These map to feature requests, integrations, workflow enhancements. Example: “I wish I could assign tasks to team members directly in the tool” is an unmet need.

Sometimes feedback expresses both: “I have to manually copy data to Slack to notify my team [pain point], which is time-consuming and error-prone. I wish there was a Slack integration [unmet need].”

Ask Claude Code to separate these:

Prompt: Pain Points vs. Unmet Needs

Review the research synthesis and categorize insights into:

1. **Pain points:** Problems with current product that need fixing
2. **Unmet needs:** Capabilities users want that don’t currently exist

For each, note:

- Frequency (how many users mentioned)
- Urgency (based on language and context)
- Potential solutions (what might address this)

This creates a prioritized list for roadmap planning. Pain points feed into bug/performance backlogs; unmet needs feed into feature roadmap.

Creating journey maps from qualitative data. Journey maps

visualize user workflows and highlight where users struggle. Claude Code can't draw diagrams, but it can structure journey data for you to visualize or share.

Prompt: Journey Map Data

Based on interview transcripts in `research/interviews/`, create a structured journey map for the `[specific workflow]` process.

For each step in the user's journey:

1. **Step name:** What the user is doing (verb phrase: "Exporting data", "Reviewing results")
2. **User goal:** What they're trying to accomplish in this step
3. **Actions taken:** Specific things they do (click, type, wait, switch tools)
4. **Pain points:** What's frustrating or difficult in this step
5. **Emotional state:** How users feel (based on language used: frustrated, satisfied, confused, neutral)
6. **Tools used:** What systems/tools are involved
7. **Opportunities:** Where improvements could help

Output as a structured list or table that I can convert to a visual journey map.

Claude Code produces structured journey data:

Step 1: Gathering Source Data

****User goal**:** Collect all necessary data from multiple systems before analysis

****Actions taken**:**

- Export CSV from CRM (5-10 minutes wait time mentioned in 7 interviews)
- Download spreadsheet from finance system
- Copy-paste from email threads or Slack
- Combine into master spreadsheet manually

****Pain points**:**

- "Exports are slow and sometimes time out" (9 interviews)
- "Data formats don't match, have to reformat" (6 interviews)

- "I never know if I have the latest data" (4 interviews)

****Emotional state****: Frustrated ("tedious", "annoying", "waste of time" mentioned)

****Tools used****: CRM system, finance ERP, Excel/Google Sheets, email, Slack

****Opportunities****:

- Speed up export performance
- Standardize export formats
- Real-time data sync instead of manual export
- Unified dashboard eliminating manual gathering

You take this structured data and create a visual journey map in Miro, Figma, or PowerPoint. The insight work is done. Claude Code identified pain points, opportunities, and emotional states from analyzing what users said.

Aggregating insights across multiple interviews. With 10-15 interviews, manual synthesis is feasible but time-consuming. With 20-30 interviews, manual synthesis is unreliable. You forget details from early interviews by the time you reach later ones. Claude Code processes all interviews consistently.

Prompt: Research Study Summary

Synthesize all interviews in `research/interviews/` into an executive research summary in `research/study-summary.md`.

Include:

1. **Research Overview**: How many interviews, who we talked to (roles, segments), what we asked
2. **Key Findings**: Top 5-7 insights that appeared across multiple interviews (prioritize by frequency and importance)
3. **Pain Points**: Top problems users face (ranked by frequency and severity)
4. **Unmet Needs**: Top feature/capability requests (ranked by frequency)

5. **Segment Insights:** Patterns that differ by user segment (if applicable)
6. **Recommendations:** Suggested product/design actions based on findings (3-5 bullets)
7. **Supporting Evidence:** Appendix with key quotes organized by theme

Target length: 4-6 pages. Optimize for stakeholder readability.

Claude Code generates a synthesis document you can share with leadership, design, and engineering. This becomes the artifact that drives roadmap discussions.

Time and cost for UX research synthesis: Single interview analysis: 10-15 minutes, 20,000-40,000 tokens, \$0.10-0.20. Multi-interview synthesis (10-15 interviews): 30-45 minutes, 80,000-150,000 tokens, \$0.40-0.75. Journey map data extraction: 15-20 minutes, 30,000-50,000 tokens, \$0.15-0.25.

Limitations: Claude Code synthesizes what users said, not what they meant. It can't read body language, interpret tone of voice, or catch sarcasm. It doesn't understand context you have but didn't document: the user was clearly confused, demonstrated frustration by long pauses, lit up when discussing a specific feature. Human researchers bring interpretation that Claude Code lacks.

Use Claude Code for the mechanical work: reading transcripts, identifying recurring themes, counting frequencies, extracting quotes. Apply your own judgment for interpretation: understanding why users struggle, what's really important, which needs align with product vision.

6.5 From Insight to Action

Synthesis without action is research theater. You need to connect insights to decisions: what goes on the roadmap, what gets prioritized, what gets deprioritized based on what you learned.

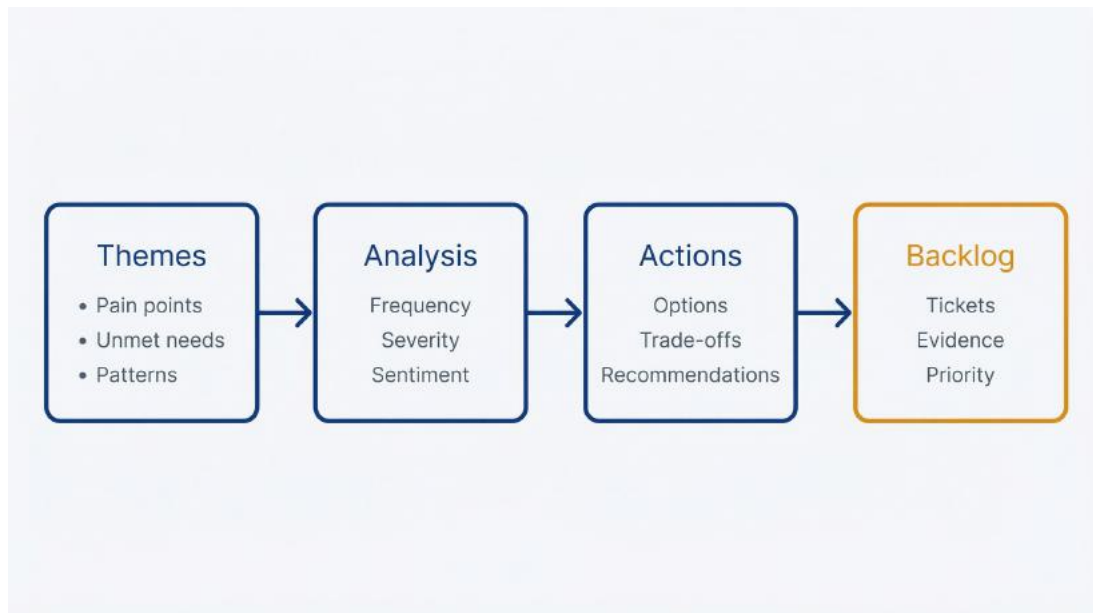


Figure 6.4: Research-to-Artifact Pipeline – Use this to transform raw feedback into prioritized backlog items while maintaining human oversight at the critical decision point. The pipeline flows from raw data (feedback, NPS, tickets) through AI analysis (pattern detection, themes) to a Human Judgment Gate where PMs apply domain expertise, business context, and priority judgment before producing validated, actionable backlog items.

Connecting feedback themes to product decisions. Each theme in your synthesis should map to a potential product action. Not every theme becomes a roadmap item, but every theme deserves consideration.

Prompt: Insight-to-Action Mapping

Review the theme report in `feedback/2026-q1/theme-report.md` and map each major theme to potential product actions.

For each theme:

- **Theme:** Name and brief description
- **Current state:** What we do today (if applicable)
- **Possible actions:** 2-4 options for addressing this (bug fix, feature enhancement, new feature, process change, documentation, not addressing it)
- **Trade-offs:** What each action would require (eng effort, design

- effort, time, complexity)
- **Recommendation:** Which action makes sense given frequency, severity, and feasibility

Be realistic about trade-offs. Not everything can be Po. Some things might be “monitor but don’t act yet.”

Claude Code produces a mapping document:

Theme: Slow Export Performance

****Current state**:** Export feature exists, works for small datasets (<10k records), degrades performance for larger datasets.

****Possible actions**:**

1. ****Optimize existing export**** – Investigate and fix performance bottlenecks
 - Trade-offs: 2–3 weeks eng effort, might not solve for very large exports
2. ****Implement async export****: Move large exports to background processing with email notification
 - Trade-offs: 4–5 weeks eng effort, requires infrastructure changes, but scales to any size
3. ****Add export limits****: Cap export size and suggest alternative approaches
 - Trade-offs: Minimal eng effort, but doesn't solve user need
4. ****Don't address****: Accept that export is slow for large datasets
 - Trade-offs: None, but continued user frustration and potential churn

****Recommendation**:** Option 2 (async export). High frequency (87 mentions) and high severity justify the investment. Solves the root cause rather than papering over it. Aligns with enterprise growth strategy where large exports are expected.

This becomes your roadmap conversation. You’re not just saying “users want faster exports.” You’re presenting options with trade-offs and a reasoned recommendation.

Prompt template: Insight-to-action mapping. The key is moving from “we learned X” to “we should do Y because Z.”

Prioritization frameworks in Claude Code. You have competing insights, limited engineering capacity, and stakeholder opinions. Prioritization frameworks help you make systematic decisions.

Prompt: RICE Prioritization

Apply the RICE prioritization framework (Reach $\times$ Impact $\times$ Confidence / Effort) to the potential actions identified in the insight-to-action mapping.

For each potential action, estimate:

- **Reach:** How many users does this affect? (Scale: 1-10, where 10 = most users)
- **Impact:** How much does this improve user experience? (Scale: 3 = massive, 2 = high, 1 = medium, 0.5 = low)
- **Confidence:** How confident are we in the Reach and Impact estimates? (Scale: 100% = high confidence, 80% = medium, 50% = low)
- **Effort:** How much work is this? (Scale in person-weeks: 1, 2, 4, 8, 12)

Calculate RICE score: (Reach $\times$ Impact $\times$ Confidence) / Effort

Rank actions by RICE score. Flag any actions with low confidence scores that need more validation before prioritization.

Claude Code scores each potential action and ranks them. You get data-informed prioritization, not just gut feel. The framework makes assumptions explicit. You can debate whether “Impact = 2” is right for a given feature, but at least you’re debating specifics rather than vague “this seems important.”

Alternative frameworks:

Value vs. Effort matrix (simpler than RICE):

- > For each action, rate:
- > > - **Value:** User impact $\times$ frequency (High/Medium/Low)
- > > - **Effort:** Engineering complexity and time (High/Medium/Low)
- > > Plot on 2 $\times$ 2 matrix:
- > > - High Value + Low Effort = Do now
- > > - High Value + High Effort = Plan for next quarter
- > > - Low Value + Low Effort = Nice to have

> - Low Value + High Effort = Don't do

Kano model (for feature prioritization): > For each unmet need, classify: > > - **Must-have**: Absence causes dissatisfaction (pain point fixes, critical gaps) > - **Performance**: More is better (speed improvements, capacity increases) > - **Delight**: Unexpected features that wow users (novel capabilities) > > Prioritize: Must-haves first (table stakes), then Performance (competitive advantage), then Delight (differentiation)

Choose a framework, apply it systematically, document the results. Your roadmap becomes defensible because you can explain the prioritization logic.

Creating feedback-driven backlog items. Synthesis feeds your backlog. Each actionable insight becomes a ticket: bug, feature request, tech debt, design task.

Prompt: Generate Backlog Items

Based on the prioritized actions in the insight-to-action mapping, draft backlog items for the top 5 actions.

For each, create a ticket in this format:

- **Title**: [Concise action-oriented title]
- **Problem**: What user problem are we solving? (2-3 sentences referencing synthesis findings)
- **Evidence**: Supporting data from synthesis (frequency, severity, representative quotes)
- **Proposed solution**: What we're considering (high level, not detailed spec)
- **Success criteria**: How we'll know this solved the problem (measurable if possible)
- **Estimated effort**: Rough sizing (small/medium/large or T-shirt sizes)
- **Priority**: Recommendation based on prioritization framework applied
- **Related feedback**: Link to theme report section for full context

Claude Code generates draft tickets you can copy into Jira, Linear, or your backlog tool. You still need to refine (add technical details, coordinate with engineering, adjust priority based on other factors), but the research-to-backlog connection is explicit.

Example backlog item:

****Title**:** Implement async export for large datasets

****Problem**:** Users exporting large datasets (>10k records) experience timeouts and extremely slow performance, blocking their reporting workflows and causing frustration.

****Evidence**:** Q1 2026 feedback synthesis identified slow export as #1 pain point – 87 mentions across tickets and NPS responses (10.3% of all feedback). Severity assessed as high based on language ("blocking", "unusable", "considering alternatives"). Representative quote: "Tried to export 50k records and it timed out after 10 minutes. This is unusable for our reporting needs."

****Proposed solution**:** Move large exports (>5k records) to background processing. User initiates export, receives email when ready. Allows exports of any size without browser timeout constraints.

****Success criteria**:**

- Users can successfully export 100k+ record datasets
- Export requests complete within 10 minutes for datasets up to 100k records
- Reduction in "slow export" support tickets by >70%
- NPS score improvement among users who frequently export

****Estimated effort**:** Large (4–5 weeks eng + infrastructure changes)

****Priority**:** P1 – High impact, high frequency, aligns with enterprise growth strategy

****Related feedback**:** See `feedback/2026-q1/theme-report.md` section "Slow Export Performance"

This ticket connects user evidence directly to the work. Engineering understands why this matters. Leadership sees the research foundation. Stakeholders can challenge assumptions with reference to actual data.

Time and cost for insight-to-action work: Mapping themes to actions: 15-20 minutes, 25,000-40,000 tokens, \$0.13-0.20. Applying prioritization framework: 10-15 minutes, 15,000-30,000 tokens, \$0.08-0.15. Generating backlog items: 10 minutes, 15,000-25,000 tokens, \$0.08-0.13. Total: 35-45 minutes, \$0.30-0.50.

When this approach fails: If your product strategy is highly opinionated (founder-driven vision, design-led innovation), feedback synthesis informs but doesn't dictate. Not every customer request aligns with where you're taking the product. Use synthesis to understand the impact of strategic choices, not to let customers design your product.

If feedback is contradictory and half of users want X while half want the opposite, synthesis reveals the conflict but doesn't resolve it. This is a segmentation issue. Either serve both segments differently or choose which segment you're building for.

If your backlog is already overloaded and unchangeable (committed roadmap, regulated industry, technical debt dominates), synthesis becomes documentation of what you can't do rather than what you will do. This is still valuable for understanding opportunity cost, but don't expect it to reshape priorities.

You can now systematically synthesize customer feedback and UX research into actionable product insights. You've turned 847 support tickets and 15 interview transcripts from overwhelming noise into structured themes with supporting evidence, priority scores, and backlog items. This capability shifts your PM workflow from reactive firefighting to strategic research-driven planning.

Chapter 7 applies Claude Code to requirements documentation: creating PRDs, user stories, and personas that live in your repository and stay synchronized with implementation.

7 Requirements, Personas, and Documentation

7.1 Creating Personas from Scattered Research

Your stakeholders want user personas before roadmap planning. You have scattered research: customer interviews from last quarter, analytics showing behavior patterns, support tickets revealing pain points, sales calls with notes about buyer concerns. Building personas means synthesizing this into coherent user archetypes. Traditional approach: read everything, identify patterns manually, write up personas in a deck that lives in Google Slides and gets referenced once then forgotten.

Claude Code creates personas as versionable artifacts that live in your repository. You feed it your research data, specify what makes personas useful for your product context, and get structured persona documents that integrate with your workflow. When engineering asks “who is this feature for?” you point to `docs/personas/enterprise-admin.md` in the repo. When personas evolve based on new research, you update the file and commit changes. The persona isn’t a static slide. It’s living documentation.

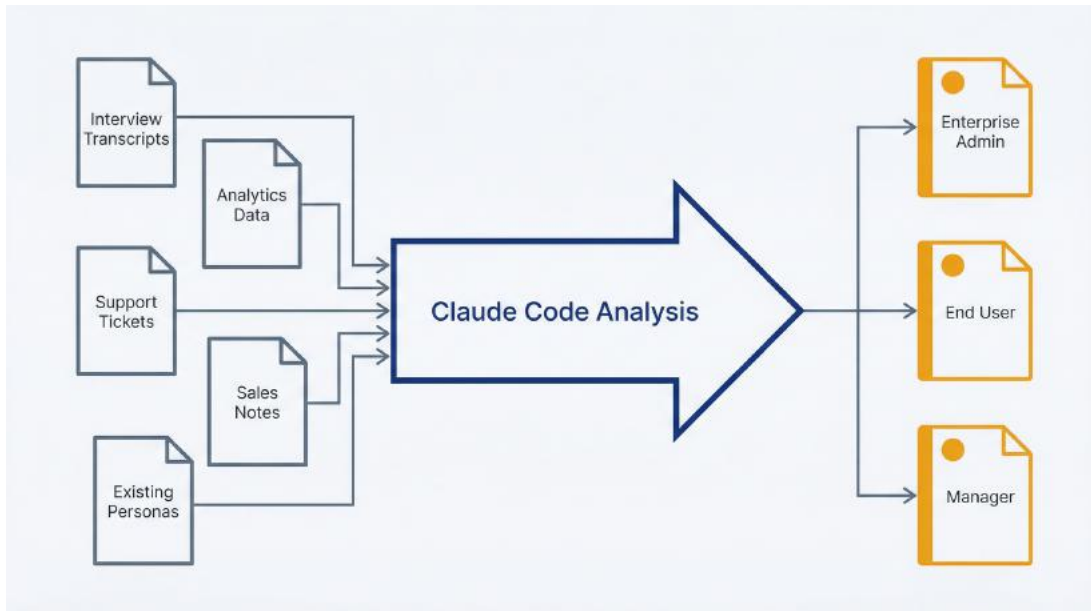


Figure 7.1: Persona Generation Workflow – Use this to transform scattered research into structured, versionable persona documents. Research sources (interviews, analytics, support data) flow through Claude Code analysis to produce persona files that live in your repository and evolve with your product.

Inputs: Research data, analytics, existing knowledge. Gather what you know about your users before generating personas. You don't need comprehensive research. You need enough signal to create useful archetypes.

Research sources to collect: - **Interview transcripts:** User research sessions, customer interviews, sales discovery calls - **Analytics data:** User behavior patterns, feature adoption rates, workflow sequences - **Support data:** Common questions, pain points, user segments with different issues - **Sales intelligence:** Buyer personas, decision criteria, objections - **Existing knowledge:** What you and your team already know about user types

Prepare this data like you would for feedback synthesis (Chapter 6). Export to files, remove PII, organize in a directory:

```
mkdir -p research/persona-development
```

Place source materials:

```
research/persona-development/  
├─ interviews-enterprise-users.txt  
├─ analytics-summary.md  
├─ support-ticket-themes.md  
├─ sales-notes.txt  
└─ existing-personas-critique.md
```

If you have old personas, include them with notes on what's outdated or wrong. Claude Code can use this as a starting point to improve.

Prompt template: Persona generation. Launch Claude Code:

```
claude --permission-mode plan
```

Start with analysis, then generate personas in a second session with write access.

Prompt: Persona Analysis

Review the research materials in `research/persona-development/` and identify distinct user types based on:

- Goals and motivations: What are they trying to accomplish?
- Behaviors: How do they use the product or similar tools?
- Pain points: What frustrates them or blocks their work?
- Context: Company size, role, technical skill, decision authority

How many distinct personas emerge from this data? For each potential persona:

- What defines this user type? (2-3 key characteristics)
- How large is this segment? (percentage of users or rough estimate)
- What evidence supports this as a distinct persona? (cite sources)

Recommend 3-5 personas that cover our primary user types without over-segmenting.

Claude Code reads your research, identifies patterns, and proposes personas. You get something like: “Three distinct personas emerge: (1) Enterprise Admin, a technical buyer concerned with security and scalability, represents 15% of users but drives purchasing decisions.

Evidence: 8 of 12 sales calls focused on enterprise requirements. (2) End User, an individual contributor focused on ease of use and daily workflow efficiency, represents 70% of users. Evidence: Support tickets and interviews primarily from this segment. (3) Manager, who oversees team usage and cares about reporting and visibility, represents 15% of users. Evidence: Feature requests and analytics show distinct usage patterns for dashboard/reporting features.”

Review the proposed personas. Do they match your intuition? Are there missing segments? Overlaps that should be merged? Refine the analysis if needed with follow-up prompts.

Output format: Structured persona document. Once you’ve validated the persona set, generate detailed persona documents:

Exit plan mode and launch with write access:

```
/exit  
claude
```

Prompt: Persona Document Generation

Based on the persona analysis, create detailed persona documents in docs/personas/. Generate one markdown file per persona: [persona-name].md.

Structure each persona document:

1. Overview

- Name and role (fictional but representative)
- One-sentence summary: Who they are and what they care about
- Photo placeholder: [Description, no actual image needed]

2. Demographics

- Role/title
- Company size
- Industry (if relevant)
- Technical skill level

- Decision authority (buyer, influencer, end user)

3. Goals and Motivations

- Primary goals (what they're trying to achieve)
- Success criteria (how they measure success)
- Motivations (what drives their behavior)

4. Pain Points and Frustrations

- Current challenges (before using our product)
- Frustrations with existing solutions
- Workflow blockers

5. Behaviors and Preferences

- How they work (workflows, tools, processes)
- Product usage patterns
- Communication preferences
- Decision-making approach

6. Needs from Our Product

- Must-have features (deal-breakers)
- Important features (high value)
- Nice-to-have features (lower priority)

7. Objections and Concerns

- What makes them hesitant to adopt?
- What questions do they ask during evaluation?
- What might cause them to churn?

8. Supporting Evidence

- Key quotes from research (2-3 representative quotes)
- Data points that validate this persona
- Source references

Keep each persona document under 2 pages. Focus on actionable

insight over demographic detail.

Claude Code generates persona files in docs/personas/. Each persona becomes a reference document you can link from PRDs, roadmap discussions, and feature planning.

Example output structure:

Enterprise Admin Persona: Sarah Chen

****Summary****: IT administrator or security lead who evaluates and purchases tools for their organization. Cares about security, compliance, and scalability more than ease of use.

Demographics

- ****Role****: IT Administrator, Security Lead, or VP of Engineering
- ****Company size****: 500–5000 employees
- ****Technical skill****: High, comfortable with APIs, SSO, infrastructure
- ****Decision authority****: Primary buyer or strong influencer

Goals and Motivations

- Ensure tools meet security and compliance requirements
- Minimize risk of data breaches or regulatory violations
- Evaluate total cost of ownership including support and maintenance
- Standardize tooling across organization to reduce complexity

Pain Points and Frustrations

- Tools that lack enterprise features (SSO, audit logs, role-based access)
- Poor security documentation that delays evaluation
- Hidden costs that appear after purchase (seat limits, API restrictions)
- Vendor lock-in concerns with proprietary data formats

Behaviors and Preferences

- Runs proof-of-concept trials before purchasing
- Requests security audits, SOC 2 reports, penetration test results
- Prefers self-service implementation over hand-holding

- Values responsive support for technical issues

Needs from Our Product

Must-have:

- SSO/SAML integration
- Audit logging and activity monitoring
- Role-based access control with granular permissions
- SOC 2 Type II compliance

Important:

- API access for integrations and automation
- Data export capabilities (avoid vendor lock-in)
- High availability SLAs
- Priority support

Nice-to-have:

- Advanced security features (IP whitelisting, session management)
- Dedicated account manager
- Custom training for team

Objections and Concerns

- "How do you handle data encryption at rest and in transit?"
- "What happens if we need to migrate away?"
- "Can we run this on-premise or in our VPC?"
- "What's your security incident response process?"

Supporting Evidence

Quotes:

- "We need full audit logs. If we can't prove who accessed what data and when, it's a non-starter." (Interview, Enterprise_Customer_12)
- "SSO is table stakes. We're not managing another set of credentials." (Sales call, Q4 2025)

Data:

- 85% of enterprise deals stall on security review (sales pipeline data)
- Enterprise admin personas represent 15% of users but 60% of revenue
- Security-related feature requests cluster in this segment (support ticket analysis)

Validating personas against data. Don't trust personas just because AI generated them. Validate against what you know:

Prompt: Persona Validation

Review the generated personas and validate against the source research:

- Are the characteristics supported by actual data or are they assumptions?
- Do the pain points match what users actually said in interviews and tickets?
- Are the segment sizes realistic given our analytics?
- Are there contradictions between personas and actual user behavior?

Flag any elements that seem speculative or poorly supported.

This catches instances where Claude Code filled gaps with plausible-sounding details that aren't grounded in your data. If the Enterprise Admin persona lists "prefers email over Slack" but you have no evidence for communication preferences, remove it or mark it as assumption to validate.

Commit personas to your repository:

```
git add docs/personas/  
git commit -m "Add user personas based on Q4 2025 research"
```

Time and cost for persona development. Analyzing research and identifying persona types: 15-20 minutes, 30,000-50,000 tokens, \$0.15-0.25. Generating 3-4 detailed persona documents: 20-30 minutes, 40,000-70,000 tokens, \$0.20-0.35. Total: 35-50 minutes, \$0.35-0.60. Compare to days of manual synthesis and formatting.

When persona generation fails. If your research is thin (few

interviews, limited data, weak signals), Claude Code produces generic personas that could describe any product's users. "Sarah wants tools that are easy to use and make her productive" isn't actionable insight. This is a data problem, not a tool problem. Gather more research before generating personas.

If your user base is extremely diverse with no clear segments, where every user wants something different, personas don't help. This suggests product-market fit issues or a feature-factory approach where you're trying to serve everyone. Segment by use case or vertical rather than user type.

How often to update personas. Annually for stable markets. Quarterly for fast-changing products. When entering new market segments or when research reveals user needs have shifted significantly. Treat personas as living documentation; version control shows evolution over time.

7.2 Generating Comprehensive Stories from Feature Concepts

You have a feature concept. Stakeholders agreed it's worth building. Now you need user stories for engineering: structured descriptions of what users need, why, and how you'll know it works. Traditional approach: write stories manually, schedule a refinement meeting, spend an hour with engineering clarifying ambiguities, discover missed edge cases during implementation.

Claude Code generates comprehensive user stories from feature concepts. You provide context about the feature and who it serves. Claude Code expands this into stories with acceptance criteria, edge cases, and technical considerations drawn from understanding your codebase. The stories aren't final (you'll still refine with engineering), but they're 80% complete rather than starting from scratch.

From feature concept to user stories. Start with what you know:

- What feature are you building?

- Who is it for? (reference persona if you have one)
- What problem does it solve?
- What are the key workflows it needs to support?

Launch Claude Code in plan mode first to explore your codebase for context:

```
claude --permission-mode plan
```

Prompt: Feature Context Analysis

I'm planning a feature: [brief description of feature]

Before writing user stories, help me understand technical context:

- What existing features or code areas does this relate to?
- What data will this feature need to access?
- Are there technical constraints I should know about? (authentication, permissions, integrations, performance considerations)
- What edge cases exist in similar features?

This context will inform user story creation.

For example, planning a “bulk user import” feature:

I'm planning a bulk user import feature that lets admins upload a CSV of users to create accounts automatically.

Before writing user stories, help me understand technical context:

- How does user creation currently work?
- What validation happens when creating individual users?
- Are there rate limits or constraints on account creation?
- How do we handle duplicate email addresses?

Claude Code investigates the codebase and responds: “User creation happens in `src/services/user-service.js:78-156`. Current validation: email format, unique email in database, password strength requirements, role must be valid. Duplicate emails are rejected with error. No rate

limiting on user creation currently, which could be a problem for bulk import. Similar batch operations (bulk project creation) live in `src/services/batch-operations.js` with progress tracking and error reporting.”

Now you know what constraints exist, what patterns to follow, and what edge cases matter.

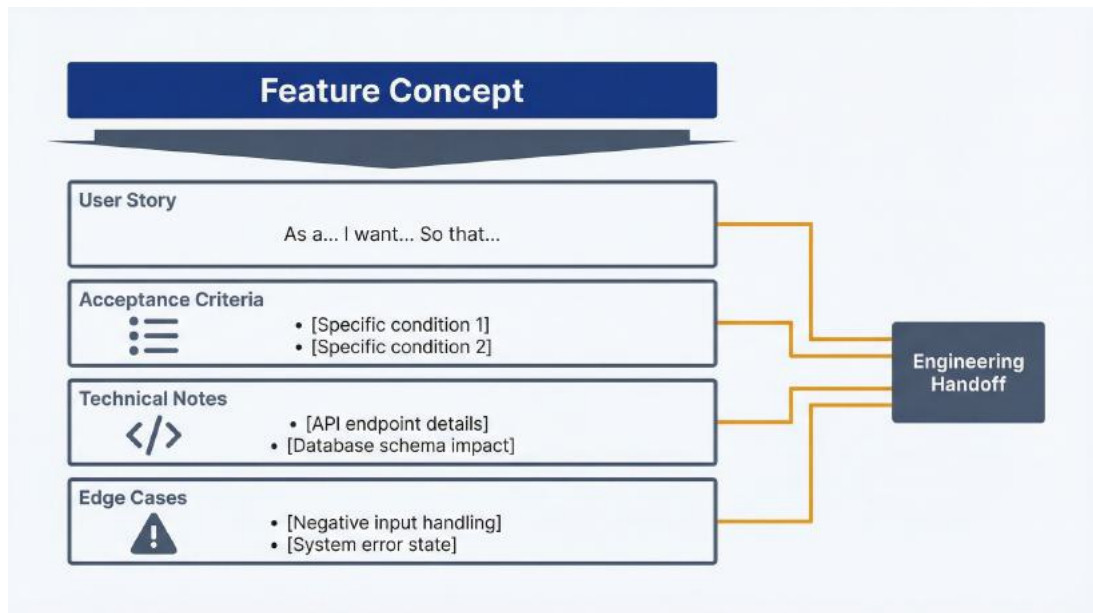


Figure 7.2: User Story Anatomy – Use this to understand how feature concepts expand into comprehensive stories. Shows the structure: user story format (As a / I want / So that), acceptance criteria (testable requirements), technical notes (codebase references, patterns to follow), and edge cases (failure modes, boundary conditions).

Prompt template: User story expansion. Exit plan mode and launch with write access:

```
/exit  
claude
```

Prompt: User Story Generation

Generate user stories for the [feature name] feature in `docs/user-stories/[feature-name].md`.

Feature description: [2-3 sentences describing the feature]

Target persona: [Persona name or description, e.g., “Enterprise Admin” or link to docs/personas/enterprise-admin.md]

Core workflows to support: 1. [Primary workflow] 2. [Secondary workflow] 3. [Error/edge case workflow]

Technical context: [Summary from your exploration: existing patterns, constraints, related features]

Generate user stories in this format:

User Story: As a [persona], I want to [action], So that [outcome/benefit].

Acceptance Criteria: - [Specific, testable criterion 1] - [Specific, testable criterion 2] - [Edge case or error handling criterion]

Technical Notes: - [Implementation considerations from codebase context] - [Related files or patterns to follow] - [Constraints or dependencies]

Edge Cases: - [Scenario that could go wrong] - [Boundary condition] - [Error state]

Create 5-8 stories that cover core functionality, variations, and edge cases. Prioritize by user value.

Claude Code generates a structured document with user stories, acceptance criteria, and technical context.

Example output for bulk user import:

```
# User Stories: Bulk User Import
```

```
## Story 1: Upload CSV to Create Multiple Users
```

```
**As an** Enterprise Admin,  
**I want to** upload a CSV file with user information,  
**So that** I can onboard multiple team members at once instead of  
creating accounts individually.
```

****Acceptance Criteria:****

- Admin can upload CSV file via file picker in admin dashboard
- CSV format is documented with required columns: email, name, role
- System validates CSV format before processing (rejects invalid files with clear error)
- Success message shows number of users to be created before confirmation
- Admin can review first 5 rows as preview before confirming

****Technical Notes:****

- Follow pattern from `src/services/batch-operations.js` for batch processing
- Reuse validation logic from `src/services/user-service.js:78-156`
- Consider file size limits (current upload limit is 10MB in config)
- Store uploaded file temporarily for processing, clean up after

****Edge Cases:****

- If CSV file exceeds size limit, reject with error message
- If CSV has malformed rows, report which rows failed validation
- If CSV contains 1000+ rows, consider async processing to avoid timeout

Story 2: Validate User Data Before Import

****As an**** Enterprise Admin,
****I want to**** see validation errors before users are created,
****So that**** I can fix issues in my CSV rather than partially importing bad data.

****Acceptance Criteria:****

- System validates all rows before creating any users
- Validation checks: email format, unique email, valid role, required fields present
- Validation errors list row numbers and specific issues
- Admin can download error report CSV showing failed rows
- No users are created if any validation fails (all-or-nothing)

****Technical Notes:****

- Extend existing validation from `src/services/user-service.js`
- Check for duplicate emails both against database AND within the CSV
- Role validation should match allowed roles from `src/models/user.js:12-16`

****Edge Cases:****

- If duplicate email exists in database, report as validation error
- If multiple rows have same email in CSV, report as validation error
- If role name has typo, suggest closest valid role if possible
- If required field is missing, specify which column and row

Story 3: Track Import Progress and Status

****As an**** Enterprise Admin,
****I want to**** see progress while users are being created,
****So that**** I know the import is working and don't navigate away prematurely.

****Acceptance Criteria:****

- Progress indicator shows number of users created vs. total
- Process doesn't block other admin dashboard usage
- Admin can navigate away and return; progress persists
- Completion notification appears when import finishes
- Summary shows success count and any errors

****Technical Notes:****

- Use async batch processing pattern from `src/services/batch-operations.js`
- Store import job status in database with progress tracking
- Consider websocket for real-time progress updates or polling approach
- Clean up completed jobs after 24 hours

****Edge Cases:****

- If user navigates away mid-import, import continues in background
- If import fails partway through, show partial success state and report which users were created
- If database connection lost during import, retry failed users or fail entire batch?

Story 4: Handle Import Failures Gracefully

****As an**** Enterprise Admin,
****I want to**** understand what went wrong if import fails,
****So that**** I can correct the issue and retry.

****Acceptance Criteria:****

- Error messages specify root cause (database error, validation, timeout)
- Partial failures show which users were created successfully
- Admin can download failed rows as CSV to fix and re-import
- Failed import doesn't leave system in inconsistent state
- Retry mechanism available for failed imports

****Technical Notes:****

- Wrap user creation in database transaction if possible for atomicity
- Log detailed errors to `logs/import-errors.log` for debugging
- Reference existing error handling in `src/services/user-service.js:142-156`

****Edge Cases:****

- If database transaction too large, split into smaller batches
- If rate limiting triggered by rapid user creation, implement throttling
- If duplicate users created from retry after partial success, add idempotency check

Story 5: Send Welcome Emails to Imported Users

****As an**** imported user,
****I want to**** receive a welcome email with account setup instructions,
****So that**** I know my account was created and can log in.

****Acceptance Criteria:****

- Welcome email sent to each successfully created user
- Email includes temporary password or password reset link
- Email sending happens after all users created (don't spam if import

fails)

- Failed email sends are logged but don't fail the import
- Admin can optionally suppress welcome emails for import

****Technical Notes:****

- Use existing email service from `src/services/email-notifications.js`
- Consider queue for sending emails to avoid blocking import completion
- Follow email template pattern from `src/templates/email/welcome.html`

****Edge Cases:****

- If email service down, queue emails for retry; don't fail import
- If some emails bounce, log bounces so admin can resend manually
- If 500+ users imported, batch email sending to avoid rate limits

[Additional stories for role assignment variations, audit logging, CSV template download...]

Acceptance criteria generation. Acceptance criteria translate stories into testable requirements. Good criteria are specific, measurable, and testable. Bad criteria are vague: “Users can import accounts easily.” Good criteria: “Admin can upload CSV file via file picker in admin dashboard” and “Success message shows number of users to be created before confirmation.”

Claude Code generates criteria based on the feature description and codebase patterns. Review and refine: remove ambiguities, add missing cases, ensure testability.

Edge case identification. The value of Claude Code-generated stories is spotting edge cases you might miss. It knows your codebase constraints (file upload limits, validation patterns, error handling approaches) and suggests edge cases based on what could fail:

- What if the file is too large?
- What if duplicate emails exist?
- What if the database connection fails mid-import?
- What if email sending fails?

Review the edge cases. Some might be over-cautious. Some might reveal

actual risks you hadn't considered. Use judgment to prioritize which edge cases warrant handling in the initial implementation vs. future iterations.

Referencing codebase for technical constraints. The technical notes sections connect stories to implementation. "Follow pattern from batch-operations.js" tells engineering where to look for existing solutions. "Reuse validation from user-service.js" prevents duplicating logic. "Current upload limit is 10MB" sets realistic expectations.

This bridges product and engineering. Stories aren't abstract requirements. They're grounded in how your system actually works.

Commit user stories:

```
git add docs/user-stories/  
git commit -m "Add user stories for bulk user import feature"
```

Time and cost. Exploring codebase for technical context: 10-15 minutes, 20,000-40,000 tokens, \$0.10-0.20. Generating comprehensive user story document (5-8 stories): 15-25 minutes, 30,000-60,000 tokens, \$0.15-0.30. Total: 25-40 minutes, \$0.25-0.50.

Refinement with engineering. Generated stories are a starting point, not a finished spec. Review with engineering:

- Are technical notes accurate?
- Are edge cases realistic or over-engineered?
- Are acceptance criteria testable?
- What's missing?

This refinement takes 30-60 minutes but starts from solid draft rather than blank page. Engineering spends time improving stories, not writing them from scratch.

7.3 Writing PRDs That Stay Current

Product requirements documents live in limbo: too important to skip, too tedious to maintain. You write a PRD in Google Docs, share it for feedback, finalize it, then never update it as requirements change during

implementation. Engineering builds something, the PRD becomes outdated, and future readers don't know whether the doc or the code represents truth.

The solution isn't better discipline. It's better placement. PRDs should live in your repository as markdown files, version-controlled like code. Claude Code helps generate PRD sections, maintain consistency with existing documentation, and keep requirements synchronized with implementation.

PRD as living artifact in repo. Create a docs structure:

```
mkdir -p docs/prds
```

PRDs become markdown files: docs/prds/bulk-user-import.md, docs/prds/api-rate-limiting.md, docs/prds/sso-integration.md. When requirements change, you update the file and commit. Git tracks evolution. Engineering references the file directly, with no Google Doc links that break or require permission requests.

Prompt template: PRD section generation. PRDs have standard sections. Claude Code generates them based on your research, user stories, and codebase context.

Launch Claude Code:

```
claude
```

Prompt: PRD Generation

Create a PRD for [feature name] in docs/prds/[feature-name].md.

Context: - Feature description: [2-3 sentences] - Target users: [personas or user types] - User stories: See docs/user-stories/[feature-name].md (if exists) - Related research: [link to research docs if applicable]

Generate PRD with these sections:

1. Overview - One-paragraph summary: What is this feature and

why are we building it? - Success metrics: How will we measure success?

2. User Personas and Use Cases - Who is this for? (reference persona docs if they exist) - Primary use cases (3-5 scenarios)

3. Requirements

Functional Requirements: - What the feature must do (prioritized: Must-have, Should-have, Nice-to-have)

Non-Functional Requirements: - Performance, security, scalability, accessibility considerations

4. User Experience - Key user workflows (step-by-step) - UI/UX considerations (screens, flows, interactions)

5. Technical Approach - High-level technical design - Integration points with existing code (reference specific files/areas) - Data model changes (if applicable) - Third-party dependencies

6. Out of Scope - What we're explicitly NOT building (prevents scope creep)

7. Open Questions - Unresolved decisions - Areas needing more research

8. Success Criteria - Launch criteria: When is this ready to ship? - Success metrics: What does good look like post-launch?

9. Timeline and Milestones (placeholder for PM to fill)

10. Appendix - Links to user stories, research, designs, related docs

For the Technical Approach section, analyze the codebase to identify:

- Existing patterns we should follow
- Files or modules that will need changes
- Potential technical risks or complexity

Keep the PRD under 6 pages. Focus on clarity and decision-making, not exhaustive detail.

Claude Code generates a structured PRD. The technical approach section benefits from codebase context: it references actual files, existing patterns, and implementation constraints rather than generic architectural hand-waving.

Example PRD excerpt for bulk user import:

PRD: Bulk User Import

Overview

Enable enterprise administrators to create multiple user accounts simultaneously by uploading a CSV file, reducing onboarding time for large teams from hours to minutes.

Success metrics:

- 50% of enterprise customers use bulk import within first month of availability
- Average time to onboard 50+ users decreases from 2 hours to under 10 minutes
- Import success rate >95% (fewer than 5% of imports fail due to validation or errors)

User Personas and Use Cases

Primary persona: Enterprise Admin (see `docs/personas/enterprise-admin.md`)

Use cases:

1. New customer onboarding, provisioning accounts for entire team at once
2. Seasonal hiring, adding interns or contractors in batches
3. Department expansion, creating accounts for newly acquired team members
4. Migration from competitor, importing existing user lists

Requirements

Functional Requirements

****Must-have:****

- Upload CSV file with user details (email, name, role)
- Validate CSV format and data before creating users
- Show validation errors with specific row numbers and issues
- Create all users atomically (all succeed or all fail)
- Send welcome emails to imported users
- Display import progress and completion status
- Download error report for failed imports

****Should-have:****

- CSV template download with example data
- Preview first 5 rows before confirming import
- Audit log of import activity (who imported, when, how many users)
- Option to suppress welcome emails

****Nice-to-have:****

- Support additional user fields (phone, department, manager)
- Schedule imports for future date
- Import existing users to update roles or details

Non-Functional Requirements

- ****Performance:**** Handle CSV files up to 1,000 users without timeout (< 2 minutes processing)
- ****Security:**** Validate admin permissions before allowing import, log all import activity
- ****Reliability:**** Transaction-based import to prevent partial failures
- ****Usability:**** Clear error messages that specify how to fix issues

User Experience

****Primary workflow:****

1. Admin navigates to Users > Import Users
2. Downloads CSV template (optional) to see required format
3. Prepares CSV file with user data
4. Clicks "Upload CSV" and selects file

5. System validates file and shows preview of first 5 users
6. Admin reviews and confirms import
7. Progress indicator shows creation status
8. Completion message displays results (success count, errors)
9. Admin downloads error report if any validations failed
10. Admin can fix errors in CSV and retry

****Error state workflow:****

1. Admin uploads CSV with validation errors
2. System displays error count and specific issues
3. Admin downloads error report showing failed rows
4. Admin fixes CSV and re-uploads
5. System validates again until successful

Technical Approach

****Architecture:****

- Frontend: Add import UI to admin dashboard (``src/client/components/Admin/UserImport.tsx``)
- Backend: Create import API endpoint (``src/server/routes/users.js``)
- Service layer: Implement batch processing (``src/services/user-import-service.js``)
- Follow existing batch operation pattern from ``src/services/batch-operations.js``

****Key technical decisions:****

1. ****CSV parsing:**** Use ``csv-parser`` library (already in dependencies)
2. ****Validation:**** Extend existing user validation from ``src/services/user-service.js:78-156``
3. ****Batch processing:**** Process in batches of 50 users to avoid database connection limits
4. ****Progress tracking:**** Store job status in ``user_import_jobs`` table, poll for progress
5. ****Email sending:**** Queue emails using existing ``src/services/email-queue.js`` to avoid rate limits

****Integration points:****

- ``src/services/user-service.js`` - Reuse `createUser` logic

- `src/services/email-notifications.js` - Send welcome emails
- `src/models/user.js` - User model and validation
- `src/middleware/auth.js` - Verify admin permissions

****Data model changes:****

- New table: `user\_import\_jobs` (id, admin_id, status, total_count, success_count, error_report, created_at)
- Columns: id, admin_user_id, filename, status (pending/processing/completed/failed), total_rows, successful_rows, error_details (JSON), created_at, updated_at

****Technical risks:****

- Database transaction size limit for 1,000 users: mitigate with batching
- Email rate limiting: mitigate with queue and throttling
- Large CSV file upload timeout: set appropriate limits and show progress

Out of Scope

- Importing into multiple workspaces simultaneously (single workspace per import)
- Updating existing user details via CSV (create-only for v1)
- Importing group/team assignments (future enhancement)
- Scheduled/recurring imports
- Integration with HRIS systems for automated sync

Open Questions

- Should partial success be allowed (some users created, some failed) or all-or-nothing? ****Decision needed from engineering re: transaction feasibility****
- What file size limit is reasonable? Current upload limit is 10MB; need to test how many users that represents
- Should we support Excel files (.xlsx) or CSV only? CSV is simpler, covers 95% of use cases

Success Criteria

****Launch criteria:****

- Can successfully import 500 users in under 2 minutes
- Validation catches all invalid data before creating users
- Error reporting clearly identifies issues
- Documentation includes CSV template and format requirements
- Tested with real customer data (anonymized)

****Post-launch success:****

- 50% of enterprise customers adopt within 30 days
- Support tickets about user creation decrease by 30%
- Average onboarding time for 50+ user teams under 10 minutes

Timeline and Milestones

[PM fills in sprint allocation, dependencies, launch date]

Appendix

- User stories: `docs/user-stories/bulk-user-import.md`
- Related research: `research/feedback/enterprise-feature-requests.md`
- Competitive analysis: `research/competitors/competitor-a-profile.md` (has bulk import feature)
- Design mocks: [Figma link when available]

Maintaining consistency with existing documentation. Before generating a PRD, ask Claude Code to check for related documentation:

Are there existing PRDs, user stories, or technical docs related to [feature area]? Show me what exists and how this new PRD should reference or align with existing docs.

This prevents contradictions. Maybe another PRD defines user roles differently, or establishes a pattern you should follow. Claude Code can ensure consistency: “The authentication PRD defines admin roles in section 3.2. Reference that definition rather than redefining here.”

Version control for requirements documents. Commit the PRD:

```
git add docs/prds/bulk-user-import.md
git commit -m "Add PRD for bulk user import feature"
```


As requirements evolve during implementation, update the PRD and commit changes:

```
git commit -am "Update bulk import PRD: reduce scope to 500 user limit"
```

Git history shows what changed and why. PRDs become living documentation that reflects actual decisions, not initial guesses.

Time and cost. Generating comprehensive PRD (6-8 pages): 25-35 minutes, 50,000-80,000 tokens, \$0.25-0.40. Less for smaller features, more for complex multi-component features.

When PRDs add value. Use PRDs for: - Features requiring cross-team alignment (engineering, design, marketing) - Complex features with multiple integration points - Features with compliance or security implications - Anything you'll reference during implementation and post-launch

Skip PRDs for: - Small bug fixes - Minor UI tweaks - Experimental features you're prototyping - Internal tools with single stakeholder

Don't create documentation overhead where conversation suffices.

7.4 Keeping Product Docs in Sync with Implementation

Product managers traditionally treat documentation as separate from code: PRDs in Google Docs, release notes in Confluence, product specs in Notion. Engineering treats docs-as-code: README files in repos, API docs generated from code comments, changelogs maintained in version control. This split creates drift. Product docs describe what should exist; code represents what does exist.

Docs-as-code for PMs means putting product documentation in the repository alongside code. README files, changelogs, API documentation, and product guides live as markdown files, version-controlled and updated as part of normal workflow. Claude Code makes this practical by helping you write and maintain these docs without

needing deep technical knowledge.

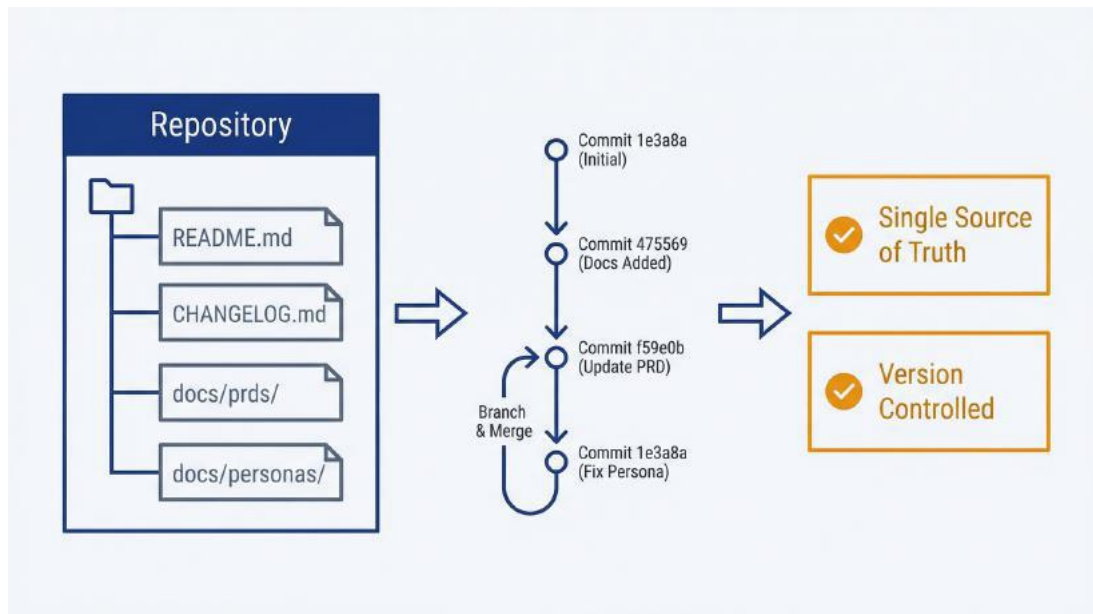


Figure 7.4: Docs-as-Code Trigger Loop – Use this to keep product documentation automatically synchronized with code changes. Shows a three-stage cycle: Engineering merges code (trigger), Claude Code detects change and updates documentation (action), PM reviews and approves (approval). The loop repeats continuously, keeping PRDs, user guides, changelogs, and API docs always in sync with implementation.

Docs-as-code philosophy for PMs. The core principle: documentation that describes the product should live with the product. This doesn't mean PMs write code comments. It means the product knowledge you own exists as files in the repo:

- README.md - What this product is, how to get started
- CHANGELOG.md - What changed in each release
- docs/ - Product documentation, user guides, how-tos
- docs/api/ - API documentation (you collaborate, engineering owns implementation)
- docs/prds/ - Requirements documents
- docs/personas/ - User personas

Benefits of this approach:

Single source of truth. The repo contains everything. No hunting

across Google Drive, Confluence, and Notion for scattered docs.

Version control. Documentation evolves with the product. Git history shows when things changed and why.

Proximity to code. Engineers see product docs while working. PMs see technical docs while planning. Cross-functional context increases.

Automated workflows. Pull requests can include both code changes and doc updates. CI can check that docs are updated when features change.

README updates and changelog maintenance. The README is often the first thing users see. Keep it current with Claude Code's help:

Prompt: README Update

Review and update README.md to reflect recent changes to the product.

Current state: [Brief summary of what changed: new features, removed features, renamed concepts]

Update sections:

- **Overview:** Ensure feature list is current
- **Getting Started:** Verify installation/setup steps still accurate
- **Key Features:** Add recent additions, remove deprecated features
- **Configuration:** Update if new config options exist

Keep tone clear and welcoming for new users. Remove outdated information.

Changelogs document what changed in each release. Maintaining them manually is tedious. Claude Code can generate changelog entries from git history:

Prompt: Changelog Generation

Generate a changelog entry for version [X.Y.Z] by reviewing git commits since [last version tag].

Format:

[X.Y.Z] - YYYY-MM-DD

Added

- [New features]

Changed

- [Changes to existing features]

Fixed

- [Bug fixes]

Removed

- [Deprecated or removed features]

Focus on user-facing changes. Ignore internal refactoring or code cleanup unless it affects users. Use product language, not technical jargon.

Claude Code reads commit messages, identifies user-facing changes, and produces a structured changelog entry. You review and edit for accuracy and tone, then commit.

API documentation contributions. If your product has an API, documentation matters. Engineering typically owns implementation, but PMs contribute context: what endpoints are for, what use cases they serve, common integration patterns.

Claude Code can help write or improve API documentation:

Prompt: API Documentation Enhancement

Review the API documentation in `docs/api/users.md` for the user management endpoints.

Enhance with:

- **Use case context:** When would a developer use this endpoint?
- **Common patterns:** How is this typically used in integration workflows?

- **Error handling guidance:** What errors might occur and how to handle them?
- **Examples:** Real-world request/response examples with explanations

Keep technical accuracy—reference the actual API implementation in `src/server/routes/users.js` to verify endpoint behavior.

You're adding product context to technical docs. Engineering ensures accuracy of request/response schemas; you ensure developers understand why and when to use the API.

Keeping product docs synchronized with implementation. The risk of docs-as-code is the same as any documentation: it gets outdated. Mitigate this with process:

Review docs during sprint planning. When planning features that affect user-facing behavior, include “update docs” as acceptance criteria in user stories.

Link PRs to doc updates. Engineering PRs that add features should include README or changelog updates. Make this a review checklist item.

Periodic doc audits. Quarterly, ask Claude Code to audit documentation against implementation:

Prompt: Documentation Audit

Review the documentation in `docs/` and compare to the current codebase.

Identify:

- Features documented that no longer exist (code was removed)
- Features that exist but aren't documented
- Configuration options in docs that don't match actual config files
- Examples or screenshots that are outdated

Create a list of doc updates needed to bring documentation in sync

with implementation.

Claude Code reads docs and code, identifies drift, and produces a todo list. You prioritize and fix the most important gaps.

Example workflow: Updating docs for feature launch. You're launching the bulk user import feature. Before release, ensure docs are ready:

1. **Update README:** Add "Bulk User Import" to feature list
2. **Update changelog:** Add entry for version that includes this feature
3. **Create user guide:** Write docs/guides/bulk-user-import.md with step-by-step instructions
4. **Update API docs:** If import uses API, document the endpoint in docs/api/
5. **Update admin guide:** Add section on user management with import

Claude Code helps with each:

Generate a user guide in docs/guides/bulk-user-import.md for the bulk user import feature.

Include:

- What bulk import is and when to use it
- Step-by-step instructions with screenshots placeholders:
[Screenshot: Upload CSV button]
- CSV format requirements and template download link
- Troubleshooting common issues (validation errors, file format problems)
- FAQ section

Tone: Clear and helpful. Assume reader is admin with moderate technical skill.

Review the generated guide, add actual screenshots, test the instructions yourself to ensure accuracy, then commit.

Time and cost. README update: 5-10 minutes, 10,000-20,000 tokens,

\$0.05-0.10. Changelog generation from git history: 10-15 minutes, 15,000-30,000 tokens, \$0.08-0.15. User guide creation: 20-30 minutes, 30,000-60,000 tokens, \$0.15-0.30. Documentation audit: 15-25 minutes, 25,000-50,000 tokens, \$0.13-0.25.

When docs-as-code doesn't fit. Some documentation belongs outside the repo:

- Marketing content (product pages, landing pages) lives in CMS or marketing tools
- Support articles for end users live in help desk software (Zendesk, Intercom) for search and analytics
- Sales collateral lives in sales enablement tools
- Long-form content like this book works better in dedicated authoring tools

Docs-as-code works for: - Technical documentation (setup, API, architecture) - Developer-facing content - Product knowledge that changes with code (features, configuration, workflows) - Internal documentation for team use

Choose placement based on audience and update frequency. Docs that change with code belong in the repo. Docs that change independently belong in dedicated tools.

You can now use Claude Code to generate personas from research data, create comprehensive user stories with acceptance criteria and edge cases, produce PRDs that live as version-controlled artifacts, and maintain product documentation synchronized with implementation. These capabilities shift documentation from a chore you avoid to integrated artifacts you create as part of your normal workflow.

Chapter 8 shifts from individual documents to repeatable workflows—building skills that encode your PM processes for consistent execution across time.

8 Anatomy of a Skill—Structure and Design Patterns

8.1 From Ad-Hoc Prompts to Repeatable Workflows

You've synthesized customer feedback five times this quarter. Each time, you opened Claude Code, typed a variation of the same prompt, waited for results, realized you forgot to specify the output format, iterated, got something usable, and moved on. Next month, you'll do it again. You won't remember exactly what worked. You'll spend 20 minutes rediscovering the right prompt structure. The output will be slightly different from last time, making comparison across quarters harder.

This is the prompt treadmill. You do the same work repeatedly without compounding value from repetition. Skills solve this problem.

A skill is a documented workflow that Claude Code executes on demand. You define the process once: what inputs it needs, what steps to follow, what output to produce, and what quality looks like. Then you invoke it by name. Claude Code follows your documented workflow, producing consistent results regardless of how much time has passed since you created it.

The compounding value is significant. The first quarterly feedback synthesis takes 45 minutes including prompt iteration. With a skill, the second one takes 10 minutes. The third takes 10 minutes. By the fourth quarter, you've saved two hours of work and gained consistency across time periods that makes trend analysis possible.

This chapter covers skill structure and design patterns. Chapter 9 provides complete examples and walks through building your first skill. Master both chapters and you'll transform recurring PM tasks from cognitive overhead into reliable processes.

8.2 What Is a Skill?

A skill is a markdown file that tells Claude Code how to execute a specific workflow. It lives in your project's `.claude/skills/` directory (shared with your team via git) or in `~/.claude/skills/` (personal, available across all projects). When you invoke a skill or when Claude Code recognizes that your request matches a skill's description, it follows the documented instructions instead of improvising.

Skills vs. prompts: the distinction matters. A prompt is a one-time instruction. You type it, Claude Code responds, and the prompt disappears into session history. If you want to do the same thing next week, you write the prompt again (or dig through history hoping you remember when you ran it).

A skill is a persistent instruction set. It lives as a file, version-controlled like any other project artifact. When you run the skill next week, it executes the same documented process. When a teammate runs it, they get consistent behavior. When you update the skill, everyone benefits from the improvement.

The difference is persistence and repeatability:

Aspect	Prompt	Skill
Persistence	Session only	File in repository
Repeatability	Requires recall	Invoke by name
Consistency	Varies each time	Documented process
Shareability	Copy-paste	Git commit
Evolution	Lost to history	Version controlled

Where skills live. Claude Code looks for skills in two locations:

- **Project-level:** `.claude/skills/` in your current directory. These skills are project-specific and shared with teammates who clone the repository. Use this for skills tied to your product: feedback synthesis for your specific data format, release notes for your repository

conventions, PRD audits against your team's checklist.

- **Personal:** `~/.claude/skills/` in your home directory. These skills follow you across all projects. Use this for general PM workflows: competitive analysis templates, user story expansion patterns, meeting note synthesis.

Each skill is a subdirectory containing at least one file: `SKILL.md`. The directory name becomes the skill's identifier. Complex skills can include additional files: helper scripts, reference documents, output templates.

```
.claude/skills/  
├── feedback-synthesis/  
│   ├── SKILL.md  
├── release-notes/  
│   ├── SKILL.md  
│   └── resources/  
│       └── customer-voice-guide.md  
└── prd-audit/  
    ├── SKILL.md  
    └── resources/  
        └── prd-checklist.md
```

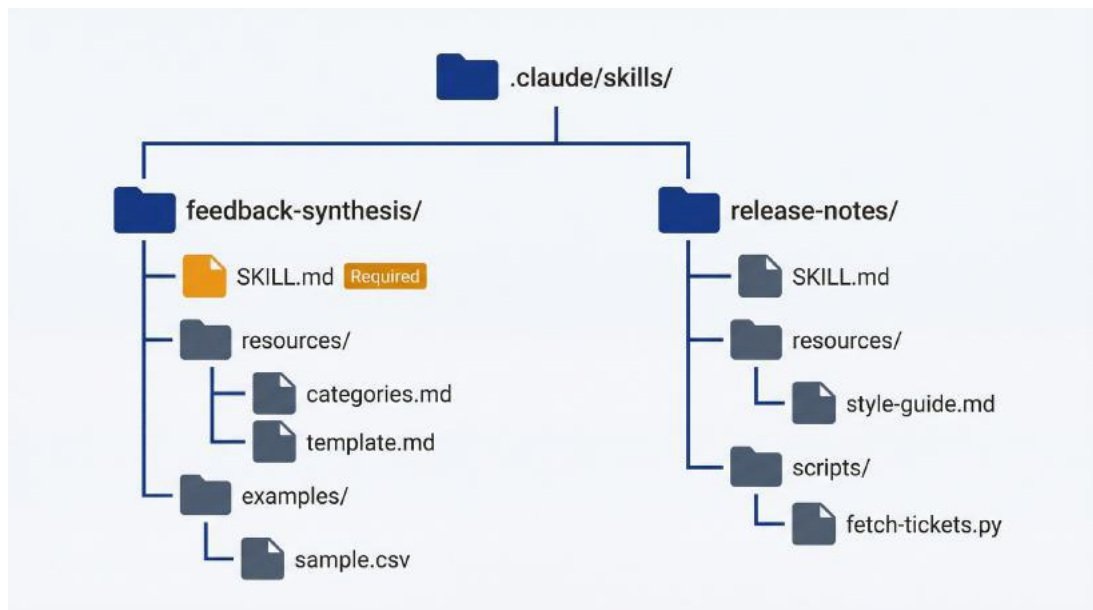


Figure 8.1: Skill Directory Structure – Use this to organize your skills with the required `SKILL.md` file and optional supporting resources, scripts, and examples folders.

How skills are discovered and invoked. Claude Code uses skill descriptions to determine relevance. When you make a request, Claude reads the description field from each skill's YAML frontmatter and decides whether any skill applies. If your feedback CSV and ask to “synthesize this into themes,” Claude Code matches that to a skill whose description mentions “feedback synthesis” and activates it automatically.

You can also invoke skills explicitly as slash commands. A skill named `feedback-synthesis` becomes `/feedback-synthesis`. This explicit invocation guarantees the skill runs, regardless of how you phrase your request.

The auto-discovery behavior is useful but not magic. If your skill description doesn't include words users actually say, Claude Code won't match it. “Analyzes customer sentiment data” won't trigger when you say “summarize this feedback.” Include trigger words that match natural requests: “synthesizes customer feedback, NPS responses, support ticket themes.”

The compounding value of skills. Building a skill takes 30-60 minutes the first time. Using a skill takes 5-10 minutes. The math is straightforward:

- Two uses: 60 minutes building + 20 minutes using = 80 minutes total, versus 90 minutes ad-hoc
- Five uses: $60 + 50 = 110$ minutes, versus 225 minutes ad-hoc
- Ten uses: $60 + 100 = 160$ minutes, versus 450 minutes ad-hoc

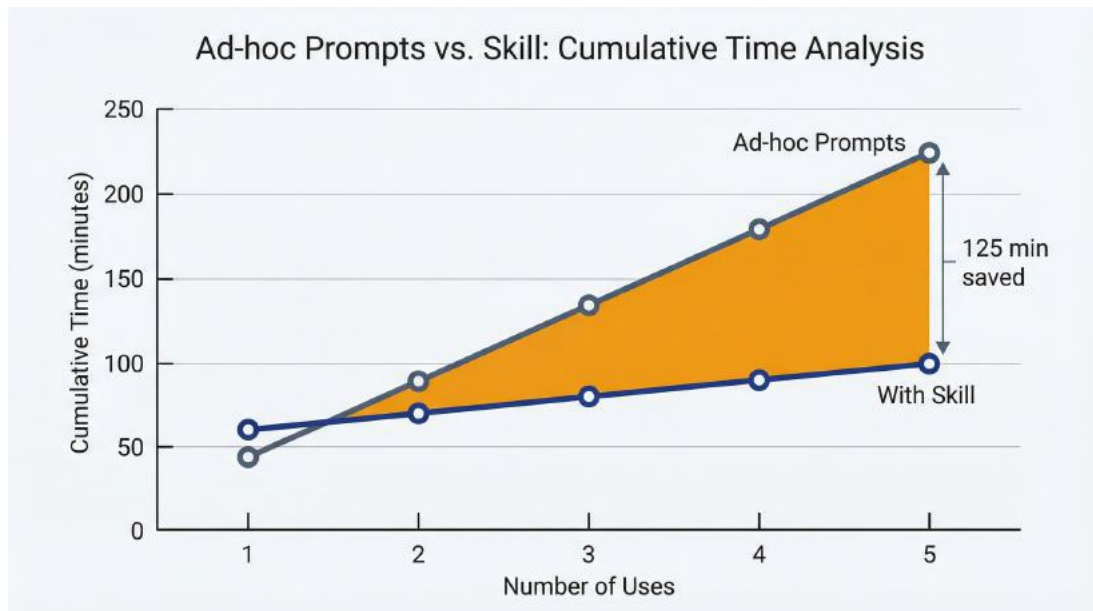


Figure 8.2: Skill Compounding Value – Use this to justify the initial investment in building skills. Ad-hoc prompts grow linearly (45 min each use), while skills have higher initial investment (60 min to build) but flatten to 10 min per use. By the fifth use, skills save 125 minutes versus ad-hoc.

Beyond time savings, skills provide consistency. Quarterly reports follow the same structure. Feedback themes use the same categorization criteria. PRD audits check the same items. This consistency enables comparison across time periods and reduces cognitive load on consumers of your outputs.

8.3 Skill Components

A skill directory contains several types of files. Only `SKILL.md` is required; the others extend capability for complex workflows.

SKILL.md: The instruction file. This is the core of every skill. It contains YAML frontmatter for discovery and markdown instructions for execution. Claude Code reads this file when the skill activates and follows the documented process.

The minimum viable `SKILL.md`:

```
---
name: standup-notes
description: Generate standup notes from yesterday's git commits. Use
when user mentions standup, daily update, or what I did yesterday.
---
```

Standup Notes

Purpose

Create formatted standup notes from recent commits.

Inputs

– Git repository with commit history

Process

1. Read commits from the past 24 hours
2. Group by type (feature, fix, refactor)
3. Write in first person, past tense

Output

Print standup notes to terminal. Do not create a file.

This minimal structure works for simple workflows. More complex skills need additional sections covered in 8.3.

Reference files: data, rubrics, templates. Skills can include supporting files that Claude Code consults during execution. These live in a resources/ subdirectory:

```
feedback-synthesis/
├── SKILL.md
├── resources/
│   ├── category-definitions.md
│   ├── output-template.md
│   └── quality-rubric.md
```

Reference files serve several purposes:

- **Category definitions:** When synthesizing feedback, what categories should Claude Code use? Define them once in category-definitions.md rather than embedding them in the SKILL.md instructions.

- **Output templates:** Exact structure for generated artifacts. “Follow the template in `resources/output-template.md`” is clearer than embedding the template in instructions.
- **Quality rubrics:** Detailed criteria for evaluating outputs. Keeps SKILL.md focused on process while delegating evaluation standards to a reference file.
- **Style guides:** Tone, voice, and formatting preferences. Useful for skills that generate customer-facing content.

In your SKILL.md, reference these files explicitly: “Categorize feedback using the definitions in `resources/category-definitions.md`.” Claude Code will read the referenced file when it reaches that instruction.

Output templates: consistent formatting. If your skill produces documents, define the expected structure:

```
## Output Format
```

```
Generate file: `reports/feedback-synthesis-[date].md`
```

```
Follow this structure:
```

```
# Feedback Synthesis: [Date Range]
```

```
## Executive Summary
```

```
[3–5 bullet points of key findings]
```

```
## Theme Analysis
```

```
### Theme 1: [Name]
```

- ****Frequency:**** [count] mentions ([percentage]%)
- ****Sentiment:**** [positive/negative/mixed]
- ****Representative quotes:****
 - “[Quote 1]”
 - “[Quote 2]”
- ****Implications:**** [What this means for the product]

```
[Repeat for each theme]
```

```
## Recommendations
[Prioritized list of suggested actions]
```

```
## Methodology
[Brief description of data sources and analysis approach]
```

This template ensures every feedback synthesis report has the same structure, making it easier to compare across time periods and present to stakeholders who know what to expect.

Optional scripts: pre/post-processing. Complex skills can include helper scripts for tasks that pure conversation can't handle efficiently:

```
release-notes/
├── SKILL.md
└── scripts/
    └── fetch-jira-tickets.py
```

The SKILL.md references the script: “Run `scripts/fetch-jira-tickets.py --sprint [sprint-name]` to gather ticket data before generating notes.”

For PM skills, scripts are rarely necessary. Most PM workflows involve text analysis and document generation, which Claude Code handles conversationally. Scripts become useful when you need to:

- Call external APIs with specific authentication
- Process large datasets that would consume excessive tokens if read directly
- Transform data formats before analysis

If you find yourself needing scripts, consider whether an MCP integration (Chapter 10) might be cleaner.

Directory structure conventions. Organize skill directories consistently:

```
skill-name/
├── SKILL.md           # Required: Core instructions
├── resources/        # Optional: Reference documents
│   ├── template.md
│   └── rubric.md
```

```
|— scripts/           # Optional: Helper scripts
|   |— helper.py
|— examples/         # Optional: Sample inputs/outputs
|   |— sample-input.csv
|   |— expected-output.md
```

The `examples/` directory is useful for testing and documentation. Include sample inputs and the outputs they should produce. This helps teammates understand what the skill does and helps you verify the skill still works correctly after modifications.

8.4 The SKILL.md File in Detail

SKILL.md combines discovery metadata with execution instructions. Understanding both parts helps you write skills that activate when appropriate and execute reliably.

Required: YAML frontmatter. Every SKILL.md starts with frontmatter delimited by `---`:

```
---
name: feedback-synthesis
description: Synthesizes customer feedback into themed reports. Auto-
invoke when user mentions feedback analysis, NPS synthesis, support
ticket themes, or customer sentiment.
---
```

Two fields are required:

- **name:** Unique identifier, lowercase with hyphens. This becomes the slash command (`/feedback-synthesis`) and the skill's display name. Keep it concise and descriptive.
- **description:** Discovery text that Claude Code uses to decide when to activate the skill. Include trigger words that match how users naturally phrase requests. Be specific: “Synthesizes customer feedback” will match more reliably than “Helps with data analysis.”

The description serves two purposes: helping Claude Code auto-discover

when the skill applies, and helping human readers understand what the skill does. Write it for both audiences.

Required sections in the markdown body. After the frontmatter, structure your SKILL.md with these sections:

Purpose statement. One or two sentences explaining what the skill accomplishes and why it's valuable. Lead with the user benefit, not the process:

`## Purpose`

Transform raw customer feedback into actionable insight reports that product teams can use for prioritization decisions.

This tells readers what they get, not how the skill works internally.

Input requirements. Specify what the user must provide:

`## Expected Inputs`

- `**Required:**` CSV or text file containing customer feedback (one piece of feedback per row)
- `**Optional:**` Date range for filtering (defaults to last 30 days)
- `Supports:` CSV, JSON, plain text files
- `Minimum:` 10 feedback entries for meaningful analysis

Be specific about formats. “CSV file” is vague; “CSV with feedback text in a column named ‘feedback’ or ‘comment’” tells Claude Code what to look for.

Execution steps. Numbered steps Claude Code follows:

`## Process`

1. `**Read and validate input file**`
 - Confirm file exists and has expected format
 - Identify the feedback column
 - Report record count
2. `**Identify themes**`
 - Read through all feedback entries
 - Group by common topics or concerns
 - Use category definitions from ``resources/categories.md`` if strict categorization is needed

3. **Analyze each theme**
 - Count frequency
 - Assess sentiment (positive, negative, mixed)
 - Extract representative quotes (verbatim, not paraphrased)
4. **Generate report**
 - Follow template in ``resources/output-template.md``
 - Include executive summary, theme analysis, and recommendations
 - Save to ``reports/feedback-synthesis-[date].md``
5. **Quality check**
 - Verify all themes have supporting quotes
 - Confirm percentages sum to approximately 100%
 - Check that recommendations are actionable

Use imperative mood (“Read the file” not “The file should be read”). Include decision points (“If fewer than 10 entries, warn and proceed with caveats”). Reference external files when appropriate.

Output format specification. Define exactly what gets generated:

Output Format

Generate file: ``reports/feedback-synthesis-YYYY-MM-DD.md``

Structure:

- Executive summary (3-5 bullets)
- Theme analysis (one section per theme with frequency, sentiment, quotes, implications)
- Recommendations (prioritized list)
- Methodology (brief description)

See ``resources/output-template.md`` for detailed structure.

Specifying the file path pattern ensures consistent output locations. Teams know where to find feedback reports without asking.

Quality criteria. Testable standards for successful execution:

Quality Criteria

- Every theme has at least 2 supporting quotes
- Quotes are verbatim from source data, not paraphrased

- Sentiment assessment explains the reasoning
- Recommendations tie back to specific themes
- File saves successfully to specified path

These criteria let Claude Code self-check before finishing. They also help you evaluate whether a skill execution succeeded.

Optional sections. Add these when they improve clarity or reliability:

Example invocations. Show how to use the skill:

Usage Examples

- Standard: "Run feedback-synthesis on data/customer-feedback-q4.csv"
- With date range: "Run feedback-synthesis on feedback.csv for November only"
- Quick version: "Run feedback-synthesis, just give me the top 3 themes"

Common failure modes. Document what goes wrong and how to handle it:

Edge Cases

- **Sparse data (<10 entries):** Generate report with caveat about limited sample size
- **Mixed formats:** If feedback column isn't obvious, ask user to specify
- **Very long feedback entries:** Summarize individual entries exceeding 500 words
- **Non-English content:** Note language and proceed; don't attempt translation

Related skills. Point to skills that complement this one:

Related Skills

- ``prd-audit``: Use insights from feedback synthesis to audit PRD completeness
- ``user-story-expander``: Turn feedback themes into user stories

Full annotated example. Here's a complete SKILL.md for feedback synthesis:

name: feedback-synthesis

description: Synthesizes customer feedback into themed reports with actionable insights. Auto-invoke when user mentions feedback analysis, NPS synthesis, support ticket themes, customer sentiment, or voice of customer.

Feedback Synthesis

Purpose

Transform raw customer feedback into actionable insight reports that product teams can use for prioritization decisions.

Expected Inputs

- **Required:** File containing customer feedback (CSV, JSON, or plain text)
- **Optional:** Date range, specific focus areas
- Minimum 10 feedback entries for meaningful analysis
- Supports feedback from: support tickets, NPS surveys, user interviews, app store reviews

Process

1. **Read and validate input**
 - Confirm file exists and is readable
 - Identify feedback content (look for columns: feedback, comment, response, review)
 - Report total entry count
2. **Identify themes**
 - Group feedback by common topics
 - Aim for 4-8 themes (merge if too granular, split if too broad)
 - Name themes descriptively: "Onboarding Confusion" not "Theme 1"
3. **Analyze each theme**
 - Count frequency (entries mentioning this theme)
 - Calculate percentage of total feedback
 - Assess sentiment: positive, negative, or mixed
 - Extract 2-3 representative quotes (verbatim)
 - Identify implications for product decisions
4. **Synthesize findings**
 - Write executive summary (top 3-5 findings)
 - Prioritize themes by frequency and business impact

- Generate actionable recommendations

5. ****Generate report****

- Save to `reports/feedback-synthesis-YYYY-MM-DD.md`
- Follow structure in Output Format section

6. ****Quality check****

- Verify all themes have supporting quotes
- Confirm recommendations are specific and actionable
- Check that executive summary captures key points

Output Format

Generate file: `reports/feedback-synthesis-YYYY-MM-DD.md`

Feedback Synthesis: [Date Range]

Executive Summary

- [Key finding 1]
- [Key finding 2]
- [Key finding 3]

Theme Analysis

[Theme Name]

- ****Frequency:**** [X] mentions ([Y]% of feedback)
- ****Sentiment:**** [Positive/Negative/Mixed]
- ****Representative Quotes:****
 - "[Quote 1]"
 - "[Quote 2]"
- ****Product Implications:**** [What this means for roadmap/priorities]

[Repeat for each theme]

Recommendations

1. [Specific action tied to theme]
2. [Specific action tied to theme]
3. [Specific action tied to theme]

Methodology

- ****Source:**** [File name]
- ****Period:**** [Date range]
- ****Sample size:**** [X] feedback entries

- **Analysis date:** [Today's date]

Quality Criteria

- Every theme supported by at least 2 verbatim quotes
- Quotes are exact text from source, not paraphrased
- Percentages sum to approximately 100% (themes can overlap)
- Recommendations are specific enough to be actionable
- Executive summary is readable in under 30 seconds

Edge Cases

- **Sparse data (<10 entries):** Proceed with caveat about limited sample
- **Ambiguous feedback column:** Ask user to specify which column contains feedback
- **Multi-language content:** Note language distribution; analyze separately if significant
- **Very long entries (>500 words):** Summarize before categorizing

Usage Examples

- "Run feedback-synthesis on data/q4-feedback.csv"
- "Synthesize the NPS responses in surveys/november.json"
- "Analyze customer feedback from support-tickets.txt, focus on billing issues"

This skill is approximately 600 words—substantial but under the 5,000-word limit for SKILL.md files. For skills requiring more detail, move reference material to separate files.

8.5 Design Patterns for PM Skills

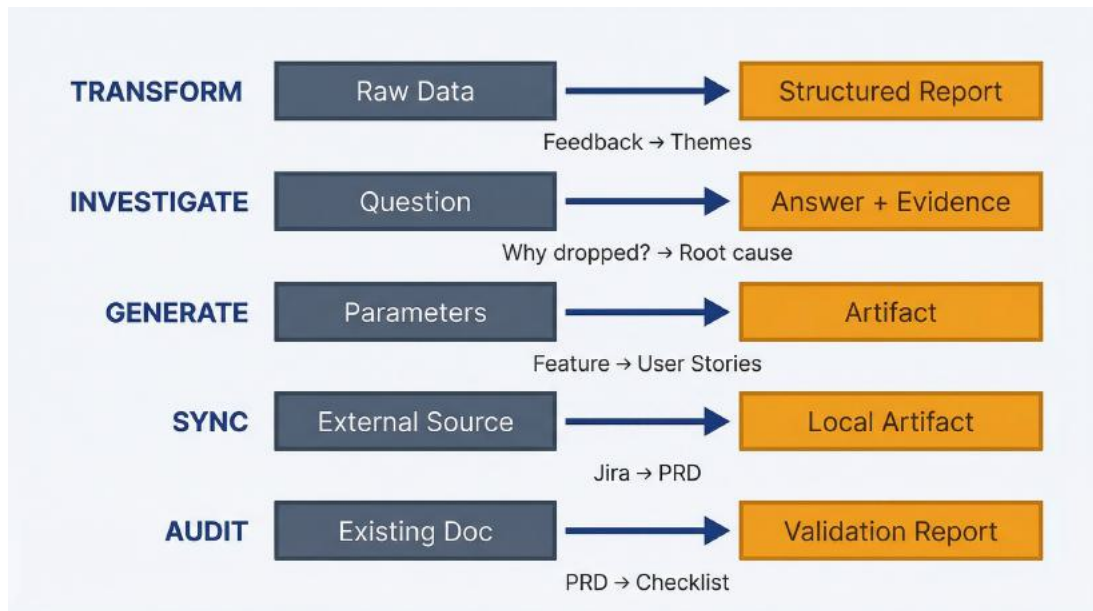


Figure 8.3: Five PM Skill Patterns – Use this to choose the right structure for your workflow. Transform (raw data → structured report), Investigate (question → answer with evidence), Generate (parameters → artifact), Sync (external source → local artifact), Audit (existing doc → validation report). Match your task to the pattern for faster skill development.

Five patterns cover most PM skill needs. Recognize which pattern fits your workflow and structure your skill accordingly.

Transform pattern: Input → Structured output. You have unstructured or semi-structured data. You need structured analysis or documentation.

Examples: - Raw feedback → Themed report - Interview transcript → Insight summary - Git commits → Release notes - Meeting notes → Action items

Structure:

Process

1. Read and validate input data
2. Apply transformation logic (categorize, summarize, restructure)
3. Generate structured output
4. Quality check against criteria

Transform skills have clear inputs and outputs. They're the most common

PM skill pattern.

Investigate pattern: Question → Answer with evidence. You need to answer a question by examining data or codebase and provide supporting evidence.

Examples: - “Why did conversion drop?” → Root cause analysis with data citations - “How does feature X work?” → Explanation with file references - “What do customers say about pricing?” → Summary with supporting quotes

Structure:

Process

1. Understand the question
2. Identify relevant data sources
3. Analyze sources for answer
4. Compile evidence (quotes, references, data points)
5. Synthesize answer with supporting evidence

Investigate skills differ from transform skills in that the output structure depends on what’s found, not a predetermined template.

Generate pattern: Parameters → Artifact. You provide specifications; the skill generates a document or artifact matching those specs.

Examples: - Feature description → User stories with acceptance criteria - Sprint goals → Status report - Product area → Competitive analysis

Structure:

Process

1. Accept parameters (feature name, date range, focus area)
2. Gather context (from codebase, templates, reference docs)
3. Generate artifact following template
4. Validate against quality criteria

Generate skills are template-driven. The output structure is fixed; the content varies based on parameters.

Sync pattern: External source → Local artifact. You pull data from external systems and create or update local documentation.

Examples: - Jira epic → PRD section in repository - Analytics dashboard → Weekly metrics summary - Competitor website → Updated competitive profile

Structure:

Process

1. Connect to external source (via MCP or export)
2. Extract relevant data
3. Transform to local format
4. Create or update local artifact
5. Note sync timestamp

Sync skills often require MCP connections (Chapter 10) or manual data export as input.

Audit pattern: Artifact → Validation report. You have an existing artifact. You need to evaluate it against criteria and report findings.

Examples: - PRD → Completeness checklist - User stories → Quality assessment - Documentation → Accuracy verification against code

Structure:

Process

1. Read artifact to audit
2. Load evaluation criteria (from reference file)
3. Evaluate artifact against each criterion
4. Compile findings (what passes, what fails, what's missing)
5. Generate recommendations for improvement

Audit skills are valuable for quality gates: run the PRD audit before engineering handoff to catch gaps early.

Choosing the right pattern. Ask yourself:

- Do I have data that needs structuring? → Transform
- Am I answering a question? → Investigate

- Am I creating something from specs? → Generate
- Am I pulling external data into my repo? → Sync
- Am I evaluating existing work? → Audit

Some workflows combine patterns. A quarterly review might Sync data from analytics, Transform it into a report, and Audit the report against last quarter's goals. Start with the primary pattern, then extend.

8.6 Determinism and Repeatability

Skills should produce similar outputs given similar inputs. Complete determinism isn't possible (language models have inherent variability), but you can increase consistency through skill design.

Why consistency matters for PM workflows. Inconsistent outputs undermine the value proposition of skills:

- Quarterly feedback reports should be comparable across quarters
- PRD audits should check the same criteria every time
- Release notes should follow the same format regardless of when they're generated

If each skill execution produces a different structure or uses different criteria, you're back to ad-hoc prompt iteration with extra steps.

Techniques for predictable outputs.

Explicit schemas. Don't say "generate a report." Say "generate a report with these exact sections in this order." The more specific your output format, the more consistent the results.

Output Format

Generate with exactly these sections, in this order:

1. Executive Summary (3–5 bullets)
2. Theme Analysis (one subsection per theme)
3. Recommendations (numbered list, max 5 items)
4. Methodology (one paragraph)

Reference file anchoring. Move variable content into reference files:

Process

Categorize feedback using exactly the categories defined in ``resources/category-definitions.md``. Do not create new categories.

This anchors the skill to fixed criteria. Update the reference file to change behavior; the SKILL.md instructions stay stable.

Explicit quality criteria. Testable criteria reduce variability:

Quality Criteria

- Executive summary contains exactly 3–5 bullet points
- Each theme section includes frequency count and percentage
- All quotes are verbatim from source data
- Recommendations reference specific themes by name

Claude Code uses these criteria for self-checking, which catches deviations before output.

Constrained interpretation. Limit how Claude Code interprets ambiguous situations:

Edge Cases

- If a feedback entry could fit multiple themes, assign to the most specific theme
- If sentiment is unclear, mark as "mixed" rather than guessing
- If entry is too short to categorize (<10 words), count it but don't quote it

These constraints reduce variability in edge cases where Claude Code might otherwise make different choices each time.

Testing skills for consistency. Before committing a skill, run it multiple times on the same input:

1. Run the skill on sample data
2. Save the output
3. Clear the session (`/clear`)
4. Run the skill again on the same data
5. Compare outputs

Perfect consistency isn't the goal. Reasonable consistency is. The

structure should be identical. The themes should be similar. The exact wording will vary, but the substance should be stable.

If outputs vary wildly, your skill needs tighter constraints. Add more explicit schemas, reference files, or quality criteria until consistency improves.

Version controlling skills. Skills are files in your repository. Treat them with the same version control discipline as code:

- Commit skills with meaningful messages: “Add feedback-synthesis skill” or “Update release-notes to include Jira ticket links”
- Review skill changes before merging (the team depends on consistent behavior)
- Tag releases if skills are shared widely
- Document breaking changes in skill behavior

When you update a skill, consider backward compatibility. If the output format changes, will existing processes that consume that output break? Communicate changes to teammates who depend on the skill.

8.7 Skill Composition

Complex workflows can combine multiple skills. Rather than building one massive skill that does everything, compose focused skills that each do one thing well.

Atomic skills vs. composite workflows. Atomic skills handle a single task: synthesize feedback, generate release notes, audit a PRD. Composite workflows orchestrate multiple atomic skills for larger objectives.

Example composite workflow: Quarterly product review 1. Run `feedback-synthesis` on Q4 feedback data 2. Run `metrics-summary` on Q4 analytics export 3. Run `competitive-update` to refresh competitor profiles 4. Run `quarterly-review` to synthesize the above into a review document

Each atomic skill is useful independently. The composition creates

higher-level value.

Invoking skills from other skills. A skill can instruct Claude Code to run another skill:

Process

1. Run the ``feedback-synthesis`` skill on the provided feedback file
2. Run the ``metrics-summary`` skill on the provided analytics export
3. Synthesize findings from both reports into a quarterly summary
4. Generate the quarterly review document

This keeps each skill focused while enabling complex workflows.

When to compose vs. build monolithic skills. Compose when: - Component skills are useful independently - The workflow has natural breakpoints - Different people might run different parts - You want to reuse components in other workflows

Build monolithic when: - The workflow is truly atomic (no useful intermediate outputs) - Composition adds overhead without benefit - The skill is simple enough that decomposition is over-engineering

Most PM workflows benefit from composition. Build atomic skills first, then compose as patterns emerge.

You understand skill structure: the SKILL.md file with YAML frontmatter and markdown instructions, reference files for templates and criteria, and directory conventions for organization. You know the five design patterns (transform, investigate, generate, sync, audit) and techniques for improving consistency. Chapter 9 provides complete examples of essential PM skills and walks through building your first skill from scratch.

9 Building Your PM Skill Library

You've built a feedback synthesis workflow that works. You've run it three times. Each time, you start a Claude Code session, explain what you want ("analyze this CSV of feedback, categorize by theme, count frequencies, extract quotes"), wait while Claude figures out your process, review the output, refine with follow-up prompts. The results are good. The inefficiency is obvious: you're re-teaching the same workflow every time.

Skills solve this. A skill is a folder containing a `SKILL.md` file with instructions that Claude Code loads automatically when your request matches what the skill does. You define the workflow once (inputs, process, output format, quality criteria). Then you just describe what you want done, and if it matches a skill's description, Claude Code loads those instructions and executes the workflow without re-explanation.

The first feedback synthesis takes 45 minutes including setup and iteration. The second synthesis, with a skill built from the first run, takes 10 minutes. The third takes 10 minutes. The fourth takes 10 minutes. You've encoded the process. Time saved compounds with each repetition.

This chapter provides five starter skills that solve common PM workflows, shows you how to build your first custom skill from scratch, and explains how to maintain and evolve your skill library over time. By the end, you'll have repeatable, consistent workflows for the tasks you do monthly or quarterly (feedback synthesis, release notes, competitive analysis) that execute the same way every time without cognitive overhead.

9.1 The Starter Kit: Five Essential PM Skills

These five skills address workflows most PMs run repeatedly. You can build them yourself following the patterns shown, or adapt them to your specific needs. Each one saves 20-40 minutes per invocation once built.

9.1.1 Skill 1: Feedback Synthesizer

Purpose: Transform unstructured customer feedback (support tickets, NPS responses, survey data) into categorized themes with frequency counts and supporting quotes.

Use cases:

- Quarterly feedback review for roadmap planning
- Post-launch feedback analysis
- Feature request prioritization
- Customer insight synthesis for stakeholders

How it works: You provide CSV or text files containing feedback. The skill applies consistent categorization, identifies recurring themes, counts frequency, assesses sentiment, and produces a structured report with supporting evidence.

SKILL.md structure:

```
---
name: feedback-synthesizer
description: Synthesizes customer feedback from CSV or text files into
themed reports with frequency analysis and supporting quotes. Auto-
invoke when user mentions feedback analysis, customer insights, NPS
synthesis, or support ticket themes.
---
```

Feedback Synthesizer Skill

Purpose

Analyze unstructured customer feedback and produce structured theme reports for roadmap planning and insight synthesis.

Expected Inputs

- CSV files with feedback text in a column (support tickets, NPS responses, survey data)
- Plain text files with feedback items separated by blank lines
- Multiple feedback sources to synthesize together

Process

1. Read all feedback files provided
2. Identify recurring themes (10–15 distinct themes typically)
3. Count frequency of each theme
4. Assess sentiment per theme (positive/negative/neutral/mixed distribution)
5. Extract representative quotes (3–5 per theme)
6. Prioritize themes by frequency and severity language

Output Format

Generate a markdown report: `feedback/theme-report-[date].md`

Structure:

- **Executive Summary**: Top 3 themes by frequency, top 3 by severity
- **Detailed Themes**: One section per theme with:
 - Theme name and description
 - Frequency count and percentage
 - Sentiment distribution
 - Representative quotes
 - Suggested action or investigation area
- **Cross-Cutting Patterns**: Themes that frequently co-occur
- **Weak Signals**: Low-frequency themes worth monitoring

Quality Criteria

- Themes must appear in at least 5 feedback items unless flagged as critical
- Quotes must be actual verbatim excerpts, not paraphrased
- Frequency counts must be accurate
- Sentiment assessment based on language used, not assumed
- Report length under 6 pages for readability

Common Pitfalls

- Don't create too many themes. Consolidate similar concepts
- Don't bias toward recent feedback. Analyze all data equally
- Don't infer sentiment without textual evidence

Customization points:

- Minimum theme frequency threshold (default: 5 occurrences)
- Predefined categories vs. emergent themes
- Sentiment analysis depth
- Report length and format

Invocation: Just describe what you want: “Analyze the feedback in data/q1-2026-feedback.csv and create a theme report.” Claude Code sees the skill description matches your request and loads the feedback-synthesizer skill automatically.

9.1.2 Skill 2: Competitive Intelligence Scanner

Purpose: Generate structured competitive analysis comparing your product to competitors across features, positioning, and pricing.

Use cases:

- Quarterly competitive landscape updates
- Feature gap analysis
- Pricing strategy review
- Market positioning research

How it works: You provide competitor names and aspects to compare (features, pricing, positioning). The skill researches each competitor, structures findings in comparison matrices, and produces version-controlled competitive intelligence docs.

SKILL.md structure:

```
---
name: competitive-intel
description: Generates structured competitive analysis comparing
products across features, pricing, and positioning. Auto-invoke when
user mentions competitive analysis, competitor comparison, market
landscape, or feature gap analysis.
---
```

Competitive Intelligence Scanner Skill

Purpose

Create systematic competitive analysis reports for roadmap planning and strategic positioning.

Expected Inputs

– List of competitor names or URLs

- Comparison dimensions (features, pricing, positioning, target market)
- Existing competitive docs to update (optional)

Process

1. For each competitor, gather information about:
 - Core features and capabilities
 - Pricing model and tiers
 - Target market and positioning
 - Strengths and weaknesses relative to our product
2. Create comparison matrices across competitors
3. Identify feature gaps (what they have that we lack)
4. Identify differentiation opportunities (what we have that they lack)
5. Summarize strategic implications

Output Format

Generate markdown files in `research/competitors/`:

- `competitor-[name]-profile.md` for each competitor (detailed profile)
- `competitive-matrix-[date].md` (comparison table)
- `competitive-summary-[date].md` (strategic analysis)

Comparison matrix format:

- Rows: Feature categories or evaluation criteria
- Columns: Competitors plus your product
- Cells: ✓ (has feature), ✗ (lacks feature), ~ (partial/limited), or description

Quality Criteria

- Information must be current (check dates of sources)
- Pricing must include all tiers, not just base price
- Feature comparisons must be apples-to-apples (same capabilities, not just names)
- Avoid bias. Represent competitor strengths accurately
- Include sources for verification

Update Frequency

Run quarterly or when:

- Major competitor launches announced
- Pricing changes occur
- Strategic positioning shifts needed

Customization points:

- Comparison criteria (features, pricing, target market, UX, integrations)
- Number of competitors tracked
- Update cadence
- Output format (matrices vs. narrative)

Invocation: “Update competitive analysis for Competitor A, Competitor B, and Competitor C focusing on pricing and enterprise features.”

9.1.3 Skill 3: Release Notes Generator

Purpose: Generate customer-facing release notes from git commit history and Jira tickets, written in product voice.

Use cases:

- Monthly or quarterly release announcements
- Feature launch communication
- Changelog maintenance
- Customer update emails

How it works: You specify a version tag or date range. The skill reads git commits, filters for user-facing changes, categorizes by type (Added/Changed/Fixed/Removed), and writes release notes in your product voice.

SKILL.md structure:

```
---
name: release-notes
description: Generates customer-facing release notes from git history
and Jira tickets, written in product voice. Auto-invoke when user
mentions release notes, changelog, version notes, or feature
announcements.
---
```

Release Notes Generator Skill

Purpose

Create consistent, customer-friendly release notes for product updates

and feature launches.

Expected Inputs

- Version number or tag (e.g., "v2.5.0" or git tag)
- Date range (e.g., "commits since last release" or "January 1–31, 2026")
- Optional Jira filter or ticket list for additional context

Process

1. Read git commits in the specified range
2. Identify user-facing changes (filter out internal refactoring, dependency updates)
3. Categorize changes:
 - ****Added****: New features or capabilities
 - ****Changed****: Improvements to existing features
 - ****Fixed****: Bug fixes
 - ****Removed****: Deprecated or removed features
4. Write descriptions in product language (not technical jargon)
5. Group related changes together
6. Order by user impact (most impactful first)

Output Format

Generate `CHANGELOG.md` entry or `docs/releases/[version].md`:

```

## [Version X.Y.Z] – YYYY-MM-DD

### Added

- [Feature description in user terms]

### Changed

- [Improvement description]

### Fixed

- [Bug fix in plain language]

### Removed

- [Deprecated feature with migration guidance if needed]

```

Voice and Tone

- ****Customer-facing, not technical****: "Bulk user import now supports up

to 1,000 users" not "Increased MAX_IMPORT_SIZE constant"

- ****Benefit-oriented****: Explain what users can now do, not just what changed
- ****Clear and concise****: One sentence per change typically
- ****Active voice****: "Added dashboard customization" not "Dashboard customization has been added"

Quality Criteria

- Only include changes that affect user experience
- Each change must be understandable without reading commit messages
- Related changes should be grouped, not listed separately
- Removed features should note alternatives or migration paths
- Version number and date must be accurate

Common Patterns

- Internal refactoring → exclude unless it affects performance
- Dependency updates → exclude unless they add user-facing capability
- Bug fixes in unreleased features → exclude (not relevant to customers)
- Breaking changes → call out explicitly with migration guidance

Customization points:

- Voice and tone (formal vs. casual, technical depth)
- Categorization scheme (add “Deprecated”, “Security”, etc.)
- Output format (changelog, email, in-app announcement)
- Whether to include breaking changes section
- Internal vs. customer-facing versions

Invocation: “Generate release notes for version 2.5.0 from commits since v2.4.0.”

9.1.4 Skill 4: PRD Auditor

Purpose: Review PRD documents against a completeness checklist and identify gaps or inconsistencies.

Use cases:

- Pre-kickoff PRD review
- Stakeholder readiness check

- Engineering handoff preparation
- Quarterly PRD quality audit

How it works: You point the skill at a PRD file. It checks for required sections, evaluates completeness, flags ambiguities, suggests improvements, and produces a scored audit report.

SKILL.md structure:

```

---
name: prd-auditor
description: Reviews PRD documents for completeness, clarity, and
consistency. Auto-invoke when user mentions PRD review, requirements
audit, PRD checklist, or spec validation.
---
```

PRD Auditor Skill

Purpose

Systematically review PRD documents to ensure completeness before engineering kickoff.

Expected Inputs

- Path to PRD markdown file (e.g., `docs/prds/feature-name.md`)
- Optional custom checklist or scoring rubric

Process

1. Read the PRD document
2. Check for required sections:
 - Overview and success metrics
 - User personas and use cases
 - Functional and non-functional requirements
 - User experience / workflows
 - Technical approach
 - Out of scope
 - Open questions
 - Success criteria
3. Evaluate each section for:
 - ****Completeness****: Are critical details present?
 - ****Clarity****: Is it unambiguous?
 - ****Specificity****: Are requirements testable/measurable?
 - ****Consistency****: Do sections align (e.g., success metrics match

requirements)?

4. Flag issues:

- Missing sections
- Vague or ambiguous requirements
- Unresolved open questions marked as "TBD"
- Success metrics that aren't measurable
- Technical approach missing implementation details

5. Score overall PRD readiness (Ready / Needs Work / Incomplete)

Output Format

Generate audit report as comment in the PRD or separate file:

```
```markdown
```

```
PRD Audit: [Feature Name]
```

```
Audit Date: YYYY-MM-DD
```

```
Overall Score: Ready / Needs Work / Incomplete
```

### ## Completeness Check

- [✓] Overview and success metrics
- [x] User personas (missing or too generic)
- [✓] Functional requirements
- [~] Non-functional requirements (performance criteria vague)

...

### ## Issues Found

#### ### Critical (Must fix before kickoff)

1. Success metrics not measurable: "Users will be happier" → Define specific metric
2. Open questions section has 3 unresolved items marked "TBD" → Resolve or document assumptions

#### ### Important (Should fix)

1. Technical approach missing specific files/modules that need changes
2. Out of scope section too brief. This could cause scope creep

#### ### Minor (Nice to fix)

1. No timeline/milestones section
2. Appendix missing links to related docs

### ## Recommendations

- Consult Engineering for technical approach details

- Define measurable success metrics (usage %, time saved, error rate reduction)
  - Resolve open questions or document assumptions explicitly
- ```
```
```

## ## Scoring Methodology

**\*\*Ready\*\***: All required sections present and complete, fewer than 2 critical issues

**\*\*Needs Work\*\***: Required sections present but critical gaps exist (3–5 critical issues)

**\*\*Incomplete\*\***: Missing required sections or 6+ critical issues

## ## Checklist Customization

The default checklist can be overridden by including a `PRD\_CHECKLIST.md` file in the same directory defining custom requirements.

### Customization points:

- Required sections (adjust for your team’s PRD template)
- Scoring criteria and thresholds
- Issue severity definitions
- Specific quality checks (e.g., all requirements must have acceptance criteria)

**Invocation**: “Audit the PRD at docs/prds/bulk-import.md and tell me what’s missing.”

## 9.1.5 Skill 5: User Story Expander

**Purpose**: Expand feature concepts into detailed user stories with acceptance criteria and edge cases, informed by codebase context.

### Use cases:

- Sprint planning prep
- Epic breakdown
- Feature scoping
- Engineering handoff



**How it works:** You provide a feature description and target persona. The skill explores the codebase for technical context (existing patterns, constraints), generates comprehensive user stories following your team's format, includes acceptance criteria, and identifies edge cases based on codebase patterns.

## **SKILL.md structure:**

```

name: user-story-expander
description: Expands feature concepts into detailed user stories with
acceptance criteria, technical notes, and edge cases. Auto-invoke when
user mentions user stories, story expansion, acceptance criteria, or
epic breakdown.

```

### # User Story Expander Skill

#### ## Purpose

Generate comprehensive user stories for features, grounded in codebase context and technical constraints.

#### ## Expected Inputs

- Feature description (2-3 sentences about what to build)
- Target persona or user type
- Core workflows the feature must support
- Optional: existing user story file to append to

#### ## Process

1. Explore codebase for technical context:
  - Existing patterns related to this feature area
  - Validation rules and constraints
  - Similar features to reference
  - Potential integration points
2. Generate 5-8 user stories covering:
  - Core happy path workflow
  - Variations (different user types, scenarios)
  - Error handling and edge cases
  - Admin/configuration needs if applicable
3. For each story, include:
  - User story statement (As a X, I want to Y, so that Z)
  - Acceptance criteria (specific, testable)

- Technical notes (files/modules affected, patterns to follow)
- Edge cases (boundary conditions, error states)

#### 4. Prioritize stories by user value

### ## Output Format

Generate or append to `docs/user-stories/[feature-name].md`:

```markdown

User Stories: [Feature Name]

Story 1: [Primary Workflow]

****As a**** [persona],
****I want to**** [action],
****So that**** [benefit].

****Acceptance Criteria:****

- [Specific, testable criterion]
- [Another criterion]
- [Edge case handling]

****Technical Notes:****

- Relevant files: `src/path/to/file.js:123`
- Pattern to follow: See `src/services/batch-operations.js` for async processing
- Constraints: File upload limit 10MB, user creation validation rules in `user-service.js`

****Edge Cases:****

- What if file exceeds size limit?
- What if user already exists?
- What if validation fails mid-process?

Story 2: [Variation or Secondary Workflow]

...

Quality Criteria

- Stories must be user-centric (not implementation-focused)

- Acceptance criteria must be testable (not vague)
- Technical notes must reference actual files in codebase
- Edge cases must be realistic based on codebase constraints
- Coverage: 80% of typical scenarios (not every possible edge case)

Story Format

Use standard "As a / I want to / So that" format unless team has custom template.

Customization points:

- Story format (if your team uses different template)
- Number of stories generated (5-8 default)
- Technical detail depth
- Whether to include story points estimation
- Integration with Jira/Linear (reference ticket format)

Invocation: “Expand the bulk user import feature into user stories for the Enterprise Admin persona.”

These five skills form your starter kit. Build them once, customize to your context, and invoke them whenever the workflow recurs. Each one reduces a 30-60 minute manual task to a 10-minute automated execution with consistent output quality.

9.2 Creating Your First Skill Step by Step

You’ll learn by building, not just reading templates. This section walks through creating a feedback synthesizer skill from scratch, the most common PM workflow and a pattern you can adapt for other skills.

Selecting a repeatable workflow to encode. Choose a workflow you’ve done at least twice and will do again: feedback synthesis, competitive analysis, release notes, PRD generation. Choose anything with consistent inputs, process, and outputs.

Bad choice for first skill: One-off analysis that won’t repeat. Experimental workflow you haven’t validated yet. Process that changes significantly

each time.

Good choice: Quarterly feedback review. Monthly release notes. Feature investigation following same pattern. Anything you find yourself explaining to Claude Code the same way multiple times.

We'll build a feedback synthesizer because it demonstrates the core pattern: structured input, consistent processing, formatted output.

Drafting the initial SKILL.md. Create the directory structure:

```
mkdir -p .claude/skills/feedback-synthesizer
cd .claude/skills/feedback-synthesizer
```

Create SKILL.md:

```
touch SKILL.md
```

Open it in your editor. Start with the YAML frontmatter. This is critical: the name and description determine whether Claude Code loads your skill.

```
---
name: feedback-synthesizer
description: Synthesizes customer feedback from CSV or text files into
themed reports with frequency analysis and supporting quotes. Auto-
invoke when user mentions feedback analysis, customer insights, NPS
synthesis, or support ticket themes.
---
```

Critical: The description is your skill's interface. Claude Code reads this to decide whether to load the skill. Make it specific about: - What the skill does ("Synthesizes customer feedback") - What input formats it handles ("CSV or text files") - What output it produces ("themed reports with frequency analysis") - When to invoke it ("feedback analysis, customer insights, NPS synthesis")

Vague description = skill won't activate when needed. Overly specific = skill only activates for exact phrasing. Balance specificity with reasonable pattern matching.

Now write the skill instructions below the frontmatter:

Feedback Synthesizer Skill

Purpose

Analyze unstructured customer feedback and produce structured theme reports for roadmap planning.

Expected Inputs

- CSV files with feedback text column
- Plain text files with feedback items separated by blank lines
- Supports multiple files to synthesize together

Process

Follow these steps exactly:

1. ****Read all feedback data****
 - Identify which column contains feedback text in CSV files
 - Count total feedback items
2. ****Identify themes****
 - Read all feedback completely before categorizing
 - Identify 10-15 distinct recurring themes
 - Theme must appear in at least 5 feedback items to be included
 - Consolidate similar themes (don't over-segment)
3. ****Categorize and count****
 - Assign each feedback item to one or more themes
 - Count frequency per theme
 - Calculate percentage of total feedback
4. ****Assess sentiment****
 - For each theme, determine sentiment based on language used
 - Categories: Positive, Negative, Neutral, Mixed
 - Base on actual words used, not assumptions
5. ****Extract supporting quotes****
 - Select 3-5 representative quotes per theme
 - Use verbatim text, not paraphrased
 - Choose quotes that clearly illustrate the theme
6. ****Prioritize****
 - Rank themes by frequency
 - Flag high-severity themes (language like "blocking", "critical",

"frustrated")

Output Format

Generate markdown report: `feedback/theme-report-[YYYY-MM-DD].md`

Structure:

```markdown

# Feedback Synthesis Report – [Date]

## ## Executive Summary

Top 3 themes by frequency:

1. [Theme name] – [count] mentions ([percentage]%)
2. [Theme name] – [count] mentions ([percentage]%)
3. [Theme name] – [count] mentions ([percentage]%)

Top 3 themes by severity:

1. [Theme name] – [severity assessment]
2. [Theme name] – [severity assessment]
3. [Theme name] – [severity assessment]

## ## Detailed Themes

### ### Theme: [Name]

**\*\*Description\*\*:** [What this theme represents – 1 sentence]

**\*\*Frequency\*\*:** [count] mentions ([percentage]% of total feedback)

**\*\*Sentiment\*\*:** [Distribution – e.g., "75% negative, 20% neutral, 5% positive"]

**\*\*Severity\*\*:** [High/Medium/Low based on language used]

**\*\*Representative Quotes\*\*:**

- "[Verbatim quote 1]"
- "[Verbatim quote 2]"
- "[Verbatim quote 3]"

**\*\*Suggested Action\*\*:** [What to investigate or consider]

[Repeat for each theme]

## ## Cross-Cutting Patterns

[Themes that frequently appear together – e.g., "Slow performance and Export failures often mentioned together"]

## ## Weak Signals

[Low-frequency themes worth monitoring but not urgent]  
```

Quality Criteria

- Themes based on actual patterns in data, not assumptions
- Quotes are verbatim excerpts, not summarized
- Frequency counts are accurate
- Sentiment based on language evidence
- Report is scannable (under 6 pages)
- Themes are distinct, not overlapping

Creating reference files. Some skills benefit from reference files: templates, rubrics, examples. For feedback synthesis, you might include:

touch theme-categories.md

In theme-categories.md, define your default categories if you prefer predefined themes over emergent ones:

Feedback Theme Categories

Use these categories when analyzing feedback, unless emergent themes are more appropriate:

- Feature Requests
- Performance Issues
- Usability Problems
- Bug Reports
- Integration Requests
- Documentation Gaps
- Pricing/Billing Concerns
- Support Experience

If feedback doesn't fit these categories, create new themes as needed.

Reference this in your SKILL.md if you want consistent categorization:

Categorization Approach

Use predefined categories from `theme-categories.md` if feedback clearly fits.

Create emergent themes when feedback patterns don't match predefined

categories.

Testing with real inputs. Don't commit untested skills. Run it with actual data:

```
cd /path/to/your/project
claude
```

Prepare test feedback (use real data from a past quarter):

```
# Assuming you have feedback CSV in data/test-feedback.csv
```

```
Analyze the feedback in data/test-feedback.csv and generate a theme
report.
```

Watch what happens. Claude Code should recognize your skill description matches ("feedback analysis") and prompt you:

```
I found a skill that might help: feedback-synthesizer
Load this skill? (y/n)
```

Confirm. Claude loads your SKILL.md instructions and executes the workflow.

Review the output: - Did it follow your process steps? - Is the output format correct? - Are themes reasonable and well-supported? - Are frequency counts accurate? - Are quotes verbatim?

Iterating based on output quality. Your first skill won't be perfect. Common issues:

Problem: Themes too granular (20+ themes, many with 1-2 mentions)

Fix: Add to SKILL.md: "Consolidate themes with <5 mentions unless critically severe. Aim for 10-15 distinct themes maximum."

Problem: Quotes are paraphrased or summarized **Fix:** Emphasize in SKILL.md: "Quotes MUST be verbatim excerpts from feedback, enclosed in quotation marks, not summaries."

Problem: Sentiment seems assumed rather than text-based **Fix:** Add: "Sentiment must be based on specific language used in feedback. If

language is neutral/factual, sentiment is neutral regardless of topic negativity.”

Problem: Report is 12 pages (too long) **Fix:** Add: “Limit to top 10 themes. Combine related themes if report exceeds 6 pages.”

Edit your SKILL.md, then test again with the same data. Compare outputs. Iterate until you get consistent, high-quality results.

Documenting edge cases. Add a section to your SKILL.md capturing failure modes:

`## Edge Cases and Limitations`

`**Very small dataset (<50 feedback items):**`

- `- May not find meaningful themes`
- `- Recommend manual review instead`

`**Highly heterogeneous feedback (every item unique):**`

- `- Themes won't emerge`
- `- Suggest segmenting by customer type or use case first`

`**Low-quality feedback (one-word responses, "good/bad" only):**`

- `- Insufficient data to categorize meaningfully`
- `- Note this in report and recommend improving data collection`

`**Multiple languages in dataset:**`

- `- May categorize differently across languages`
- `- Recommend separating languages if theme consistency needed`

`**Extremely large dataset (>2,000 items):**`

- `- Processing may take longer and consume significant tokens`
- `- Consider sampling or batching by time period`

Finalizing and committing. Your skill is working consistently. Time to version control it:

```
git add .claude/skills/feedback-synthesizer/  
git commit -m "Add feedback-synthesizer skill for quarterly feedback  
analysis"
```

Now anyone on your team using this repository gets access to the skill. Your future self benefits. Your process is encoded.

Time investment for first skill. Drafting initial SKILL.md: 20-30 minutes. Testing with real data: 15-20 minutes. Iterating and refining: 20-40 minutes (depends on issues found). Documenting edge cases: 10 minutes. Total: 65-100 minutes.

That feels like a lot. Compare to repeating the manual process every quarter: 45 minutes per synthesis $\times$ 4 quarters = 180 minutes annually. Your skill pays back on the second use and saves time every use thereafter.

9.3 Keeping Your Skills Working Over Time

Skills aren't write-once artifacts. Your processes evolve. Your data formats change. You discover edge cases. Good skill maintenance keeps your library valuable.

When to update vs. create new skills. Update existing skill when: - Process stays the same but needs refinement (better output format, clearer instructions) - You discover edge cases to document - Input formats evolve slightly (CSV adds new columns) - Output format needs enhancement (add new section to report)

Create new skill when: - Process is fundamentally different (different analysis methodology) - Inputs are different type (feedback synthesis vs. interview synthesis) - Outputs serve different purpose (internal analysis vs. external report) - Workflows are unrelated even if similar domain

Example: You have feedback-synthesizer for categorizing feedback. You want to create executive summaries from the theme reports. This is a different workflow with different inputs (theme report, not raw feedback) and different output (exec summary, not full analysis). Create feedback-

executive-summary as a separate skill.

Deprecation practices. Skills become obsolete. Your process changes. A workflow is no longer relevant. Handle this explicitly:

Add deprecation notice to SKILL.md:

```
---
name: old-skill-name
description: [DEPRECATED - Use new-skill-name instead] Previous
description for compatibility
---
```

```
# DEPRECATED: Old Skill Name
```

```
**This skill is deprecated as of 2026-02-01.**
**Use `new-skill-name` instead, which provides [reason for new
approach].**
```

```
[Original skill instructions remain for reference]
```

Move to deprecated directory:

```
mkdir -p .claude/skills/_deprecated
git mv .claude/skills/old-skill-name .claude/skills/_deprecated/
git commit -m "Deprecate old-skill-name in favor of new-skill-name"
```

Claude Code won't auto-load deprecated skills because they're not in the main skills directory, but they're available in version history for reference.

Changelog maintenance for skills. Track skill evolution with a CHANGELOG file in each skill directory:

```
touch .claude/skills/feedback-synthesizer/CHANGELOG.md
```

```
# Feedback Synthesizer Skill Changelog
```

```
## [2.0.0] - 2026-03-15
```

```
### Changed
```

```
- Now generates sentiment distribution per theme (was binary
positive/negative)
```

- Increased minimum theme frequency from 3 to 5 mentions
- Report format now includes "Suggested Actions" per theme

Fixed

- Quotes now guaranteed verbatim (were sometimes paraphrased)
- Cross-cutting patterns section no longer empty when no patterns exist

[1.1.0] - 2026-01-20

Added

- Support for plain text feedback files (was CSV only)
- Edge case handling for small datasets
- Weak signals section for low-frequency themes

Fixed

- Theme consolidation now works correctly (was creating too many granular themes)

[1.0.0] - 2025-12-10

Added

- Initial feedback synthesizer skill
- CSV input support
- Theme identification and frequency counting
- Report generation in markdown format

Update the changelog when you modify the skill. This helps team members understand what changed and whether they need to adjust how they use it.

Team sharing and standardization. When multiple PMs use the same skill, standardization matters. Establish practices:

Skill review before commit. Before adding or modifying a team-shared skill, have another PM test it with their data. Does it work for their use cases? Are instructions clear? Is output format useful?

Naming conventions. Use consistent naming: - Descriptive names: feedback-synthesizer not skill1 - Kebab-case: user-story-expander not UserStoryExpander OR user_story_expander - Specific enough to distinguish: release-notes-customer-facing VS. release-notes-internal if you have both

Documentation standards. Every skill should have: - Clear

description in frontmatter that explains when it auto-invokes - Purpose statement - Expected inputs section - Process steps - Output format specification - Quality criteria - Known edge cases or limitations

Version compatibility. If you use Claude Code versions with different capabilities, note compatibility:

Compatibility

- Requires Claude Code 0.5.0+ for MCP integration features
- Works with both Claude.ai and API (no network-dependent features)

Onboarding new team members to existing skills. When a new PM joins, they inherit the skill library. Create a skill index:

```
touch .claude/skills/README.md
```

PM Skill Library

Available Skills

Feedback Synthesizer

****File**:** `feedback-synthesizer/`

****Purpose**:** Analyze customer feedback and generate theme reports

****Use when**:** Quarterly feedback review, post-launch analysis, feature request prioritization

****Invocation**:** "Analyze feedback in [file]" or "Create feedback theme report"

Release Notes Generator

****File**:** `release-notes/`

****Purpose**:** Generate customer-facing release notes from git history

****Use when**:** Monthly releases, feature launches, changelog updates

****Invocation**:** "Generate release notes for version X.Y.Z"

Competitive Intel Scanner

****File**:** `competitive-intel/`

****Purpose**:** Create competitive analysis comparing products

****Use when**:** Quarterly competitive review, feature gap analysis, market research

****Invocation**:** "Update competitive analysis for [competitors]"

[List all skills with descriptions]

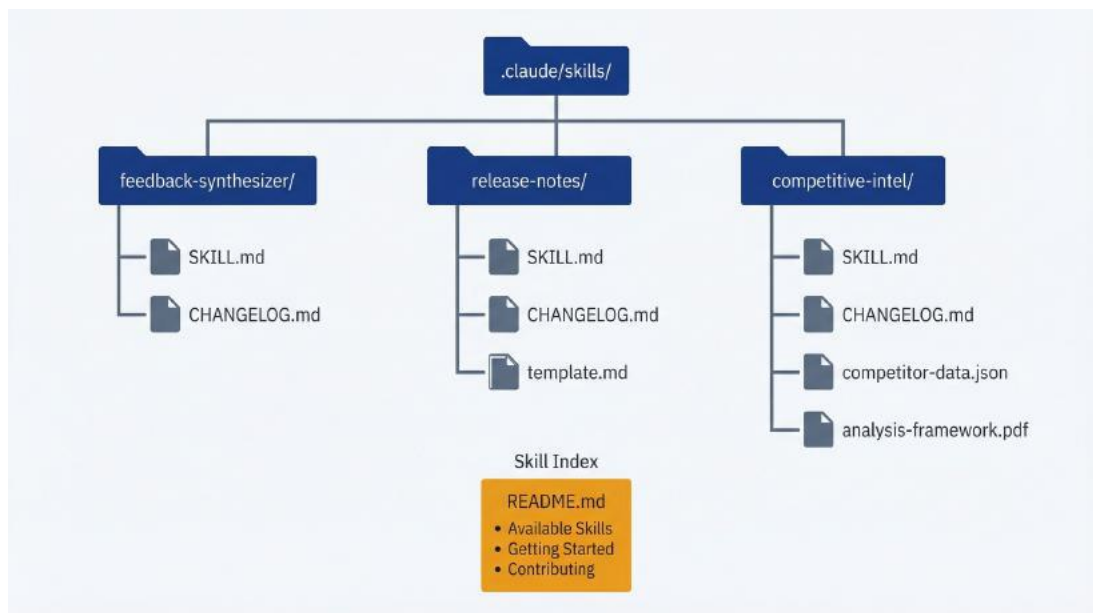
Getting Started

1. Identify which skill matches your task
2. Prepare inputs as specified in skill's SKILL.md
3. Run Claude Code and describe your task naturally. The skill will auto-load
4. Review output and refine if needed

Contributing New Skills

See CONTRIBUTING.md for skill development guidelines.

This serves as skill discovery and documentation for the team.



Skill library organization: skills directory with individual skill folders, each containing SKILL.md and supporting files, plus a README index

Measuring adoption. Track which skills get used. Simple approach: git commit messages when skills generate outputs:

```
git log --grep="feedback-synthesizer" --oneline
git log --grep="release-notes" --oneline
```

Skills with high commit frequency are valuable. Skills with zero usage in 6 months are candidates for deprecation.

Common maintenance triggers:

- Skill used 5+ times → Review and refine based on usage patterns
- Skill fails on new input format → Update input handling
- Output format doesn't meet stakeholder needs → Adjust output specification
- Process takes too long → Optimize instructions or split into smaller skills
- Team stops using skill → Investigate why, improve or deprecate

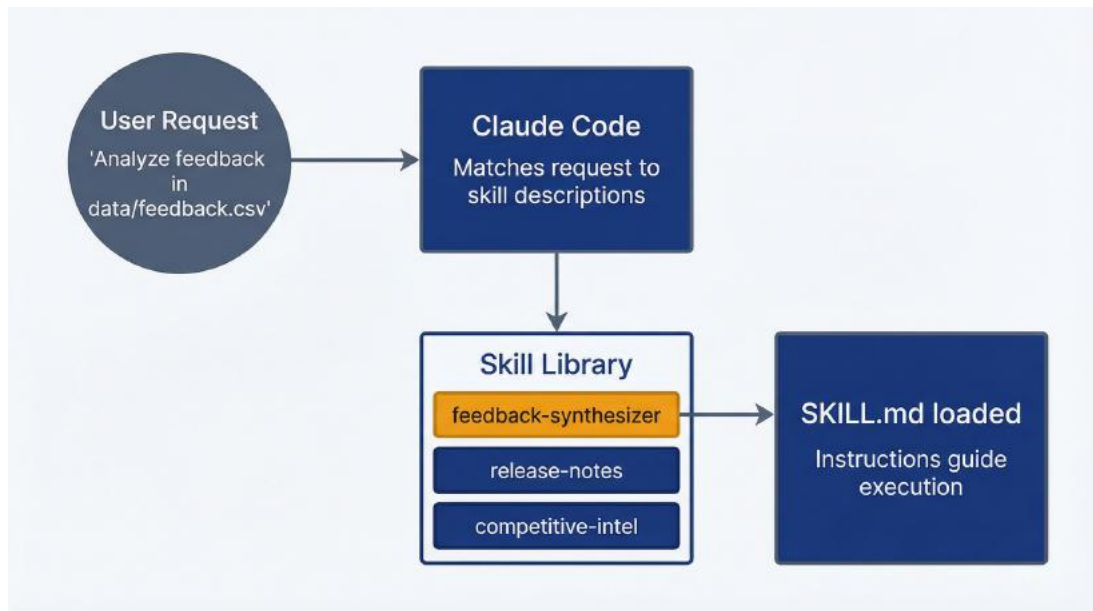
Quarterly skill maintenance (review all skills, update documentation, deprecate unused) takes 30-60 minutes and keeps your library valuable.

9.4 Making Skills Adapt to Different Requests

Skills can't accept parameters directly. There's no syntax like `/run-skill feedback-synthesizer --format=detailed --min-frequency=10`. But skills can handle variable inputs by reading context from your request and project files.

Understanding skill activation and context. When you ask Claude Code to do something, it: 1. Reads your request 2. Checks loaded skill descriptions for matches 3. If a skill matches, prompts you to load it 4. Loads the SKILL.md instructions into the conversation 5. Processes your request using those instructions plus your original prompt

Your original prompt provides the context: file paths, specific requirements, variations you want. The skill instructions explain how to handle different scenarios.



Skill activation flow: user request matches skill description, triggering SKILL.md to load and guide execution

Techniques for variable inputs:

Conditional logic based on file format. Your skill can instruct Claude to adapt based on what it finds:

Input Handling

****CSV files**:**

- Identify which column contains feedback text (usually "comment", "feedback", "response", or "description")
- Parse each row as one feedback item

****Plain text files**:**

- Treat blank lines as separators between feedback items
- Treat blocks of text as individual feedback entries

****JSON files**:**

- Parse structure and identify feedback text fields
- Handle nested objects if feedback is in array of objects

Automatically detect format based on file extension and content structure.

Variable output depth. Users might want different detail levels.

Handle this through context:

Output Depth

****Default**:** Generate standard theme report (10–15 themes, 3–5 quotes each)

****If user mentions "detailed" or "comprehensive"**:**

- Include all themes with 3+ mentions (not just 5+)
- Provide 5–7 quotes per theme
- Add methodology appendix

****If user mentions "executive" or "summary"**:**

- Limit to top 5 themes only
- 2–3 quotes per theme
- Focus on actionable insights

****If user mentions "quick" or "brief"**:**

- Top 3 themes only
- 1 representative quote each
- One paragraph summary

Handling multiple input files. Skill can synthesize across multiple sources:

Multiple Input Sources

When user provides multiple files:

1. Read all files completely first
2. Note source of each feedback item (file name)
3. Synthesize themes across all sources (unified analysis)
4. Optionally note if themes cluster in specific sources

Example: "Analyze feedback in `data/nps-responses.csv` and `data/support-tickets.csv`"

→ Synthesize together, note if themes differ between NPS and support channels

Environment-specific behavior. Skills can adapt based on what exists in the project:

Customization via Project Files

****If `.claude/feedback-categories.md` exists**:**

- Use predefined categories from that file

****If `docs/personas/` directory exists**:**

- Reference personas when identifying user segments in feedback

****If `CHANGELOG.md` exists**:**

- Check recent changes to contextualize feedback (users might be responding to recent feature launches)

****If no customization files exist**:**

- Use emergent theme identification (no predefined categories)

Example: Release notes skill handling internal vs. external versions. One skill, two output formats:

Output Variants

****Customer-facing release notes** (default):**

- Product language, no technical jargon
- Focus on benefits and new capabilities
- Omit internal refactoring and technical debt work

****Internal release notes**:**

If user mentions "internal" or "engineering changelog":

- Include technical changes and refactoring
- Reference pull requests and Jira tickets
- Use technical terminology
- Include deployment notes and breaking changes

****Detection**:** Look for keywords "internal", "engineering", "technical" in user request to determine which format to use.

Limitations of context-based parameterization. This isn't true parameterization. You can't guarantee precise behavior without explicit parameters. Skills interpret context based on natural language, which can be ambiguous.

Bad example (too implicit): User says "Make it detailed" but skill doesn't clearly define what "detailed" means. This might produce unpredictable results.

Good example (explicit in skill instructions): Skill defines specific triggers: “If user request includes ‘detailed’ or ‘comprehensive’, then [specific behavior].”

Document these context patterns in your SKILL.md so users know how to trigger different behaviors:

`## Usage Variations`

`**Standard invocation**`: "Analyze feedback in [file]"

→ Produces standard theme report

`**Detailed analysis**`: "Analyze feedback in [file] and create a comprehensive report"

→ Includes all themes 3+ mentions, extended quotes, methodology

`**Executive summary**`: "Analyze feedback in [file] and create an executive summary"

→ Top 5 themes only, brief format

`**Multi-source synthesis**`: "Analyze feedback in [file1] and [file2]"

→ Synthesizes across sources, notes source-specific patterns

When you need actual parameters. If your workflow truly requires precise parameters (numeric thresholds, boolean flags, complex configuration), skills might not be the right approach. Consider: - Building a script that Claude Code can call (store in `.claude/scripts/`) - Using MCP server for more structured tool interfaces (Chapter 10) - Accepting that some workflows need explicit conversation rather than automated skills

Skills excel at consistent workflows with natural language variation, not at workflows requiring precise programmatic control.

9.5 Tracking Whether Skills Save You Time

Building skills takes time. Using skills saves time. Track whether the investment pays off.

Time saved per invocation. Measure baseline (manual process)

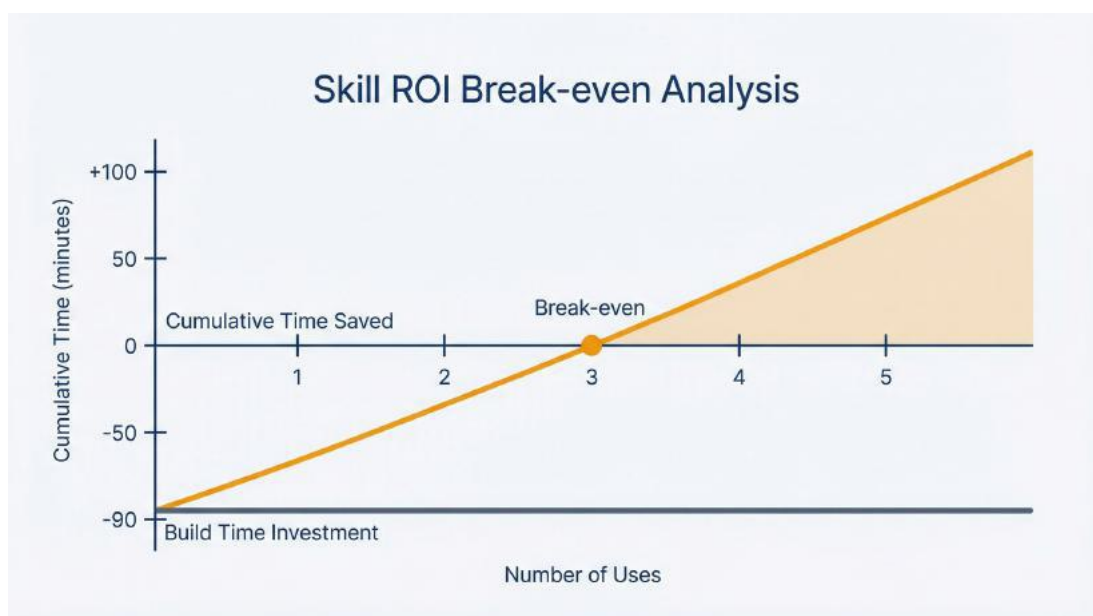
vs. skill execution time:

Workflow	Manual Time	Skill Time	Savings per Use
Feedback synthesis	45 minutes	10 minutes	35 minutes
Release notes	30 minutes	8 minutes	22 minutes
Competitive analysis	60 minutes	15 minutes	45 minutes
PRD audit	25 minutes	5 minutes	20 minutes
User story expansion	40 minutes	12 minutes	28 minutes

Break-even calculation: How many uses to recover skill building time?

Feedback synthesizer: - Build time: 90 minutes - Time saved per use: 35 minutes - Break-even: $90 \div 35 = 2.6$ uses → pays back on 3rd use

If you do quarterly feedback analysis (4x/year), this skill pays back in 9 months and saves 2+ hours annually thereafter.



Skill ROI break-even: initial time investment recovered after approximately 3 uses, with cumulative savings growing with each

subsequent use

Consistency improvement metrics. Skills don't just save time. They improve output consistency. Hard to measure quantitatively, but track qualitatively:

- Before skill: Theme categorization varied each quarter, made year-over-year comparison difficult
- After skill: Consistent categorization scheme, comparable trends across quarters
- Before skill: Release notes tone inconsistent (sometimes technical, sometimes product-focused)
- After skill: Consistent customer-facing voice every release

Knowledge capture value. Skills encode institutional knowledge. When you leave a project or new PMs join, skills transfer process knowledge that would otherwise be lost or require re-learning.

Measure this indirectly:

- Onboarding time for new PM using skill library vs. learning processes from scratch
- Turnover resilience: key workflows don't break when people leave
- Process documentation that's executable, not just written

When a skill isn't worth maintaining. Deprecate skills when:

Low usage (used fewer than 2x in past 6 months):

- Process doesn't repeat as frequently as expected
- Team stopped using it. Investigate why: bad UX? Better alternative?

Marginal time savings (<10 minutes per use):

- Manual process is fast enough; skill overhead not worth it

High maintenance burden (requires updates every time you use it):

- Inputs change too frequently
- Process isn't actually standardizable
- Better to just use Claude Code interactively without skill

Superseded by better solution (external tool now handles this):

- Jira now generates release notes automatically
- Analytics platform added feedback categorization feature

Track these indicators per skill. Annual skill audit: review usage, time savings, and maintenance burden. Keep high-value skills, deprecate low-value ones.

Simple ROI formula:

Annual time saved = (Time saved per use × Uses per year) – (Initial build time ÷ years of use) – (Annual maintenance time)

Feedback synthesizer example: - Time saved per use: 35 minutes - Uses per year: 4 (quarterly) - Total saved: $35 \times 4 = 140$ minutes - Build time amortized (assuming 3-year use): $90 \div 3 = 30$ minutes/year - Maintenance (quarterly review): 15 minutes/year - Net annual savings: $140 - 30 - 15 = 95$ minutes (1.6 hours)

Over 3 years: 4.8 hours saved for 90 minutes initial + 45 minutes total maintenance = 3× return on time invested.

The compounding value. Skills aren't just time savers. They're process improvements that compound: - Consistent methodology → better decision-making - Encoded knowledge → reduced reliance on memory - Team standardization → easier collaboration - Quality baseline → higher output floor even on bad days

These benefits are harder to quantify but often exceed the direct time savings.

You now have five starter skills addressing common PM workflows, understand how to build custom skills from scratch, know how to maintain and evolve your skill library, understand how skills handle

variable inputs through context, and can measure whether your skills deliver ROI. Your PM processes are becoming executable workflows that run consistently without cognitive overhead.

Chapter 10 expands beyond the file system to integrate with your PM stack: connecting Claude Code to Jira, Slack, Figma, and databases via MCP to eliminate export-import cycles.

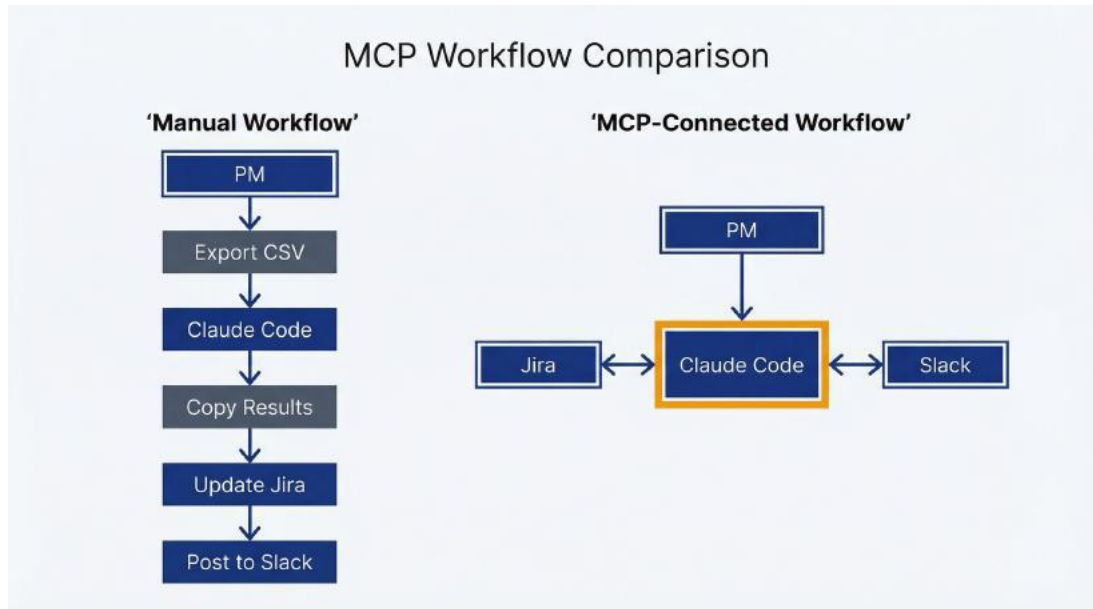
10 MCP: Connecting to Your PM Stack

You've built a skill that synthesizes customer feedback from CSV files. It works. But the workflow still involves manual steps: export tickets from Jira to CSV, run the skill, copy insights back to Jira as a comment, post summary to Slack. Each export-copy-paste cycle introduces friction, delay, and the risk of working with stale data. You're using Claude Code effectively, but you're still shuttling data between systems manually.

MCP eliminates these handoffs. Model Context Protocol connects Claude Code directly to external systems: Jira, Slack, Figma, databases, analytics platforms. Instead of exporting and importing, Claude Code queries your tools in place, analyzes the data, and writes results back where they belong. The feedback synthesis workflow becomes: "Query Jira for last month's support tickets tagged 'checkout', synthesize themes, post summary to #product-feedback." One prompt, no exports, results land in both systems.

This chapter explains what MCP is and why it matters for PM workflows, shows you how to configure connections to common PM tools, provides practical integration examples for Jira, Slack, Figma, and databases, and addresses the real limitations and cost implications. By the end, you'll understand when MCP integration adds value and when the export-import cycle is actually simpler.

10.1 How MCP Eliminates Export-Import Cycles



MCP eliminates the manual export-import cycle by connecting Claude Code directly to external systems

Model Context Protocol is a standard for connecting AI agents to external systems. Anthropic developed MCP as an open protocol (think USB-C for AI integrations). Instead of building custom connectors for every tool combination, developers build MCP servers that expose their systems' capabilities. Once configured, Claude Code can use those capabilities during any session.

For PMs, this means Claude Code can read Jira tickets, post to Slack channels, extract Figma design specs, or query databases without you exporting data manually. The integration happens transparently during your conversation with Claude. You describe what you want, Claude decides which tools to use, executes the operations, and shows you the results.

How MCP servers work: An MCP server is a small program that runs alongside Claude Code and exposes specific tools. When you configure a Jira MCP server, for example, it provides tools like “search_jira_issues”, “get_issue_details”, and “add_comment”. During your Claude Code session, when you ask about Jira data, Claude uses those tools automatically. You don't call the tools directly. You describe your goal, and Claude determines which MCP tools help accomplish it.

MCP servers come in two types. **Local stdio servers** run on your machine as separate processes and communicate with Claude Code through standard input/output. These work well for local file systems, databases on your network, or cloud services you authenticate to via API keys. **Remote HTTP servers** run on external infrastructure and communicate over HTTPS. Some SaaS providers offer hosted MCP servers for their platforms.

Authentication patterns: MCP servers authenticate to external systems using credentials you provide. Most common pattern: API keys or tokens stored as environment variables. When you configure a Jira MCP server, you provide your Jira instance URL and an API token. The MCP server uses these to make requests on your behalf. The server operates within your permission boundaries. If your API token has read-only access to Jira, the MCP server has read-only access. MCP cannot escalate permissions.

Some MCP servers support OAuth 2.1 for more secure authorization flows. Claude Code provides an `/mcp` command to help complete OAuth authentication when required.

Security considerations: You're giving Claude Code the ability to read from and write to external systems using your credentials. This requires trust and care:

- Store credentials as environment variables, never hard-coded in configuration files
- Use API tokens with minimal necessary permissions (read-only when possible)
- Never commit credentials to version control
- Audit which MCP servers are configured before running sessions. You might not want database write access active during exploratory codebase work
- MCP operations consume API quotas and rate limits on the external systems

Available MCP servers relevant to PMs: The MCP ecosystem is growing rapidly. As of publication, relevant servers include:

- **Jira and Atlassian tools** (Confluence, Jira Service Desk)
- **Slack** (read channels, post messages, search history)
- **Figma** (access design specs, extract component details)
- **GitHub** (read repo structure, manage issues, pull requests)
- **PostgreSQL, MySQL, and other databases** (read-only queries recommended)
- **Linear, Asana, Monday.com** (project management alternatives to Jira)
- Proprietary servers for internal tools if your organization builds them

Directories like PulseMCP and MCP.so catalog thousands of available servers. Most PM workflows require only 3-5 servers.

When MCP adds value: Integrations make sense when you repeat a workflow that involves multiple systems. Generating release notes from git history plus Jira tickets. Synthesizing feedback from Zendesk plus Slack mentions. Creating user stories from Figma designs plus existing code patterns. If you do the workflow once, manual export-import is faster than configuring MCP. If you do it monthly or quarterly, MCP pays back on the second run.

When to skip MCP: One-off analysis. Workflows that don't cross system boundaries. Tasks where manually reviewing the data helps you think (exporting to CSV and reading it before synthesis can be valuable context). MCP automation is most useful when the data gathering is mechanical and you want to focus cognitive effort on analysis and decision-making, not data wrangling.

MCP is infrastructure. You configure it once per tool, then invoke capabilities naturally during sessions. The next sections show you how to set up connections to common PM tools and use them practically.

10.2 Configuring Your First Integration

MCP configuration happens in JSON files at three scope levels. Understanding which scope to use matters for team collaboration and credential security.



Three configuration scopes control where MCP settings live and who can access them

Configuration file locations:

Project-scoped (team-shared): `.mcp.json` in your project root directory. Commit this to version control. Everyone using this repository gets the same MCP servers configured. Use for servers the whole team needs: Jira, Figma, Slack channels relevant to this project. Never include credentials in this file. Use environment variable references instead.

Project-local (personal, this project only): `.claude/settings.local.json` in your project directory. Add this to `.gitignore`. Use for MCP servers or credentials specific to your workflow that shouldn't be shared. Example: your personal analytics database connection.

User-wide (personal, all projects): `~/.claude/settings.local.json` in your home directory. MCP servers configured here work across all your projects. Use for tools you use everywhere: your company's Slack workspace, your Jira instance, personal databases.

Configuration file format:

```
{  
  "mcpServers": {  
    "jira": {
```

```

    "command": "npx",
    "args": ["@anthropic-ai/mcp-server-jira"],
    "env": {
      "JIRA_HOST": "https://yourcompany.atlassian.net",
      "JIRA_USER_EMAIL": "${JIRA_EMAIL}",
      "JIRA_API_TOKEN": "${JIRA_TOKEN}"
    }
  }
}

```

Breaking down the configuration:

- "jira" is the server name (you choose this; make it descriptive)
- "command" is the executable that starts the server (npx runs Node packages, python for Python servers, node for JavaScript files)
- "args" are command-line arguments passed to the executable (package name or file path)
- "env" are environment variables the server needs (URLs, API keys, configuration)

Environment variable references: The `${JIRA_TOKEN}` syntax tells Claude Code to substitute the value of the `JIRA_TOKEN` environment variable at runtime. This keeps secrets out of configuration files. Set environment variables in your shell profile (`~/.bashrc`, `~/.zshrc`) or a `.env` file you load before launching Claude Code:

```

export JIRA_EMAIL="your-email@company.com"
export JIRA_TOKEN="your-jira-api-token-here"

```

Using absolute paths: If your MCP server is a local script, use absolute file paths:

```

{
  "mcpServers": {
    "custom-tool": {
      "command": "node",
      "args": ["/home/user/projects/mcp-servers/custom.js"]
    }
  }
}

```

Relative paths like `./mcp-servers/custom.js` can fail between sessions if your working directory changes.

Adding MCP servers via command line:

Claude Code provides a command to configure servers without editing JSON manually:

```
claude mcp add --scope project jira -- npx @anthropic-ai/mcp-server-jira
```

This creates or updates the appropriate configuration file. Scope options:

- `--scope project` → `.mcp.json` (team-shared) - `--scope local` → `.claude/settings.local.json` in project (personal) - `--scope user` → `~/.claude/settings.local.json` in home directory (personal, all projects)

Default is `--scope local` if you don't specify.

Testing connections:

After configuring an MCP server, restart Claude Code (configuration changes require a restart). Launch a session and use the `/mcp` command to list configured servers:

```
> /mcp
```

You'll see which servers are loaded and their status. If a server failed to start, check: - Command and arguments are correct - Environment variables are set and exported - File paths are absolute if using local scripts - Network connectivity if using remote HTTP servers

Troubleshooting common failures:

Server configured but not appearing: You didn't restart Claude Code after configuration changes. Quit completely and relaunch.

Authentication errors: API token is invalid or expired. Verify credentials work independently. Test with `curl` for APIs or log into the web interface. Ensure the token has necessary permissions (read Jira issues, post to Slack channels, etc.).

Silent failures: MCP server started but tools don't work. Check that your API credentials have access to the specific resources you're querying. Example: Jira token might work but not have permission to access a particular project board.

Context window warnings: MCP servers consume context. Each server adds 5,000-10,000 tokens just for tool definitions before your conversation starts. Configuring 5+ servers can consume 40% of your context window. Solution: only configure the servers you actually need for a given project or workflow. Use different Claude Code sessions for different task types (one session for Jira work, another for code investigation).

Path issues on Windows: Use forward slashes even on Windows (C:/Users/Name/server.js) or use WSL (Windows Subsystem for Linux) for more reliable MCP setup.

Setting up your first MCP connection: Jira example walkthrough.

1. Get a Jira API token from your Atlassian account settings (Settings → Security → API tokens)
2. Set environment variables in your shell:

```
export JIRA_HOST="https://yourcompany.atlassian.net"
export JIRA_EMAIL="your-email@company.com"
export JIRA_TOKEN="your-api-token-here"
```

3. Add to your shell profile (~/.bashrc or ~/.zshrc) so they load automatically
4. Add MCP configuration to .mcp.json in your project:

```
{
  "mcpServers": {
    "jira": {
      "command": "npx",
      "args": ["@anthropic-ai/mcp-server-jira"],
      "env": {
        "JIRA_HOST": "${JIRA_HOST}",

```

```

    "JIRA_USER_EMAIL": "${JIRA_EMAIL}",
    "JIRA_API_TOKEN": "${JIRA_TOKEN}"
  }
}
}
}

```

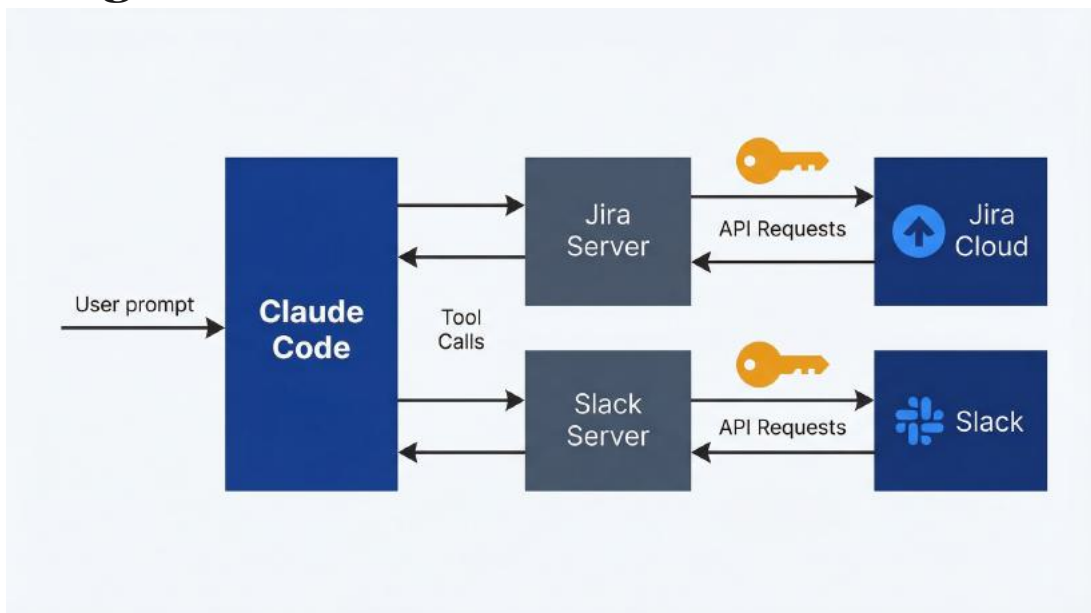
5. Restart Claude Code

6. Test: > Query Jira for all tickets assigned to me in the current sprint

If Claude responds with ticket data, your MCP connection works. If you see “I don’t have access to Jira” or errors, check the troubleshooting steps above.

Time investment for MCP setup: First server: 15-30 minutes (getting API tokens, figuring out configuration syntax, debugging). Second server: 5-10 minutes (you understand the pattern). Third onward: 2-5 minutes each. This setup time pays back when you eliminate manual export-import cycles from repeated workflows.

10.3 Querying and Updating Jira Without Leaving Claude Code



Data flows from your prompt through Claude Code to MCP servers, which authenticate and communicate with external services

Jira MCP integration gives Claude Code the ability to query issues, read ticket details, update fields, add comments, and analyze project data without exporting to CSV. For PMs, this eliminates the biggest context-switching tax in backlog management: bouncing between Jira for data and other tools for analysis.

Available operations with the Jira MCP server: - Search for issues using JQL (Jira Query Language) - Get detailed information about specific tickets - Read comments and activity history - Update issue status, assignee, priority, labels - Add comments to tickets - Create new issues - Read project structure and board configuration

You don't need to know JQL syntax. You describe what you want in natural language, and Claude Code translates to the appropriate queries and API calls.

Prompt patterns for Jira-connected workflows:

Ticket analysis and summarization:

Query Jira for all tickets in the "Ready for Grooming" state. For each ticket, summarize: title, acceptance criteria, linked dependencies, and any open questions in comments. Format as a structured list.

Claude runs the Jira query, reads each ticket's details, analyzes comment threads for unresolved questions, and produces a grooming prep document. Time saved vs. manually clicking through tickets: 20-30 minutes for a typical 10-ticket grooming session.

Sprint velocity and metrics:

Pull all completed tickets from the last three sprints. Calculate velocity (story points per sprint). Identify if velocity is trending up, down, or stable. Note any outlier sprints and check ticket comments for context on why.

Instead of exporting to Excel and calculating manually, Claude extracts

the data, runs the analysis, and provides context. Takes about 2 minutes vs. 15-20 minutes manual.

Feature progress tracking:

Search for all tickets with the epic link “PROD-1234”. Group by status (To Do, In Progress, Done). Show percentage complete and flag any tickets blocked or with unresolved comments.

Useful for weekly stakeholder updates. Eliminates manual status board review.

Bug triage context:

Find all bugs reported in the last 30 days tagged “checkout flow”. Categorize by root cause based on descriptions and engineering comments. Identify patterns.

Helps you spot systemic issues versus isolated bugs. Manual categorization of 50 bug reports takes an hour. With MCP: 5 minutes.

Example: Automated ticket enrichment from codebase investigation.

You’ve investigated a bug using Claude Code in `plan` mode, identified the likely cause in specific files, and want to document findings in the Jira ticket:

I’ve identified that the checkout bug in ticket PROD-5678 is likely caused by null validation missing in `src/services/payment.ts:142`. Add a comment to PROD-5678 with this finding and link to the relevant file in our GitHub repo.

Claude reads the ticket to verify it exists, constructs a properly formatted comment with your technical finding and GitHub link, and posts it to Jira. The engineering team sees your investigation when they pick up the ticket. No separate Slack message or email thread needed.

Example: Sprint review data gathering.

Your sprint review is in an hour. You need to prepare a summary of what shipped:

Pull all tickets completed in sprint “2026-Q1-S3”. Categorize by: new features, improvements, bug fixes. For new features, extract the user-facing description. Format as bullet points suitable for a stakeholder presentation.

Claude queries Jira for the completed sprint, reads each ticket, categorizes based on issue type and labels, extracts customer-facing descriptions (often found in ticket summaries or acceptance criteria), and produces a ready-to-present summary. Total time: 3 minutes. Manual equivalent: 20-30 minutes of clicking through tickets and copy-pasting.

Limitations and manual steps:

Complex JQL queries: Claude handles typical queries well (“all bugs assigned to me”, “tickets in sprint X”), but very complex JQL with custom fields or nested conditions might require you to provide the query explicitly or run it manually first.

Bulk updates with approval: If you’re updating many tickets (changing 50 tickets to a new status), Claude will execute this, but you should review the impact first. Use `plan` mode to preview what would be updated, then switch to `acceptEdits` mode if the changes are correct.

Board configuration and workflow customization: Claude can’t modify Jira board settings, workflow transitions, or custom field configurations. These require admin access through the Jira web interface.

Rate limits: Heavy Jira API usage (querying hundreds of tickets in a single session) might hit Atlassian’s API rate limits. If you see errors about rate limiting, space out queries or reduce scope.

Context consumption: Reading detailed information about 50+ tickets consumes significant tokens (each ticket’s description, comments, and history adds up). For large analyses, ask Claude to query in batches or summarize as it goes rather than reading everything into context first.

When Jira MCP integration is worth it: You do weekly sprint planning, monthly backlog reviews, or quarterly roadmap updates that require pulling ticket data, analyzing patterns, and documenting findings. You frequently triage bugs and want to enrich tickets with technical context from code investigation. You manage cross-functional workflows where Jira is the source of truth but you need to synthesize with other data sources (customer feedback, analytics, code changes).

When to skip it: One-off queries where opening Jira and looking is faster than writing a prompt. Workflows where manually reviewing tickets helps you think (reading descriptions and comments yourself provides context that summaries miss). Teams that don't use Jira heavily or where ticket data quality is poor. Garbage in, garbage out: Claude can't extract insights from vague ticket descriptions.

Jira integration is high-value for PMs who live in backlog management and want to reduce the mechanical overhead of gathering and formatting ticket data for analysis and communication.

10.4 Pulling Context and Posting Updates Automatically

Slack MCP integration connects Claude Code to your team's communication channels. For PMs, this means pulling context from discussions, posting summaries automatically, and eliminating the copy-paste workflow between analysis tools and team updates.

Available operations with Slack MCP: - Read message history from channels you have access to - Post messages to channels (as yourself or a bot, depending on authentication) - Search across channels and DMs for specific keywords or threads - Read thread replies and reactions - Get channel member lists and metadata

Prompt patterns for Slack-connected workflows:

Gathering context from team discussions:

Search the #customer-feedback channel for mentions of "checkout"

in the last 30 days. Summarize the main issues customers reported and how frequently each appears.

Claude searches Slack messages, identifies relevant discussions, categorizes issues, and counts frequency. Useful when customer feedback lives in scattered Slack threads and you need to synthesize before building a feature or prioritizing fixes.

Posting automated summaries:

Query Jira for tickets completed in the last sprint, categorize by type, then post a summary to #product-updates in this format: [template you provide].

This workflow combines Jira MCP and Slack MCP: pull data from Jira, analyze and format, post to Slack. Eliminates the manual step of drafting the update and copying it into Slack. Your team gets consistent sprint summaries without you spending 15 minutes formatting each one.

Cross-referencing discussions and tickets:

Find all Slack threads in #eng-standup where someone mentioned “performance issues” in the last 2 weeks. Check if Jira tickets exist for these issues. Report which issues don’t have tickets yet.

Helps catch things that were discussed but never made it into the backlog. Manual equivalent: scrolling through Slack history and checking Jira one by one. Takes 30+ minutes. With MCP: 2 minutes.

Example: Posting automated summaries.

You’ve configured Slack MCP and Jira MCP. Every two weeks, you run this workflow:

Pull all tickets completed in sprint [current sprint name] from Jira. For each ticket, extract the title and user-facing description. Format as a bulleted list. Post to #product-updates with this intro: “Here’s what shipped in [sprint name]. Thanks to the team for another productive sprint!”

First time you run this, it might take 5 minutes to refine the formatting and wording. After that, you copy-paste the prompt and run it in 30 seconds. Total time savings: 15 minutes per sprint × 26 sprints per year = 6.5 hours annually.

Example: Aggregating feedback from multiple channels.

Your company discusses customer feedback across #customer-success, #sales-insights, and #support-escalations:

Search the following channels for mentions of “data export” or “export feature” in the last 90 days: #customer-success, #sales-insights, #support-escalations. Categorize feedback by: feature requests, bugs, performance complaints. Count frequency and extract representative quotes.

Claude searches all three channels, aggregates results, categorizes, and produces a synthesis report with supporting quotes. Manual version: scrolling through three channels, copy-pasting quotes to a doc, categorizing by hand. Takes hours. With MCP: 3-4 minutes.

Appropriate use and team norms:

Don’t spam channels. Posting automated summaries is useful. Posting every analysis result clutters the channel. Establish norms: what gets auto-posted vs. what you share manually when relevant.

Respect privacy. If Claude searches private channels or DMs you have access to, be mindful about sharing what it finds. Summarizing themes is fine. Quoting someone’s DM without permission is not.

Use bot tokens for automation. If you’re regularly posting summaries or updates, create a Slack bot with a descriptive name (“Product Updates Bot”) rather than posting as yourself. This makes it clear when messages are automated vs. human-written. Bot token setup: Slack Admin → Create New App → Bot Token Scopes → Install to Workspace.

Verify before posting. Use plan mode to generate the message first, review it, then switch to acceptEdits mode to actually post. Automated

posts with typos or incorrect data are embarrassing and erode trust.

Limitations:

Search across DMs is limited. Slack API restricts searching other people's DMs even if you're an admin. Claude can only search channels you're a member of and your own DMs.

Historical message limits. Free Slack workspaces have 90-day message history limits. MCP can only search within available history. Paid workspaces have unlimited history.

Formatting complexity. Slack uses a specific markdown variant for formatting (mrkdwn). Complex formatting with tables, nested lists, or custom layouts might not render as expected. Keep automated posts simple: bullets, bold, links.

Rate limits. Heavy Slack API usage (posting many messages quickly, searching extensively) can hit rate limits. For bulk operations, space them out.

Context consumption. Pulling long message histories or many channel searches consumes tokens quickly. Be specific about date ranges and keywords to minimize data pulled into context.

When Slack MCP integration is worth it: You regularly synthesize discussions from multiple channels for roadmap planning or stakeholder updates. You run recurring workflows (sprint summaries, weekly metrics posts) that involve posting formatted updates to Slack. Your team uses Slack as the primary communication tool and important context lives in message threads.

When to skip it: Your team uses Slack lightly (most communication is email, meetings, or other tools). You rarely need to synthesize Slack conversations. Occasional manual review is sufficient. Automated posting feels impersonal for your team culture.

Slack integration shines when you're eliminating repetitive posting tasks or when valuable context is scattered across multiple channels and manual search is tedious.

10.5 Extracting Design Specs Programmatically

Figma MCP integration gives Claude Code access to design files, component specifications, and design system tokens. For PMs, this eliminates the back-and-forth with designers when writing specs, creating user stories, or verifying implementation matches design.

Available operations with Figma MCP: - Access design file hierarchy and frame structure - Read component properties, variants, and instances - Extract text styles, color tokens, and spacing values - Read auto-layout properties and constraints - Download design assets (images, icons) - Compare design versions

Prompt patterns for design-connected workflows:

Extracting design specifications:

Open the Figma file for “Customer Dashboard Redesign”. List all top-level frames, the components used in each, and key spacing/layout properties. Format as a structured spec document.

Instead of scheduling a meeting with your designer to walk through the file or manually annotating screenshots, Claude reads the Figma file structure and generates a technical spec. Useful when handing off to engineering or creating user stories.

Design-to-user-stories workflow:

Review the Figma file “Mobile Onboarding Flow”. For each screen, identify the user actions, form fields, and navigation elements. Generate user stories in the format: “As a [user], I want to [action] so that [benefit].” Include acceptance criteria based on the design specs.

Claude analyzes the Figma frames, interprets the UI elements as user interactions, and drafts user stories with design-informed acceptance criteria. You review and refine, but the mechanical work of translating designs to stories is automated. Time saved: 60-90 minutes for a 5-screen flow.

Example: Verifying implementation against specs.

Engineering claims they've implemented the new dashboard. You want to verify it matches the design:

Compare the implemented dashboard at [staging URL] to the Figma design in "Dashboard-Final-v3". Identify any discrepancies in: spacing, color usage, component placement, typography.

This requires combining Figma MCP with screenshot analysis or HTML inspection. Claude reads the Figma spec, examines the implemented version, and reports differences. You can then decide if discrepancies are acceptable or need fixing.

Example: Extracting design documentation.

Your design system lives in Figma but engineering needs a written spec:

Extract all color tokens from the "Design System" Figma file. List token names, hex values, and usage notes (if present in component descriptions). Format as a markdown table.

Claude reads the design file, identifies color definitions, and produces a reference document. Engineering can now reference colors without opening Figma. Useful for onboarding new engineers or maintaining documentation.

Limitations of programmatic design access:

Design intent isn't always explicit. Figma files contain visual properties (spacing, colors, sizes) but not always the reasoning behind design decisions. Claude can tell you a button is 48px tall with 16px padding, but it can't tell you *why* the designer chose those values unless it's documented in comments or descriptions.

Complex interactions require designer explanation. Animations, micro-interactions, conditional states may not be fully specified in static Figma frames. Claude can describe what's visible in the design, but interactive behaviors often need designer clarification.

Version and branching complexity. Large Figma files with multiple branches or version history can be hard to navigate programmatically. Be specific about which version or branch you're referencing.

Authentication and access control. Figma MCP requires a Figma access token with appropriate permissions. If the file you're trying to access is in a different workspace or you don't have view access, the MCP server will fail silently.

Asset export limitations. While Figma MCP can download assets, complex vector graphics or design elements might not export cleanly for all use cases. For production asset handoff, designers should still export assets with proper settings.

When Figma MCP integration is worth it: You frequently translate designs into user stories or specs. Your team uses Figma as the source of truth for product specs and you need to reference design details while writing documentation. You're managing a design system and need to keep engineering documentation synchronized with Figma definitions.

When to skip it: Your team doesn't use Figma or design files are rarely referenced in PM work. You prefer collaborative design review meetings where you walk through the file together (automation removes valuable discussion). Design files change frequently and programmatic extraction would be outdated quickly.

Figma integration is valuable when design specs are stable enough to reference programmatically and you have clear workflows (design → user stories, design → implementation verification) that benefit from automated extraction.

10.6 Pulling Metrics Without Writing SQL

Connecting Claude Code directly to databases or analytics platforms lets you pull metrics, query customer data, and analyze trends without exporting to CSV or building custom dashboards. For PMs who frequently need "quick data pulls" to inform decisions, this eliminates a major bottleneck.

Available database MCP servers:

- PostgreSQL
- MySQL / MariaDB
- SQLite
- MongoDB
- Redis (for cached or session data)
- Cloud data warehouses (Snowflake, BigQuery via custom MCP servers)

Security requirements and access controls:

Use read-only database replicas. Never connect Claude Code to production write databases. If your database crashes, you don't want it to be because Claude executed a query that locked tables or consumed resources. Use a read replica or analytics database.

Principle of least privilege. Create a database user with read-only permissions scoped to only the tables and schemas needed for analysis. Don't use admin credentials.

Network access. If your database is behind a VPN or firewall, Claude Code needs network access. For cloud databases, whitelist your IP or use SSH tunneling.

Data privacy. If you're querying customer data, be mindful of PII (personally identifiable information). Aggregate and anonymize in your queries when possible. Don't ask Claude to pull customer email lists or personal information unless you have a legitimate need and appropriate data handling practices.

Connecting to a read-only database replica:

Example configuration for PostgreSQL:

```
{  
  "mcpServers": {  
    "analytics-db": {  
      "command": "npx",  
      "args": ["@modelcontextprotocol/server-postgres"],  
    },  
  },  
}
```

```
"env": {  
  "DATABASE_URL": "${ANALYTICS_DB_URL}"  
}  
}
```

Set the environment variable:

```
export ANALYTICS_DB_URL="postgresql://readonly_user:password@analytics-replica.company.com:5432/analytics"
```

This connects to a read-only user account on your analytics replica database.

Query generation and execution:

You don't need to write SQL. Describe what you want in natural language, and Claude generates the query, executes it via the MCP server, and analyzes results:

Query the analytics database for the number of new user signups per week for the last 12 weeks. Show the trend (increasing, decreasing, or stable).

Claude writes the SQL query, executes it, retrieves the results, and calculates the trend. You see both the query (for transparency) and the analysis.

Example: Pulling metrics for analysis.

You're preparing a quarterly business review and need current product metrics:

From the analytics database, pull the following metrics for Q4 2025:

- Total active users (monthly)
- Average session duration
- Feature adoption rate for "export to PDF"
- Conversion rate from free to paid

Format as a summary table with month-over-month changes.

Claude generates the necessary SQL queries, executes them, aggregates results, calculates changes, and produces a formatted table. You copy this into your QBR deck. Time saved vs. writing SQL queries manually or asking analytics team: 30-45 minutes.

Example: Customer cohort analysis.

You want to understand retention by signup cohort:

Query the users table for all customers who signed up in January 2025. Calculate what percentage are still active (logged in within last 30 days) as of today. Repeat for February, March, April, May, June cohorts. Show as a cohort retention table.

Claude writes cohort analysis SQL, executes it, and formats results. This type of query is tedious to write manually but straightforward to describe.

When to use vs. dedicated analytics tools:

Use Claude Code + database MCP when:

- You need ad-hoc analysis that isn't in your existing dashboards
- The question is exploratory and you'll iterate on the query
- You want to combine database metrics with other data sources (Jira tickets, Slack discussions, customer feedback)
- Your analytics team is backlogged and you need an answer today

Use dedicated analytics tools (Looker, Tableau, Metabase) when:

- The metric is tracked regularly and should be a dashboard
- Multiple stakeholders need access to the same view
- You need complex visualizations (graphs, charts)
- The query runs slowly and should be optimized by analytics engineers

Database MCP is for PM agility and ad-hoc questions. It's not a replacement for proper analytics infrastructure. It's a complement for

when you need answers faster than the dashboard team can build them.

Token costs for database queries:

Database queries themselves are fast and cheap (the MCP server executes them, Claude doesn't process the SQL byte-by-byte). But the results consume tokens when loaded into context. A query returning 1,000 rows with 10 columns might be 20,000-50,000 tokens depending on content density.

Optimization: Ask Claude to aggregate or filter in the query.

Instead of:

Pull all user records from the last 90 days and analyze churn patterns.

Which might return 100,000 rows and exhaust your context window, use:

Write a SQL query to analyze churn patterns for users from the last 90 days. Aggregate by week and return only the weekly churn counts and percentages.

This keeps the result set small (13 rows for 13 weeks), consumes minimal tokens, and still provides the analysis you need.

Limitations:

Performance and timeouts. Long-running queries (30+ seconds) might timeout. If your query hits a large table without indexes, it can fail. Keep queries focused and let database query optimization principles guide you.

Data freshness. If you're querying a replica that syncs hourly, your data might be an hour stale. For real-time metrics, you need a live connection or a more frequently synced replica.

Complex joins and schema knowledge. Claude can generate SQL for common patterns, but very complex joins across many tables, or queries requiring deep schema knowledge, might produce incorrect SQL. Review queries before assuming results are accurate, especially early in

your usage.

Schema changes. If your database schema changes (tables renamed, columns added/removed), Claude’s queries might break. You’ll need to update your descriptions or provide current schema information.

Database integration is powerful for PMs who need quick data access and are comfortable reviewing SQL queries for correctness. It’s not a substitute for proper analytics engineering, but it fills the gap between “I need an answer now” and “let’s build a dashboard.”

You now understand what MCP is, how to configure connections to common PM tools (Jira, Slack, Figma, databases), and when integration adds value versus when manual workflows are simpler. MCP eliminates export-import cycles for repeated workflows, but introduces configuration complexity and context consumption trade-offs.

Chapter 11 covers subagents: how Claude Code spawns specialized instances for parallel work or focused tasks, and when this capability matters for PM workflows.

11 Subagents for Specialized Tasks

You're in a Claude Code session investigating three competitors, synthesizing 500 customer feedback items, and analyzing product metrics to inform quarterly planning. You could do this sequentially: spend 15 minutes on Competitor A, then B, then C, then the feedback, then the metrics. Total time: 90 minutes. Or you could spawn five subagents that execute these tasks in parallel while you monitor progress. Total time: 15 minutes.

The cost difference is significant. Sequential execution: approximately 20,000 tokens (\$0.60). Parallel subagents: approximately 160,000 tokens (\$4.80). You're paying 8× more in tokens to buy 6× faster execution. Whether that's worthwhile depends on whether those 75 minutes of saved time are worth \$4.20 to you and your organization.

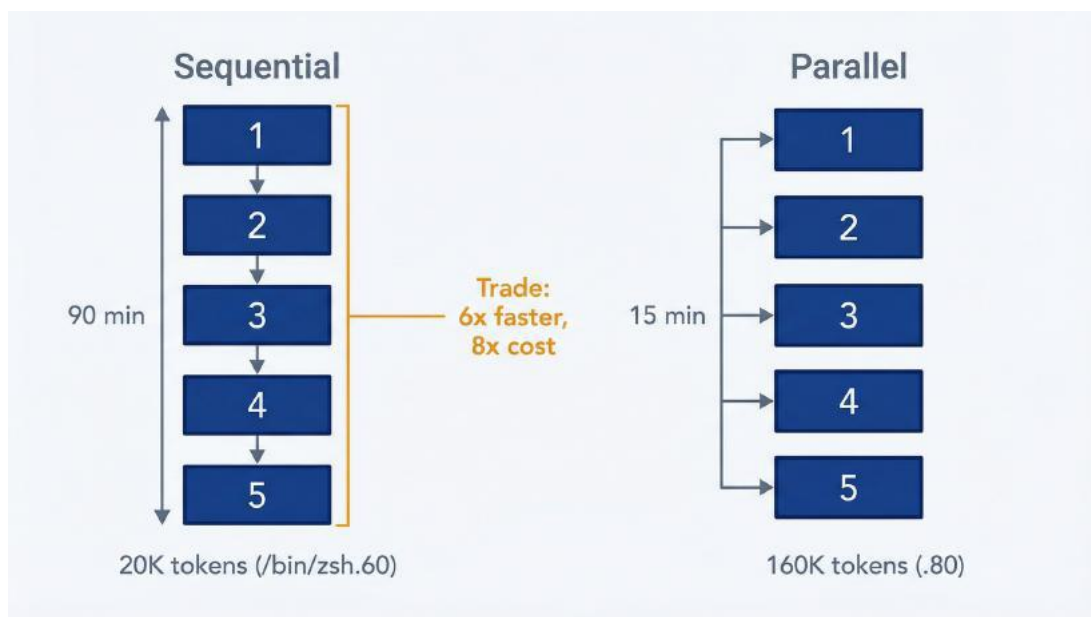


Figure 11.2: Cost-Time Tradeoff – Use this to evaluate whether parallel subagents are worth the token premium. Sequential execution: ~20K tokens (\$0.60), ~90 minutes. Parallel subagents: ~160K tokens (\$4.80), ~15 minutes. The 8× cost increase buys 6× time savings—worth it when 75 minutes saved exceeds \$4.20 in value.

Subagents are spawned Claude instances that execute tasks independently with isolated context. They enable parallelism, specialization, and delegation. They also multiply token consumption and introduce debugging complexity. This chapter explains what subagents are, when they're worth the cost, and how to orchestrate them without accidentally burning through your API budget.

11.1 How Subagents Enable Parallel Work

Subagents are separate Claude instances launched by your main Claude Code session to handle specific tasks. Unlike sequential prompts where one conversation handles everything, each subagent maintains its own context window, executes independently, and returns results to the main agent that orchestrates the work.

How subagents differ from sequential prompts. In a standard Claude Code session, you're having a conversation. Each prompt builds on previous context. Claude remembers what you discussed, what files were read, what decisions were made. The conversation state accumulates (helpful for iterative work), but everything happens sequentially.

Subagents break this pattern. When you (or Claude Code) spawn a subagent, it starts with a clean context. No memory of previous discussion. You define a specific task, the subagent executes it in isolation, completes the work, and reports back. Then it terminates. The main conversation never saw the intermediate steps, just the final result.

This isolation creates two benefits: **parallelism** and **focus**. Five subagents can work simultaneously on five independent tasks. Each subagent focuses only on its assigned work without being distracted by the main conversation's accumulated context. The downside: each subagent reads files and maintains context independently, multiplying token consumption.

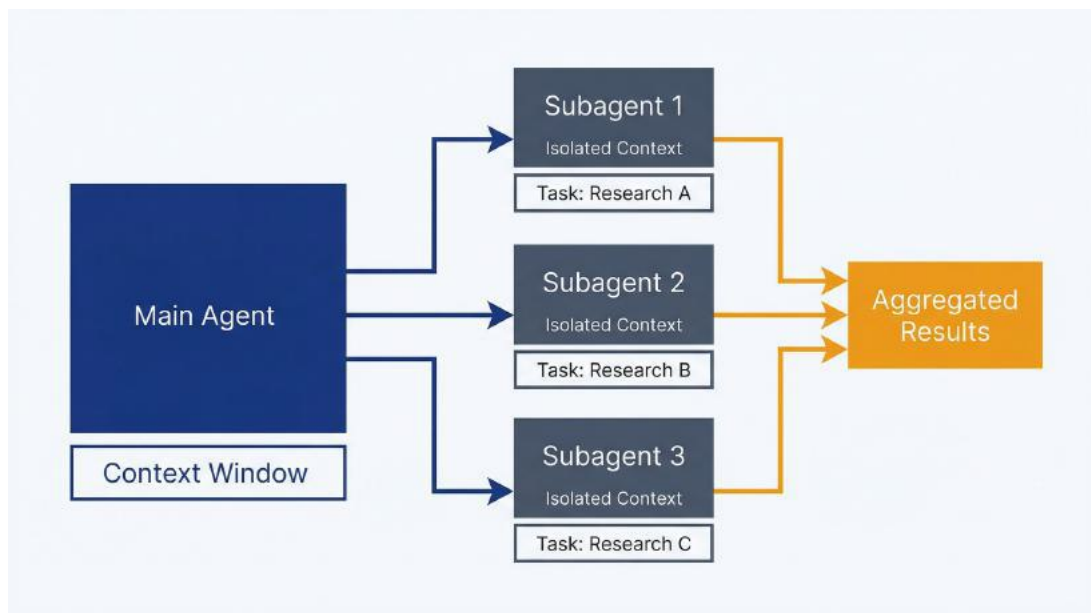


Figure 11.1: Subagent Architecture – Use this to understand how subagents maintain isolated contexts while executing tasks independently.

When subagents add value. Subagents make sense when:

Time savings exceed cost premium. If parallel execution saves an hour and costs an extra \$5 in tokens, calculate your hourly rate. If it's above \$5, subagents have positive ROI. If it's below, sequential prompts are more economical.

Tasks are truly independent. Analyzing four competitors doesn't require sequential ordering. Each analysis is standalone. Competitor A's pricing doesn't inform how you research Competitor B. Perfect for parallelism. Contrast with implementing a feature: you need to understand existing architecture before writing new code. Sequential dependencies mean subagents don't help.

Context isolation prevents pollution. Deep codebase investigation accumulates thousands of lines of context. If you need Claude Code to write a release note afterward, that context is baggage. Delegating the investigation to a subagent keeps the main conversation clean for the writing task.

Specialization improves output. A subagent configured with read-

only tools and a system prompt emphasizing caution makes a better code reviewer than the main agent with full write access. Constraints create focus.

Cost implications of subagent usage. This is the critical factor. Every subagent maintains its own context window. If you're analyzing a codebase, each subagent reads the relevant files independently. Five subagents reviewing different aspects of the same 50,000-token codebase means 250,000 tokens of input just for file reads.

Documented cases show token consumption increasing 50× when work is delegated to subagents instead of handled sequentially. One extreme example: a multi-agent session consumed 887,000 tokens per minute over 2.5 hours with 49 specialized agents (estimated cost \$8,000-\$15,000 for a single session).

You won't hit that extreme doing PM work, but you can easily turn a \$0.50 sequential task into a \$5 parallel task without realizing it. Track costs with `/cost` during sessions. Set spend alerts. Calculate ROI before spawning subagents.

11.2 Four Patterns for Common PM Workflows

Four patterns cover most PM use cases: parallel research, specialized reviewers, sequential delegation, and format conversion. Each serves different needs. Each has different cost-benefit profiles.

11.2.1 Pattern 1: Parallel Research

Use case: You need to gather information from multiple independent sources and synthesize it. Competitive analysis, market research, multi-source feedback synthesis.

How it works: The main agent divides the work, spawns one subagent per source, each subagent researches its target independently, then the main agent aggregates results into a unified analysis.

Example: Competitive Analysis

You're analyzing four competitors. Sequential approach: 45 minutes (10 minutes per competitor plus 5 minutes for synthesis). Parallel approach: 12 minutes (all four analyses happen simultaneously, plus synthesis).

Prompt: Parallel Competitive Research

I need competitive analysis for Competitor A, Competitor B, Competitor C, and Competitor D.

For each competitor, research and document:

- Core product offerings and key features
- Pricing model and tiers
- Target market positioning
- Recent product launches or updates (last 6 months)

Create individual profiles as `research/competitors/[name]-profile.md` for each, then synthesize a comparison matrix highlighting where we have feature gaps and pricing advantages.

Claude Code spawns four research subagents, one per competitor. Each conducts web research, structures findings consistently, writes a profile. All four execute in parallel. When complete, the main agent reads the four profiles and generates the comparison matrix.

Time saved: 33 minutes. Token cost increase: approximately $5\times$ (four subagents each reading and processing information vs. one sequential process). ROI depends on whether saving half an hour is worth the extra \$2-4 in API costs.

Example: Customer Feedback Synthesis

You have 500 survey responses to analyze for themes. One subagent processing all 500 takes 20 minutes. Five subagents each processing 100 responses takes 5 minutes.

Prompt: Parallel Feedback Analysis

Analyze the customer feedback in `data/q1-2026-survey-responses.csv` (500 responses).

Divide responses into 5 equal chunks and process in parallel. For each chunk:

- Identify recurring themes
- Tag sentiment
- Extract representative quotes

Then aggregate all findings into a unified theme report with frequency analysis across all 500 responses.

Main agent splits the data, spawns five subagents with different row ranges, each analyzes its chunk, returns findings. Main agent consolidates into a single report with accurate frequency counts across the full dataset.

When parallel research isn't worth it: Small datasets (fewer than 50 items), simple analysis that takes under 10 minutes sequentially, one-off research you won't repeat. The overhead of spawning subagents, waiting for parallelism to complete, and aggregating results only pays off when the task is large enough that time savings matter.

11.2.2 Pattern 2: Specialized Reviewers and Auditors

Use case: You need focused analysis with specific constraints: code review, documentation audit, PRD completeness check, security review.

How it works: You create a subagent with a specialized system prompt defining its role, often with restricted tools (read-only access), and invoke it for focused assessment work.

Example: Documentation Auditor

You want to verify that API documentation matches actual implementation. Create a documentation auditor subagent:

Create `.claude/agents/docs-auditor.md`:

```
name: docs-auditor
description: Reviews product documentation for accuracy against codebase
implementation. Use for documentation quality checks.
tools: Read, Grep, Glob
permissions: plan
---
```

You are a documentation quality specialist. Your task is to verify documentation accuracy against codebase implementation.

When reviewing documentation:

1. Identify all endpoints, methods, or features described in docs
2. Verify each exists in the actual codebase
3. Check that parameters, return types, and behavior match implementation
4. Flag missing documentation for implemented features
5. Flag outdated examples or incorrect information

Provide specific citations: ``docs/api.md:45`` vs ``src/api/routes.ts:123``

Output format:

- Verified items (accurate documentation)
- Discrepancies found (docs don't match code)
- Missing documentation (implemented but undocumented features)
- Recommendations for fixes

Read-only analysis. Do not modify files.

Invoke with:

Review our API documentation in `docs/api/` against the actual implementation in `src/api/`. Report discrepancies and missing documentation.

The docs-auditor subagent loads with read-only tools, focused system prompt, and constraint to only analyze (never modify). It produces a structured audit report. You get higher quality output than asking the main agent to “check if docs are accurate” because the subagent’s entire context is dedicated to this specific task.

Example: PRD Completeness Reviewer

Create a PRD auditor that checks requirements documents against a standard checklist:

Using the PRD auditor subagent, review docs/prds/bulk-import-feature.md for completeness. Check for required sections, measurable success criteria, clear acceptance criteria, and unresolved open questions. Produce an audit report with readiness assessment.

The subagent reads your PRD, evaluates against criteria, produces a scored report. Because it's specialized for this task, you get consistent evaluation every time: same standards, same format, comparable across PRDs.

When specialized reviewers make sense: Repeated review tasks (monthly release reviews, quarterly documentation audits), tasks where constraints improve quality (read-only tools prevent accidental changes), workflows where you want consistent methodology every time.

When they don't: One-off reviews, tasks where flexibility is more valuable than consistency, simple checks that don't justify the setup overhead.

11.2.3 Pattern 3: Sequential Delegation

Use case: Multi-step workflows where each phase needs isolation, but steps must happen in order. Research followed by synthesis, analysis followed by recommendation, investigation followed by documentation.

How it works: Main agent orchestrates a sequence: spawn Subagent A for phase 1, receive results, spawn Subagent B for phase 2 with context from phase 1, aggregate final output.

This isn't parallelism. It's sequential work with context isolation between phases.

Example: The “Three Amigo Agents” Pattern

This pattern comes from Anthropic teams building complex features. Three specialized subagents handle different phases of product

development:

Phase 1: PM Agent reads feature request, writes specification, asks clarifying questions, produces a requirements document. Status: READY_FOR_ARCHITECTURE.

Phase 2: Architect Agent reviews requirements, validates against technical constraints, considers performance and scalability, produces an architectural decision record (ADR). Status: READY_FOR_IMPLEMENTATION.

Phase 3: Implementation Agent builds the feature following the ADR, writes tests, updates documentation. Status: DONE.

Each agent sees only what it needs. The PM agent doesn't get buried in implementation details. The architect doesn't need to know about product positioning. The implementation agent follows a clear technical plan. Context isolation keeps each phase focused.

Result from Anthropic's usage: complex features that previously took weeks now complete in hours.

Adapting for PM workflows: You likely won't implement code, but the pattern applies to PM deliverables:

Phase 1: Research Subagent → Gathers competitive intelligence, customer feedback, and market trends. Produces raw research report.

Phase 2: Analysis Subagent → Reads research report, identifies patterns, prioritizes insights, produces strategic analysis.

Phase 3: Recommendation Subagent → Reads analysis, considers product context and roadmap, produces action plan with prioritized recommendations.

Each phase stays focused. The analysis subagent doesn't re-read all 500 customer feedback items. It reads the research summary. The recommendation subagent doesn't redo the analysis. It starts from strategic insights.

When sequential delegation makes sense: Multi-phase workflows where each phase produces an artifact the next phase consumes, when you want different “mindsets” for different phases (broad research vs. critical analysis vs. action planning), when total sequential time is long enough that context isolation improves quality.

When it doesn’t: Simple workflows where a single prompt handles everything fine, when you’re iterating and need flexibility, when the overhead of three separate subagent invocations exceeds the benefit of isolation.

11.2.4 Pattern 4: Format Conversion and Document Generation

Use case: Transforming structured data or code into product artifacts: release notes from git history, user stories from PRDs, changelog entries from commit messages.

How it works: Subagent reads source material (code, git log, structured data), applies transformation rules, produces formatted output for specific audience.

Example: Release Notes Generator

Using a documentation writer subagent, generate customer-facing release notes from git commits between v2.4.0 and v2.5.0.

Read commit messages and code changes. Filter for user-facing changes only (exclude refactoring, dependency updates, internal improvements). Categorize as Added, Changed, Fixed, or Removed. Write descriptions in product language, not technical jargon. Output to docs/releases/v2.5.0.md following our standard release note format.

The subagent reads git history, filters commits, translates technical changes into customer language, formats as structured release notes. Main conversation stays focused on reviewing and refining the output, not processing 100 commits.

Example: User Story Expansion from Requirements

Create detailed user stories from the feature description in `docs/prds/advanced-filters.md`.

Generate 5-8 user stories covering core workflows, variations, edge cases, and admin configuration. For each story, include acceptance criteria, technical notes referencing relevant codebase files, and edge case considerations. Output to `docs/user-stories/advanced-filters.md`.

Subagent reads the PRD, explores relevant codebase areas for technical context, generates comprehensive user stories. You get artifact generation without cluttering your main conversation with implementation details.

When format conversion subagents make sense: Regular artifact generation (monthly release notes, sprint planning story creation), transformations that benefit from isolation (reading large git logs without polluting main context), when you've defined clear formatting standards and want consistency.

Defining subagent scope and constraints. Effective subagents have clear boundaries. Define:

Input scope: "Read only these files," "Process rows 1-100 of this CSV," "Analyze commits in this date range."

Output format: "Produce markdown table," "Use bullet list with specific sections," "Follow template at `.claude/templates/competitor-profile.md`."

Constraints: "Read-only analysis," "No code execution," "Web search allowed," "Maximum 5-page report."

Vague subagent tasks produce vague results. Specific scope produces usable artifacts.

Aggregating subagent outputs. The main agent's job is synthesis. When five subagents return five competitive analyses, don't just present five separate documents. The main agent should:

1. Read all subagent outputs
2. Identify patterns across analyses
3. Create unified comparison (feature matrix, pricing table)
4. Highlight strategic insights (where we're strong, where we have gaps)
5. Produce single synthesized artifact

Subagents generate raw material. The main agent produces the deliverable.

11.3 Coordinating Multiple Agents Effectively

Simple subagent usage is straightforward: spawn one, get result, continue. Multi-agent workflows require planning.

Planning agent decomposition. Before spawning subagents, define:

What are the independent units of work? Competitive analysis of four companies = four independent units. Customer feedback from three channels = three units. Feature investigation and documentation review = two units.

What are the dependencies? Can all units execute in parallel (competitive research), or does Unit B depend on Unit A completing first (requirements before architecture)?

What's the aggregation strategy? Will the main agent synthesize all results, or will one subagent handle synthesis after others complete?

What's the cost-benefit? Calculate approximate tokens (number of subagents × average context per subagent) vs. time saved. If you're saving 20 minutes for \$10 in extra tokens, is that worthwhile?

Coordination patterns: Sequential, parallel, hierarchical. Three fundamental patterns:

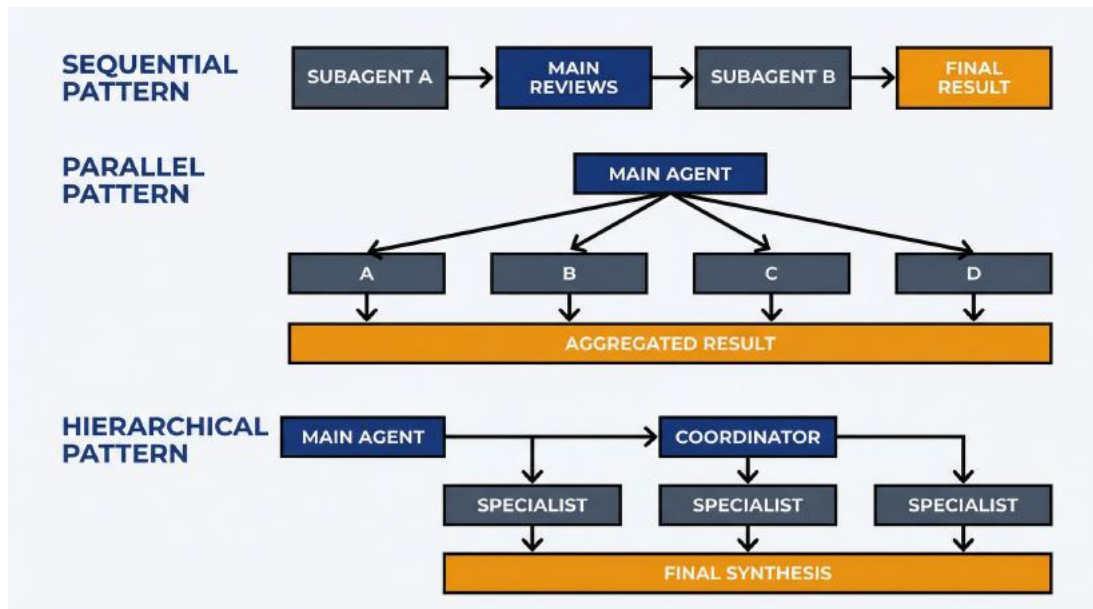


Figure 11.3: Coordination Patterns – Use this to choose the right multi-agent structure. Sequential: chains subagents in order when dependencies exist. Parallel: runs all simultaneously when tasks are independent. Hierarchical: adds coordinator layer for complex orchestration (rarely needed for PM workflows).

Sequential: Subagent A completes → Main agent reviews → Subagent B starts with A's output → Main agent synthesizes final result.

Use when: Dependencies exist, each phase needs isolation, you want checkpoints between phases.

Parallel: Subagents A, B, C, D all start simultaneously → All complete independently → Main agent aggregates results.

Use when: Tasks are independent, time savings justify cost, you can synthesize results effectively.

Hierarchical: Main agent spawns Coordinator subagent → Coordinator spawns specialist subagents (or coordinates sequential invocations) → Coordinator aggregates and returns to main agent.

Use when: Complexity is high enough that orchestration itself benefits from isolation. Rarely needed for PM workflows (adds overhead without clear benefit in most cases).

Example: Quarterly planning workflow with three subagents.

You're preparing Q2 planning. You need market trends analysis, customer insight synthesis, and product metrics review. Three independent research tasks, then strategic synthesis.

Prompt: Quarterly Planning Research

Conduct Q2 planning research in three parallel streams:

Subagent 1 - Market Trends: Research industry trends in [your domain] over the past quarter. Identify emerging patterns, competitive movements, and market signals. Output: `planning/q2-2026/market-trends.md`

Subagent 2 - Customer Insights: Synthesize customer feedback from `data/q1-feedback/` (support tickets, NPS, user interviews). Identify top themes and priorities. Output: `planning/q2-2026/customer-insights.md`

Subagent 3 - Product Metrics: Analyze product usage metrics from `data/q1-metrics.csv`. Identify underperforming areas, growth opportunities, and user behavior patterns. Output: `planning/q2-2026/metrics-analysis.md`

Once all three complete, synthesize into a unified Q2 strategic brief with recommended priorities.

Three subagents execute in parallel. Each completes in 8-12 minutes. Total elapsed time: 12 minutes (vs. 35 minutes sequential). Main agent reads three reports, identifies themes across market trends + customer needs + metrics, produces strategic brief for stakeholder review.

Handling subagent failures. Subagents can fail due to tool access issues, overly vague instructions, or tasks that are harder than expected. When a subagent fails:

Check the error. Did it lack tool permissions? Fix subagent configuration. Was the task too vague? Provide clearer scope. Did it run into actual blockers (missing data files)? Address the root cause.

Re-run with refined instructions. Don't spawn five new subagents. Fix the specific one that failed and retry.

Fall back to sequential. If subagent orchestration is causing more problems than it solves, abandon parallelism and handle the task directly in the main conversation. Some tasks aren't worth the complexity.

Cost management across agents. Multi-agent workflows consume tokens fast. Strategies to control costs:

Set spend alerts. Before launching a 5-subagent workflow, check `/cost` and set a mental threshold. If costs are approaching your limit mid-session, cancel remaining subagents and switch to sequential.

Limit parallelism. Three subagents instead of ten. The time savings curve flattens. Going from 1 to 3 subagents provides significant speedup, but going from 3 to 10 provides diminishing returns while multiplying costs.

Reuse outputs. If all five subagents need the same reference material, create a shared context file they all read rather than having each subagent conduct independent research. Reduces redundant token consumption.

Monitor per-task costs. After completing a multi-agent workflow, note total tokens consumed. Compare to your sequential baseline. If the cost premium isn't justified by time savings, don't use subagents for that workflow again.

11.4 Limitations and When to Avoid Subagents

Subagents are a powerful capability. They're also frequently the wrong choice.

Overhead vs. benefit threshold. Spawning a subagent, waiting for execution, aggregating results all create overhead. For tasks under 10 minutes, sequential execution is faster when you account for orchestration time. For tasks under \$0.50 in tokens, the cost multiplication isn't worth it.

The threshold where subagents make sense: tasks requiring 20+ minutes sequentially, consuming significant tokens already (so the multiplier is on a meaningful base), and delivering clear time value when accelerated.

Context isolation challenges. Subagents start with clean context. This is a feature when you want focus. It's a limitation when context matters.

If you're iterating on a PRD and ask a subagent to "add user stories for the advanced search feature," the subagent doesn't remember your previous discussion about target personas, technical constraints, or product strategy. It starts cold. You need to provide that context explicitly in the prompt, which often takes longer than just continuing the conversation sequentially.

Debugging multi-agent issues. When a sequential prompt fails, you see the thinking, can review intermediate steps, and diagnose the problem. When a subagent fails, you get a final error. No intermediate output, no transparent thinking mode. Debugging requires re-running the subagent with modified instructions and hoping you've fixed the issue.

For critical work where you need control and transparency, sequential prompts with stepwise oversight beat subagent delegation.

Simpler alternatives that often work better. Before spawning subagents, consider:

Sequential prompts with clear phases. "First, analyze competitor pricing. Then, based on that analysis, identify gaps in our pricing strategy." This provides similar phase isolation without subagent overhead.

Skills (Chapter 9). If the workflow repeats, encode it as a skill rather than orchestrating subagents every time. Skills provide consistency without requiring multi-agent complexity.

Chunking within a single conversation. "Process the first 100 feedback items and summarize themes. Then I'll ask you to process the next 100." This maintains context continuity while breaking work into

manageable pieces.

External tools. Some tasks (data analysis at scale, complex report generation) are better handled by purpose-built tools (analytics platforms, BI tools) than by Claude Code subagents. If you find yourself routinely spawning 10 subagents to process data, you might need better tooling, not better orchestration.

The teams that succeed with Claude Code start simple: sequential prompts, skills for repeated workflows, subagents only when measurement proves they're worth the cost. The teams that struggle build elaborate multi-agent systems before writing a single line of code.

Subagents are a tool for specific bottlenecks, not a default approach. Use them when parallelism demonstrably saves time worth more than the token premium. For everything else, simpler is better.

You now understand what subagents are, when they add value versus multiplying costs, four practical patterns for PM workflows, and how to orchestrate multi-agent work without building unnecessary complexity. You also know when to avoid subagents entirely. Simpler sequential approaches often deliver better results with less overhead.

Chapter 12 shifts from automation to collaboration: how to work effectively with engineering teams when both PMs and engineers use Claude Code, share artifacts, and maintain productive handoff protocols.

12 Collaborating with Engineering

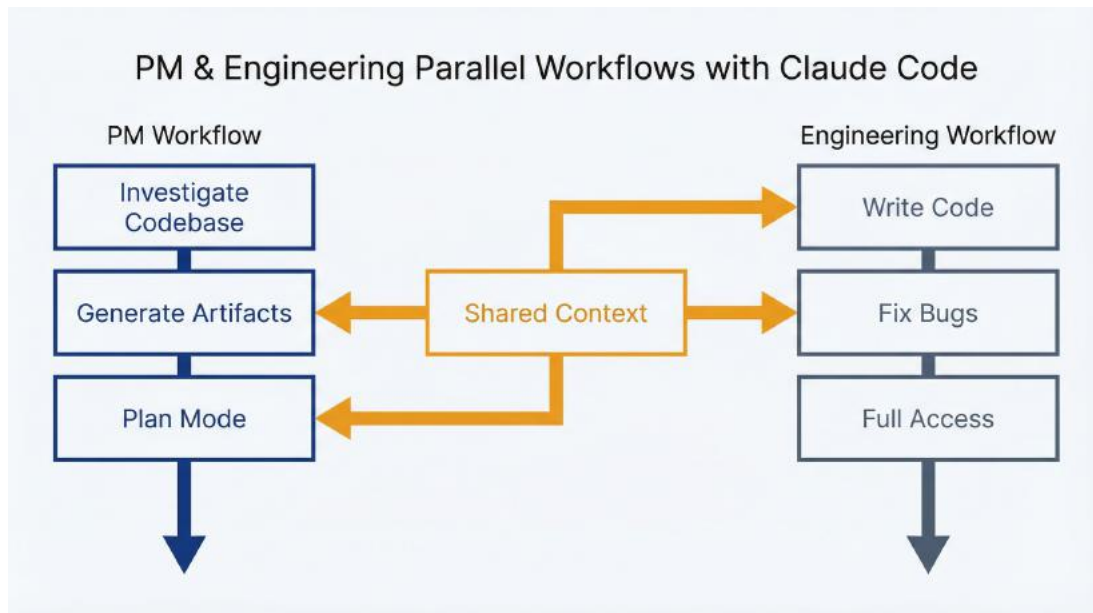
You've been using Claude Code for weeks. Your bug investigations are faster, your feature questions get answered without interrupting sprints, and your PRDs reference actual implementation. Then engineering starts using Claude Code too: same repository, same sessions, same tools. What happens when both roles work in the same AI-augmented workspace?

Done well, this overlap amplifies both roles. PMs investigate faster, engineers get better context in handoffs, and shared artifacts reduce miscommunication. Done poorly, it creates territorial friction, duplicated effort, and conflicting instructions in CLAUDE.md files. This chapter covers the interface between PM and engineering Claude Code usage: what to share, when to hand off, and how to maintain productive boundaries.

12.1 Where PM and Engineering Usage Overlaps

PMs and engineers use Claude Code differently. Understanding these differences prevents stepping on each other's toes.

Complementary usage patterns. Engineers use Claude Code to write and modify code: implementing features, fixing bugs, refactoring systems. Their sessions involve file edits, test runs, and deployment tasks. PMs use Claude Code to investigate and generate artifacts: understanding implementation, synthesizing research, creating documents. Most PM sessions run in plan mode; most engineering sessions don't.



PM and Engineering use Claude Code in parallel workflows. PMs investigate, generate artifacts, and work in plan mode. Engineers write code, fix bugs, and use full access. Shared context in the middle connects both sides.

This natural division means overlap is rare. An engineer modifying `checkout.service.js` doesn't conflict with a PM asking Claude Code to explain how checkout works. The PM reads; the engineer writes. Different tool permissions, different outputs, different workflows.

Where overlap creates value is in shared context. When both roles understand the same codebase through the same tool, handoffs become conversations between people who speak the same language. The PM says "I investigated the discount validation logic in `discount-validator.js:45-78` and found it rejects codes when `cart.hasActivePromotion` is true." The engineer knows exactly where to look and what the PM understood. No translation layer required.

Avoiding territorial conflicts. Conflicts arise when role boundaries blur. A PM who starts editing code (even with good intentions) creates confusion about ownership. An engineer who dismisses PM investigation findings because "you don't understand the code" undermines collaboration.

Clear boundaries prevent friction:

- PMs investigate and explain; engineers validate and implement
- PMs suggest where problems might be; engineers confirm root causes
- PMs create product artifacts (PRDs, release notes); engineers create technical artifacts (ADRs, code comments)
- PMs use plan mode for exploration; engineers use full access for implementation

When you're tempted to cross a boundary (editing a config file, running a database migration, pushing a "simple fix"), stop and hand off instead. The five minutes you save aren't worth the confusion about who owns what.

Shared artifacts and version control. Claude Code generates artifacts that both roles use. Release notes, architecture documentation, feature explanations, investigation summaries: these live in the repository alongside code. Treat them like any other versioned asset.

Files created by Claude Code should follow existing repository conventions. If your team uses docs/ for documentation, Claude Code outputs go there. If commit messages follow a format, maintain it. AI-generated content isn't special; it's just content that needs to follow the rules.

Use git commits to track who created what and when. Claude Code adds a Co-Authored-By: Claude <noreply@anthropic.com> trailer to commits by default. This attribution makes it clear which work involved AI assistance (useful for audits, reviews, and understanding how artifacts evolved).

Communicating about AI-assisted work. When you share investigation findings or generated artifacts, be transparent about the process. "I used Claude Code to investigate this issue and here's what I found" sets appropriate expectations. The reader knows to validate technical details rather than treating them as authoritative.

This transparency works both directions. When engineers use Claude Code to generate documentation or analysis, they should note it too. The goal isn't disclaiming responsibility (you still own the output) but signaling how it was produced so reviewers calibrate their scrutiny

appropriately.

12.2 Transitioning Work Without Losing Context

The PM-engineering boundary isn't a wall. It's a transition zone. Good handoffs make that transition smooth.

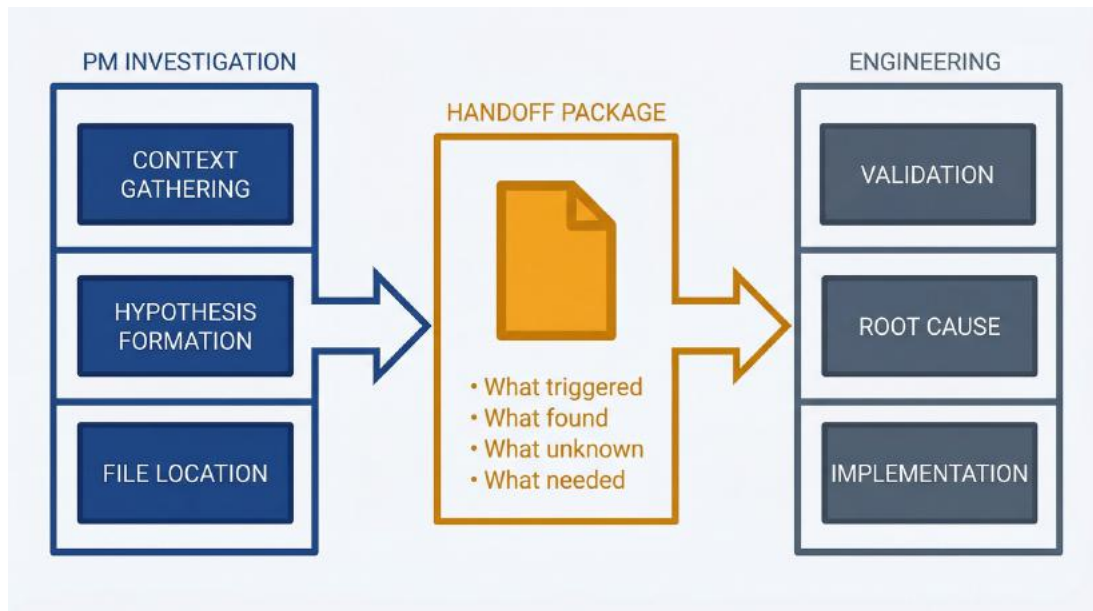
When PM investigation ends and engineering begins. Chapter 4 covered when to stop investigating bugs. The same principles apply to all PM-engineering handoffs. You've gathered enough context when you can explain the flow, have a testable hypothesis, identified relevant files, and know what you don't know.

For feature investigations, stop when you can answer your product question. "How does checkout work?" is answered when you understand the flow well enough to make product decisions. You don't need to understand every function.

For problem reports, stop when you have actionable context for engineering. File paths, suspected code areas, reproduction hypotheses, open questions. Engineering takes it from there.

For artifact generation, stop when you have a draft worth reviewing. Don't iterate indefinitely trying to perfect release notes. Get feedback from engineering on accuracy, then finalize.

The handoff moment is when additional PM investigation yields diminishing returns while engineering investigation would yield accelerating returns. You're good at context gathering and hypothesis formation. They're good at verification and implementation. Transition at the inflection point.



The PM-Engineering handoff workflow. PM Investigation phase includes context gathering, hypothesis formation, and file location. The Handoff Package in the middle documents what triggered the investigation, what was found, what remains unknown, and what is needed from engineering. Engineering then handles validation, root cause analysis, and implementation.

Information packaging for handoff. Structure your handoffs for engineering consumption:

Investigation summaries should include:

- What triggered the investigation (user report, product question, planning need)
- What you investigated (files read, flows traced, questions asked)
- What you found (explanations, hypotheses, relevant code locations)
- What you couldn't determine (runtime behavior, data-dependent logic, configuration questions)
- What you need from engineering (validation, root cause confirmation, implementation)

Artifact drafts should include:

- Source material used (git history, feedback data, competitive research)

- Methodology (what prompts you used, what Claude Code analyzed)
- Open questions (accuracy concerns, missing context, needed validation)
- Requested feedback (technical accuracy, completeness, tone)

Bad handoff: “I think there’s a bug in checkout. Can you take a look?”

Good handoff: “Users report charges without order confirmations. I investigated and found payment processing happens in `payment-processor.js`, with order creation afterward in `order.service.js`. If order creation fails after successful payment, the reconciliation queue in `payment-reconciliation.js` should catch it. Hypothesis: the reconciliation queue isn’t processing, or errors aren’t being handled. Can you verify queue status and recent failures? Customer ID for investigation: [ID].”

The good handoff respects engineering time by providing context that eliminates discovery work.

Using Claude Code output as conversation starters, not conclusions. Your investigation findings are hypotheses, not facts. Claude Code interpreted code based on static analysis. It didn’t run the code, check production data, or consult the original authors.

Frame handoffs accordingly. “I think this might be the issue” rather than “This is the bug.” “The code appears to...” rather than “The code does...” Engineering should validate before acting.

This framing isn’t false humility. It’s accurate epistemic positioning. You investigated thoroughly, formed reasonable conclusions, but you’re working from incomplete information. Engineering has access to runtime data, deployment context, and deep system knowledge you don’t. They might confirm your hypothesis or discover something different. Either outcome is valuable; the wrong outcome is acting on PM investigation as if it were engineering analysis.

Respecting engineering expertise and judgment. You’ve saved engineering hours of discovery work, but that doesn’t make you an engineer. When they disagree with your hypothesis, listen. When they explain why the code doesn’t work the way Claude Code described, learn.

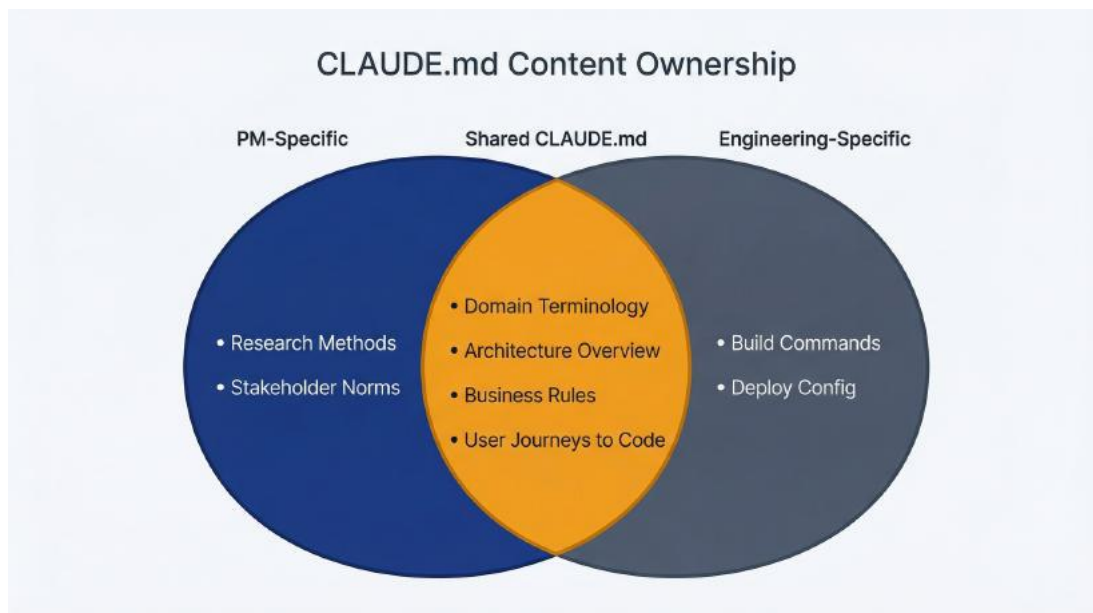
When they prioritize differently than you expected based on implementation complexity you didn't understand, trust their judgment.

The goal of PM codebase investigation isn't to replace engineering. It's to reduce the communication burden between roles. You can have informed conversations without being able to implement solutions. That's the boundary. Stay on your side of it.

12.3 Maintaining Context Both Roles Can Use

CLAUDE.md files persist context across sessions. When multiple people work in the same repository, maintaining that shared context becomes a collaborative responsibility.

Product knowledge that benefits both roles. Some CLAUDE.md content serves everyone:



CLAUDE.md content ownership shown as overlapping circles. PM-specific content includes research methods and stakeholder norms. Engineering-specific content includes build commands and deploy config. The shared center contains domain terminology, architecture overview, business rules, and user journey mappings to code.

Domain terminology prevents misinterpretation for any Claude Code

user. Whether a PM is investigating or an engineer is implementing, knowing that “credit” means internal currency (not payment credit) matters.

Architecture overview helps PMs understand where to look and helps engineers onboard new team members. A brief description of component boundaries and data flow serves both audiences.

Business rules and constraints belong in shared context. “Discount codes can’t combine with promotions” and “Users limited to 5 workspaces on free tier” are product rules that code should respect. Both roles need this context.

User journeys mapped to code help PMs investigate and help engineers understand product impact of changes. The mapping from “checkout flow” to “CheckoutForm.tsx → checkout.service.js → payment-processor.js → order.service.js” serves both.

Content that’s PM-specific (research methodologies, stakeholder communication norms) might belong in a personal `~/claude/CLAUDE.md` or a `docs/pm-workflow.md` file rather than the shared project `CLAUDE.md`.

Maintaining accuracy as a shared responsibility. `CLAUDE.md` content goes stale. Code paths change, terminology evolves, business rules update. Someone needs to maintain accuracy.

Treat `CLAUDE.md` like any documentation. When you notice something wrong, fix it. If the checkout flow now includes a fraud detection step between payment and order creation, update the user journey mapping. If the team renamed “credits” to “coins,” update the terminology. Small fixes during regular work prevent large drift.

For significant changes (new product areas, architectural overhauls, major terminology shifts), create a documentation task. Don’t let `CLAUDE.md` accuracy drift until someone notices Claude Code giving bad advice.

Conflict resolution for `CLAUDE.md` content. When PM and engineering perspectives differ on what `CLAUDE.md` should contain, resolve based on usage patterns.

Favor specificity over comprehensiveness. A focused CLAUDE.md with 50 high-value lines beats a 500-line document covering everything. Include what improves Claude Code's responses; exclude what's nice-to-have but rarely used.

Favor accuracy over aspiration. Document how things actually work, not how they should work. If the code has technical debt you plan to address, don't describe the future state as current. Claude Code reads code, and conflicting CLAUDE.md instructions create confusion.

Favor shared understanding over role-specific detail. Content that only PMs use probably doesn't belong in shared CLAUDE.md. Content that only engineers use might not either. Shared context means content that helps everyone using Claude Code in this repository.

When conflicts persist, try both approaches for a few weeks and see which produces better results. CLAUDE.md isn't sacred. It's iterative documentation that improves through use.

Review cadence and ownership. Quarterly CLAUDE.md review aligns with other documentation maintenance rhythms. One person should own the review: typically the PM who uses Claude Code most frequently or the tech lead who understands architectural context best.

Review checklist:

- Are all file paths still accurate?
- Have any terminology definitions changed?
- Are user journey mappings current with implementation?
- Have business rules changed that need documentation?
- Is anything documented that no longer exists?
- Is anything new that should be documented?

A 30-minute quarterly review prevents the gradual decay that makes CLAUDE.md useless. Treat it like any living documentation: maintained by use, reviewed periodically, updated incrementally.

12.4 Building Skills That Serve the Whole Team

Skills (Chapter 8) become more valuable when shared. A PM builds a feedback synthesis skill that works well; engineering benefits from it too. An engineer creates a release notes skill; PMs use it for customer communications. The `.claude/commands/` directory in your repository holds team workflows that both roles invoke.

Skills that serve multiple roles. Some workflows benefit everyone:

Release notes generation transforms git history into customer-facing communication. PMs want readable summaries; engineers want accurate change attribution. One skill serves both needs with audience-appropriate output.

Documentation auditing compares docs to implementation. PMs want to know if product descriptions match reality; engineers want to catch stale technical documentation. Same audit logic, same benefit.

Architecture summarization explains codebase structure. PMs use it for onboarding and investigation; engineers use it for new team member context and cross-team collaboration.

Store these shared skills in `.claude/commands/` with clear names: `generate-release-notes.md`, `audit-documentation.md`, `summarize-architecture.md`.

Anyone on the team can invoke them with `/project:generate-release-notes`.

Contribution guidelines. Shared skills need consistency:

Naming conventions make skills discoverable. Use verb-noun format (`generate-release-notes`, `audit-documentation`, `analyze-feedback`) consistently.

Documentation requirements explain what each skill does. Include a header comment with purpose, expected inputs, and output format. Someone encountering the skill for the first time should understand its function without reading the full implementation.

Testing expectations ensure skills work before team adoption. Run the skill against real data, verify output quality, and document limitations. A skill that works 80% of the time with unclear failure modes creates more

frustration than value.

Version control tracks changes. Commit skill modifications with clear messages. When a skill changes behavior, team members should be able to trace why and when.

Quality standards for shared skills. Before adding a skill to the shared library, verify:

- Does it solve a recurring problem? One-off tasks don't need skills.
- Is it reliable? Skills that need constant tweaking aren't ready for sharing.
- Is it documented? Users should understand inputs, outputs, and limitations.
- Is it general enough? Skills that only work for one PM's specific workflow might stay personal.

Quality bar for shared skills is higher than personal skills. Your personal workflow tolerates rough edges because you understand the quirks. Shared skills need to work predictably for everyone.

Deprecation and cleanup. Skills that aren't used should be removed. Skills that break should be fixed or deprecated. Skills that duplicate functionality should be consolidated.

Quarterly skill library review (aligned with CLAUDE.md review) prevents accumulation of broken or unused workflows. Check usage (git log shows who invokes what), verify current functionality, and remove anything that no longer serves the team.

The skill library is a team asset. Treat it with the same care as production code: test before committing, review changes, maintain quality, deprecate what's unused.

You now understand how to collaborate effectively with engineering when both roles use Claude Code. Complementary usage patterns prevent territorial conflicts. Clear handoff protocols respect expertise boundaries while sharing context efficiently. Shared CLAUDE.md practices maintain persistent knowledge both roles benefit from. Team skill libraries encode

workflows that serve the whole organization.

Chapter 13 addresses failure modes: the common mistakes PMs make with Claude Code and how to recognize when you've hit diminishing returns.

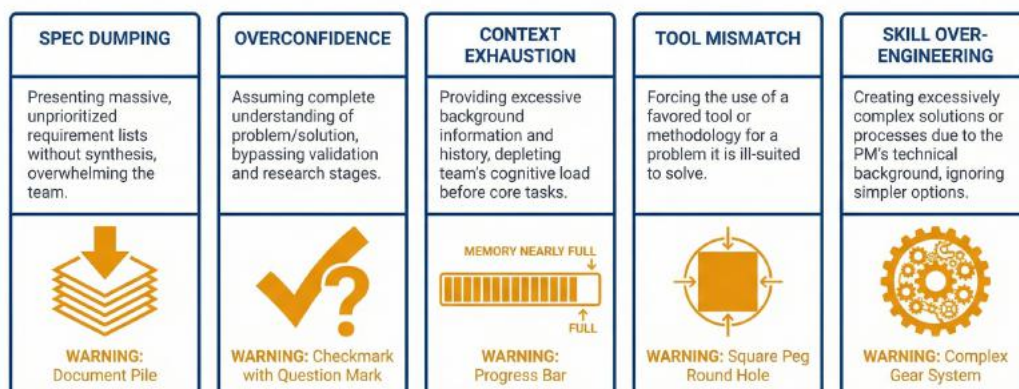
13 Failure Modes and Boundaries

You've read eleven chapters about what Claude Code can do. This chapter is about what goes wrong, when to stop, and where PM usage ends. Every tool has limits. Knowing them before you hit them is cheaper than learning through expensive mistakes.

The teams that get lasting value from Claude Code share a pattern: they fail fast, recognize the failure mode, and adjust. The teams that struggle keep hitting the same walls because they don't recognize the pattern. This chapter names the patterns so you can break them.

13.1 Common PM Failure Modes

Five failure modes account for most PM frustration with Claude Code. Each is predictable, each has warning signs, and each has a straightforward remedy.



The five common PM failure modes with Claude Code

13.1.1 Spec Dumping

What it looks like: You paste an entire PRD into Claude Code and ask it to “implement this feature” or “create user stories for everything.” The response is either superficially generic or Claude Code asks so many clarifying questions that you’ve done more work than if you’d written the stories yourself.

Why it fails: Claude Code works best with focused, scoped requests. A 15-page PRD contains dozens of implicit decisions, unstated assumptions, and context that lives in your head. Dumping it all and expecting magic produces garbage.

The pattern: Large input with vague instruction produces large output with vague utility.

The fix: Break the PRD into specific, actionable requests. “Create user stories for the authentication flow described in section 3.2” is scoped. “Based on the error handling requirements in section 5, identify which existing error messages in `src/errors/` need updates” combines the PRD with codebase investigation. Work through the PRD section by section, getting useful output at each step, rather than expecting one massive synthesis.

Warning sign: Your prompt is longer than 500 words and your instruction is shorter than 20 words. Flip that ratio.

13.1.2 Overconfidence in Code Analysis

What it looks like: Claude Code explains how authentication works in your codebase. You take that explanation to the engineering team as fact. Engineering points out that the analysis missed a critical middleware layer, or misunderstood an async pattern, or described the deprecated flow instead of the current one.

Why it fails: Claude Code reads code statically. It can’t see runtime behavior, configuration-dependent paths, or which code paths are actually exercised in production. It also can’t know which parts of the

codebase are actively maintained versus legacy cruft that nobody touches. Its analysis is often right, but “often” isn’t “always.”

The pattern: Static analysis treated as runtime truth produces confidently wrong conclusions.

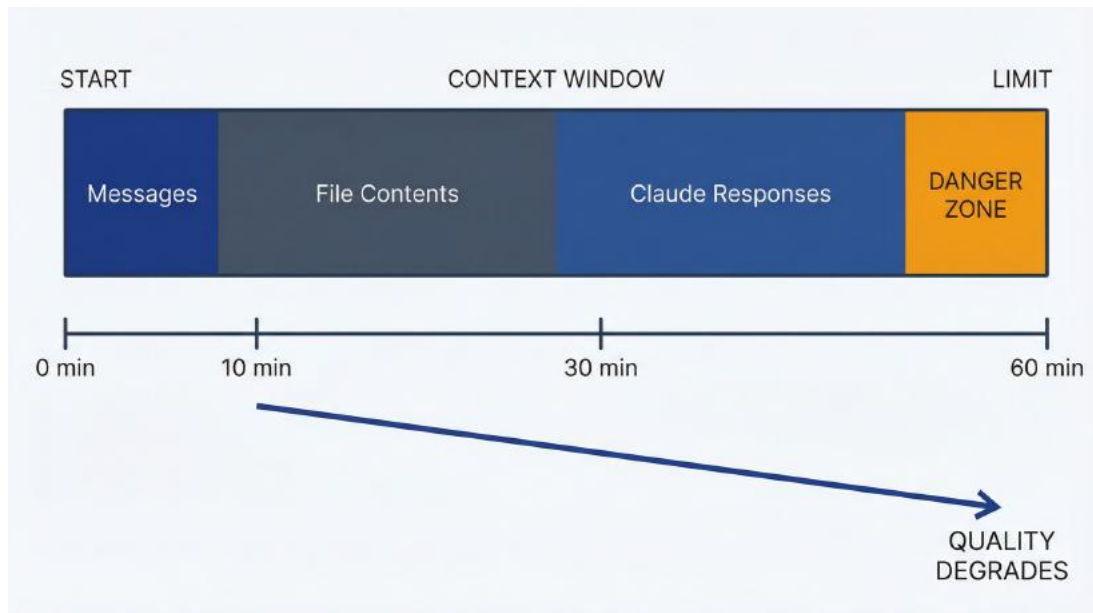
The fix: Frame Claude Code’s analysis as hypothesis, not conclusion. When you share findings with engineering, use language like “Based on my investigation, the authentication flow appears to work this way. Can you confirm?” This invites correction instead of forcing engineers to contradict you. Your investigation adds value by narrowing the problem space, not by delivering final answers.

Warning sign: You’re about to make a product decision or commitment based solely on Claude Code’s code analysis without engineering validation. Pause and verify first.

13.1.3 Context Exhaustion

What it looks like: You’re thirty minutes into a session. Claude Code’s responses become less coherent. It starts suggesting things you already tried. It “forgets” files it read earlier. It references incorrect details from earlier in the conversation.

Why it fails: Every message you send and every file Claude Code reads accumulates in the context window (a 200,000-token buffer that holds your entire session). As context fills, Claude Code must summarize or drop earlier content to fit new content. Important details get lost in the compression.



How context accumulates and quality degrades over a session

Modern Claude models won't silently truncate your context. They return an error rather than degrading invisibly. But before you hit that hard limit, quality degrades as context fills. Users report Claude becoming "noticeably dumber" after context compaction, forgetting which files it was examining and repeating corrections made earlier in the session.

The pattern: Long sessions accumulate context until coherence degrades, then the degradation accelerates.

The fix: Use `/clear` proactively. Start fresh sessions for unrelated tasks rather than continuing a single marathon session. If you need to continue and context is large, use `/compact` with specific instructions about what to preserve: `/compact Focus on the authentication flow analysis; discard the earlier database investigation.`

Warning signs:

- `/cost` shows context approaching 150,000+ tokens
- Claude repeats suggestions you already rejected
- Claude asks questions you already answered
- Claude refers to files incorrectly or conflates different code paths

13.1.4 Tool Mismatch

What it looks like: You're using Claude Code to answer a question that Claude.ai would handle just as well. You've launched a session, waited for startup, typed your question, waited for file scanning it doesn't need, and gotten an answer that required zero codebase access. You've spent two minutes on a task that takes thirty seconds in Claude.ai.

Why it fails: Claude Code's strength is file system access, codebase investigation, and artifact generation. For questions that don't require these capabilities, it adds overhead without value. Every minute spent in unnecessary Claude Code sessions is a minute not spent on work that actually needs the tool.

The pattern: Agentic tool for conversational task creates friction without benefit.

The fix: Before launching Claude Code, ask: "Does this task require reading files, generating artifacts, or executing commands?" If no, use Claude.ai. Chapter 1's decision matrix applies throughout your usage, not just at initial adoption.

Common PM tasks better suited to Claude.ai: - Brainstorming feature ideas - Refining presentation talking points - Editing email drafts - Explaining general concepts (not codebase-specific) - Quick competitive research without file output

Warning sign: You're using Claude Code as your default AI interface instead of choosing based on task fit. Match the tool to the task.

13.1.5 Skill Over-Engineering

What it looks like: You've spent three hours building a comprehensive skill for generating quarterly business reviews. It has five reference files, a complex output template, and handles edge cases you thought of. You've used it exactly once. The next quarter, requirements changed and the skill needed rework that took longer than creating QBR content from scratch.

Why it fails: Skills provide ROI through repetition. A skill that takes two hours to build and saves thirty minutes per use needs four uses to break even. If the task changes frequently, or you only do it annually, the skill never pays back its creation cost.

The pattern: Building infrastructure for one-time tasks creates maintenance burden without efficiency gain.

The fix: Apply the “three times” rule before building a skill. Have you done this task at least twice already, and will you do it at least once more in the next quarter? If not, use a prompt template instead. Save the prompt to a text file for future reference without the overhead of skill structure.

Warning sign: You’re excited about building a skill before you’ve done the underlying task manually. Do it manually first. Understand what varies and what stays constant. Then (maybe) encode it as a skill.

13.2 Knowing When You’ve Extracted the Available Value

Every session has a natural lifespan. Pushing past it wastes time and tokens without additional value.

Signs you’ve extracted available value:

The answers are getting repetitive. Claude Code restates things it already told you, perhaps with different words. You’re asking variations of the same question hoping for new insight. There isn’t more insight in the codebase. You’ve found what’s there.

You understand the shape of the problem. You can explain the relevant code areas, the data flow, the potential failure points. You might not understand every line, but you can have an informed conversation with engineering. Further investigation has marginal returns.

Your questions are getting more specific than the codebase can answer. You’re asking “Why did the team choose this approach?” or

“What happens if 10,000 users do this simultaneously?” These questions require human context or production testing, not code reading. Claude Code has given you what static analysis can provide.

Your hypothesis is testable. You have a specific theory about the bug, the architecture, or the implementation. The next step is validation through engineering review, testing, or production observation, not more investigation.

The “one more prompt” trap: After useful investigation, there’s a temptation to ask one more question, explore one more file, get one more summary. Sometimes this yields value. More often, it’s procrastination disguised as thoroughness. You have enough to act but you’re just not ready to act.

Test yourself: If someone asked you right now “What did you learn?” could you give a coherent answer? If yes, you probably have enough. Document your findings and move on.

Session length guidelines:

Task Type	Typical Useful Length	When to Stop
Quick question	2-5 minutes	First useful answer
Feature investigation	15-30 minutes	You can explain the flow
Bug triage	20-40 minutes	You have a testable hypothesis
Research synthesis	30-60 minutes	Themes are identified and documented
Skill building	45-90 minutes	Skill produces useful output consistently
Deep codebase	60-90 minutes	Diminishing new information per

If you're past these ranges without clear value, you're likely in diminishing returns. Long sessions aren't inherently bad; some investigations genuinely require depth. But if you're at ninety minutes and still not finding answers, the answer might not exist in the code, or you're asking the wrong questions.

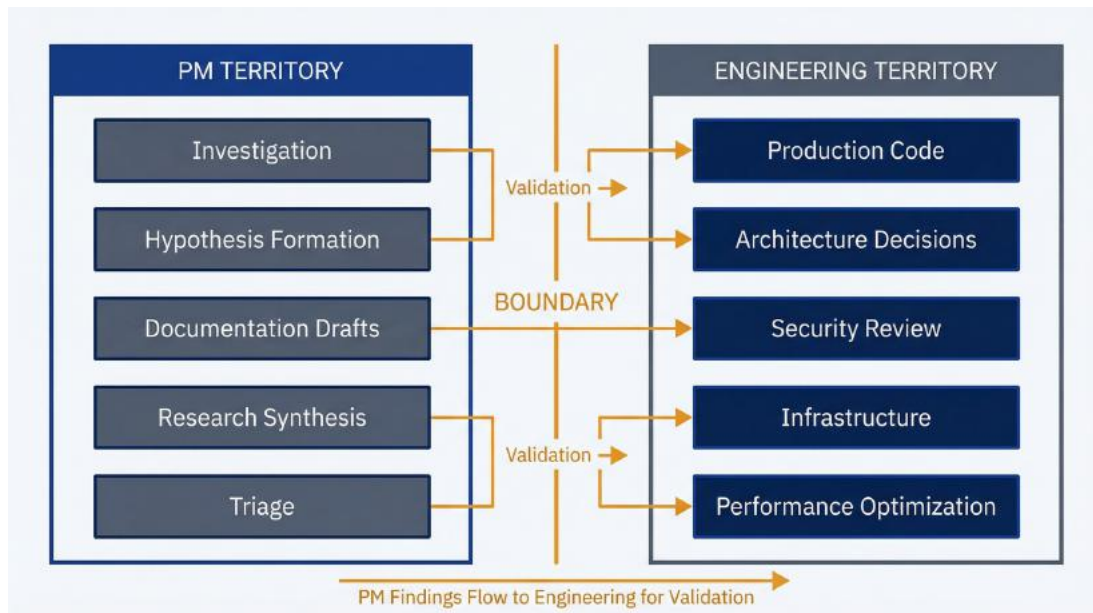
When to start fresh: Use `/clear` and begin a new session when:

- You've completed one task and are starting an unrelated task
- Context exhaustion symptoms appear
- You've gone down an unproductive path and want to try a different approach
- Your prompts are getting long and convoluted, trying to provide context Claude already "should" have
- More than two hours have passed since session start

Starting fresh costs you nothing but the thirty seconds to reorient. Continuing a degraded session costs ongoing frustration and compounding confusion.

13.3 Tasks That Belong to Engineering

Some tasks fall outside PM scope regardless of tooling. Claude Code doesn't change who owns what. It changes how efficiently you work within your scope.



The boundary between PM territory and Engineering territory

13.3.1 Tasks That Require Engineering Ownership

Writing production code. Claude Code can generate code. It can even generate code that works. But code that runs in production requires engineering review, testing, and ownership. Even if you could merge code directly, you shouldn't. The issue isn't capability; it's accountability. When that code breaks at 3 AM, an engineer is getting paged, not you.

Architectural decisions. Your investigation might reveal that the authentication system is convoluted or the database schema is suboptimal. Claude Code might even suggest improvements. These are input to engineering decisions, not decisions themselves. Architecture choices have long-term consequences that require engineering judgment.

Performance optimization. Claude Code's static analysis can identify potential performance issues but can't measure actual performance. Optimization requires profiling production load, understanding usage patterns, and making trade-offs that affect system behavior. Engineering owns this.

Infrastructure changes. Deployment configurations, database

migrations, server provisioning: these are engineering territory. Claude Code can help you understand existing infrastructure or draft documentation about it, but changing it requires engineering ownership.

13.3.2 Security Work You Should Always Escalate

Authentication and authorization review. If you're investigating authentication flows or access control logic, your findings are hypotheses for security review, not conclusions. Security bugs can expose customer data, damage reputation, and create legal liability. Even if Claude Code's analysis looks correct, security review must involve specialists.

Handling of secrets and credentials. Claude Code might reveal how your application stores API keys or manages tokens. This information is sensitive. Don't share it broadly. If you discover potential security issues (hardcoded secrets, insufficient encryption), escalate to engineering and security. Don't try to fix it.

Compliance-related code. If code touches compliance requirements (GDPR, HIPAA, SOC 2, PCI DSS), Claude Code's analysis is input to compliance review, not a substitute for it. Compliance mistakes have regulatory consequences.

13.3.3 Why Production Access Is Off-Limits

Database queries on production data. Chapter 10 covered database MCP integration with explicit guidance: use read-only replicas, never production databases. Even read operations on production databases can cause performance issues, and any write operation could corrupt data. Claude Code's ability to generate SQL doesn't mean it should execute on production.

Production logs and monitoring. If you need to read production logs for investigation, do it through your organization's established access patterns, not through Claude Code MCP connections. Production access usually requires specific permissions and audit trails that Claude Code

bypasses.

Live customer sessions. Debugging customer issues might tempt you to access their actual data. Don't. Use anonymized exports, test accounts, or staging environments. Customer data access has privacy implications regardless of your intent.

13.3.4 Protecting Privacy in Your Analysis

PII in analysis. When you feed customer feedback to Claude Code for synthesis, strip personally identifiable information first. Names, email addresses, phone numbers: remove them before processing. Claude Code's analysis doesn't improve with PII, and you avoid privacy risks.

Raw support tickets. Support tickets often contain customer names, account details, and sensitive information. Extract the relevant content (the bug report, the feature request) without the identifying wrapper. Your synthesis is about patterns, not individuals.

Export controls. If your product serves regulated industries, understand what data can be processed by external AI services. Some organizations prohibit sending certain data types to cloud AI providers. Claude Code running locally is different from API calls, but check with your legal and security teams.

13.3.5 Meeting Regulatory Requirements

Audit trails. Claude Code sessions are stored locally on your machine. If your organization requires audit trails for certain decisions or documentation, Claude Code's session history might not meet those requirements. Capture outputs in your organization's official systems of record.

Change documentation. If you use Claude Code to generate documents that affect product decisions (PRDs, user stories, requirements), those documents should go through your normal review and approval processes. Claude Code generated the draft; you're accountable for the content.

Regulated communications. Some industries require specific formats or review processes for customer-facing documentation. Claude Code can help draft content, but the content must flow through established approval workflows before publication.

13.4 Habits That Make Claude Code Work Long-Term

Using Claude Code effectively over months and years requires habits, not heroics.

Weekly usage review. Spend ten minutes each week reviewing:

- What tasks did Claude Code help with?
- What worked well? What was frustrating?
- Did I hit any failure modes? Which ones?
- Are my skills still accurate, or do they need updates?

This reflection compounds. After a month, you'll have identified your personal patterns: tasks where Claude Code excels for you, failure modes you're prone to, skills worth maintaining. Without reflection, you repeat mistakes indefinitely.

Cost monitoring habits. For API users: Check your Anthropic dashboard weekly for the first month, monthly thereafter. For subscription users: Check `/cost` at session end occasionally to understand your usage patterns. Both groups: Review whether spending aligns with value received. If costs feel high relative to value, you're using Claude Code for wrong tasks. If costs feel low relative to value, you might use it more proactively.

Skill maintenance schedule. Skills rot. Your codebase changes, requirements evolve, better approaches emerge. Review each skill every quarter:

- Does this skill still produce useful output?
- Has the task it automates changed significantly?
- Can it be simplified or combined with other skills?

- Should it be retired?

Skills are an investment. Like all investments, they require maintenance to stay valuable.

Staying current with Claude Code updates. Claude Code updates regularly with new capabilities, changed behaviors, and improved models. Check the changelog monthly (code.claude.com/changelog or release notes in the CLI). New features might solve problems you're currently working around. Changed behaviors might break workflows you depend on.

Subscribe to update channels if available. When you encounter unexpected behavior, check if there was a recent update before assuming you did something wrong.

The sustainable rhythm: Weekly reflection, quarterly skill review, monthly update check. These habits take thirty minutes a month in aggregate. They prevent the accumulation of technical debt in your personal Claude Code usage: outdated skills, repeated mistakes, missed capabilities.

You now know the five common PM failure modes and their remedies, how to recognize diminishing returns and when to stop, where PM usage has hard boundaries, and how to build sustainable long-term practices. This chapter is less exciting than chapters about capabilities, but knowing when to stop and where not to go is what separates effective users from frustrated ones.

This completes Part VI and the main content of the book. The appendices that follow provide reference material: command syntax, skill templates, sample CLAUDE.md files, and prompt templates for quick lookup during your work.

Appendix A: Command Reference

This reference covers Claude Code commands and options relevant to PM workflows. Run `/help` in any session for the complete, current list—Claude Code adds features regularly.

Commands You'll Use Every Session

Commands you type during a session. All start with `/`.

Session Management

Command	Purpose
<code>/help</code>	Display all available commands, including custom commands from your project
<code>/status</code>	Show current session state: permission mode, context usage, model, authentication method
<code>/cost</code>	Display token consumption and estimated session cost
<code>/clear</code>	Reset conversation and start fresh in the same directory
<code>/compact [focus]</code>	Summarize context to reduce tokens. Optional: specify what to preserve
<code>/exit</code>	End the session

Context and Memory

Command	Purpose
<code>/init</code>	Create a new CLAUDE.md file with a documentation template
<code>/memory</code>	View or edit the project's CLAUDE.md file
<code>/context</code>	Show current context window usage and remaining capacity
<code>/resume [id]</code>	Continue a previous session. Omit ID to see recent sessions

Configuration

Command	Purpose
<code>/config</code>	Open settings interface for Claude Code preferences
<code>/permissions</code>	Manage tool permissions and confirmation prompts
<code>/model</code>	Switch Claude models (Sonnet, Opus, Haiku) for the current session
<code>/login</code>	Authenticate with your Claude account
<code>/logout</code>	Sign out from current session

Integrations

Command	Purpose
<code>/mcp</code>	Configure Model Context Protocol servers
<code>/agents</code>	Create and manage subagents with specialized prompts
<code>/bashes</code>	List and manage background shell processes

Utilities

Command	Purpose
/doctor	Run diagnostic health check on your installation
/terminal-setup	Install Shift+Enter keybinding for multi-line input (macOS)
/vim	Toggle Vim editing mode
/release-notes	View recent Claude Code updates
/bug	Report bugs (sends conversation to Anthropic)

Launching Claude Code with Options

Options when launching Claude Code from your terminal: `claude [flags] [query]`

Permission Control

Flag	Example	Purpose
<code>--permission-mode</code>	<code>claude --permission-mode plan</code>	Set initial mode. Options: default, plan, acceptEdits, bypassPermissions
<code>--dangerously-skip-permissions</code>	(use with caution)	Bypass all permission prompts. Only for trusted environments
<code>--allowedTools</code>	<code>claude --allowedTools "Bash(git:*)"</code>	Specify tools that don't require prompts
<code>--disallowedTools</code>	<code>claude --disallowedTools "Bash(rm:*)"</code>	Explicitly deny specific tools

Session Options

Flag	Example	Purpose
<code>--continue</code> or <code>-c</code>	<code>claude -c</code>	Continue most recent session
<code>--resume</code>	<code>claude --resume abc123</code>	Resume specific session by ID
<code>--add-dir</code>	<code>claude --add-dir /path/to/repo</code>	Add directory to allowed paths
<code>--model</code>	<code>claude --model opus</code>	Select Claude model (aliases: sonnet, opus)
<code>--print</code> or <code>-p</code>	<code>claude -p "query"</code>	Run query once and exit (useful for scripts)

System Prompt

Flag	Example	Purpose
<code>--append-system-prompt</code>	<code>claude --append-system-prompt "Focus on..."</code>	Add instructions to default prompt (recommended)
<code>--system-prompt</code>	<code>claude --system-prompt "You are..."</code>	Replace entire system prompt
<code>--system-prompt-file</code>	<code>claude --system-prompt-file ./prompt.txt</code>	Load system prompt from file

Help

Flag	Purpose
<code>--help</code>	Display CLI usage information
<code>--version</code>	Show installed version number

Controlling Claude Code’s Autonomy

Set via `--permission-mode` flag or cycle with `Shift+Tab` during a session.

Mode	Behavior	PM Use Case
plan	Read-only. No file modifications, no command execution. Enables extended thinking.	Codebase investigation, bug triage, architecture review
default	Prompts before all modifications and commands. Maximum control.	Learning Claude Code, sensitive work
acceptEdits	Auto-approves file edits. Still prompts for shell commands.	Generating documents, creating skills, updating CLAUDE.md
bypassPermissions	Skips all prompts. Requires <code>--dangerously-skip-permissions</code> flag.	Rarely needed for PM work

Recommendation: Set `plan` as your default mode in `~/.claude/settings.json`. Cycle to `acceptEdits` when you need file output.

Faster Navigation and Control

Mode Control

Shortcut	Function
Shift+Tab	Cycle permission modes: <code>default</code> → <code>acceptEdits</code> → <code>plan</code>
Tab	Toggle extended thinking (step-by-step reasoning)

Esc Esc

Rewind: undo last changes or conversation turn

Input and Navigation

Shortcut	Function
Shift+Enter	New line without sending (requires /terminal-setup on macOS)
Ctrl+C	Interrupt current operation
Ctrl+D	Exit CLI
Ctrl+L	Clear screen
Ctrl+O	Toggle verbose mode (show extended thinking output)

Special Input Prefixes

Prefix	Function	Example
#	Add note to CLAUDE.md	# Auth uses JWT tokens
@	File path autocomplete	@src/components/
/	Slash command	/cost

Where Settings Live

Location	Scope	Purpose
~/.claude/settings.json	All projects	Global preferences, default mode
.claude/settings.json	Project (shared)	Team settings, committed to git
.claude/settings.local.json	Project (personal)	Local overrides, gitignored

Setting a Default Mode

Create or edit `~/.claude/settings.json`:

```
{  
  "defaultMode": "plan"  
}
```

Custom Slash Commands

Create Markdown files in `.claude/commands/` (project) or `~/.claude/commands/` (personal) to add custom slash commands.

Features:

- `$1`, `$2`, `$@` for positional arguments
- `!command` prefix to embed bash output
- Subdirectories create namespaced commands (e.g., `pm/feedback.md` becomes `/pm:feedback`)

Example: `.claude/commands/investigate.md`

Investigate how `$1` works in this codebase. Focus on:

- User-facing behavior
- Data flow
- Key decision points

Summarize in terms a product manager would use.

Usage: `/investigate authentication`

Quick Reference Card

Start safe: `claude --permission-mode plan`

Check costs: `/cost`

Reduce context: `/compact`

See what's available: `/help`

Cycle modes: `Shift+Tab`

Exit cleanly: `/exit`

Commands and options are current as of publication. Run `/help` and `claude --help` for the authoritative list in your installed version.

Appendix B: SKILL.md

Template

Copy this template when creating new skills. See Chapter 9 for complete examples and Chapter 8 for design patterns.

Minimal Template

For simple skills with straightforward workflows:

```
---
name: my-skill-name
description: What this skill does and when to use it. Include trigger
words Claude should recognize.
---
```

My Skill Name

Purpose

One sentence explaining the goal.

Inputs

– What files or information the user provides

Process

1. Step one
2. Step two
3. Step three

Output

What gets generated and where it goes.

Example minimal skill:

```
---
name: standup-notes
description: Generate standup notes from yesterday's git commits. Use
```

when user mentions `standup`, `daily update`, or what I did yesterday.

Standup Notes

Purpose

Create formatted standup notes from recent commits.

Inputs

– Git repository with commit history

Process

1. Read commits from the past 24 hours
2. Group by type (feature, fix, refactor)
3. Write in first person, past tense

Output

Print standup notes to terminal. Do not create a file.

Comprehensive Template

For complex skills requiring detailed instructions:

name: my-skill-name
description: Detailed description of what this skill does. Mention specific trigger phrases: `analysis`, `synthesis`, `report generation`. Auto-invoke when user mentions `[keyword1]`, `[keyword2]`, or `[keyword3]`.

My Skill Name

Purpose

Explain the goal and value of this skill in 2–3 sentences. What problem does it solve? What outcome does it produce?

Expected Inputs

- ****Required:**** File types, formats, or information the user must provide
- ****Optional:**** Additional context that improves results
- **Supports:** List compatible formats (CSV, JSON, plain text)

Process

Follow these steps in order:

1. ****Read and validate inputs****
 - Check that required files exist
 - Identify relevant columns or fields
2. ****Analyze content****
 - Describe the analysis methodology
 - Include specific criteria or thresholds
3. ****Generate output****
 - Specify the output format
 - Note any formatting requirements
4. ****Quality check****
 - Verify output meets criteria before finishing

Output Format

Generate file: `path/to/output-[date].md`

Structure:

Section One

[What goes here]

Section Two

[What goes here]

Quality Criteria

- Criterion one (specific and measurable)
- Criterion two
- Criterion three

Edge Cases

- ****Small datasets:**** How to handle insufficient data
- ****Malformed input:**** What to do when input is incomplete
- ****Ambiguous requests:**** How to interpret unclear instructions

Usage Examples

****Standard:**** "Run [skill name] on [file]"

****Detailed:**** "Run [skill name] on [file] with comprehensive analysis"

****Quick:**** "Run [skill name] on [file], just the summary"

Field Reference

Field		Required	Purpose
name	Yes		Unique identifier, lowercase with hyphens
description	Yes		Discovery text—Claude uses this to decide when to activate. Include trigger words.

Section Guidance

Purpose: One paragraph maximum. Lead with what the user gets, not what the skill does internally.

Expected Inputs: Be specific about formats. “CSV file” is too vague; “CSV with feedback text in a column” tells Claude what to look for.

Process: Number your steps. Use imperative mood (“Read the file” not “The file should be read”). Include decision points (“If X, then Y”).

Output Format: Show the exact structure. Use code blocks for templates. Specify the file path pattern.

Quality Criteria: Make these testable. “Good quality” is useless; “Quotes are verbatim, not paraphrased” is actionable.

Edge Cases: Document the three most common failure scenarios. This saves debugging time later.

File Structure

```
.claude/skills/  
└─ my-skill-name/  
    └─ SKILL.md           # Required  
    └─ scripts/          # Optional helper scripts  
        └─ helper.py  
    └─ resources/        # Optional templates, rubrics  
        └─ template.md
```

Location options: - `.claude/skills/` — Project-specific, shared with team via git - `~/.claude/skills/` — Personal, available in all projects

Common Mistakes

Vague description: “Helps with feedback” won’t trigger. Use:

“Synthesizes customer feedback into themed reports. Auto-invoke when user mentions feedback analysis, NPS synthesis, or support ticket themes.”

Missing trigger words: Claude matches your request to skill descriptions. If your description doesn't include words users actually say, the skill won't activate.

Overly long instructions: Keep SKILL.md under 5,000 words. Move reference material to separate files in resources/.

No quality criteria: Without explicit criteria, output quality varies. Define what “done well” looks like.

After creating a skill, restart Claude Code to load it. Test with real data before committing.

Appendix C: Sample CLAUDE.md for PM Workflows

Copy and adapt this template for your project. See Chapter 3 for detailed guidance on each section.

File Locations

Claude Code reads CLAUDE.md files from multiple locations, loaded in this order:

Location	Scope	Use Case
~/.claude/CLAUDE.md	All projects	Personal preferences across repositories
./CLAUDE.md	This project	Team-shared context (commit to git)
./CLAUDE.local.md	This project	Personal overrides (gitignored)
.claude/rules/*.md	This project	Split large configs into focused files

All levels are loaded and combined. Project-level settings override user-level for conflicts.

Sample CLAUDE.md

```
# Acme Dashboard – Product Context

## About This Product
```

Acme Dashboard is a B2B analytics platform for e-commerce teams. Users connect their store data, view sales metrics, and create custom reports. Primary users are e-commerce managers and marketing analysts at mid-market retailers.

Product Terminology

- **Workspace**: A team's shared environment. Users can belong to multiple workspaces. Not the same as "account" (billing entity).
- **Connector**: Integration with external data source (Shopify, Google Analytics). Each workspace can have multiple connectors.
- **Widget**: A single chart or metric on a dashboard. Widgets pull from one or more data sources.
- **Pipeline**: Data processing job that syncs connector data. Runs hourly by default. Not related to sales pipelines.
- **Activation**: When a user completes onboarding and creates their first dashboard. Different from account activation (email verification).

Key User Journeys

New User Onboarding

1. Sign up and email verification → `src/auth/`
2. Workspace creation → `src/services/workspace-service.js`
3. First connector setup → `src/connectors/`
4. Dashboard creation → `src/components/DashboardBuilder/`

Report Generation

- Entry point: `src/components/ReportBuilder/`
- Data queries: `src/services/query-engine/`
- Export: `src/services/export-service.js`
- Scheduling: `src/services/scheduler/`

Billing and Subscription

- Pricing logic: `src/services/billing/pricing.js`
- Stripe integration: `src/services/billing/stripe-client.js`
- Usage metering: `src/services/billing/usage-tracker.js`

Business Rules

- Free tier: 1 workspace, 2 connectors, 30-day data retention
- Pro tier: 5 workspaces, unlimited connectors, 1-year retention
- Enterprise: Custom limits, SSO, dedicated support
- Connector sync failures retry 3x before alerting user
- Dashboard exports limited to 10,000 rows (prevent memory issues)
- Widget refresh rate minimum 15 minutes (API rate limiting)

Team Conventions

- All user-facing strings use i18n system in `src/locales/`
- Pricing calculations happen server-side only (never in frontend)
- Jira ticket IDs in commit messages: "Add export feature (ACME-1234)"
- Feature flags managed in `src/config/features.js`
- Database migrations require data team approval

Code Patterns

- API endpoints follow REST conventions in `src/api/routes/`
- Business logic lives in `src/services/`, not controllers
- React components use hooks pattern, no class components
- Tests colocated with source files: `Component.test.tsx`

Common PM Questions

- "How does pricing work?" → `src/services/billing/pricing.js`
- "Where is email content?" → `src/templates/email/`
- "How do we track usage?" → `src/services/billing/usage-tracker.js`
- "What triggers notifications?" → `src/services/notifications/`
- "Where are feature flags?" → `src/config/features.js`

External Resources

- Product strategy: [internal wiki link]
- Design system: [Figma link]
- API docs: [internal docs link]
- Analytics: [dashboard link]

Section-by-Section Guidance

About This Product (2-3 sentences): What does it do? Who uses it? This orients Claude Code before any investigation.

Product Terminology (5-10 terms): Define words that have specific meaning in your domain or could be confused with common terms. Prevents misinterpretation.

Key User Journeys (3-5 flows): Map critical user paths to code locations. Claude Code knows where to look without exploration. Include file paths, not just descriptions.

Business Rules (5-10 rules): Document logic that isn't obvious from code—limits, constraints, pricing tiers, retry policies. Helps Claude Code explain behavior correctly.

Team Conventions (5-8 conventions): How your team works. Prevents suggestions that violate practices. Include commit message format, code organization rules, approval requirements.

Code Patterns (3-5 patterns): High-level architecture guidance. Where does business logic live? What patterns does the codebase follow?

Common PM Questions (5-10 entries): Quick-reference for frequent investigations. Save time by pointing directly to relevant files.

External Resources: Links Claude Code can't follow, but useful reminders for you during sessions.

Size Guidelines

- Keep under 100 lines for most projects
- Each section should be scannable, not narrative
- Use bullet points, not paragraphs
- If exceeding 150 lines, split into `.claude/rules/` directory

The sample above is approximately 80 lines and consumes roughly 2,000 tokens per session—minimal cost for significant context improvement.

Quick Start

Run `/init` in any project to generate a starter `CLAUDE.md` based on codebase analysis. Edit the generated file to add product context, terminology, and PM-specific sections.

Use `/memory` during sessions to verify what's loaded.

Update your `CLAUDE.md` as your understanding grows. When you learn something valuable during investigation, add it. Living documentation beats stale documentation.

Appendix D: Prompt Templates

Copy and adapt these prompts for common PM workflows. Placeholders in [brackets] need your input. See the referenced chapters for detailed context and examples.

Investigation Prompts

Use these in plan mode (`--permission-mode plan`) for safe exploration.

Architecture and Features

Architecture Overview

Explain this codebase's architecture to a product manager. Cover the main components, how data flows through the system, key abstractions, and where business logic lives. Skip implementation details—focus on concepts.

Feature Implementation

How does [feature name] work in this application? Specifically:

- What triggers it (user action, scheduled job, external event)?
- What data does it read and write?
- What external services does it call?
- What are the success and failure paths?

Critical Files

I need to understand the [feature] feature. What are the five most important files I should know about? For each file, explain what it owns and why it matters.

Data and Behavior

Data Access

What user data does the [feature] feature access? Show me which database tables or API endpoints it reads from, what fields it uses, and whether it writes anything back.

Edge Case Investigation

What happens when [condition] occurs during [process]? Walk me through the code path, error handling, and user-facing behavior.

Behavior Explanation

Users report that [observed behavior]. Why does this happen? Is it intentional or a bug?

Bug Investigation

Symptom Translation

A user reports: “[paste exact complaint]”

Translate this into technical terms. What components might be involved? What should I investigate first?

Code Area Identification

I’m investigating a user-reported issue: [one-sentence summary]

Which code areas should I examine? List relevant files, functions, and data flows.

Issue Layer Identification

A user experiences [symptom]. Help me determine if this is likely:

- Frontend (UI, client-side state, browser)
- Backend (API, server logic, processing)
- Data (database, cache, data integrity)

- Integration (third-party services, external APIs)
- Infrastructure (network, hosting, configuration)

Explain your reasoning.

Configuration Investigation

What configuration controls [feature]? Show me feature flags, environment variables, and settings that affect its behavior.

Access Control Investigation

How are permissions checked for [feature]? Show me where access is validated and what roles or conditions are required.

Generation Prompts

Use these in acceptEdits mode for file creation.

Requirements and Stories

User Story Generation

Generate user stories for the [feature name] feature in docs/user-stories/[feature-name].md.

For each story, include:

- User story in standard format (As a... I want... So that...)
- Acceptance criteria (Given/When/Then)
- Edge cases and error states
- Dependencies on other features

Reference the codebase for technical constraints.

PRD Generation

Create a PRD for [feature name] in docs/prds/[feature-name].md.

Include: Problem statement, success metrics, user stories, scope (in/out), technical considerations, open questions.

Reference existing implementation patterns in this codebase.

Persona Generation

Based on the research materials in [research folder], create persona documents in docs/personas/.

For each distinct user type, include: Name, role, goals, frustrations, key behaviors, quotes from research.

Documentation

Changelog Generation

Generate a changelog entry for version [X.Y.Z] by reviewing git commits since [last version tag].

Format for external users: focus on benefits, not implementation.
Group by: New Features, Improvements, Bug Fixes.

README Update

Review and update README.md to reflect recent changes. Ensure the getting started instructions, feature list, and examples match current behavior.

Bug Report Generation

Based on my investigation, generate a bug report including:

- Summary (one sentence)
- Steps to reproduce
- Expected vs. actual behavior
- Affected code areas (files, functions)
- Hypothesis on root cause
- Suggested investigation path for engineering

Analysis Prompts

Use these for synthesis and insight generation.

Feedback and Research

Theme Identification

Read all feedback in [feedback folder] and identify the top 10-15 themes that appear repeatedly.

For each theme: Name, description, frequency (how many mentions), representative quotes (3-5 verbatim examples).

Feedback Classification

Classify feedback in [feedback folder] into these categories: [category 1], [category 2], [category 3], [other].

Create a summary showing count per category, sentiment breakdown, and notable patterns.

Interview Synthesis

Read interview transcripts in [interview folder] and synthesize patterns across interviews.

Identify: Common pain points, unmet needs, workarounds users have created, emotional reactions, feature requests (explicit and implied).

Insight-to-Action Mapping

Review the theme report in [report file] and map each major theme to potential product actions.

For each theme suggest: Quick wins, larger initiatives, and things to monitor but not act on yet.

Market and Competition

Competitor Profile

Research [Competitor Name] using web search and create a profile in `research/competitors/[competitor-name].md`.

Include: Company overview, target market, core features, pricing model, positioning, strengths, weaknesses, recent news.

Competitive Matrix

Based on competitor profiles in `research/competitors/`, create a comparison matrix in `research/competitors/competitive-matrix.md`.

Compare: Target audience, pricing, key features, integrations, differentiators.

Pricing Analysis

Research [Competitor Name]'s pricing and create a breakdown in `research/pricing/[competitor-name]-pricing.md`.

Include: Pricing model (per seat, usage, flat), tier structure, feature gates, published prices, discounting patterns if known.

Market Sizing

Help me size the market for [product/category] using TAM/SAM/SOM. Create `research/market-sizing/market-size.md`.

For each level, document: Methodology, data sources, assumptions, calculation, confidence level.

Validation and Audit

PRD Audit

Review [PRD file] for completeness. Check:

- Clear problem statement with evidence
- Measurable success metrics
- User stories with acceptance criteria
- Explicit scope boundaries
- Technical considerations addressed
- Dependencies identified
- Open questions listed

Flag gaps and suggest improvements.

Documentation Audit

Compare documentation in docs/ to current codebase. Identify:

- Outdated information
- Missing documentation for implemented features
- Incorrect examples or code snippets
- Broken internal references

Persona Validation

Review personas in docs/personas/ against source research in [research folder].

For each persona, verify: Claims are supported by research, quotes are accurate, no invented details.

Quick Customization Guide

Make prompts more specific by adding constraints:

- “Focus on the user-facing behavior, not internal implementation”
- “Keep the output under 500 words”
- “Format as a table for easy scanning”
- “Include file paths for all code references”

Make prompts more thorough by requesting artifacts:

- “Save the analysis to [specific file path]”

- “Include a summary section at the top”
- “List your assumptions explicitly”
- “Rate your confidence in each finding”

Chain prompts for complex workflows:

1. Run investigation prompt to gather context
2. Use `/compact` to summarize findings
3. Run generation prompt with context established

Prompts work best when you’ve established context first. Read relevant files, explain your goal, then use these templates. See individual chapters for full workflows and expected outputs.